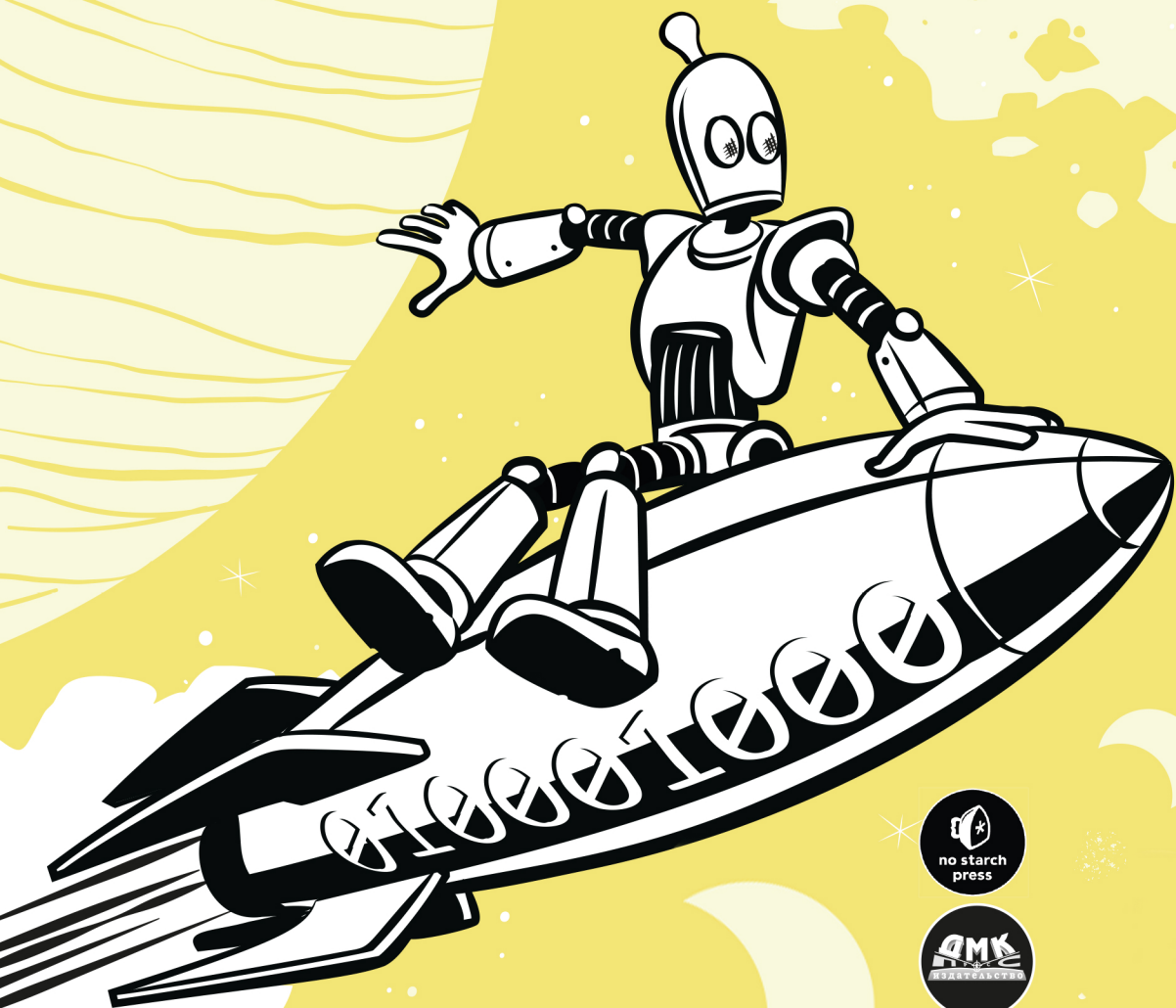


ИСКУССТВО WEBASSEMBLY

Рик Баттаљини



THE ART OF WEBASSEMBLY

**Build Secure, Portable,
High-Performance
Applications**

Rick Battagline



**no starch
press**

San Francisco

ИСКУССТВО WEBASSEMBLY

**Создание безопасных
межплатформенных
высокопроизводительных
приложений**

Рик Батталини



Москва, 2022

УДК 004.9
ББК 32.072
Б28

Баттальяни Р.

Б28 Искусство WebAssembly / пер. с англ. П. М. Бомбаковой. – М.: ДМК Пресс, 2021. – 310 с.: ил.

ISBN 978-5-97060-976-7

В книге подробно рассматриваются принципы работы WebAssembly – компактной межплатформенной технологии, которая оптимизирует производительность ресурсоемких веб-приложений и программ.

Вы узнаете, как оптимизировать, компилировать и отлаживать низкоуровневый код, сравнивать его производительность с JavaScript, а также представлять код в удобном для прочтения текстовом формате WebAssembly Text (WAT). Затем сможете создать программу обнаружения столкновений на базе браузера, поработать с технологиями рендеринга в браузере для создания графики и анимации и выяснить, как WebAssembly взаимодействует с другими языками программирования.

Книга адресована веб-разработчикам, желающим понять, как создавать и разворачивать приложения на основе WebAssembly, а также пользователям, которые хотят изучить и применять эту технологию.

УДК 004.9
ББК 32.072

Title of English-language original: The Art of WebAssembly: Build Secure, Portable, High-Performance Applications, ISBN 9781718501447, published by No Starch Press Inc. 245 8th Street, San Francisco, California United States 94103. The Russian-Language 1st edition Copyright © 2021 by ДМК Пресс Publishing under license by No Starch Press Inc. All rights reserved.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 978-1-7185-0144-7 (англ.)
ISBN 978-5-97060-976-7 (рус.)

© Rick Battagline, 2021
© Перевод, издание,
оформление, ДМК Пресс, 2021

*В память о моей бабушке Сью Баттальяни (Sue Battagline).
Я очень скучаю по тебе.*

СОДЕРЖАНИЕ

<i>От издательства</i>	10
<i>Об авторе</i>	11
<i>О техническом рецензенте</i>	11
<i>Предисловие</i>	12
<i>Благодарности</i>	13
<i>Введение</i>	14
Глава 1. Введение в WebAssembly	19
Что такое WebAssembly?	20
Причины использовать WebAssembly	21
Повышение производительности	22
Интеграция существующих библиотек	22
Портируемость на другие платформы и безопасность	23
Противники JavaScript	23
Связь WebAssembly с JavaScript	24
Зачем учить WAT?	25
Стили кодирования WAT	26
Среда встраивания	30
Браузер	31
WASI	31
Visual Studio Code	32
Node.js	33
Наше первое приложение WebAssembly с помощью Node.js	35
Вызов модуля WebAssembly из Node.js	36
Синтаксис <code>.then</code>	37
Удачное время	38
Глава 2. Основы работы с WebAssembly Text	39
Написание простейшего модуля	40
Hello World в WebAssembly	40
Создание WAT-модуля	41
Создание файла JavaScript	43
Переменные WAT	46
Глобальные переменные и преобразование типов	46
Локальные переменные	50
Распаковка S-выражений	52
Переменные с индексами	54
Преобразование между типами	54
Условные операторы <code>if/else</code>	56
Операторы цикла и блока	58

Оператор блока (block)	59
Оператор цикла (loop)	61
Совместное использование операторов блока и цикла	62
Переход с помощью br_table	64
Заключение	66
Глава 3. Функции и таблицы	67
Когда следует вызывать функции из WAT	68
Разработка функции is_prime	68
Передача параметров	68
Создание внутренних функций	69
Функция is_prime	71
Код на стороне JavaScript	75
Объявление импортированной функции	77
Числа JavaScript	78
Передача типов данных	78
Объекты в WAT	78
Влияние вызовов внешних функций на производительность	79
Таблицы функций	83
Создание таблицы функций в WAT	83
Заклучение	92
Глава 4. Низкоуровневые битовые операции	93
Системы счисления: двоичная, десятичная и шестнадцатеричная	94
Арифметические операции над целыми числами и числами с плавающей запятой	95
Целые числа	96
Числа с плавающей запятой	98
Биты старшего и младшего разрядов	101
Битовые операции	103
Сдвиг и вращение битов	103
Маскирование битов с помощью AND и OR	105
Инверсия битов с помощью XOR	108
Обратный vs. прямой порядок байтов	109
Заклучение	110
Глава 5. Строки в WebAssembly	111
ASCII и Unicode	112
Строки в линейной памяти	112
Передача длины строки в JavaScript	113
Строки с завершающим нулем	114
Строки с префиксом длины	117
Копирование строк	120
Создание числовых строк	126
Создание шестнадцатеричной строки	131
Создание двоичной строки	136
Заклучение	139

Глава 6. Линейная память	140
Линейная память в WebAssembly	141
Страницы	142
Указатели	144
Объект памяти JavaScript	146
Создание объекта памяти WebAssembly	146
Запись в консоль в цвете	148
Создание JavaScript в store_data.js	149
Обнаружение столкновений	151
Начальный адрес, шаг и сдвиг	152
Загрузка структур данных из JavaScript	153
Отображение результатов	155
Функция обнаружения столкновений	156
Заключение	164
 Глава 7. Веб-приложения	166
DOM	167
Создание и настройка простого сервера Node	167
Первое веб-приложение WebAssembly	169
Определение HTML-заголовка	170
JavaScript	170
HTML-тег <body>	173
Готовое веб-приложение	173
Шестнадцатеричные и двоичные строки	175
HTML	175
WAT	178
Компиляция и запуск	183
Заклучение	184
 Глава 8. Работа с Canvas	186
Рендеринг HTML-страницы на холсте	187
Определение холста в HTML	187
Определение констант JavaScript в HTML	188
Создание случайных объектов	190
Данные растрового изображения	191
Функция requestAnimationFrame	192
Модуль WAT	194
Импортируемые значения	194
Очистка холста	195
Функция вычисления абсолютного значения	196
Установка цвета пикселя	197
Рисуем объект	200
Установка и получение атрибутов объекта	202
Функция \$main	204
Компиляция и запуск приложения	213
Заклучение	214

Глава 9. Оптимизация производительности	216
Использование профилировщика	217
Профилировщик Chrome	217
Профилировщик Firefox	224
wasm-opt	228
Установка Binaryen	228
Запуск wasm-opt	228
Взглянем на оптимизированный код WAT	230
Приемы повышения производительности	231
Встраивание функций	231
Умножение и деление vs. сдвиг	235
DCE	237
Сравнение приложения обнаружения столкновений с JavaScript	238
Оптимизация WAT вручную	241
Запись производительности в лог	242
Более сложное тестирование с помощью benchmark.js	247
Сравнение WebAssembly и JavaScript с флагом --print-bytecode	253
Заключение	256
 Глава 10. Отладка WebAssembly	 258
Отладка из консоли	259
Запись сообщений в консоль	264
Предупреждения об ошибках	268
Трассировка стека	269
Отладчик Firefox	275
Отладчик Chrome	279
Заключение	282
 Глава 11. AssemblyScript	 283
Интерфейс командной строки в AssemblyScript	284
Приложение Hello World на AssemblyScript	286
Код JavaScript для приложения Hello World	288
Приложение Hello World в загрузчике AssemblyScript	290
Объединение строк AssemblyScript	291
Объектно-ориентированное программирование на AssemblyScript	293
Приватные атрибуты	295
Среда встраивания JavaScript	297
Загрузчик AssemblyScript	298
Расширение классов в AssemblyScript	301
Сравнение производительности загрузчика и прямых вызовов	
WebAssembly	303
Заключение	306
 <i>Послесловие</i>	 307
<i>Предметный указатель</i>	308

От издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге, – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Скачивание исходного кода примеров

Скачать файлы с дополнительной информацией для книг издательства «ДМК Пресс» можно на сайте www.dmkpress.com на странице с описанием соответствующей книги.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг, мы будем очень благодарны, если вы сообщите о ней главному редактору по адресу dmkpress@gmail.com. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательства «ДМК Пресс» и No Starch Press очень серьезно относятся к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Об авторе

Рик Батталлини (Rick Battagline) – разработчик игр и автор книги «Hands-On Game Development with WebAssembly» (Packt Publishing, 2019). Занимается браузерными технологиями с 1994 г. В 2006 г. он основал независимую студию разработки веб-игр BattleLine Games LLC. Разработанная студией игра Epoch Star была номинирована на награду конкурса Slamdance Guerilla Games Competition. На сегодняшний день под его авторством созданы сотни игр с использованием различных веб-технологий, таких как WebAssembly, HTML5, WebGL, JavaScript, TypeScript, ActionScript и PHP.

О техническом рецензенте

Конрад Уатт (Conrad Watt) – научный сотрудник Питерхауса, Кембриджский университет (Peterhouse, University of Cambridge). Прежде чем получить степень доктора компьютерных наук в Кембриджском университете, он успешно прошел обучение по программе бакалавриата в Имперском колледже Лондона (Imperial College London). Исследования Конрада Уатта сосредоточены преимущественно на формальной семантике и характеристиках безопасности WebAssembly. Он является разработчиком WasmCert-Isabelle – первой в мире механизации семантики языка WebAssembly с помощью инструмента Isabelle/HOL.

Конрад является активным членом WebAssembly Community Group и продолжает участвовать в создании спецификаций новых языковых возможностей, делая упор на многопоточность и параллелизм. Его исследование характеристик слабых моделей памяти JavaScript и WebAssembly является ключевым компонентом спецификации потоков WebAssembly. Вне профессиональной жизни он увлекается хоровым пением, а также много путешествует (напоминаем, что ввиду ситуации, сложившейся в мире на момент написания этой книги, заниматься тем и другим не рекомендуется).

ПРЕДИСЛОВИЕ

Сегодня, когда большинство языков успешно компилируются в JavaScript, WebAssembly представляет собой новый виток развития технологий, который позволит выйти за пределы устоявшихся рамок. WebAssembly является универсальным эффективным инструментом для выполнения кода на вашем любимом языке в браузере, который к тому же позволяет переосмыслить способы взаимодействия многократно используемых программных компонентов не только в сети, но и на других платформах, начиная от блокчейна и заканчивая граничными вычислениями интернета вещей (IoT – Internet of things).

Безусловно, WebAssembly – это молодая технология, на развитие которой уйдет немало времени. Однако уже сегодня ее явный потенциал вдохновил огромное количество самых разных людей. В качестве примера хотелось бы привести разрабатываемый совместно с Риком (Rick) проект AssemblyScript. Здесь мы рассматриваем WebAssembly не как язык системного программирования, а как инструмент для объединения лучших концепций данной технологии с JavaScript. AssemblyScript позволяет компилировать варианты кода JavaScript, схожие с TypeScript, в WebAssembly, формируя при этом сверхмалые и эффективные модули. Все это дает возможность ощутить преимущества WebAssembly тем, кто работает с JavaScript.

WebAssembly обладает множеством интересных функций и аспектов, которые будут интересны тем, кто стремится исследовать новые технологии и внести свою лепту в их развитие. Книга «Искусство WebAssembly» поможет овладеть базовыми знаниями для дальнейшего более глубокого погружения в тонкости работы технологий, способных совершить революцию в области вычислений и во всей сети.

– ДЭНИЕЛ ВИРТЦ (DANIEL WIRTZ),
создатель AssemblyScript

БЛАГОДАРНОСТИ

В первую очередь хотелось бы сказать спасибо Лиз Чедвик (Liz Chadwick), работавшей над первыми набросками этой книги. Благодаря неутомимой работе и огромному труду она воплотила мой смутный поток мыслей в связную письменную форму и превратила его в полноценный проект. Если вам понравится читать эту книгу, будьте уверены, что это заслуга Лиз (Liz) и результат ее усилий, приложенных на каждом этапе создания.

Я хочу поблагодарить Конрада Уатта (Conrad Watt), который является приглашенным экспертом рабочей группы W3C WebAssembly, за предоставление экспертных технических рецензий. Я искренне восхищаюсь его талантом. Его технический опыт в данной области невозможно переоценить. Конрад Уатт (Conrad Watt) провел глубокий и доскональный технический анализ данной книги.

Также я хочу сказать спасибо Катрине Тейлор (Katrina Taylor) и Энн Мари Уокер (Anne Marie Walker). Я искренне признателен за вашу работу по подготовке книги к печати. И конечно, я благодарю своих друзей Винита Капура (Vineet Kapur), Стива Тэка (Steve Tack) и Терри Коэна (Terri Cohen), которые нашли время, чтобы ознакомиться с моим первым черновиком и дать по этому поводу свои комментарии. Все вы помогли мне сделать эту книгу лучше.

Особую благодарность хотелось бы выразить Биллу Поллоку (Bill Pollock), чья помощь в некоторых критических моментах помогла мне продвинуться дальше и завершить эту книгу.

ВВЕДЕНИЕ



Добро пожаловать в «Искусство WebAssembly». Цель этой книги – показать, как читать, понимать и писать код WebAssembly на уровне виртуальной машины. Здесь вы узнаете, как WebAssembly взаимодействует с JavaScript, веб-браузером и средой встраивания. После прочтения книги вы поймете, что такое WebAssembly, узнаете об оптимальных вариантах его применения, а также научитесь писать коды WebAssembly с высокой скоростью их исполнения в браузере.

Для кого предназначена эта книга

Данная книга предназначена для веб-разработчиков, желающих понять, когда и зачем стоит использовать WebAssembly. Если вы действительно хотите понять WebAssembly, вам необходимо изучить его подробно. На сегодняшний день существует множество книг о различных наборах инструментальных средств WebAssembly. Однако данная книга создана не для того, чтобы научить вас писать коды на C/C++, Rust или другом языке. Вместо этого она расскажет вам о механизмах и возможностях WebAssembly.

Также данная книга предназначена для пользователей, которые хотят понять, что такое WebAssembly, на что он способен и как применять его наиболее оптимальным образом. WebAssembly превосходит JavaScript по части производительности, объема загружаемых данных и потребления памяти. Однако разработка высокопроизводительных приложений WebAssembly представляет собой более сложный процесс, нежели просто написание приложения на C++/Rust или AssemblyScript и его дальнейшая компиляция в WebAssembly. Для создания приложения, в три раза превосходящего по скорости свой эквивалент в JavaScript, необходимо погрузиться в работу WebAssembly чуть глубже.

Читателю необходимы базовые знания о веб-технологиях, таких как JavaScript, HTML и CSS. При этом быть экспертом в какой-либо из них вовсе необязательно. Изучать WebAssembly в нынешнем воплощении, не имея фундаментальных знаний о работе сетей, будет крайне непросто. Безусловно, в рамках этой книги у меня не получится раскрыть все аспекты функционирования веб-страниц. Однако я предполагаю, что большинство читателей уже знакомы с данной темой.

Чем интересен WebAssembly для пользователей

На первом саммите WebAssembly Эшли Уильямс (Ashley Williams) (@ag_dubs) представила результаты проведенного ею опроса в Твиттере (Twitter). Первый вопрос для пользователей WebAssembly звучал как «Чем вас привлекает WebAssembly?». Результаты были следующие:

- возможность поддержки нескольких языков, 40,1 %;
- сокращение объема кода и более быстрое его исполнение, 36,8 %;
- изолированность (безопасность), 17,3 %.

Затем она попросила пользователей, выбравших первый вариант (возможность поддержки нескольких языков), рассказать, почему для них важен этот аспект. В результате ответы были следующими:

- недостаточность JavaScript для удовлетворения всех потребностей, 43,5 %;
- возможность повторного использования существующих библиотек, 40,8 %;
- наличие дистрибутива, 8,1 %.

Далее пользователей, выбравших первый вариант ответа (недостаточность JavaScript для удовлетворения всех потребностей), спросили, почему они так считают. Среди причин были:

- низкая или нестабильная производительность, 42 %;
- неудовлетворенность экосистемой, 17,4 %;
- нежелание или невозможность разобраться, 31,3 %.

Вы можете ознакомиться с докладом Эшли (Ashley) «Why the #wasmsummit Website Isn't Written in Wasm» на YouTube по адресу <https://www.youtube.com/watch?v=J5Rs9oG3FdI>.

Несмотря на то что описанный выше опрос не был частью научно-го исследования, он все же представляется достаточно информативным. Как минимум, по его результатам выяснилось, что более 55 % пользователей заинтересованы в повышении производительности своих приложений посредством WebAssembly. И несомненно, повышение производительности кода с помощью WebAssembly более чем возможно. В свою очередь, реализация WebAssembly – это не волшебство; для этого просто необходимо знать, что и зачем вы делаете.

После прочтения данной книги вы будете знать о WebAssembly достаточно, чтобы значительно повысить производительность ваших веб-приложений.

Почему миру необходим WebAssembly

Я занимаюсь разработкой веб-приложений с середины 1990-х гг. Изначально веб-страницы были не более чем документами с изображениями. Ситуация изменилась с появлением Java и JavaScript. В то время JavaScript был «языком-игрушкой», который позволял добавлять эффекты при наведении курсора на кнопки на веб-страницах. Большинство считало Java весьма стоящим продуктом, а виртуальную машину Java (JVM – Java virtual machine) новой захватывающей технологией. Однако Java так и не смогла реализовать весь свой потенциал на веб-платформе. Дело в том, что Java требовала использования встраиваемых расширений, которые в конечном итоге вышли из моды, поскольку очень часто они были источником вредоносных ПО и не соответствовали требованиям безопасности.

К сожалению, Java является проприетарной технологией, что не позволяет интегрировать поддержку этого языка напрямую в веб-браузер. WebAssembly же отличается своим открытым форматом. Данная технология является продуктом сотрудничества между многими поставщиками аппаратного и программного обеспечения, такими как Google, Mozilla, Microsoft и Apple. Она не требует встраиваемых расширений и может быть успешно интегрирована во все современные веб-браузеры. Объединив использование данной технологии с Node.js, вы можете создавать аппаратно-независимое ПО. Ввиду открытого формата технология может быть использована без каких-либо лицензионных отчислений или разрешения на любой аппаратной либо программной платформе. Так что WebAssembly является воплощением мечты 1990-х – один двоичный код на все случаи жизни (one binary to rule them all).

Структура книги

В этой книге мы расскажем о работе WebAssembly на низком уровне, познакомив вас с текстовым форматом WebAssembly Text. Мы рассмотрим множество вопросов низкоуровневой реализации, а также покажем, как WebAssembly взаимодействует с JavaScript в Node.js и веб-приложениях. Данную книгу следует читать последовательно, поскольку описываемые концепции опираются друг на друга. Также здесь содержатся ссылки на примеры кода, которые можно найти на <https://wasmbk.com>.

Глава 1 «Введение в WebAssembly»

В этой главе мы подробно рассмотрим, что представляет собой технология WebAssembly, а также поговорим о наиболее оптимальных

вариантах ее применения. Вы познакомитесь с WebAssembly Text (WAT), который позволяет понять, как работает WebAssembly на самом низком уровне. Также стоит отметить, что мы создали специальную среду, включающую все примеры из данной книги.

Глава 2 «Основы работы с WebAssembly Text»

Здесь мы рассмотрим основы WAT и его взаимосвязь с высокоуровневыми языками, которые могут быть развернуты в WebAssembly. Также вы напишете свою первую программу на WAT, а затем мы обсудим такую фундаментальную концепцию, как контроль потока с помощью переменных.

Глава 3 «Функции и таблицы»

В этой главе мы рассмотрим создание функций в модулях WebAssembly и их вызов в JavaScript. Затем вы напишете программу для проверки простых чисел, что послужит отличной иллюстрацией для описанных концепций. После мы исследуем вызов функций из таблиц и влияние данного процесса на производительность.

Глава 4 «Низкоуровневые битовые операции»

Здесь вы узнаете о низкоуровневых концепциях, которые можно использовать для повышения производительности модулей WebAssembly. Среди них: системы счисления, побитовое маскирование и дополнительный код.

Глава 5 «Строки в WebAssembly»

В WebAssembly нет встроенной поддержки строкового типа данных. В этой главе вы узнаете, как представлены строки в WebAssembly и как ими управлять.

Глава 6 «Линейная память»

В этой главе вы познакомитесь с линейной памятью и тем, как модули WebAssembly используют ее для обмена большими наборами данных в JavaScript или альтернативной среде. Здесь вы начнете создание программы обнаружения столкновений, которая заставляет объекты перемещаться случайным образом и проверяет наличие столкновений между ними. Мы будем обращаться к данной программе на протяжении всей книги.

Глава 7 «Веб-приложения»

Здесь вы узнаете, как создаются простые веб-приложения с помощью HTML, CSS, JavaScript и WebAssembly.

Глава 8 «Работа с canvas»

В данной главе мы обсудим, как использовать элемент HTML canvas в WebAssembly для создания быстрой веб-анимации. Также мы научимся применять canvas для улучшения созданной ранее программы обнаружения столкновений.

Глава 9 «Оптимизация производительности»

В этой главе вы узнаете, почему WebAssembly хорошо подходит для выполнения ресурсоемких задач, таких как обнаружение столкновений. Также вы научитесь использовать профилировщики Chrome и Firefox и другие инструменты оптимизации для повышения производительности приложений.

Глава 10 «Отладка WebAssembly»

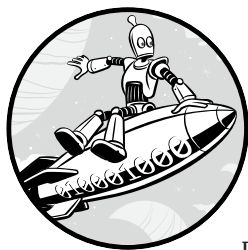
Здесь мы рассмотрим основы отладки, такие как вывод данных в веб-консоль посредством уведомлений и трассировки стека. Также вы научитесь использовать отладчики в Chrome и Firefox для пошагового исполнения кода WebAssembly.

Глава 11 «AssemblyScript»

В этой главе мы обсудим использование WAT с точки зрения понимания высокоуровневых языков и оценки AssemblyScript – высокоуровневого языка, разработанного для эффективного развертывания в WebAssembly.

1

ВВЕДЕНИЕ В WEBASSEMBLY



В этой главе вы получите базовые знания о технологии WebAssembly и изучите инструменты, которые понадобятся вам для начала работы с ней и ее текстовым воплощением – WebAssembly Text (WAT). Мы обсудим преимущества WebAssembly, такие как повышение производительности, возможность интеграции уже существующих библиотек, портируемость на другие платформы, безопасность и использование WebAssembly в качестве альтернативы JavaScript. Мы рассмотрим взаимосвязь JavaScript и WebAssembly, а также то, чем же на самом деле является WebAssembly. Вы научитесь синтаксису встроенных WAT- и S-выражений (символьные выражения). Мы познакомимся с концепциями среды встраивания и обсудим встраивание WebAssembly в веб-браузеры, Node.js и системный интерфейс WebAssembly (WASI – WebAssembly System Interface).

Затем мы рассмотрим преимущества использования Visual Studio Code в качестве среды разработки для WAT. Вы познакомитесь с основами Node.js и научитесь встраивать в данную платформу Web-

Assembly. Мы покажем, как использовать пакетный менеджер npm (Node Package Manager) для установки инструмента wat-wasm, с помощью которого вы сможете создавать приложения WebAssembly на основе WAT. Кроме того, вы создадите свое первое приложение WebAssembly, встроенное в среду Node.js.

Что такое WebAssembly?

WebAssembly – это технология, которая в перспективе следующих нескольких лет позволит значительно повысить производительность веб-приложений. Поскольку WebAssembly является молодой технологией и требует некоторых пояснений, многие люди используют ее не совсем правильно. Данная книга расскажет вам, что представляет собой технология WebAssembly и как использовать ее для создания высокопроизводительных веб-приложений.

WebAssembly – это *виртуальная архитектура набора команд (ISA – Instruction Set Architecture)* для стековой машины. Как правило, ISA представляется в двоичном формате и предназначается для конкретной машины. В свою очередь, WebAssembly разработан для работы на *виртуальной* машине, то есть не предназначен для физического аппаратного обеспечения. Виртуальная машина позволяет WebAssembly работать на разнообразном аппаратном обеспечении компьютеров и цифровых устройствах. ISA для WebAssembly предполагает компактность, межплатформенность и безопасность, а также наличие небольших двоичных файлов, что позволяет сократить время загрузки при развертывании в качестве части веб-приложения. Байт-код легко перенести на самые разные аппаратные платформы, а также он может быть безопасно развернут в интернете.

Большинство современных браузеров уже поддерживают WebAssembly. По данным Mozilla Foundation, код WebAssembly работает на 10–800 % быстрее, чем эквивалентный код JavaScript. К примеру, исполнение одного проекта eBay с помощью WebAssembly было в 50 раз быстрее, чем с JavaScript. В последующих главах мы расскажем, как создать программу обнаружения столкновений, которая может использоваться для измерения производительности. При запуске проведенные тесты производительности показали, что код обнаружения столкновений WebAssembly работает более чем в четыре раза быстрее того же JavaScript в Chrome и более чем в два раза быстрее, чем JavaScript в Firefox.

WebAssembly предлагает наиболее значительное повышение производительности в сети с момента появления динамических JIT-компиляторов (JIT – just-in-time) JavaScript. Современные браузерные движки JavaScript могут загружать и анализировать двоичный формат WebAssembly на порядок быстрее, чем JavaScript. Дело в том, что WebAssembly является генератором двоичного целевого кода для среды выполнения, а не языком программирования, как JavaScript, что

позволяет разработчикам выбирать языки программирования, которые лучше всего соответствуют потребностям его приложения. Сегодня многие говорят, что «JavaScript – это ассемблерный язык для сети», но на самом деле формат JavaScript является не лучшей целевой платформой для компиляции. JavaScript значительно менее эффективен, чем двоичный формат WebAssembly. Также любой код целевой платформы JavaScript должен уметь справляться со всеми особенностями языка JavaScript.

WebAssembly позволяет значительно повысить производительность веб-приложений в двух аспектах. Один из них – скорость запуска. В настоящее время наиболее компактным форматом JavaScript является минифицированный JavaScript, который позволяет уменьшить размеры загружаемых приложений, но при этом должен анализировать, интерпретировать, осуществлять JIT-компиляцию и оптимизировать код JavaScript. Для более компактного двоичного файла WebAssembly перечисленные выше действия не требуются. WebAssembly по-прежнему нуждается в парсинге, однако этот процесс происходит значительно быстрее, ввиду того что байт-код анализировать гораздо проще, чем текстовый формат. Также WebAssembly требуется оптимизация веб-движками, однако и этот процесс происходит намного быстрее ввиду большей четкости языка.

Более того, WebAssembly предлагает значительное улучшение пропускной способности. WebAssembly упрощает оптимизацию движка браузера. JavaScript – это очень динамичный и гибкий язык программирования, который полезен для разработчика JavaScript, но абсолютно не подходит для оптимизации кода. WebAssembly не имеет никаких веб-предпочтений (несмотря на свое название) и может использоваться вне браузера.

В скором времени WebAssembly сможет делать все, что умеет JavaScript. К сожалению, текущая версия – приложение с минимальным функционалом (MVP – Minimum Viable Product) версии 1.0 – на это неспособна. MVP-версия WebAssembly подходит для выполнения конкретных задач, но она не предназначена для замены JavaScript или фреймворков, таких как Angular, React или Vue. Если вы хотите работать с WebAssembly прямо сейчас, у вас должен быть конкретный проект с интенсивными вычислениями, который требует очень высокой производительности. Онлайн-игры, WebVR, 3D-математика и криптовалюта являются теми областями, где может эффективно применяться WebAssembly.

Причины использовать WebAssembly

Прежде чем перейти к детальному изучению WebAssembly, давайте рассмотрим несколько причин, по которым вам может быть интересно его использовать. Вы поймете, что такое WebAssembly, а также для чего и как его использовать.

Повышение производительности

JavaScript требует от разработчиков программного обеспечения делать выбор, который повлияет на то, как они проектируют движок JavaScript. Например, вы можете оптимизировать движок JavaScript для достижения максимальной производительности с помощью оптимизирующего компилятора JIT, который может выполнять код быстрее, но требует больше времени на запуск. В качестве альтернативы вы можете использовать интерпретатор, который сразу же запускает выполнение кода, но не достигает максимальной производительности оптимизирующего компилятора JIT. Решение, которое большинство разработчиков движков JavaScript используют в своих веб-браузерах, заключается в реализации обоих подходов, но для этого требуется гораздо больший объем памяти. Каждое ваше решение – это компромисс.

WebAssembly позволяет ускорить запуск и повысить пиковую производительность без чрезмерного увеличения объема памяти. Но, к сожалению, вы все же не сможете просто переписать свой JavaScript на AssemblyScript, Rust или C++ и ожидать, что это произойдет без каких-либо дополнительных действий. WebAssembly – это не волшебство, поэтому простой перенос программы JavaScript на другой язык и его компиляция без понимания того, что делает WebAssembly на более низком уровне, могут привести к разочаровывающим результатам. Написание кода C++ и его компиляция в WebAssembly с помощью флагов оптимизации обычно занимают меньше времени, чем JavaScript. Иногда программисты жалуются на то, что они потратили весь день на переписывание своего приложения на C++, а оно работает лишь на 10 % быстрее. Если это так, то, вероятно, этим приложениям не очень поможет перенос в WebAssembly, и их код на C++ пусть и дальше компилируется в JavaScript. Выделите некоторое количество времени, чтобы изучить WebAssembly, а не C++, и заставьте свои веб-приложения работать с молниеносной скоростью.

Интеграция существующих библиотек

Две популярные библиотеки для переноса существующих библиотек на WebAssembly – это `wasm-pack` для Rust и `Emscripten` для C/C++. Использование WebAssembly идеально подходит тогда, когда у вас есть существующий код, написанный на C/C++ или Rust, который вы хотите сделать доступным для веб-приложений, или же хотите перенести любые существующие настольные приложения, чтобы сделать их доступными в сети. Набор инструментальных средств `Emscripten` особенно эффективен при переносе существующих настольных приложений C++ в сеть с помощью WebAssembly. Если вы следуете этому пути, то, вероятно, захотите, чтобы ваше приложение после переноса работало как можно ближе к исходной скорости вашего существующего приложения, что может быть осуществимо, только если прило-

жение не потребляет много ресурсов. Однако у вас также может быть приложение, для которого придется повозиться с оптимизацией быстродействия, чтобы оно работало так, как на компьютере. К концу этой книги вы сможете оценить модуль WebAssembly, созданный вашим набором инструментальных средств на основе существующего кода.

Портируемость на другие платформы и безопасность

Мы объединили их в один раздел, потому что они часто зависят друг от друга. WebAssembly изначально был лишь технологией для запуска в браузере, а затем стал быстро расширяться, превращаясь в изолированную среду, которую можно запускать где угодно. Рабочая группа WebAssembly создает высокозащищенную среду выполнения, которая не позволяет злоумышленникам скомпрометировать ваш код – от серверного кода WASI до WebAssembly для встраиваемых систем и интернета вещей (IoT). Я рекомендую послушать превосходный доклад Лин Кларк (Lin Clark) о безопасности WebAssembly и повторном использовании пакетов на первом саммите WebAssembly (<https://www.youtube.com/watch?v=IBZFzGnBoU/>).

Несмотря на то что рабочая группа WebAssembly внимательно относится к безопасности, ни одна система не является полностью безопасной. Изучение WebAssembly на низком уровне подготовит вас к любым будущим угрозам безопасности.

Противники JavaScript

Некоторым людям просто не нравится JavaScript, и они не хотят, чтобы он был доминирующим языком веб-программирования. К сожалению, WebAssembly все же не в состоянии свергнуть JavaScript. Сегодня JavaScript и WebAssembly должны мирно сосуществовать и хорошо работать вместе, как показано на рис. 1.1.

Но в мире есть и хорошие новости для противников JavaScript: наборы инструментов WebAssembly предлагают множество вариантов написания веб-приложений без использования JavaScript. Например, Emscripten позволяет писать веб-приложения на C/C++ с очень малым количеством кода JavaScript, если таковой потребуется, конечно. Вы также можете писать целые веб-приложения, используя Rust и wasm-pack. Эти наборы инструментов не только генерируют WebAssembly, но также создают обширный связующий код JavaScript для вашего приложения. Причина в том, что в настоящее время возможности WebAssembly ограничены, а наборы инструментов заполняют эти пробелы с помощью кода JavaScript. Прелесть полноценных наборов инструментов, таких как Emscripten, в том, что они делают все за вас. Если вы разрабатываете с помощью одного из этих наборов инструментов, полезно понимать, когда ваш код превратится в WebAssembly, а когда – в JavaScript. Эта книга поможет вам узнать, когда это произойдет.

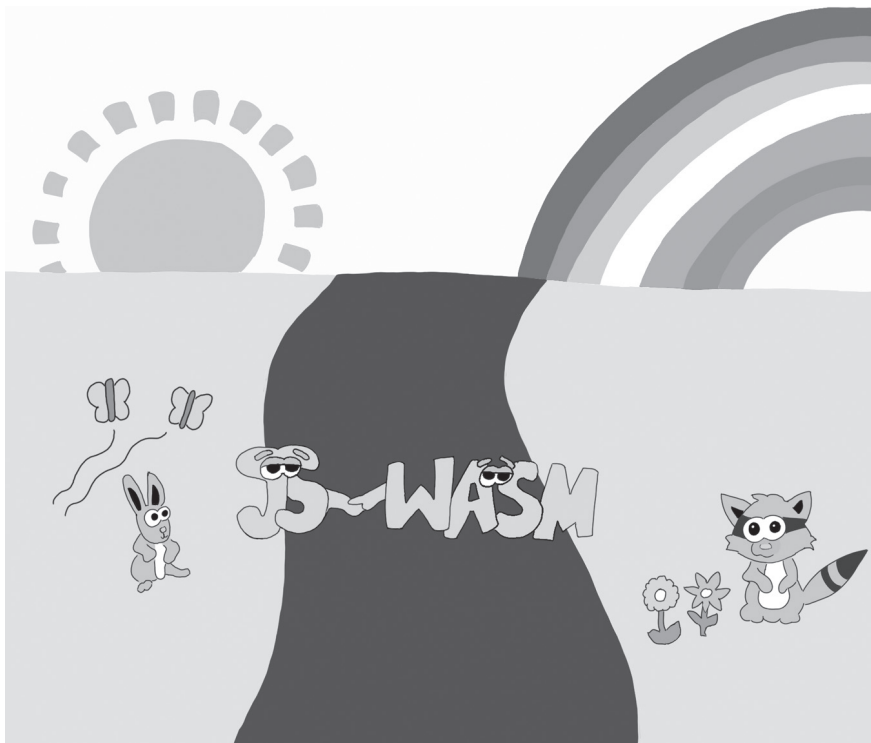


Рис. 1.1. JavaScript и WebAssembly могут сосуществовать в гармонии

Связь WebAssembly с JavaScript

Важно прояснить, как WebAssembly используется и соотносится с JavaScript. WebAssembly не является прямой заменой JavaScript; скорее, WebAssembly:

- быстрее загружается, компилируется и выполняется;
- позволяет писать приложения для сети на языках, отличных от JavaScript;
- при правильном использовании может обеспечивать скорость, близкую к исходной для вашего приложения;
- работает с JavaScript для повышения производительности ваших веб-приложений, когда используется надлежащим образом;
- не является ассемблерным языком, хотя с ним связан псевдоассемблерный язык (WAT);
- предназначен не только для сети и также может выполняться небраузерными движками Javascript, такими как Node.js, или может выполнять байт-код в средах выполнения, реализующих WASI;
- пока что не универсальное решение для создания веб-приложений.

WebAssembly – это результат сотрудничества всех основных производителей браузеров над созданием новой платформы для распространения приложений через интернет. Язык JavaScript эволюционировал от потребностей веб-браузеров в конце 1990-х гг. до полноценного языка сценариев, которым он является сегодня. Хотя JavaScript стал достаточно быстро работающим языком, веб-разработчики заметили, что иногда он показывает плохие результаты. WebAssembly – это решение многих проблем с производительностью JavaScript.

Несмотря на то что WebAssembly не может делать абсолютно все, что умеет JavaScript, он может выполнять определенные операции намного быстрее, чем JavaScript, при этом потребляя меньше памяти. В этой книге мы сравниваем код JavaScript с WebAssembly. Мы будем многократно тестировать и профилировать код для сравнения. К концу книги вы будете уметь определять, когда вам следует использовать WebAssembly, а когда имеет смысл продолжать пользоваться JavaScript.

Зачем учить WAT?

Многие книги и руководства по WebAssembly сосредоточены на конкретных наборах инструментов, таких как вышеупомянутый `wasm-pack` для Rust или `Emscripten` для C/C++. Наборы инструментов для других языков, таких как `AssemblyScript` (разновидность `TypeScript`) и `Go`, в настоящее время находятся в разработке. Эти наборы инструментов – основная причина, по которой программисты обращаются к WebAssembly, и постоянно становятся доступными все новые языковые наборы инструментов WebAssembly. В будущем веб-разработчики смогут выбирать язык, на котором они будут разрабатывать, исходя из потребностей проекта, а не доступности языков.

Одним из важных факторов для любого из этих языков является понимание того, что делает WebAssembly на самом низком уровне. Глубокое понимание WAT даст вам ответ на вопрос, почему код может работать не так быстро, как бы вам хотелось. Также поможет понять, как WebAssembly взаимодействует со своей средой встраивания. Написание модуля в WAT – лучший способ работать в веб-браузере как можно ближе к «железу» (на низком уровне). Знание WAT поможет вам создавать максимально производительные веб-приложения, разбирать и оценивать любое веб-приложение, написанное для платформы WebAssembly. Вы сможете оценить любые потенциальные риски безопасности в будущем. Кроме того, он позволяет писать код, который максимально приближен к скорости обычного приложения, но без написания специфичного кода для программно-аппаратной платформы.

Так что же такое WAT? WAT похож на ассемблерный язык для виртуальной машины WebAssembly. В практическом смысле это означает, что при написании программ WebAssembly на таком языке, как Rust

или C++, используется набор инструментов, который, как упоминалось ранее, компилирует двоичный файл WebAssembly, а также связующий код JavaScript и HTML, в который встроен модуль WebAssembly. Файл WebAssembly очень похож на машинный код, поскольку он включает разделы, коды операций и данные, которые хранятся в виде последовательности двоичных чисел. Когда у вас есть исполняемый файл в машинном коде, вы можете деассемблировать этот файл в машинный ассемблерный язык, который находится на самом низком уровне языков программирования. Ассемблирование заменяет числовые коды операций в двоичном формате мнемоническими кодами, которые читабельны для человека. WAT действует как ассемблерный язык для WebAssembly.

Стили кодирования WAT

Есть два основных стиля кодирования WAT. Первый – это *стиль линейного набора команд*. Такой стиль кодирования требует, чтобы разработчик параллельно отслеживал элементы в стеке. Большинство команд WAT проталкивают элементы в стек, выталкивают их из стека или и то, и другое. Если вы решите писать в стиле линейного набора команд, то должны знать, что существует неявный стек, в котором параметры ваших команд должны быть помещены перед вызовом самих команд. Другой стиль кодирования называется *S-выражениями*. S-выражения представляют собой древовидную структуру кодирования, в которой параметры передаются в дерево способом, который похож на вызовы функций в JavaScript. В случае если у вас есть проблемы с визуализацией стека и его элементов, синтаксис S-выражения, скорее всего, подойдет больше. Вы также можете комбинировать два данных стиля в зависимости от неявного стека для менее сложных команд, а когда становится сложно отслеживать количество параметров – использовать S-выражение.

Применение стиля линейного набора команд

Рассмотрим простую функцию сложения в листинге 1.1, которая написана на языке JavaScript.

Листинг 1.1. Код JavaScript, выполняющий сложение переменных a_val и b_val

```
function main() {  
    let a_val = 1;  
    let b_val = 2;  
    let c_val = a_val + b_val;  
}
```

После выполнения этих строк значение переменной c_val теперь равно 3, что является результатом сложения a_val и b_val. Чтобы

выполнить ту же задачу в WAT, вам понадобится больше строк кода. В листинге 1.2 показана та же программа, использующая WAT.

Листинг 1.2. WebAssembly прибавляет `$a_val` к `$b_val`

```
(module
  ❶ (global $a_val (mut i32) (i32.const 1))
  ❷ (global $b_val (mut i32) (i32.const 2))
    (global $c_val (mut i32) (i32.const 0))
    (func $main (export "main")
      global.get $a_val
      global.get $b_val

      i32.add
      global.set $c_val
    )
)
```

Листинг 1.2 содержит больше строк кода, потому что WAT должен быть более развернутым, чем JavaScript. Пока код не будет запущен, JavaScript понятия не имеет, являются ли типы в предыдущих двух примерах данными в форме с плавающей запятой, целыми числами, строками или комбинацией. WebAssembly заранее компилируется в байт-код, и при компиляции необходимо знать, какие типы он использует. JavaScript должен провести синтаксический анализ и выделить лексемы, прежде чем JIT-компилятор сможет преобразовать его в байт-код. Как только оптимизирующий компилятор начинает работать с этим байт-кодом, компилятор должен следить за тем, являются ли переменные последовательными целыми числами. Если это так, JIT может создать байт-код, который делает это предположение.

Однако у JavaScript нет уверенности в том, что в конечном итоге он не получит строковые данные или данные с плавающей запятой, хотя ожидал целые числа; поэтому в любой момент он должен быть готов сбросить свой оптимизированный код и начать все сначала. Код WAT может быть и сложнее написать и понять, но веб-браузеру его намного проще запустить. WebAssembly передает большую часть работы из браузера в компилятор средства разработки или разработчику. Отсутствие необходимости выполнять столько работы делает браузеры счастливыми, и приложения работают быстрее.

Стековые машины

Как упоминалось ранее, WebAssembly – это виртуальная стековая машина. Давайте разберемся, что это значит. Представьте стопку тарелок. Каждая тарелка в этой метафоре – фрагмент данных. Когда вы добавляете тарелку в стопку, вы кладете ее поверх уже имеющихся тарелок. Вы убираете тарелку из стопки сверху, а не снизу. По этой причине последняя тарелка, которую вы кладете в стопку, становится первой, которую вы уберете. В информатике это называется «послед-

ним пришел – первым вышел» (LIFO – last-in, first-out). Добавление данных в стек называется *проталкиванием*, а извлечение данных из стека – *выталкиванием*. Когда вы используете стековую машину, почти все команды взаимодействуют со стеком, либо добавляя дополнительные данные в верхнюю часть стека с помощью проталкивания, либо удаляя данные сверху с помощью выталкивания. На рис. 1.2 показано взаимодействие со стеком.

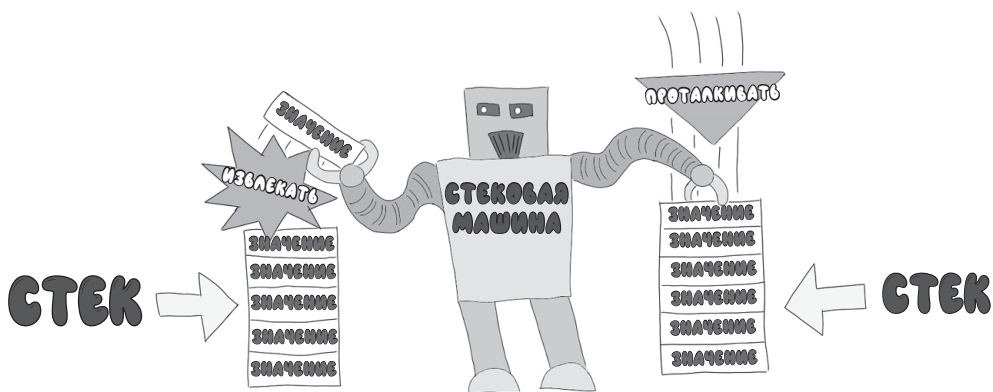


Рис. 1.2. Стекковая машина выталкивает и проталкивает значения в стек

Как вы могли увидеть, первые две строки функции `$main` в листинге 1.2 проталкивают `$a_val` в верхнюю часть стека ❶, а затем проталкивают `$b_val` в верхнюю часть стека ❷. В результате получается стек с двумя значениями на нем. Нижняя часть стека имеет значение `$a_val`, потому что мы добавили ее первой, а верхняя часть имеет значение `$b_val`, потому что она была добавлена последней.

Важно проводить различие между ISA для стековой машины (WebAssembly) и ISA для регистровой машины (x86, ARM, MIPS, PowerPC или любой другой популярной аппаратной архитектурой за последние 30 лет). Регистровые машины должны перемещать данные из памяти в регистры ЦП, чтобы выполнять над ними математические операции. WebAssembly – это виртуальная стековая машина, которая должна работать на регистровых машинах. Когда мы будем писать код в формате WAT, вы увидите это взаимодействие подробнее.

Для выполнения вычислений стековые машины проталкивают в стек и выталкивают данные из него. Аппаратные стековые машины – редкая разновидность компьютеров. Виртуальные стековые машины, такие как WebAssembly, более распространены (например, JVM Java, AVM2 Adobe Flash player, EVM Ethereum и интерпретатор байт-кода CPython). Преимущество машин виртуального стека заключается в том, что они создают байт-код меньшего размера, что удобно для любого байт-кода, предназначенного для загрузки или потоковой передачи через интернет.

Стековые машины не делают никаких предположений о количестве регистров общего назначения, доступных среде встраивания. Это

позволяет оборудованию выбирать, какие регистры использовать и когда. Код WAT может немного сбить с толку, если вы не знаете, как работает стековая машина, поэтому давайте еще раз взглянем на первые две строки функции `$main` с учетом стека (листинг 1.3).

Листинг 1.3. Получение `$a_val` и `$b_val`, а затем их проталкивание в стек

```
global.get $a_val ;; проталкиваем $a_val в стек
global.get $b_val ;; проталкиваем $b_val в стек
```

Первая строка получает значение `$a_val`, которое мы определяем как глобальное значение, а вторая строка получает глобальную переменную `$b_val`. Оба элемента попадают в стек в ожидании обработки.

Функция `i32.add` берет из стека две 32-битные целочисленные переменные, складывает их вместе, а затем проталкивает результат обратно в верхнюю часть стека. Как только два значения окажутся в стеке, мы можем вызвать `i32.add`. Если вы запустите функцию, которая выталкивает из стека больше значений, чем доступно, инструменты, которые вы используете для преобразования вашего WAT в двоичный файл WebAssembly, не позволят этого и вызовут ошибку компилятора. Мы используем последнюю строку в функции `$main`, чтобы установить для переменной `$c_val` значение в стеке. Это значение является результатом вызова функции `i32.add`.

Применение стиля S-выражений

S-выражения – это стиль кодирования с вложенной древовидной структурой, используемый в языках программирования, таких как Lisp. В листинге 1.3 мы применяли стиль линейного набора команд для написания WAT. В этом стиле используется неявный стек для каждого вызываемого оператора вызова и выражения. Тем, у кого есть некоторый опыт работы с ассемблерным языком, этот метод может показаться удобным. Но если вы пришли к WebAssembly с языка высокого уровня, такого как JavaScript, синтаксис S-выражений для WAT, вероятно, покажется вам более по душе. S-выражения организуют ваши обращения к операторам и выражениям WAT во вложенной структуре. Линейный же стиль требует, чтобы вы мысленно проталкивали элементы в стек и выталкивали их при написании кода. S-выражения больше похожи на вызовы функций JavaScript, чем на линейный стиль.

В листинге 1.2 мы присваиваем `c_val` значение `a_val + b_val`, используя стек. Код в листинге 1.4 – это фрагмент кода в листинге 1.2, где мы складываем два значения:

Листинг 1.4. Добавление и установка `$c_val` в WebAssembly

```
❶ global.get $a_val ;; проталкиваем $a_val в стек
   global.get $b_val ;; проталкиваем $b_val в стек
```

```
❷ i32.add          ;; выталкиваем два значения и помещаем результат в стек
global.set $c_val ;; выталкиваем значение из стека и получаем $c_val
```

Мы помещаем в стек две 32-битные целочисленные переменные, которые получили из глобальных переменных с помощью `global.get` ❶. Затем мы извлекли эти два значения из стека с помощью вызова `i32.add`. После сложения этих двух значений функция `i32.add` ❷ поместила полученное значение обратно в стек. Так работает стековая машина. Каждый инструмент либо проталкивает значение в стек, либо выталкивает значение, либо и то, и другое.

В листинге 1.5 показана та же функция с использованием альтернативного синтаксиса S-выражения.

Листинг 1.5. Модуль WebAssembly для сложения двух значений

```
(module
  (global $a_val (mut i32) (i32.const 1))
  (global $b_val (mut i32) (i32.const 2))
  (global $c_val (mut i32) (i32.const 0))
  (func $main (export "main")
    ❶ (global.set $c_val
      (i32.add (global.get $a_val) (global.get $b_val))
    )
  )
)
```

Пусть вас не сбивают с толку круглые скобки: они работают так же, как символы `{ }` во многих языках для создания блоков кода. При написании WAT-функции мы заключаем ее в круглые скобки. Когда вы помещаете соответствующую круглую скобку под открывающую скобку с таким же отступом, это то же самое, если сделать отступ для символов `{ }` в JavaScript. Например, обратите внимание, что отступ перед вызовом `(global.set` ❶ на одном уровне со скобкой закрытия под ним.

Этот код больше похож на обычный язык программирования, чем тот, что в листинге 1.2, потому что он в явном виде передает параметры в функцию, а не проталкивает и выталкивает значения из стека. Этот код компилируется в тот же двоичный файл. При написании своего кода в стиле S-выражений вы по-прежнему проталкиваете элементы в стек и выталкиваете из него. Такой стиль написания WAT – всего лишь синтаксический «сахар» (синтаксический прием, упрощающий чтение кода). Когда вы научитесь деассемблировать файлы WebAssembly в WAT, вы обнаружите, что синтаксис S-выражений не поддерживается дизассемблерами, такими как `wasm2wat`.

Среда встраивания

Как упоминалось ранее, WebAssembly не взаимодействует напрямую с аппаратным оборудованием. Вы должны встроить двоич-

ный файл WebAssembly в среду хоста, которая управляет загрузкой и инициализацией модуля WebAssembly. В этой книге мы работаем с движками JavaScript, такими как Node.js, и веб-браузерами как средами для встраивания. Другие среды, например среда выполнения *wasmtime*, включают в себя WASI. Но даже несмотря на то, что мы все же обсудим WASI, мы не будем использовать его в этой книге, потому что он все еще очень новый и находится в стадии разработки. Реализовать стек-машину должна среда встраивания. Поскольку современное оборудование обычно представляет собой регистровую машину, среда встраивания управляет стеком с помощью аппаратных регистров.

Браузер

Скорее всего, ваш интерес к WebAssembly вызван стремлением к повышению производительности ваших веб-приложений. Все современные браузерные движки JavaScript реализуют WebAssembly. В настоящее время Chrome и Firefox имеют лучшие инструменты для отладки WebAssembly, поэтому мы предлагаем выбрать для разработки один из этих браузеров. Ваши приложения WAT также должны нормально работать в Microsoft Edge, а Internet Explorer больше не вводит новые функции, и поэтому, к сожалению, Internet Explorer никогда не будет поддерживать WebAssembly.

Когда вы пишете WAT для веб-браузера, очень важно понимать, какие части приложения вы можете писать в WAT, а какие лучше в JavaScript. Также может случиться, что повышение производительности, которое вы получаете с помощью WebAssembly, может не стоить траты дополнительного времени на разработку. Вы сможете принимать эти решения, если разбираетесь в WAT и WebAssembly. Когда вы работаете с WebAssembly, вам часто приходится во время разработки жертвовать производительностью или циклами процессора ради памяти, или наоборот. Оптимизация производительности должна быть превыше всего.

WASI

WASI – это спецификация среды выполнения для приложений WebAssembly и стандарт взаимодействия WebAssembly с операционной системой. Он позволяет WebAssembly использовать файловую систему, выполнять системные вызовы и обрабатывать входные и выходные данные. Mozilla Foundation создали среду выполнения WebAssembly под названием *wasmtime*, которая реализует стандарт WASI. С WASI WebAssembly может делать все, что и исходное приложение, но более безопасным и независимым от платформы способом. Он делает все это с тем же уровнем производительности, что и исходное приложение.

Node.js также может запускать экспериментальный предварительный просмотр WASI с помощью командной строки `--experimental-wasi`

mental-wasi-unstable-preview1. Вы можете использовать его для запуска приложений WebAssembly, которые взаимодействуют с операционной системой вне веб-браузера. Windows, macOS, Linux или любая другая операционная система может реализовать среду выполнения WASI, поскольку она предназначена для обеспечения портируемости, безопасности и в конечном итоге универсальности WebAssembly.

Visual Studio Code

Visual Studio Code (VS Code) – это интегрированная среда разработки (IDE) с открытым исходным кодом, которую я использовал для написания примеров в этой книге. VS Code доступен для Windows, macOS и Linux по адресу <https://code.visualstudio.com/download>. Мы используем VS Code с расширением WebAssembly, написанным Дмитрием Цветчих (Dmitriy Tsvettsikh), которое доступно по адресу <https://marketplace.visualstudio.com/items?itemName=dtsvet.vscode-wasm>. Расширение обеспечивает подсветку синтаксиса кода для формата WAT, а также дает несколько других полезных пунктов в меню. Например, если у вас есть файл WebAssembly, вы можете дизассемблировать его в WAT, кликнув на файл правой кнопкой мыши и выбрав пункт меню **Show WebAssembly**. Это очень полезно, если вы хотите посмотреть код WebAssembly, который был написан кем-то другим, или код, который был скомпилирован с использованием набора инструментов. Расширение также может компилировать ваши файлы WAT в двоичный файл WebAssembly. Вы можете щелкнуть правой кнопкой мыши файл `.wat` и выбрать **Save as WebAssembly binary file**. Появится окно запроса на сохранение файла, где можно указать имя файла, в который вы хотите сохранить новый файл WebAssembly.

На рис. 1.3 показан снимок экрана с данным расширением.

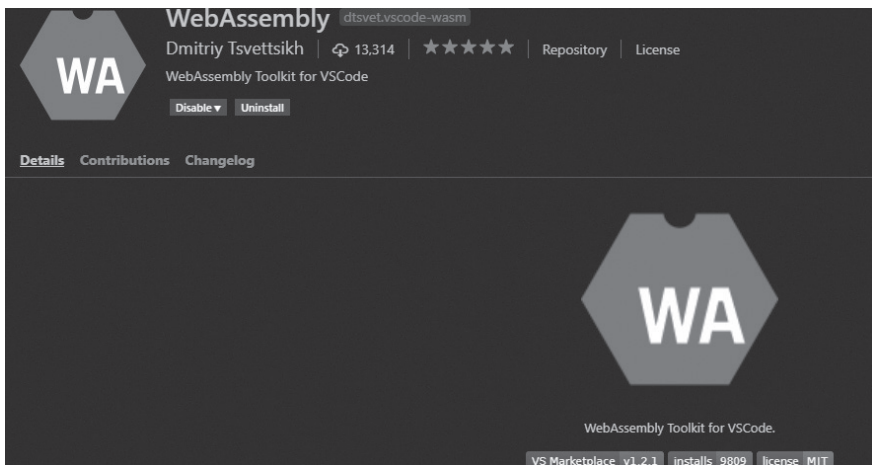


Рис. 1.3. Установка расширения WebAssembly для VS Code

Node.js

Node.js – отличный инструмент для тестирования производительности модулей WebAssembly по сравнению с существующими модулями JavaScript, а также это среда выполнения JavaScript, которую мы будем часто использовать в данной книге. Node.js поставляется с *npm* (*Node Package Manager*), который можно использовать для простой установки пакетов кода. WebAssembly – отличная альтернатива написанию исходного модуля в Node.js, который блокирует использование определенного аппаратного оборудования. Если вы хотите создать модуль npm для общего использования, написав его под WebAssembly, то можете получить производительность исходного модуля в сочетании с портируемостью и безопасностью модуля JavaScript. Мы будем выполнять многие приложения, описанные в этой книге, с помощью Node.js.

Node.js – наш предпочтительный инструмент разработки для выполнения WebAssembly, будь то из JavaScript или через веб-сервер. Мы начнем с использования Node.js для выполнения модулей WebAssembly из JavaScript, а в главе 7 напишем простой веб-сервер для обслуживания веб-приложений WebAssembly.

Node.js поставляется с npm, что упрощает установку некоторых инструментов, которые используются для разработки WebAssembly. Далее мы покажем вам, как использовать npm для установки модуля *wat-wasm*, инструмента для компиляции, оптимизации и дизассемблирования WebAssembly. Также рассмотрим, как использовать Node.js для написания простого приложения WebAssembly. Многие читатели, возможно, уже знакомы с Node.js, но если нет, существует масса документации по Node.js, которую стоит прочитать.

Установка Node.js

У вас должен быть заранее установлен Node.js, чтобы выполнять примеры по мере прочтения. К счастью, сделать это несложно. Если вы используете Windows или macOS, установщики для обеих операционных систем доступны по адресу <https://nodejs.org/en/download/>.

Для Ubuntu Linux вы можете установить Node, используя следующую команду apt:

```
sudo apt install nodejs
```

После установки Node выполните следующую команду из командной строки (на любой платформе), чтобы убедиться, что все установлено должным образом:

```
node -v
```

Если все правильно установлено, вы увидите установленную версию Node.js. Когда мы запускаем команду `node -v` на нашей машине с Windows, она выдает следующий вывод:

Это означает, что мы работаем с версией 12.14.0.

Примечание Если у вас возникли какие-либо проблемы с выполнением кода из этой книги, вы можете установить конкретную версию *Node.js* со страницы **Предыдущие выпуски** по адресу <https://nodejs.org/ru/download/Release/>.

Установка *wat-wasm*

На сегодняшний день доступно множество инструментов для преобразования кода WAT в двоичный файл WebAssembly. При написании этой книги я использовал многие из этих инструментов. В конце концов, я написал *wat-wasm* поверх *WABT.js* и *Binaryen.js*, чтобы уменьшить количество пакетов, необходимых для функций, которые я хотел продемонстрировать. Чтобы установить *wat-wasm*, выполните следующую команду `npm`:

```
npm install -g wat-wasm
```

Флаг `-g` устанавливает *wat-wasm* глобально. На протяжении всей книги мы будем использовать в окне терминала инструменты командной строки, такие как *wat2wasm*. Чтобы использовать инструменты и дальше, вам необходимо установить его глобально. После установки *wat-wasm* убедитесь, что вы можете запустить его через команду *wat2wasm* из командной строки:

```
wat2wasm
```

После этого вы должны увидеть вывод *wat-wasm* в вашу консоль. Также вы увидите множество флагов, о которых узнаете позже в этой книге.

Вы можете протестировать *wat2wasm*, создав простейший модуль WAT, как показано в листинге 1.6. Создайте новый файл с именем *file.wat* и введите в него следующий код:

Листинг 1.6. Простейший возможный модуль WebAssembly

```
(module)
```

После установки *wat-wasm* вы можете использовать команду из листинга 1.7 для компиляции файла *file.wat* в *file.wasm*, который является двоичным файлом WebAssembly:

```
wat2wasm file.wat
```

В этой книге мы будем использовать Node.js для запуска приложений командной строки WebAssembly и обслуживания веб-приложений WebAssembly для открытия в браузере. В следующем разделе мы напишем первое приложение WebAssembly с помощью Node.js.

Наше первое приложение WebAssembly с помощью Node.js

Мы начнем книгу с использования Node.js в качестве среды встраивания вместо веб-браузера, чтобы исключить необходимость в HTML и CSS в примерах кода и сделать их проще. Позже, когда вы поймете основу, рассмотрим использование браузера в качестве среды встраивания.

Код WAT в наших приложениях Node.js будет работать так же, как и в браузере. Механизм WebAssembly внутри Node.js такой же, как и в Chrome, а часть приложения WebAssembly вообще не знает, в какой среде оно работает.

Давайте начнем с создания простого файла WAT и его компиляции с помощью *wat2wasm*. Создайте файл с именем *AddInt.wat* и добавьте к нему код WAT из листинга 1.8.

Листинг 1.8. Модуль WebAssembly с функцией, которая складывает два целых числа

```
AddInt.wat (module
  (func (export "AddInt")
    (param $value_1 i32) (param $value_2 i32)
    (result i32)
      local.get $value_1
      local.get $value_2
      i32.add
    )
  )
)
```

Сейчас вы должны понять этот код. Выделите время, чтобы внимательно просмотреть код и понять его логику. Это простой модуль WebAssembly с единственной функцией *AddInt*, которую мы экспортируем в среду встраивания. Теперь скомпилируйте *AddInt.wat* в *AddInt.wasm* с помощью *wat2wasm*, как показано в листинге 1.9.

Листинг 1.9. Компиляция *AddInt.wat* в *AddInt.wasm*

```
wat2wasm AddInt.wat
```

Теперь мы готовы написать фрагмент JavaScript нашего первого приложения Node.js.

Вызов модуля WebAssembly из Node.js

Мы можем вызвать модуль WebAssembly из Node.js с помощью JavaScript. Создайте файл с именем *AddInt.js* и добавьте код JavaScript, как в листинге 1.10.

Листинг 1.10. Вызов функции *AddInt* WebAssembly из асинхронного IIFE

```
AddInt.js ❶ const fs = require ('fs');
           const bytes = fs.readFileSync (__dirname+'/AddInt.wasm');
           ❷ const value_1 = parseInt (process.argv[2]);
           const value_2 = parseInt (process.argv[3]);
           ❸ (async () => {
               ❹ const obj = await WebAssembly.instantiate (
                                   new Uint8Array (bytes));
               ❺ let add_value = obj.instance.exports.AddInt( value_1,value_2 );
               ❻ console.log( `${value_1}+${value_2}=${add_value}` );
           })();
```

Node.js может читать файл WebAssembly прямо с жесткого диска, на котором запущено приложение, благодаря встроенному модулю *fs* ❶, который считывает файлы из локального хранилища. Мы загружаем этот модуль с помощью функции *require* в Node.js. Также используем его для чтения файла *AddInt.wasm* с помощью функции *readFileSync*. Мы также извлекаем два аргумента из командной строки, используя массив *process.argv* ❷. В массиве *argv* есть все аргументы, переданные из командной строки в Node.js. Запустим функцию из командной строки; *process.argv[0]* будет содержать командный узел, а *process.argv[1]* будет содержать имя файла JavaScript *AddInt.js*. Когда мы запускаем программу, то передаем в командную строку два числа, которые устанавливают *process.argv[2]* и *process.argv[3]*.

Для создания экземпляра модуля WebAssembly, вызова функции WebAssembly и вывода результатов на консоль мы используем асинхронное *немедленно вызываемое функциональное выражение* (IIFE – *immediately invoked function expression*). Для тех, кто незнаком с синтаксисом IIFE: это средство, с помощью которого JavaScript может дожидаться обещания (promise), перед тем как выполнить остальную часть кода. Создание экземпляра модуля WebAssembly требует определенного количества времени, поэтому вы не захотите задерживать браузер или узел в ожидании завершения этого процесса. Синтаксис *(async () =>{ })()*; ❸ сообщает движку JavaScript: «объект-обещание в пути, так что можно сделать что-нибудь еще, пока ждешь». Внутри IIFE мы вызываем *WebAssembly.instantiate* ❹, передавая байты, полученные из файла WebAssembly ранее, с вызо-

вом `readFileSync`. После создания экземпляра модуля мы вызываем функцию `AddInt` ❶, экспортированную из кода WAT. Затем вызываем оператор `console.log` ❷ для вывода добавляемых значений и результата.

Теперь, когда у нас есть модуль `WebAssembly` и файл `JavaScript`, мы можем запустить приложение из командной строки, используя вызов `Node.js`, как показано в листинге 1.11.

Листинг 1.11. Запуск `AddInt.js` с использованием `Node.js`

```
node AddInt.js 7 9
```

Выполнение этой команды приводит к следующему результату:

```
7 + 9 = 16
```

Сложение двух целых чисел происходит в `WebAssembly`. Прежде чем мы продолжим, давайте быстро посмотрим, как использовать синтаксис `.then` в качестве альтернативы асинхронному IIFE.

Синтаксис `.then`

Другой широко используемый синтаксис ожидания возврата обещаний – это синтаксис `.then`. Предпочтительнее использовать синтаксис IIFE из листинга 1.10, но любой из этих синтаксисов вполне приемлем.

Создайте файл с именем `AddIntThen.js` и добавьте код из листинга 1.12, чтобы заменить асинхронный синтаксис IIFE в листинге 1.10 кодом `.then`.

Листинг 1.12. Использование синтаксиса `.then` для вызова функции `WebAssembly`

```
AddIntThen.js const fs = require ('fs');
const bytes = fs.readFileSync (__dirname + '/AddInt.wasm');
const value_1 = parseInt(process.argv[2]);
const value_2 = parseInt (process.argv[3]);

❶ WebAssembly.instantiate (new Uint8Array (bytes))
❷ .then (obj => {
    let add_value = obj.instance.exports.AddInt(value_1, value_2);
    console.log(`${value_1} + ${value_2}=${add_value}`);
});
```

Основное различие здесь заключается в функции `WebAssembly.instantiate` ❶, за которой следует `.then` ❷ и содержащей обратный вызов функции стрелки, который передает объект `obj`.

Удачное время

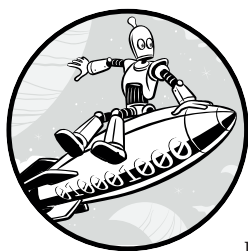
Сейчас самое время, чтобы изучать WAT. На момент написания этой книги текущая версия WebAssembly 1.0 имела относительно небольшой набор инструментов, в общей сложности 172 различных кода операций в двоичном файле WebAssembly, и вам нет необходимости запоминать их все. WebAssembly поддерживает четыре разных типа данных: `i32`, `i64`, `f32` и `f64`, и многие коды операций являются повторяющимися командами для каждого типа (например, `i32.add` и `i64.add`). Так, для устранения повторяющихся кодов операций вам нужно знать всего около 50 различных мнемоник, чтобы знать весь язык. Количество кодов операций, поддерживаемых WebAssembly, со временем будет увеличиваться. Изучение WebAssembly сейчас, когда он только зарождается, дает вам преимущество, ведь в будущем запоминание каждого кода операции станет крайне трудно или невозможно.

Как упоминалось ранее, написание ваших модулей в WAT – лучший способ работать в веб-браузере как можно ближе к «железу». То, как сегодня реализован JavaScript в браузере, может привести к провалам в производительности в зависимости от множества факторов. WebAssembly может устранить эти провалы, а WAT может помочь вам оптимизировать код, чтобы сделать его максимально быстрым.

Чтобы использовать набор инструментальных средств Emscripten, вам понадобится только базовое понимание платформы WebAssembly. Однако лишь базовых знаний вряд ли будет достаточно, чтобы повысить производительность вашего приложения до максимума, и как следствие вы подумаете, что WebAssembly не стоит потраченных усилий. В таком случае эта мысль будет ошибкой. Если вы хотите получить максимально возможную производительность от своего веб-приложения, то должны узнать как можно больше о WebAssembly. Так, необходимо знать, что он может и что лучше не стоит делать. Вы должны понимать, в каких случаях лучше выбрать WebAssembly, а когда JavaScript. Лучший способ получить эти знания – написать код WAT. В конце концов, вы можете и не писать свое приложение в WAT, но знание языка поможет вам понять WebAssembly и веб-браузер.

2

ОСНОВЫ РАБОТЫ С WEBASSEMBLY TEXT



В этой главе мы углубимся в основы кода WAT. Бóльшая часть кода в этой книге написана на WAT, самом низкоуровневом языке программирования для развертывания в WebAssembly (хотя для опытных программистов на языке ассемблера он может показаться языком довольно высокого уровня).

Здесь мы ответим на несколько вопросов. Начнем с демонстрации двух стилей комментариев в WebAssembly. Далее напишем первое приложение в стиле «Hello World». Порядок именно такой, потому что работать со строками в WAT сложнее, чем вы могли бы представить.

Затем обсудим, как импортировать данные из JavaScript в наш модуль WebAssembly с помощью объекта импорта. Рассмотрим именованные и безымянные глобальные и локальные переменные, а также типы данных, которые поддерживает WebAssembly. Также обсудим синтаксис S-выражений и то, как компилятор `wat2wasm` распаковывает эти S-выражения при компиляции вашего кода. Вы углубитесь в логику условного перехода, включая операторы `if/else` и таблицы переходов, и узнаете, как использовать циклы и блоки в сочетании с логикой условного перехода.

К концу этой главы вы сможете писать простые приложения WebAssembly, которые можно будет запускать из командной строки в среде Node.js.

Написание простейшего модуля

Каждое приложение WAT должно быть модулем, поэтому сначала рассмотрим синтаксис модуля. Модуль объявим в блоке, как в листинге 2.1.

Листинг 2.1. Однострочный комментарий WAT

```
(module
  ;; Здесь начинается код модуля.
)
```

Мы объявляем модуль с помощью ключевого слова `module`, и все, что находится внутри круглых скобок, является частью модуля. Чтобы добавить комментарий, мы используем две точки с запятой `;;`, и все, что находится после этого знака, является комментарием. WAT также имеет синтаксис многострочного комментария; открыть и закрыть его можно с помощью символов `( ; и ; )` соответственно, как показано в листинге 2.2.

Листинг 2.2. Многострочный комментарий WAT

```
(module
  ( ;
  Это модуль с многострочным комментарием. Как и в комментариях, обозначенных
  символами /* и */ в JavaScript, между открывающей и закрывающей скобками
  может быть сколько угодно строк
  ; )
)
```

Поскольку этот модуль пока что ничего не делает, мы не будем его компилировать. Вместо этого перейдем к написанию нашего первого приложения Hello World.

Hello World в WebAssembly

WAT не имеет встроенной поддержки строк, поэтому работа со строками требует, чтобы вы работали непосредственно с памятью как с массивом символьных данных. Затем эти данные памяти необходимо преобразовать в строку кода JavaScript, потому что оперировать строками при помощи JavaScript намного проще.

При работе со строками в WAT необходимо указать массив символьных данных, которые хранятся в *линейной памяти* WebAssembly.

Тему линейной памяти мы подробно обсудим в главе 6, а на данном этапе знайте, что линейная память похожа на кучу (неупорядоченный массив) памяти в исходных приложениях или гигантский типизированный массив в JavaScript.

Вам также потребуется вызвать импортированную функцию JavaScript из WebAssembly для обработки операций ввода-вывода. В отличие от исходного приложения, где ввод-вывод обычно обрабатывает операционная система, в модуле WebAssembly ввод-вывод должен обрабатываться средой встраивания, независимо от того, является ли эта среда веб-браузером, операционной системой или средой выполнения.

Создание WAT-модуля

Здесь мы создадим простой модуль WebAssembly, который в свою очередь создаст строку `hello world!` в линейной памяти и вызовет JavaScript для записи этой строки в консоль. Создайте новый файл WAT и назовите его `helloworld.wat`. Откройте этот файл и добавьте код WAT, как в листинге 2.3.

Листинг 2.3. Импорт функции

```
helloworld.wat (module
  (import "env" "print_string" (func $print_string( param i32 )))
)
```

Этот код сообщает WebAssembly, что ожидается импорт объекта `env` из нашей среды встраивания и что внутри этого объекта мы ожидаем функцию `print_string`. После того как напишем наш код JavaScript, мы создадим этот объект `env` с функцией `print_string`, которая будет передана нашему модулю WebAssembly после реализации.

Мы также настроили сигнатуру как запрашивающую единственный параметр `i32`, представляющий длину нашей строки. Назовем эту функцию `$print_string`, чтобы получить к ней доступ непосредственно из нашего кода WAT.

Затем добавим импорт для нашего буфера памяти. Добавьте строку, выделенную жирным шрифтом в листинге 2.4.

Листинг 2.4. Импорт функции и буфера памяти

```
helloworld.wat (module
  (import "env" "print_string" (func $print_string( param i32 ))
  (import "env" "buffer" (memory 1))
)
```

Этот новый импорт сообщает нашему модулю WebAssembly, что мы будем импортировать буфер памяти из объекта `env`, и этот буфер будет называться `buffer`. Оператор `(memory 1)` указывает, что буфер будет одной страницей линейной памяти: *страница* – это наименьший

фрагмент памяти, который вы можете одновременно выделить для линейной памяти. В WebAssembly размер страницы составляет 64 КБ, что даже больше, чем нам нужно для этого модуля, поэтому нам понадобится всего одна страница.

Затем, как в листинге 2.5, мы добавляем несколько глобальных переменных в *helloworld.wat*.

Листинг 2.5. Добавление глобальных переменных

```
helloworld.wat (module
  (import "env" "print_string" (func $print_string( param i32 ))
  (import "env" "buffer" (memory 1))
  ❶(global $start_string (import "env" "start_string") i32)
  ❷(global $string_len i32 (i32.const 12))
)
```

Первая глобальная переменная ❶ – это число, импортированное из нашего объекта импорта JavaScript; оно сопоставляется с переменной *env* в JavaScript (которую мы еще не создали). Это значение будет начальной ячейкой памяти нашей строки и может быть в любом месте на странице линейной памяти до максимального значения 65 535. Конечно, лучше не выбирать значение, близкое к концу линейной памяти, потому что это ограничит длину строки, которую вы можете сохранить. Если переданное значение = 0, вы сможете использовать для своей строки все 64 КБ. Если вы передали значение 65 532, тогда сможете использовать только последние четыре байта для хранения символьных данных. Если вы попытаетесь записать в область памяти, размер которой больше, чем было выделено, то получите ошибку памяти в консоли JavaScript. Вторая глобальная переменная, *\$string_len* ❷, является константой, представляющей длину строки, которую мы определим. Установим ее равной 12.

В листинге 2.6 мы определяем нашу строку в линейной памяти с помощью выражения данных.

Листинг 2.6. Добавление строки данных

```
helloworld.wat (module
  (import "env" "print_string" (func $print_string( param i32 ))
  (import "env" "buffer" (memory 1))
  (global $start_string (import "env" "start_string") i32)
  (global $string_len i32 (i32.const 12))
  (data (global.get $start_string) "hello world!")
)
```

Сначала мы передаем место в памяти, куда модуль будет записывать данные. Данные хранятся в глобальной переменной *\$start_string*, которую модуль импортирует из JavaScript. Вторым параметром – это строка данных, которую мы определяем как строку "hello world!".

Теперь мы можем определить нашу функцию "helloworld" и добавить ее в модуль, как показано в листинге 2.7.

Листинг 2.7. Добавление функции «helloworld» в модуль WebAssembly

```
helloworld.wat (module
  (import "env" "print_string" (func $print_string( param i32 ))
  (import "env" "buffer" (memory 1))
  (global $start_string (import "env" "start_string") i32)
  (global $string_len i32 (i32.const 12))
  (data (global.get $start_string) "hello world!")
  ❶(func (export "helloworld")
    ❷(call $print_string (global.get $string_len))
  )
)
```

Мы определяем и экспортируем нашу функцию как "helloworld", чтобы использовать в JavaScript ❶. Единственное, что делает эта функция, – это вызывает импортированную функцию \$print_string ❷, передавая ей длину строки, которую мы определили как глобальную. Теперь мы можем скомпилировать наш модуль WebAssembly, введя такую команду:

```
wat2wasm helloworld.wat
```

Запуск wat2wasm создает модуль *helloworld.wasm*. Чтобы выполнить модуль WebAssembly, нам нужно создать файл JavaScript.

Создание файла JavaScript

Теперь мы создадим *helloworld.js* для запуска модуля WebAssembly. Создайте и откройте файл JavaScript в текстовом редакторе и добавьте константы файла Node.js, а также три переменные, как показано в листинге 2.8.

Листинг 2.8. Объявление переменных JavaScript

```
helloworld.js const fs = require('fs');
const bytes = fs.readFileSync(__dirname + '/helloworld.wasm');

❶ let hello_world = null; // функция будет задана позже
❷ let start_string_index = 100; //линейная ячейка памяти строки
❸ let memory = new WebAssembly.Memory({initial: 1}); // линейная память
...
```

Примечание Обратите внимание на троеточия ... в последней строке кода в листинге 2.8. В этой книге мы будем использовать ..., чтобы указать, что в следующих нескольких разделах будут добавлены

дополнительные строки к этому файлу. Если ... появляется в начале блока кода, это означает, что это продолжение кода из предыдущего раздела, который закончился троеточием.

Переменная `hello_world` ❶ в конечном итоге будет указывать на функцию `helloworld`, экспортируемую нашим модулем `WebAssembly`, поэтому пока что устанавливаем для нее значение `null`. Переменная `start_string_index` ❷ – это начальная позиция нашей строки в линейном массиве памяти. Мы произвольно установили здесь значение 100, чтобы не приближаться к пределу в 64 КБ. Вы можете выбрать любое значение, если размер используемой памяти не превысит 64 КБ.

Последняя переменная содержит объект `WebAssembly.Memory` ❸. Переданное число представляет количество страниц, которые вы хотите выделить. Мы присваиваем начальное значение в соответствии с размером в одну страницу, передавая `{initial: 1}` в качестве единственного параметра. Вообще, так можно выделить до двух гигабайт, но слишком большое значение может привести к ошибке, если браузер не сможет найти достаточно непрерывной памяти для выполнения запроса.

В листинге 2.9 показана следующая переменная, которую нам нужно указать, `importObject`, которая будет передана в модуль `WebAssembly` после реализации.

Листинг 2.9. Объявление `importObject` в JavaScript

```
helloworld.js ...
let importObject = {
  ❶ env: {
    ❷ buffer: memory,
    ❸ start_string: start_string_index,
    ❹ print_string: function (str_len) {
      const bytes = new Uint8Array (memory.buffer,
        start_string_index, str_len);
      const log_string = new TextDecoder('utf8').decode(bytes);
      console.log (log_string);
    }
  }
};
...
```

Внутри нашего `importObject` добавляем объект `env` ❶ (сокр. от *environment*), хотя вы можете дать этому объекту любое имя, если оно совпадает с именем, указанным внутри объявления импорта `WebAssembly`. Это значения, которые будут переданы в модуль `WebAssembly` при его создании. Если есть какая-либо функция или значение из среды встраивания, которую вы хотите сделать доступной для модуля `WebAssembly`, передайте их сюда. Объект `env` содержит буфер памяти ❷ и начальную позицию ❸ нашей строки в буфере. Третий параметр

в `env` ❹ содержит нашу функцию JavaScript `print_string`, которую модуль `WebAssembly` будет вызывать, как указано в листинге 2.9. Эта функция извлекает длину строки в нашем буфере памяти и использует ее в сочетании с начальным индексом строки для создания строкового объекта. Затем приложение отображает строковый объект в командной строке. Кроме того, мы добавляем IIFE, который асинхронно загружает наш модуль `WebAssembly`, а затем вызывает функцию `helloWorld`, как показано в листинге 2.10.

Листинг 2.10. Реализация модуля `WebAssembly` в асинхронном IIFE

```
helloworld.js ...
( async () => {
  let obj = await
❶ WebAssembly.instantiate(new Uint8Array (bytes), importObject);
❷ ({helloWorld: hello_world} = obj.instance.exports);
❸ hello_world();
})();
```

Первая строка модуля `async` ожидает вызова функции `WebAssembly.instantiate` ❶, но, в отличие от простого примера сложения из листинга 1.1, мы передаем этой функции объект `importObject`, который был объявлен ранее. Затем извлекаем функцию `helloWorld` из `obj.instance.exports`, используя синтаксис деструктурирующего присваивания, чтобы присвоить переменной `hello_world` функцию объекта `obj.instance.exports` ❷.

Последняя строка нашего IIFE вызывает функцию `hello_world` ❸. Стрелочную функцию заключаем в круглые скобки, а затем добавляем скобки вызова функции в конец объявления этой функции, что означает ее немедленное выполнение.

Примечание Мы объявляем асинхронный код, используя синтаксис IIFE `async/await` в функции `async`; в качестве альтернативы вы можете создать функцию `async`, которая вызывается сразу после объявления.

Теперь, когда у вас есть файлы JavaScript и `WebAssembly`, выполните следующий вызов `node` из командной строки:

```
node helloworld.js
```

В командной строке вы должны увидеть такой вывод:

```
hello world!
```

Итак, мы создали традиционное приложение `hello world`! Теперь, когда у вас есть это приложение, мы рассмотрим переменные и то, как они работают в WAT.

Переменные WAT

WAT обрабатывает переменные немного иначе, чем другие языки программирования, поэтому здесь стоит предоставить вам некоторые подробности. Однако браузер управляет локальными или глобальными переменными WAT так же, как и переменными JavaScript.

WAT имеет четыре типа глобальных и локальных переменных: `i32` (32-битное целое число), `i64` (64-битное целое число), `f32` (32-битное число с плавающей запятой) и `f64` (64-битное число с плавающей запятой). Строками и другими более сложными структурами данных необходимо управлять непосредственно в линейной памяти. Линейную память и использование более сложных структур данных в WAT мы рассмотрим в главе 6. А сейчас давайте рассмотрим каждый тип переменной.

Глобальные переменные и преобразование типов

Как и следовало ожидать, вы можете получить доступ к глобальным переменным в WAT из любой функции, и обычно глобальные переменные используются как константы. *Изменяемые глобальные переменные* (*Mutable globals*) могут изменяться после установки, поэтому их использование обычно не приветствуется, поскольку они могут вызвать побочные эффекты (ошибки) в функциях, которые их используют. Импортировать глобальные переменные можно из JavaScript, позволяя JavaScript-части вашего приложения задавать значения констант внутри модуля.

При импорте глобальных переменных имейте в виду, что на момент написания этой книги стандартные числовые переменные JavaScript не поддерживают 64-битные целочисленные значения. Числа в JavaScript – это 64-битные переменные с плавающей запятой. Используя такую переменную, можно представить любое 32-битное целое число, поэтому у JavaScript нет проблем с подобными преобразованиями. Однако представить все возможные 64-битные целочисленные значения с помощью 64-битного значения с плавающей запятой невозможно. К сожалению, это означает, что вы можете работать с 64-битными целыми числами в WebAssembly, но если вы захотите отправить 64-битные значения в JavaScript, это потребует дополнительных усилий, которые выходят за рамки данной книги.

Примечание Существует идея добавить в WebAssembly поддержку типов JavaScript `BigInt`. Это упростит для JavaScript обмен 64-битными целыми числами с модулем WebAssembly. На момент написания этой книги поддержка `BigInt` находится на завершающей стадии разработки рабочей группой WebAssembly.

Еще одна деталь, которую вы должны знать о типах данных в WebAssembly и JavaScript, заключается в том, что JavaScript обрабатывает все числа как 64-битные числа с плавающей запятой. Когда вы вызы-

вае функцию JavaScript из WebAssembly, движок JavaScript выполняет неявное преобразование в 64-битное число с плавающей запятой, независимо от того, какой тип данных вы передаете. Однако WebAssembly определит импортируемую функцию как имеющую определенное требование к типу данных. Даже если вы передадите одну и ту же функцию в модуль WebAssembly три раза, вам нужно будет указать тип, который параметр передал из WebAssembly.

Давайте создадим модуль с именем *globals.wat*, который импортирует три числа из JavaScript. Файл WAT в листинге 2.11 объявляет глобальные переменные для 32-битного целого числа, 32-битного числа с плавающей запятой и 64-битного числового значения с плавающей запятой.

Листинг 2.11. Импорт альтернативных версий функции JavaScript

```
globals.wat (module
  ❶ (global $import_integer_32 (import "env" "import_i32") i32)
    (global $import_float_32 (import "env" "import_f32") f32)
    (global $import_float_64 (import "env" "import_f64") f64)

  ❷ (import "js" "log_i32" (func $log_i32 (param i32)))
    (import "js" "log_f32" (func $log_f32 (param f32)))
    (import "js" "log_f64" (func $log_f64 (param f64)))

    (func (export "globaltest")
      ❸ (call $log_i32 (global.get $import_integer_32))
      ❹ (call $log_f32 (global.get $import_float_32))
      ❺ (call $log_f64 (global.get $import_float_64))
    )
)
```

Сначала мы объявляем глобальные переменные, включая их типы и место импорта ❶. Также импортируем функцию `log` из JavaScript. WebAssembly требует от нас указать типы данных, поэтому мы импортируем три функции, каждая из которых с разными типами параметра: 32-битное целое число, 32-битное число с плавающей запятой и 64-битное число с плавающей запятой ❷.

В `$log_f64` передается переменная (`global.get $import_float_64`), сообщающая WebAssembly, что переменная, которую мы помещаем в стек, является глобальной. Если вы хотите протолкнуть в стек локальную переменную `$x`, вам нужно будет выполнить операцию (`local.get $x`). Локальные переменные мы рассмотрим чуть позже в этой главе.

В JavaScript все эти функции принимают динамическую переменную. В таком случае функции JavaScript будут практически идентичными. В функции `globaltest` мы вызываем 32-битную целочисленную версию функции `log` – (`$log_i32`) ❸, за которой следует 32-битная версия с плавающей запятой (`$log_f32`) ❹ и 64-битная версия с плавающей запятой (`log_f64`) ❺. Эти функции будут записывать в лог три

разных сообщения, чтобы продемонстрировать риски перехода между исходными 64-битными значениями с плавающей запятой в JavaScript и типами данных, поддерживаемыми WebAssembly. Прежде чем мы посмотрим на результат, нам нужно создать файл JavaScript для запуска модуля WebAssembly. Начнем с объявления переменной `global_test`, за которой следует функция `log_message`, которая будет вызываться для каждого из наших типов данных, как показано в листинге 2.12.

Листинг 2.12. Установка функций и значений `importObject`

```
globals.js const fs = require('fs');
const bytes = fs.readFileSync('./globals.wasm');
let global_test = null;

let importObject = {
  js: {
    log_i32: (value) => { console.log ("i32: ", value) },
    log_f32: (value) => { console.log ("f32: ", value) },
    log_f64: (value) => { console.log ("f64: ", value) },
  },
  env: {
    import_i32: 5_000_000_000, // _ игнорируется в числах JS и WAT
    import_f32: 123.0123456789,
    import_f64: 123.0123456789,
  }
};
...
```

В листинге 2.12 показано, что в модуль WebAssembly с помощью `importObject` передаются три разные функции JavaScript: `log_i32`, `log_f32` и `log_f64`, – которые являются оболочкой для функции `console.log`. Функции передают строку значению из модуля WebAssembly в качестве префикса. Эти функции принимают только один параметр с именем `value`. JavaScript не назначает тип параметру, как это делает WebAssembly, поэтому одну и ту же функцию можно было использовать трижды. Единственная причина, по которой мы этого не сделали, заключается в том, что мы хотели изменить строку, которая предшествовала значениям, чтобы вывод был понятным.

В листинге 2.12, чтобы продемонстрировать ограничения каждого типа данных, мы выбрали определенные значения. Так, для глобальной переменной `import_int32` мы установили значение 5,000,000,000, которое передаем в WebAssembly как 32-битное целое число. Это значение больше, чем может вместить 32-битное целое число. Далее установили глобальную переменную `import_f32` на 123.0123456789, которая имеет более высокий уровень точности, чем поддерживается 32-битной переменной с плавающей запятой, установленной в нашем модуле WebAssembly. Последняя глобальная переменная, установленная в `importObject`, – `import_f64`, которая, в отличие от двух

предыдущих переменных, достаточно велика, чтобы вместить переданное в нее значение.

Код в листинге 2.13 реализует наш модуль WebAssembly и выполняет функцию `globaltest`.

Листинг 2.13. Реализация модуля WebAssembly в асинхронном IIFE

```
globals.js ...
( async () => {
  let obj = await WebAssembly.instantiate(new Uint8Array (bytes),
                                          importObject);
  ({globaltest: global_test} = obj.instance.exports);

  global_test();
})();
```

Теперь, когда у нас есть весь код в файлах JavaScript и WAT, мы можем скомпилировать файл WAT в `globals.wasm`, используя следующий вызов `wat2wasm`:

```
wat2wasm globals.wat
```

После компиляции `globals.wasm` мы запускаем наше приложение, используя следующую команду `node`:

```
node globals.js
```

При запуске файла JavaScript с помощью `node` вы должны увидеть вывод, записанный в консоль, как в листинге 2.14.

Листинг 2.14. Вывод, записанный в консоль из `globals.js`

```
i32: 705032704
f32: 123.01234436035156
f64: 123.0123456789
```

Используя `importObject`, мы передали значение 5,000,000,000, но наши выходные данные показывают значение 705,032,704. Причина этому то, что 32-битное целое число без знака имеет максимальное значение 4 294 967 295. Если вы добавите 1 к этому числу, 32-битное целое число вернется к значению 0. Итак, если вы возьмете переданное нами число 5 000 000 000 и вычтете из него 4 294 967 296, результат будет 705 032 704. Урок в том, что если вы имеете дело с числами, превышающими несколько миллиардов, то не сможете работать с 32-битными целыми числами. К сожалению, как упоминалось ранее, вы не можете передавать из WebAssembly 64-битные целые числа в JavaScript. Если вы все-таки захотите это сделать, в таком случае вам

нужно будет преобразовать их в 64-битные числа с плавающей запятой или передать их как два 32-битных целых числа.

Следующим шагом мы передали модулю WebAssembly значение 123.0123456789. Но поскольку 32-битное число с плавающей запятой имеет ограниченную точность, лучшее, что может сделать модуль, – взять приближенное значение этого числа, а справляется он с этим не очень хорошо. 32-битное число с плавающей запятой в JavaScript и WebAssembly использует 23 бита для представления числа и умножает его на двойку, возведенную в 8-битное значение степени. Все числа с плавающей запятой являются приближенными значениями, с которыми лучше всего справляются 64-битные числа с плавающей запятой. Различия в производительности, которые вы увидите при использовании 32-битных и 64-битных чисел с плавающей запятой, зависят от вашего оборудования. Если хотите использовать 32-битные числа с плавающей запятой для повышения производительности вашего приложения, лучше всего применять для разработки и тестирования целевое оборудование. Некоторые мобильные устройства могут получить больший прирост производительности при использовании 32-разрядных чисел с плавающей запятой.

Последнее сообщение показывает 64-битное значение с плавающей запятой, возвращаемое в JavaScript как f64: 123.0123456789.

Как видите, это первое число, которое осталось неизменным по сравнению с тем, что мы до этого передали в модуль WebAssembly. Но это никоим образом не означает, что вы всегда должны использовать 64-битные числа с плавающей запятой. Сложение, вычитание и умножение обычно выполняются в три–пять раз быстрее с целыми числами. Деление на числа, представленные степенями двойки, также выполняется в несколько раз быстрее. Однако деление на что угодно, кроме степени двойки, может происходить быстрее с числами с плавающей запятой.

Эти типы данных мы более подробно рассмотрим в главе 4. Теперь, когда вы лучше понимаете глобальные переменные и их типы, давайте рассмотрим локальные переменные.

Локальные переменные

В WebAssembly значения, хранящиеся в локальных переменных и параметрах, проталкиваются в стек при помощи операции `local.get`. В главе 1 мы написали небольшую функцию, выполняющую добавление двух параметров, переданных в функцию, которая выглядит как в листинге 2.15.

Листинг 2.15. Модуль WebAssembly с 32-битным целочисленным сложением

```
AddInt.wat (module
  (func (export "AddInt")
    (param $value_1 i32) (param $value_2 i32)
```

```

    (result i32)
      local.get $value_1
      local.get $value_2
      i32.add
    )
  )
)

```

Внесем несколько изменений в код. Чтобы продемонстрировать, как можно использовать локальные переменные, возведем в квадрат значение суммы, возвращаемой `AddInt`. Создайте новый файл с именем *SumSquared.wat* и добавьте код из листинга 2.16. Изменения обозначены цифрами.

Листинг 2.16. Битовый целочисленный параметр и определение локальной переменной

```

SumSquared.wat (module
  (func (export ❶ "SumSquared")
    (param $value_1 i32) (param $value_2 i32)
    (result i32)
    ❷ (local $sum i32)

    ❸ (i32.add (local.get $value_1) (local.get $value_2))
    ❹ local.set $sum

    ❺ (i32.mul (❻ local.get $sum) (local.get $sum))
  )
)

```

Сначала мы меняем имя в экспорте на `SumSquared` ❶. Затем добавляем локальную переменную `$sum` ❷, которую будем использовать для хранения результата вызова `i32.add` ❸. Меняем `i32.add`, чтобы использовать синтаксис S-выражения. Сразу после этого мы вызываем `local.set $sum`, чтобы вытолкнуть значение из стека и установить новую локальную переменную `$sum` ❹. Далее вызываем `i32.mul` ❺, используя синтаксис S-выражения, передавая значение `$sum` для двух параметров. Это делается через вызов `local.get` ❻.

Чтобы протестировать эту функцию, создайте новый файл JavaScript с именем *SumSquared.js* и добавьте код из листинга 2.17.

Листинг 2.17. JavaScript, который выполняет SumSquared.js модуля WebAssembly

```

const fs = require('fs');
const bytes = fs.readFileSync(__dirname + '/SumSquared.wasm');
const val1 = parseInt(process.argv[2]);
const val2 = parseInt(process.argv[3]);

(async () => {
  const obj =
    await WebAssembly.instantiate(new Uint8Array (bytes));

```

```

let sum_sq =
  obj.instance.exports.SumSquared(val1, val2);
  console.log (
    `(${val1} + ${val2}) * (${val1} + ${val2}) = ${sum_sq}`
  );
})();

```

Создав функцию *SumSquared.js*, вы сможете запустить ее так же, как запускали файл *AddInt.js* ранее, но не забудьте передать ей два дополнительных параметра, представляющих значения, которые вы хотите суммировать, а затем возвести в квадрат. Следующая команда сложит 2 и 3, а затем возведет результат в квадрат:

```
node SumSquared.js 2 3
```

Результат будет следующим:

```
(2 + 3) * (2 + 3) = 25
```

Теперь вы должны понимать, как установить локальную переменную из значения в стеке и как добавить значение в стек из глобальной переменной. Далее давайте научимся распаковывать синтаксис S-выражения.

Распаковка S-выражений

До сих пор мы использовали S-выражения совместно с линейным синтаксисом WAT. Отладчик браузера не сохраняет ваши S-выражения в неизменном виде во время отладки; вместо этого он их распаковывает. Чтобы использовать WAT для декомпиляции и отладки WebAssembly, вам необходимо понимать процесс распаковки. Мы рассмотрим процесс, который компилятор `wat2wasm` совершает для распаковки небольшого фрагмента WAT-кода. Так, процесс распаковки определяет значения выражений сначала выборочно, а затем по порядку. Сначала он углубляется в каждое S-выражение в поисках подвыражений (вторичные выражения). Если подвыражения имеются, первым делом определяются именно они. А если два выражения находятся на одинаковой глубине, то определяются они по порядку. Обратите внимание на листинг 2.18.

Листинг 2.18. Использование синтаксиса S-выражения

```

❶ (i32.mul          ;; выполняет 7-е (последнее)
  ❷ (i32.add        ;; выполняет 3-е
    ❸ (i32.const 3) ;; выполняет 1-е
    ❹ (i32.const 2) ;; выполняет 2-е
  )
  ❺ (i32.sub ;; выполняет 6-е

```



```

    ⑥(i32.const 9) ;; выполняет 4-е
    ⑦(i32.const 7) ;; выполняет 5-е
  )
)

```

Во-первых, нам нужно зайти в наше выражение `i32.mul` ①, чтобы увидеть, содержит ли оно какие-либо подвыражения. Там мы находим два подвыражения – выражения `i32.add` ② и `i32.sub` ③. Заходим внутрь первого `i32.add` ②, вычисляя `(i32.const 3)` ④, тем самым проталкивая 32-битное целое число 3 в стек. Поскольку внутри этого оператора нечего определять, мы переходим к определению `(i32.const 2)` ⑤, которое помещает в стек 32-битное целое число 2. Затем S-выражение выполняет `i32.add` ②. Первые три строки, выполняемые в S-выражении, показаны в листинге 2.19.

Листинг 2.19. Код из `i32.add` после распаковки

```

i32.const 3
i32.const 2
i32.add

```

Теперь, когда `i32.add` выполнено, следом нужно распаковать фрагмент `i32.sub`. Точно так же код сначала входит в S-выражение и выполняет `(i32.const 9)` ⑥, далее делает то же самое со следующим фрагментом `(i32.const 7)` ⑦. Как только эти две константы окажутся в стеке, код выполняет `i32.sub`. Распакованное подвыражение выглядит как в листинге 2.20.

Листинг 2.20. Код из `i32.sub` после распаковки S-выражения

```

i32.const 9
i32.const 7
i32.sub

```

После выполнения S-выражений `i32.add` и `i32.sub` их распакованная версия выполняет команду `i32.mul`.

Полностью распакованная версия S-выражения показана в листинге 2.21.

Листинг 2.21. Пример использования стека WAT

```

i32.const 3 ;; стек = [3]
i32.const 2 ;; стек = [2, 3]
i32.add    ;; 2 и 3 выталкиваются из стека, их сумма 5 проталкивается в стек [5]

i32.const 9 ;; стек = [9,5]
i32.const 7 ;; стек = [7,9,5]
i32.sub    ;; 7 и 9 выталкиваются из стека. 9-7=2 проталкиваются в стек [2,5]

i32.mul    ;; 2,5 выталкиваются из стека, 2x5=10 проталкиваются в стек [10]

```

Поначалу принцип работы стековой машины может показаться немного сложным, но через некоторое время она станет более привычной. Мы рекомендуем использовать S-выражения, пока вы не привыкнете к стековой машине. Синтаксис S-выражения – отличный способ облегчить вам работу в WAT, если только вы знакомы с языками более высокого уровня, конечно же.

Переменные с индексами

WAT не требует, чтобы вы присваивали имена переменным и функциям. Вместо этого можно использовать номера индексов для ссылки на функции и переменные, которым вы еще не присвоили имена. Время от времени вам может встречаться код WAT, который использует эти индексированные переменные и функции. Иногда этот код получается в результате дизассемблирования.

Код, вызывающий `local.get`, за которым следует число, извлекает локальную переменную в зависимости от порядка, в котором она отображается в коде WebAssembly. К примеру, мы могли бы написать файл *AddInt.wat* из листинга 2.21 как код в листинге 2.22.

Листинг 2.22. Использование переменных

```
(module
  (func (export "AddInt")
    ❶ (param i32 i32)
    (result i32)
      ❷ local.get 0
      ❸ local.get 1
      i32.add
    )
  )
)
```

Как видите, мы не присваиваем имена параметрам в выражении `param` ❶. Удобство этого стиля кода заключается в том, что вы можете объявить несколько параметров в одном выражении, добавив больше типов. При вызове `local.get` нам нужно передать нулевое индексированное число, чтобы получить заданный параметр. Первый вызов `local.get` ❷ извлекает первый параметр, передавая 0. Второй вызов `local.get` ❸ извлекает второй параметр, передавая 1. Такой синтаксис также можно использовать для функций и глобальных переменных. Лично мне этот синтаксис трудно читать, поэтому я не буду использовать его в этой книге. Однако я счел необходимым показать его, потому что его используют некоторые отладчики.

Преобразование между типами

Разработчики в JavaScript не выполняют преобразования между разными числовыми типами. Все числа в JavaScript – это 64-битные числа с плавающей запятой, что упрощает программирование для раз-

работчиков, но ведет к снижению производительности. При работе с WebAssembly вам необходимо знать свои числовые данные. Для выполнения числовых операций между двумя переменными разных типов данных вам нужно будет выполнить определенные преобразования. В табл. 2.1 представлены функции преобразования, которые вы можете использовать в WAT для преобразования между различными числовыми типами данных.

Таблица 2.1. Функции преобразования числового типа

Функция	Действие
i32.trunc_s/f64 i32.trunc_u/f64	Преобразовать 64-битное число с плавающей запятой в 32-битное целое число
i32.trunc_s/f32 i32.trunc_u/f32 i32.reinterpret/f32	Преобразовать 32-битное число с плавающей запятой в 32-битное целое число
i32.wrap/i64	Преобразовать 64-битное целое число в 32-битное целое число
i64.trunc_s/f64 i64.trunc_u/f64 i64.reinterpret/f64	Преобразовать 64-битное число с плавающей запятой в 64-битное целое число
i64.extend_s/i32 i64.extend_u/i32	Преобразовать 32-битное целое число в 64-битное целое число
i64.trunc_s/f32 i64.trunc_u/f32	Преобразовать 32-битное число с плавающей запятой в 64-битное целое число
f32.demote/f64	Преобразовать 64-битное число с плавающей запятой в 32-битное число с плавающей запятой
f32.convert_s/i32 f32.convert_u/i32 f32.reinterpret/i32	Преобразовать 32-битное целое число в 32-битное число с плавающей запятой
f32.convert_s/i64 f32.convert_u/i64	Преобразовать 64-битное целое число в 32-битное число с плавающей запятой
f64.promote/f32	Преобразовать 32-битное число с плавающей запятой в 64-битное число с плавающей запятой
f64.convert_s/i32 f64.convert_u/i32	Преобразовать 32-битное целое число в 64-битное число с плавающей запятой
f64.convert_s/i64 f64.convert_u/i64 f64.reinterpret/i64	Преобразовать 64-битное целое число в 64-битное число с плавающей запятой

В этой таблице я опустил довольно много не столь важной информации, чтобы сосредоточиться на самом главном. Суффиксы `_u` и `_s` в выражениях `convert`, `trunc` и `extend` сообщают WebAssembly, какие именно целые числа вы используете – без знака (не могут быть отрицательными) или со знаком (могут быть отрицательными). Выражение `trunc` отбрасывает дробную часть числа с плавающей запятой при ее преобразовании в целое число. Числа с плавающей запятой можно расширить с `f32` до `f64` или сократить с `f64` до `f32`. Целые числа легко преобразуются в числа с плавающей запятой. Команда `wrap` помещает 32 бита 64-битного целого числа в `i32`. Чтобы преобразовать один тип данных в другой с сохранением количества битов, необходимо использовать команду `reinterpret`, которая сохраняет количество битов значений целого числа или числа с плавающей запятой одинаковыми.

Условные операторы if/else

Одним из отличий WAT от ассемблерного языка является то, что он содержит некоторые операторы потока управления более высокого уровня, такие как `if` и `else`. WebAssembly не поддерживает данные логического типа; вместо этого для представления логических значений он использует значения `i32`. Оператор `if` требует, чтобы значение `i32` находилось наверху стека для определения потока команд управления. Также оператор `if` определяет любое ненулевое значение как истинное, а нулевое как ложное. Синтаксис операторов `if/else` с использованием S-выражений выглядит как в листинге 2.23.

Листинг 2.23. Синтаксис if/else с использованием S-выражений

```
;; Этот код предназначен для демонстрации и не является частью большого
;; приложения
(if (local.get $bool_i32)
  (then
    ;; сделайте что-нибудь, если $bool_i32 не равно 0
    ;; пор - это код операции "no operation".
    пор ;; Я использую его как замену рабочего кода.
  )
  (else
    ;; сделайте что-нибудь, если $bool_i32 равно 0
    пор
  )
)
```

Далее давайте посмотрим, как выглядит распакованная версия операторов `if/else`. Обратите внимание, что в распакованной версии нет выражения `(then)`. В листинге 2.24 показано, как код из листинга 2.23 будет выглядеть после распаковки.

Листинг 2.24. Операторы if/else с использованием линейного синтаксиса

```
;; Этот демонстрационный код не является частью большого приложения
local.get $bool_i32

if
  ;; сделайте что-нибудь, если $bool_i32 не равно 0
  пор
else
  ;; сделайте что-нибудь, если $bool_i32 равно 0
  пор
end
```

S-выражение `then` представляет собой исключительно «синтаксический сахар» (syntactic sugar, прием, облегчающий восприятие текста программы) и не прописано в распакованной версии нашего кода.

Для распакованной версии требуется оператор `end`, которого нет в синтаксисе *S*-выражения.

Для того чтобы написать программы на языке высокого уровня, необходимо использовать алгебру логики с операторами `if/else`. В JavaScript есть оператор `if`, который выглядит примерно так:

```
if( x > y && y < 6 )
```

Чтобы воспроизвести то же самое в WebAssembly, нужно будет использовать выражения, которые условно возвращают 32-битные целые числа. В листинге 2.25 показан пример, как реализовать логику оператора `if` с помощью `x` и `y` вместо 32-битных целых чисел в JavaScript.

Листинг 2.25. if и i32.and с использованием синтаксиса S-выражения

```
;; Этот демонстрационный код не является частью большого приложения
if
  (i32.and
    (i32.gt_s (local.get $x) (local.get $y) ) ;; знаковое "больше, чем"
    (i32.lt_s (local.get $y) (i32.const 6) ) ;; знаковое "меньше, чем"
  )
  (then
    ;; x больше, чем y, а y меньше 6
    nop
  )
)
```

В сравнении этот вариант выглядит немного сложнее. Выражение `i32.and` выполняет побитовую операцию И (bitwise AND) с 32-битными целыми числами, а `i32.gt_s` и `i32.lt_s` возвращают истинное значение 1 или ложное значение 0. Работая в WebAssembly, нельзя забывать про побитовый характер операций И/ИЛИ (AND/OR); использование `i32.and` при значениях 2 и 1 приведет к 0 в соответствии с логикой И. Возможно, вам нужна логическая операция И, но увы, `i32.and` все же выполняет побитовую двоичную операцию И. Двоичные операции И/ИЛИ (AND/OR) мы обсудим более подробно в главе 4. В определенном смысле сложные `if` выражения выглядят лучше, когда они распакованы. В листинге 2.26 показана полная версия кода из листинга 2.25.

Листинг 2.26. if и i32.and с использованием синтаксиса стека

```
;; Этот демонстрационный код не является частью большого приложения
local.get $x
local.get $y
i32.gt_s      ;; проталкивает 1 в стек, если $x > $y

local.get $y
i32.const 6
```

```

i32.lt_s      ;; проталкивает 1 в стек, если $y < 6

i32.and      ;; выполнить побитовое and над последними двумя значениями в стеке

if
  ;; x больше, чем y, а y меньше, чем 6
  por
end

```

В следующем листинге 2.27 показано, что есть похожие выражения, которые можно использовать, при условии что \$x и \$y являются 64-битными или 32-битными числами с плавающей запятой.

Листинг 2.27. Применение операций сравнения f32, но с заключительной операцией i32.and

```

;; Этот код предназначен для демонстрации и не является частью большого
;; приложения
(if
  (i32.and
    ❶ (f32.gt (local.get $x) (local.get $y) )
    ❷ (f32.lt (local.get $y) (f32.const 6) )
  )
  (then
    ;; x больше, чем y, а y меньше 6
    por
  )
)

```

Обратите внимание, что мы заменили i32.gt_s и i32.lt_s на f32.gt ❶ и f32.lt ❷ соответственно. Большинство целочисленных операций с помощью суффикса _s указывают, поддерживают ли они отрицательные числа. Для чисел с плавающей запятой этого делать не нужно, потому что они все со знаком и имеют специальный знаковый бит.

В WebAssembly есть 40 операторов сравнения. В табл. 2.2 показаны операции, которые используются вместе с if/else. Если нет иных указаний, эти операции выталкивают два значения из стека, сравнивают их и, если значение истинно, проталкивают в стек 1, и 0, если ложно.

Операторы цикла и блока

В WAT операторы перехода отличаются от операторов перехода, которые присутствуют в ассемблерном языке. Разница в том, что выражения перехода предотвращают появление спагетти-кода (запутанный код с большим набором функций, который трудно разобрать) в результате переходов в произвольные позиции. Чтобы ваш код возвращался назад, его нужно поместить в цикл. Если же вы хотите, чтобы ваш код двигался вперед, нужно поместить его в блок. В языках про-

Таблица 2.2. Операции, которые используются вместе с условными операторами if/else

Функция	Действие
i32.eq i64.eq f32.eq f64.eq	Проверка на равенство
i32.ne i64.ne f32.ne f64.ne	Неравенство
i32.lt_s i32.lt_u i64.lt_s i64.lt_u f32.lt f64.lt	Проверка «меньше, чем». Суффикс <code>_s</code> указывает на сравнение со знаком; <code>_u</code> на сравнение без знака
i32.le_s i32.le_u i64.le_s i64.le_u f32.le f64.le	Проверка «меньше или равно». Суффикс <code>_s</code> указывает на сравнение со знаком; <code>_u</code> на сравнение без знака
i32.gt_s i32.gt_u f32.gt f64.gt i64.gt_s i64.gt_u	Проверка «больше, чем». Суффикс <code>_s</code> указывает на сравнение со знаком; <code>_u</code> на сравнение без знака
i32.ge_s i32.ge_u i64.ge_s i64.ge_u f32.ge f64.ge	Проверка «больше или равно». Суффикс <code>_s</code> указывает на сравнение со знаком; <code>_u</code> на сравнение без знака
i32.and i64.and	Побитовое И (AND)
i32.or i64.or	Побитовое ИЛИ (OR)
i32.xor i64.xo	Побитовое исключающее ИЛИ (OR)
i32.eqz i64.eqz	Проверка числа с плавающей запятой на наличие нулевого значения

граммирования высокого уровня операторы цикла и блока используются вместе для большей функциональности. Давайте рассмотрим эти структуры с помощью нескольких демонстрационных примеров кода, которые не будут частью большого приложения.

Оператор блока (block)

Сначала мы рассмотрим оператор `block`. Операторы блока и цикла в WAT работают примерно так же, как операторы `goto` в ассемблере, а еще в некоторых языках программирования низкого уровня. Однако код может перейти к концу блока, только если он находится внут-

ри этого блока, что предотвращает произвольные переходы кода из любой точки программы к метке `block`. Для того чтобы код перешел в конец блока, он должен располагаться внутри этого блока. Пример показан в листинге 2.28.

Листинг 2.28. Объявление оператора `block` в WAT

```
;; Этот демонстрационный код не является частью большого приложения
❶ (block $jump_to_end
  ❷ br $jump_to_end

  ;; код ниже ветки не выполняется. br переходит в конец блока
  ❸ por
)

;; здесь оператор br перескакивает к
❹ por
```

Итак, `br` ❷ – это оператор перехода, который просит программу перейти в другую часть кода. Можно предположить, что `br` вернется к метке в начале блока ❶. Но этого не произойдет. Если использовать оператор `br` внутри блока для перехода к метке, он выйдет из этого блока и сразу начнет выполнять код за пределами блока ❹. Это означает, что код непосредственно под оператором `br` ❸ никогда не выполнится. Как упоминалось ранее, этот код не предназначен для использования, он только демонстрирует, как работают операторы `block` и `br`.

То, как мы используем здесь оператор `br`, неправильно. Так как он всегда переходит до конца блока с меткой, он должен выполнять этот переход условно.

В отличие от кода в листинге 2.28, операция условного перехода `br_if` в листинге 2.29 используется для перехода по заданному условию.

Листинг 2.29. Переход к концу блока с помощью `br_if`

```
;; Этот демонстрационный код не является частью большого приложения
(block $jump_to_end
  ❶ local.get $should_I_branch
  ❷ br_if $jump_to_end

  ;; код под переходом будет выполнен, если $should_I_branch = 0
  ❸ por
)

❹ por
```

Новая версия кода проталкивает 32-битное целочисленное значение `$should_I_branch` в стек ❶. Оператор `br_if` выталкивает верхнее значение из стека ❷ и, если это значение не 0, переходит к концу блока `$jump_to_end` ❹. Если `$should_I_branch` = 0, тогда в блоке выполняется код под оператором `br_if` ❸.

Оператор цикла (loop)

Оператор `block` при условном ветвлении всегда переходит в конец блока. Если вам нужно перейти к началу блока кода, используйте оператор цикла (`loop`). В листинге 2.30 показано, как он работает в WAT. Стоит отметить, что этот код не выполняется в бесконечном цикле.

Листинг 2.30. Оператор `loop`, который в данном случае не работает бесконечно

```
;; Этот демонстрационный код не является частью большого приложения
(loop $not_gonna_loop
  ;; этот код будет выполняться только один раз
  ❶пор
)

;; поскольку в нашем цикле нет перехода, он выходит из блока цикла в конце
❷пор
```

Фактически выражение цикла в WAT не работает само по себе; ему нужен оператор перехода, расположенный внутри цикла, чтобы вернуться к началу выражения цикла. Код внутри цикла `loop` ❶ будет выполнен точно так же, как оператор `block`, и без перехода завершится в конце блока ❷.

Если по какой-либо причине вы захотите создать бесконечный цикл, вам нужно использовать оператор `br` в конце цикла, как показано в листинге 2.31.

Листинг 2.31. Бесконечный цикл переходов

```
;; Этот демонстрационный код не является частью большого приложения
(loop $infinite_loop
  ;; этот код будет выполняться в бесконечном цикле
  пор

  ❶br $infinite_loop
)

;; этот код никогда не будет выполнен, потому что цикл выше бесконечен
❷пор
```

С каждой итерацией оператор `br` ❶ всегда возвращается к началу блока `$infinite_loop`. Код под `loop` ❷ никогда не будет выполняться.

Примечание При выполнении WAT в браузере не рекомендуется использовать бесконечный цикл, потому что WebAssembly не оставляет ресурсы под рендеринг браузера; соответственно, вам нужно будет вернуть управление браузеру, иначе он зависнет.

Совместное использование операторов блока и цикла

Чтобы цикл мог прерываться и продолжаться, выражения `loop` и `block` следует использовать вместе. Давайте соберем небольшой модуль `WebAssembly` и приложение `JavaScript`, которое находит факториалы. Программа будет запускать цикл до тех пор, пока мы не получим факториал числа, переданного в функцию. Это позволит нам проверить функциональность операторов `continue` и `break` нашего выражения `loop`. Наш простой цикл будет вычислять значение факториала для каждого числа вплоть до некоторого значения параметра `$n`, которое мы передадим из `JavaScript`. Затем значение факториала `$n` будет возвращено в `JavaScript`.

Создайте новый файл с именем `loop.wat` и добавьте код из листинга 2.32.

Листинг 2.32. Переход вперед и назад с помощью `loop` и `block`

```
loop.wat (module
  ❶(import "env" "log" (func $log (param i32 i32)))

  (func $loop_test (export "loop_test") (param $n i32)
    (result i32)

    (local $i          i32)
    (local $factorial i32)

    (local.set $factorial (i32.const 1))

    ❷(loop $continue (block $break ;; цикл $continue и блок $break
      ❸(local.set $i          ;; $i++
        (i32.add (local.get $i) (i32.const 1))
      )

      ❹;; значение факториала $i
      (local.set $factorial ;; $factorial = $i * $factorial
        (i32.mul (local.get $i) (local.get $factorial))
      )

      ;; вызвать $log, передавая параметры $i, $factorial
      ❺(call $log (local.get $i) (local.get $factorial))

      ❻(br_if $break
        (i32.eq (local.get $i) (local.get $n))) ;; if $i==$n выйти из цикла
      ❼br $continue ;; переход к вершине цикла
    ))

    ❽local.get $factorial ;; вернуть $factorial вызывающему коду JavaScript
  )
)
```

Первое выражение в этом модуле – это импорт функции `$log` ❶. Чуть позже мы напишем эту функцию в `JavaScript` и будем вызывать

ее при каждом проходе цикла, чтобы зарегистрировать значение факториала `$i` для каждого прохода. Цикл `$continue` ❷ и блок `$break` ❷ мы поместили потому, что переход на `$continue` продолжит выполнение цикла, а переход на `$break` прервет цикл. Можно было сделать это и без использования блока `$break`, но мы ведь хотим продемонстрировать, как цикл может работать вместе с блоком. Такой метод с операторами `break` и `continue` позволяет вашему коду работать как на языке программирования высокого уровня.

Далее цикл увеличивает `$i` ❸ и вычисляет новое значение `$factorial`, умножая `$i` на старое значение `$factorial` ❹. Затем он вызывает лог с параметрами `$i` и `$factorial` ❺. Далее мы используем `br_if` для выхода из цикла, если `$i == $n` ❻. В случае если мы не выходим из цикла, тогда возвращаемся к началу цикла ❼. После завершения цикла значение `$factorial` помещается в стек ❽, для того чтобы можно было вернуть это значение вызывающему коду JavaScript.

Получив файл WAT, скомпилируйте его в файл WebAssembly с помощью следующей команды:

```
wat2wasm loop.wat
```

Теперь создадим файл JavaScript для выполнения WebAssembly. Создайте файл `loop.js` и введите код из листинга 2.33.

Листинг 2.33. Вызов `loop_test` из JavaScript

```
loop.js const fs = require('fs');
const bytes = fs.readFileSync(__dirname + '/loop.wasm');
❶ const n = parseInt(process.argv[2] || "1"); // будем повторять n раз
let loop_test = null;

let importObject = {
  env: {
    ❷ log: function(n, factorial) { // запишем n и factorial для вывода в консоль
      console.log(`${n}! = ${factorial}`);
    }
  }
};

( async() => {
  ❸ let obj = await WebAssembly.instantiate( new Uint8Array(bytes),
                                             importObject );

  ❹ loop_test = obj.instance.exports.loop_test;

  ❺ const factorial = loop_test(n); // вызов loop test
  ❻ console.log(`result ${n}! = ${factorial}`);
  ❼ if (n > 12) {
    console.log(`
=====
Факториалы больше 12 слишком велики для 32-битного целого числа.
`);
  }
});
```

```

=====
    `)
  }
})());

```

Функция `log` ❷, которую будет вызывать наш код WAT, записывает в консоль строку со значениями `n` ❶ и `n factorial`, переданными из цикла WAT. При реализации модуля ❸ мы передаем значение `n` функции `loop_test` ❹. В ❺ функция `loop_test` находит факториал. Затем мы вызываем `console.log` ❻ для отображения значения `n` и факториала `n`. В конце мы сделали проверку, чтобы убедиться, что введенное нами число не превышает 12 ❼, потому что 32-битные целые числа со знаком поддерживают числа только до 2 млрд. Запустите `loop.js` с помощью `node` и выполните в командной строке следующую команду:

```
node loop.js 10
```

В листинге 2.34 показан вывод, который вы должны увидеть в командной строке.

Листинг 2.34. Вывод из loop.js

```

1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
result 10! = 3628800

```

Теперь, когда вы знаете, как работают циклы в WAT, давайте посмотрим на таблицы переходов.

Переход с помощью br_table

Другой способ использовать операторы `block` в WAT – это соединить его с *выражением* `br_table`, которое реализует оператор `switch`. Это выражение предназначено для достижения производительности *таблицы переходов*, равной той, которую вы получаете, используя оператор `switch` при большом количестве переходов. Также выражение `br_table` записывает блоки и индексы в список блоков. Затем оно выходит из того блока, на который указывает ваш индекс. Неудобство использования таблицы переходов заключается в том, что код может выйти только из того блока, в котором он находится. Это означает, что

вы должны заранее объявить все свои блоки. В листинге 2.35 показано построение `br_table` в WAT.

Листинг 2.35. Применение синтаксиса `br_table` из WAT

```
;; Этот демонстрационный код не является частью большого приложения
❶ (block $block_0
  (block $block_1
    (block $block_2
      (block $block_3
        (block $block_4
          (block $block_5
            ❷ (br_table $block_0 $block_1 $block_2 $block_3 $block_4 $block_5
              (local.get $val)
            )
            ❸ ) ;; блок 5
            i32.const 55
            return

          ) ;; блок 4
          i32.const 44
          return

        ) ;; блок 3
        i32.const 33
        return

      ) ;; блок 2
      i32.const 22
      return

    ) ;; блок 1
    i32.const 11
    return

  ) ;; блок 0
  i32.const 0
  return
```

Здесь перед выражением `br_table` ❶ определяются все элементы `block`. При использовании выражения `br_table` ❷ не всегда понятно, из какого блока кода оно будет совершать переход. Именно поэтому мы добавили в код комментарии, начиная с ❸, которые указывают, какой фрагмент `block` завершает работу.

При большом количестве переходов выражение `br_table` обеспечивает некоторое повышение производительности по сравнению с выражением `if`. В нашем демонстрационном коде наличие выражения `br_table` не имело смысла до тех пор, пока не появилось множество переходов. Конечно, окончательный результат зависит от среды встраивания и оборудования, на котором она работает. Даже при таком количестве переходов `br_table` все еще работает медленнее в Chrome, чем операторы `if`. Firefox же при подобном количестве переходов работал заметно быстрее с выражением `br_table`.

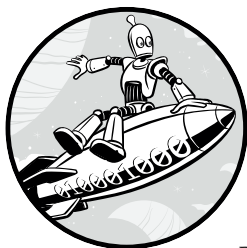
Заключение

В этой главе мы рассмотрели несколько базовых составляющих программирования WAT. В главе 1 вы научились создавать и выполнять модуль WebAssembly, а здесь перешли к созданию классического приложения hello world. Можно сказать, что создание приложения hello world в WAT немного сложнее, чем в большинстве языков программирования.

Написав несколько простых программ, вы изучили некоторые из основных функций WAT и их отличия от традиционного языка высшего уровня, такого как JavaScript. Мы изучили переменные и константы и то, как их можно поместить в стек с помощью команд WAT, обсудили синтаксис S-выражений и приемы их распаковки. Также вы познакомились с синтаксисом индексированных локальных переменных и функций. Вы узнали об основных структурах перехода и цикла, а также о том, как использовать их в синтаксисе WAT. В следующей главе мы исследуем функции и таблицы функций в WAT, а также их взаимодействие с JavaScript и другими модулями WAT.

3

ФУНКЦИИ И ТАБЛИЦЫ



В этой главе мы подробно изучим функции в WebAssembly: как и когда следует импортировать функции из JavaScript или другого модуля WebAssembly, как экспортировать функции WebAssembly в среду встраивания и вызывать эти функции из JavaScript. Вы узнаете о таблицах в WebAssembly и о том, как вызывать функции, определенные в таблицах. А также исследуем влияние на производительность вызова функций, которые мы определяем внутри и вне модуля WebAssembly.

С помощью импортированных и экспортированных функций модуль WebAssembly взаимодействует со средой встраивания. Чтобы WebAssembly смог использовать функции и предоставлять их вызывающей странице, сначала нужно импортировать их из среды встраивания. А можно, наоборот, экспортировать разработанные вами функции из модуля WebAssembly в среду встраивания с помощью оператора `export`. Иначе функции по умолчанию будут использоваться только внутри модуля.

Вызов функций всегда приводит к потере некоторых вычислительных циклов. Важно знать, что модуль WebAssembly потеряет *больше* циклов при вызове импортированной функции JavaScript, чем при вызове функции, определенной внутри вашего модуля WebAssembly.

Когда следует вызывать функции из WAT

Каждая определенная нами ранее функция включает в себя оператор экспорта функции (`export`), чтобы JavaScript мог ее вызвать. Однако не каждая функция должна использовать `export`. Как правило, выполняющие только небольшие задачи функции WAT не экспортируются, так как каждый вызов функции WebAssembly из JavaScript влечет за собой накладные расходы. Чтобы уменьшить накладные расходы, небольшие функции, которые используют мало вычислительных циклов, лучше хранить в JavaScript. Прежде чем вернуться к JavaScript, убедитесь, что ваш WAT-код работает корректно; меньшие функции не должны использовать `export`.

Функции WAT, которые перебирают и обрабатывают большой объем данных, наиболее подходят для экспорта. Мы рекомендуем использовать как можно больше функций WAT именно в ранних версиях вашего кода, чтобы облегчить процесс отладки. Пошаговое выполнение кода WAT в отладчике будет легче, если код разбит на множество небольших функций. Отладка кода WebAssembly подробно рассматривается в главе 10. Во время отладки своего кода для повышения быстродействия вы можете убрать вызовы некоторых функций, просто встроив их в код, откуда бы они ни вызывались. Любая внутренняя функция, которая многократно вызывается из экспортированной функции WebAssembly, является хорошим кандидатом для встраивания. Повышение производительности подробно описано в главе 9.

Разработка функции `is_prime`

В предыдущих главах вы увидели примеры экспорта функций в JavaScript, но эти функции абсолютно не подходили для повышения производительности WebAssembly. Здесь мы напишем намеренно медленный алгоритм, определяющий, является ли введенное пользователем число простым. Эта функция является хорошим кандидатом для создания в WebAssembly и повышает производительность по сравнению с JavaScript, поскольку в JavaScript требуется значительное количество вычислений. Чтобы приступить к разработке приложения, создайте файлы `is_prime.js` и `is_prime.wat`.

Передача параметров

Давайте начнем с основ. Итак, сначала мы создаем модуль, а затем создаем внутри этого модуля функцию, которую можно экспортировать как `is_prime`. Эта функция принимает один 32-битный целочисленный параметр и возвращает 32-битное целое число, равное 1, если переданное число является простым, и 0, если это не так. Скопируйте код из листинга 3.1 в файл `is_prime.wat`.

Листинг 3.1. Заглушка функции WebAssembly *is_prime*

```
(module
  (func (❶export "is_prime") (❷param $n i32) (❸result i32)
    ❹i32.const 0 ;; убрать позже
  )
)
```

Мы экспортируем функцию в JavaScript как *is_prime* ❶. JavaScript передает единственный параметр *param \$n* как *i32* ❷. По завершении функция возвращает 32-битное целое число в JavaScript ❸. Когда функция завершится, она должна найти в стеке 32-битное целое число, поэтому для правильной компиляции мы добавляем строку *i32.const 0* ❹. Без этой строки при запуске *wat2wasm* компилятор выдаст ошибку, потому что по завершении функции он ожидает вернуть число, которое должно быть в стеке.

Данная функция предназначена только для экспорта и не имеет метки для внутреннего использования. За оператором *(func)* следует выражение *export*, а не метка *\$is_prime*. Если бы мы хотели вызвать эту функцию из модуля WebAssembly или из JavaScript, мы бы поместили ее, как показано в листинге 3.2.

Листинг 3.2. Экспорт функции

```
(func ❶$is_prime (export "is_prime") (param $n i32) (result i32)
```

Логические значения *truthy*

Функция *\$is_prime* ❶ должна возвращать *i32* вызывающему коду JavaScript, но мы знаем, что на стороне JavaScript нам понадобится логическое значение *true* (истина) или *false* (ложь). Однако в настоящее время WebAssembly не имеет логического типа данных. Вместо этого он использует *i32*, который должен интерпретироваться JavaScript как *true* или *false*. Для нелогических переменных в JavaScript используется не стандартная для всех языков программирования концепция, называемая *truthy*, означающая что-то вроде истины, но не явную истину. Для того чтобы значение, возвращаемое из WebAssembly, оценивалось как логическое, рекомендуется использовать оператор JavaScript *!!*, который переводит значение *0* в логическое значение *FALSE* (ложь), а любое другое число будет преобразовано в *TRUE* (истина).

Создание внутренних функций

Давайте добавим еще немного кода в файл *is_prime.wat*. Так как никакие простые числа не являются четными, за исключением числа 2, мы

напишем функцию, которая проверяет, является ли введенное число четным, глядя на последний бит целого числа. Все нечетные числа в младшем бите целочисленного представления имеют значение 1. В листинге 3.3 представлен код с функцией `$seven_check`, которую мы добавим в файл `is_prime.wat`.

Листинг 3.3. Определение функции `$seven_check`

```
is_prime.wat (module
(часть 1 из 4)   ;; добавьте функцию $seven_check в начало модуля
                 (func $seven_check (❶ param $n i32) (result i32)
                   local.get $n
                   i32.const 2
                   ❷ i32.rem_u   ;; если взять остаток от деления на 2
                   i32.const 0 ;; четные числа будут иметь остаток 0
                   ❸ i32.eq      ;; $n % 2 == 0
                 )
                 ...
```

Функция `$seven_check` принимает единственный параметр `$n` ❶ и возвращает значение 1, если `$n` четное, и 0, если нечетное. Операция остатка `i32.rem_u` ❷ делит `$n` на 2 и находит остаток. Остаток четного числа будет = 0, поэтому мы сравниваем остаток, возвращенный из `i32.rem_u`, с 0, используя выражение `i32.eq` ❸.

Теперь давайте создадим еще одну простую функцию для обработки исключения, если переданное число = 2, что является единственным четным простым числом. Она принимает один параметр и возвращает 1 (истина), если переданное число равно 2, или 0 (ложь), если не равно. Мы назовем эту функцию `$eq_2`, как показано в листинге 3.4.

Листинг 3.4. Функция `$eq_2` проверяет, равно ли переданное значение двум

```
is_prime.wat ...
(часть 2 из 4) ;; добавьте функцию $eq_2 после $seven_check
                 (func $eq_2 (param $n i32) (result i32)
                   local.get $n
                   i32.const 2
                   ❶ i32.eq    ;; возвращает 1, если $n == 2
                 )
                 ...
```

С помощью `i32.eq` ❶ мы определяем, имеет ли `$n` значение 2, и таким образом возвращаем 1, если да, и 0, если нет. Строго говоря, без функции `$eq_2` вполне можно обойтись, но поскольку мы демонстрируем, как работают вызывающие функции, еще один пример не навредит.

В листинге 3.5 мы добавляем функцию `$multiple_check`, которая проверяет, является ли первый параметр `$n` кратным второму па-

параметру `$m`. Если введенное число кратно второму числу, это означает, что оно делится без остатка и, следовательно, не может быть простым.

Листинг 3.5. Определение функции `$multiple_check`, которая проверяет, кратны ли `$n` и `$m`

```
is_prime.wat ...
(часть 3 из 4) ;; добавьте $multiple_check после $seq_2
                (func $multiple_check (param $n i32) (param $m i32) (result i32)
  ❶ local.get $n
  ❷ local.get $m
  ❸ i32.rem_u      ;; получаем остаток от $n / $m
    i32.const 0    ;; проверяем, равен ли остаток 0
  ❹ i32.eq        ;; это скажет нам, делится ли $n на $m
  )
  ...
```

Функция `$multiple_check` принимает два параметра – целое число `$n` ❶ и второе целое число `$m` ❷ – и проверяет, кратны ли `$n` и `$m`. Для этого с помощью `i32.rem_u` ❸ мы получаем остаток от `$n / $m`, а затем проверяем, равен ли этот остаток 0, используя `i32.eq` ❹. Если остаток равен нулю, функция `$multiple_check` возвращает 1. Если остаток другой, `$multiple_check` возвращает 0.

Функция `is_prime`

Теперь, когда у нас определены все внутренние функции, давайте изменим определение экспортируемой функции `is_prime` так, чтобы она возвращала 1, если переданное ей число является простым, и 0 в остальных случаях. В листинге 3.6 показана новая версия функции `is_prime`.

Листинг 3.6. Определение функции `$is_prime`

```
is_prime.wat ...
(часть 3 из 4) ;; добавьте экспортируемую функцию is_prime после $multiple_check
                (func (export "is_prime") (param $n i32) (result i32)
  ❶ (local $i i32)
    ❷ (if (i32.eq (local.get $n) (i32.const 1)) ;; 1 не простое
      (then
        i32.const 0
        return
      ))
    (if (call $seq_2 (local.get $n)) ;; проверяем, $n == 2?
      (then
        i32.const 1 ;; 2 - простое число
        return
      )
    )
  )
```

```

(block $not_prime
  (call $even_check (local.get $n))
  br_if $not_prime ;; четные числа не простые (кроме 2)

  (local.set $i (i32.const 1))

  (loop $prime_test_loop

    (local.tee $i (i32.add (local.get $i) (i32.const 2) ) ) ;; $i += 2
    local.get $n ;; stack = [$n, $i]

    i32.ge_u ;; $i >= $n
    if ;; if $i >= $n, $n - простое число
      i32.const 1
      return
    end

    (call $multiple_check (local.get $n) (local.get $i))
    br_if $not_prime ;; если $n кратно $i, это не простое число
    br $prime_test_loop ;; вернуться к началу цикла
  ) ;; конец цикла $prime_test_loop
) ;; конец блока $not_prime

i32.const 0 ;; вернуть false
)
) ;; конец модуля

```

На самом деле до добавления кода из листинга 3.6 функция `is_prime` не делала проверку на простые числа. Раньше эта функция всегда возвращала 0 (что интерпретировалось как ложь). Теперь, когда мы закодировали функцию `is_prime`, она вернет 1, если переданное число простое, и 0, если нет. В начале этой функции мы создаем локальную переменную `$i` ❶, которую позже будем использовать как счетчик цикла. Далее проверяем, равно ли `$n` 1, и если да, выполняем `return 0` ❷, потому что число 1 не является простым числом. Затем мы отбрасываем половину чисел, чтобы проверить наличие четного числа 2. Если число – 2, то оно простое; если четное, но не 2, значит, не простое. Полученное число делим на каждое нечетное число от 3 до `$n-1`. Если `$n` делится на любое из этих чисел без остатка, оно не простое. Если же оно не делится без остатка ни на одно из этих чисел, оно простое. Функция `$is_prime` довольно большая, поэтому мы рассмотрим ее по частям.

В листинге 3.7 представлена часть кода, которая проверяет, равно ли полученное число двум.

Листинг 3.7. Проверка числа `$eq_2` из `$is_prime` листинга 3.6

```

...
;; начало функции $is_prime в листинге 3.6
(if ❶(call $eq_2 (local.get $n)) ;; проверка, равно ли $n двум
  ❷(then
    i32.const 1 ;; 2 - простое число

```

```

        return
    )
)
...

```

Оператор `if` вызывает определенную ранее функцию `$eq_2` ❶ и передает ей `$n`. Если значение `$n` равно 2, эта функция возвращает 1; в противном случае возвращается 0. Если значение, возвращаемое вызовом, = 1, выполняется выражение `then` ❷, указывая на то, что число является простым.

Затем мы начинаем блок кода под названием `$not_prime`. Если в какой-то момент становится ясно, что число не является простым, мы выходим из этого блока, в результате чего функция завершается и возвращает ноль; это означает, что введенное число не является простым. В листинге 3.8 показано начало этого блока.

Листинг 3.8. Если число четное, перейдите к `$not_prime` из `$is_prime` в листинге 3.6

```

...
;; код из функции $is_prime в листинге 3.6
❶(block $not_prime
  ❷(call $even_check (local.get $n))
  ❸br_if $not_prime ;; четные числа не являются простыми (кроме 2)
...

```

В этом блоке, чтобы узнать, четное ли число, сначала вызывается `$even_check` ❷. Поскольку мы уже проверили, что это число не равно 2, любое другое четное число не будет простым. Если `$even_check` возвращает 1, код покидает блок `$not_prime` ❶ с помощью оператора `br_if` ❸.

Затем в листинге 3.9 начинается цикл, в котором проверяются числа, на которые может делиться `$n`.

Листинг 3.9. Цикл тестирования простого числа из `$is_prime` в листинге 3.6

```

...
;; код из функции $is_prime в листинге 3.6
❶(local.set $i (i32.const 1))

❷(loop $prime_test_loop
  ❸(local.tee $i
    (❹i32.add (local.get $i) (i32.const 2) ) );;$i += 2
    ❺local.get $n    ;; стек = [$n, $i]

    ❻i32.ge_u      ;; $i >= $n
    ❼if            ;; if $i >= $n, $n - простое число
      i32.const 1

```

```

        return
    end
...

```

Итак, перед началом цикла мы устанавливаем значение `$i = 1` ❶, потому что в теле цикла перебираем нечетные значения. Далее вызываем цикл с помощью `$prime_test_loop` ❷, поэтому при переходе к `$prime_test_loop` ❷ он возвращается к этой метке. Для увеличения значения `$i` на 2 (потому что мы проверяем только нечетные числа) используем команду `local.tee` ❸ в сочетании с командой `i32.add` ❹, и когда `$i` присвоено значение, оставляем в стеке результат вычисления `i32.add`. Сходство команд `local.tee` и `local.set` в том, что они присваивают переданной им переменной значение, расположенное на вершине стека. А различие в том, что `local.set` выталкивает это значение из вершины стека, тогда как `local.tee` оставляет значение на прежнем месте. Чтобы сохранить новое значение для `$i` в стеке, нужно сравнить его значение с `$n`, которое мы в следующей строке проталкиваем в стек, используя выражение `local.get` ❺.

Выражение `i32.ge_u` ❻ выталкивает последние два значения из стека и проверяет, больше ли значение, которое было в `$i`, или равно значению в `$n` (потому что число не может делиться нацело на большее число), и предполагает, что это целые числа без знака (поскольку отрицательные числа не могут быть простыми). Если это условие истинно, выражение проталкивает в стек 1, а если ложно, проталкивает 0.

Следующий оператор `if` ❼ извлекает одно значение из стека, а затем, если значение, извлеченное из стека, не равно 0, выполняет код между операторами `if` и `end`. Если `$i` больше или равно `$n` – число простое. Это означает, что выполнять код между операторами `if` и `end` стоит только в том случае, если не нашлось число, которое нацело делит `$n`, поэтому мы увеличиваем `$i` до тех пор, пока его значение не станет больше или равно `$n`.

В листинге 3.10 показан код, который проверяет, делится ли `$n` на `$i` без остатка, и если да, то `$n` не является простым числом.

Листинг 3.10. Вызов `$multiple_check` внутри основного цикла проверки из `$is_prime` в листинге 3.6

```

...
;; код из функции $is_prime в листинге 3.6
❶ (call $multiple_check (local.get $n) (local.get $i))
  ❷ br_if $not_prime ;; если $n кратно $i, это не простое число
  ❸ br $prime_test_loop ;; вернуться к началу цикла
) ;; конец цикла $prime_test_loop
) ;; конец блока $not_prime
❹ i32.const 0 ;; вернуть false
)

```

Сначала выражение `call ❶` вызывает `$multiple_check`, передавая ему `$n` и `$i`. Затем возвращает 1, если `$n` делится на `$i` без остатка, и 0, если это не так. Если возвращаемое значение равно 1, оператор `br_if ❷` переходит в конец блока `$not_prime`, в результате чего функция `$is_prime` возвращает 0 ❸ (ложь). В случае если `$multiple_check` возвращает 0, мы возвращаемся к началу `loop` ❹. Делается это для того, чтобы продолжить проверку чисел до тех пор, пока `$i` не станет больше `$n` или пока мы не найдем `$i`, которое делит `$n` без остатка.

Теперь, когда у нас готов весь WAT-код, мы можем использовать `wat2wasm` для компиляции модуля WebAssembly:

```
wat2wasm is_prime.wat
```

Код на стороне JavaScript

После компиляции модуля WebAssembly создайте файл JavaScript с именем `is_prime.js` и вставьте в него код из листинга 3.11 для загрузки и вызова функции WebAssembly `is_prime`.

Листинг 3.11. Вызов функции WebAssembly `is_prime` в файле `is_prime.js`

```
is_prime.js const fs = require('fs');
const bytes = fs.readFileSync(__dirname + '/is_prime.wasm');
const value = parseInt(process.argv[2]);

(async () => {
  const obj =
    await WebAssembly.instantiate(new Uint8Array(bytes));
  if(❶!!obj.instance.exports.is_prime(value)) {
    ❷console.log(`
      ${value} – простое число!
    `);
  }
  else {
    ❸console.log(`
      ${value} – НЕ простое число.
    `);
  }
})();
```

Когда мы вызываем функцию `is_prime` и передаем ей число через аргумент командной строки, оператор `!!` ❶ переводит возвращенное значение из целого числа в логическое значение *истина* или *ложь*. Если возвращенное значение `true`, мы выводим в консоль сообщение о том, что число является простым ❷. Если возвращаемое значение `false`, в сообщении говорится, что число не является простым ❸. Теперь мы можем запустить функцию JavaScript, используя `node`, чтобы проверить, является ли 7 простым числом:

Вот результат, который вы должны увидеть:

7 – простое число!

Итак, мы разработали несколько функций в составе модуля WebAssembly для проверки, является ли число простым. Теперь вы знакомы с основами создания функций и вызова этих функций из модуля WAT.

Доступ к стеку из функций

Если выполняющийся код находится внутри функции, компилятор `wat2wasm` выдаст ошибку, когда код попытается извлечь из стека значение, которое не было передано вашей функцией. Вот небольшой модуль `WebAssembly`, пытающийся извлечь из стека значение, которое было помещено в него вызывающей функцией (листинг 3.12).

Листинг 3.12. WAT может обращаться только к переменным стека, добавленным в текущую функцию

```
;; wat2wasm выдаст ошибку
(module
  (func $inner
    (result i32)
    (local $l i32)
    ;; число 99 находится в стеке вызывающей функции
    local.set $l

    i32.const 2
  )
  (func (export "main")
    (result i32)

    i32.const 99 ;; протолкнуть 99 в стек -[99] ❶
    call $inner ;; 99 в стеке ❷
  )
)
```

Функция `main` сначала проталкивает в стек значение 99 ❶. Когда мы вызываем функцию с меткой `$inner`, значение 99 находится в стеке ❷. Первым оператором `$inner` является `local.set`, который требует, чтобы при вызове оператора в стеке был хотя бы один элемент. Однако для `wat2wasm` не имеет значения, что вы поместили элемент в стек перед вызовом функции `$inner`. Если

вы не можете определить параметр, `wat2wasm` предполагает, что у вас ничего нет в стеке, и выдает ошибку. Чтобы функция имела доступ к переменной, при определении функции следует создать параметр. Поскольку у вас нет доступа к данным в стеке, помещенным туда вызывающей функцией, передавать переменные нужно в качестве параметров.

Объявление импортированной функции

В этом разделе мы более подробно рассмотрим объявление функций как импортируемых. Допустим, нам нужно импортировать функцию `print_string` из JavaScript, как показано в листинге 3.13.

Листинг 3.13. Объявление импортированной функции `print_string`

```
(import "env" "print_string" (func $print_string( param i32 )))
```

Оператор `import` в листинге 3.13 сообщает модулю, что нужно импортировать объект с именем `print_string`, переданный внутри объекта с именем `env`. Эти имена должны соответствовать именам в коде JavaScript. В JavaScript вы можете называть объект как угодно, но как только вы дадите ему имя в JavaScript, вы должны будете использовать то же имя при импорте в WebAssembly.

В листинге 3.14 показано, как объект импорта выглядел в файле `helloworld.js`.

Листинг 3.14. Определение `importObject`

```
helloworld.js let importObject = {
  ❶ env: {
    ❷ buffer: memory,
    ❸ start_string: start_string_index,
    ❹ print_string: function(str_len) {
      const bytes = new Uint8Array( memory.buffer,
                                   start_string_index, str_len );
      const log_string = new TextDecoder('utf8').decode(bytes);
      console.log(log_string);
    }
  }
};
```

Для определения и хранения строковых данных в приложении "hello world" мы используем объект `buffer` в памяти ❷. Также мы сообщаем модулю WebAssembly расположение этих данных внутри буфера памяти в (`start_string`) ❸. Мы помещаем оба объекта в объект `env`, чтобы отделить их от функции JavaScript. Затем создаем функ-

цию `print_string` ❹ внутри `env` ❶ как функцию обратного вызова JavaScript, которая печатает строку из линейной памяти. На момент написания этой книги стандартизированного соглашения для именования объектов внутри `importObject` не существовало. Для представления объектов, связанных со средой встраивания, и для обратных вызовов функций мы выбрали объект `env`, но вы можете реализовать `importObject` любым другим способом.

Числа JavaScript

При создании функции обратного вызова JavaScript единственный тип данных, который может получать функция, – это число JavaScript. Например, функция `print_string`, которую мы создали в приложении "hello world", передала единственную переменную `str_len`, что обозначает длину строки в байтах, отображаемой в консоли. К сожалению, функциям JavaScript в качестве параметров можно передавать только числа. В главе 5 подробно объясняется, что нужно сделать, чтобы передать в JavaScript другие типы данных.

Передача типов данных

WebAssembly может передавать три из четырех основных типов данных в функции, импортированные из JavaScript: 32-битные целые числа, 32-битные числа с плавающей запятой и 64-битные числа с плавающей запятой. На момент написания этой книги передавать 64-битные целые числа в функцию JavaScript невозможно. Ситуация должна измениться после реализации предложения `BigInt` WebAssembly. А пока выберите тип данных, в который вы хотите преобразовать значение, чтобы выполнить преобразование внутри модуля WebAssembly. Если вы передаете в JavaScript 32-битное целое число или число с плавающей запятой, JavaScript преобразует его в 64-битное число с плавающей запятой, которое является исходным числовым типом JavaScript.

Объекты в WAT

WAT не поддерживает *объектно-ориентированное программирование* (ООП – *object-oriented programming*). Создание классов и объектов с использованием WAT потенциально может быть реализовано с применением комбинации структур данных, таблицы функций и косвенного выполнения функций, но это выходит за рамки данной книги. Создание более сложных структур данных в линейной памяти, которые позволят вам сгруппировать данные в линейную память способом, аналогичным использованию `struct` в C/C++, мы рассмотрим в главе 6. К сожалению, реализация функций ООП, таких как методы объектов, наследование классов и полиморфизм, выходит за рамки этой книги.

Влияние вызовов внешних функций на производительность

В этом разделе мы исследуем влияние вызова импортированных и экспортированных функций на производительность. При вызове функции JavaScript в WAT теряются несколько циклов из-за накладных расходов. Хотя количество этих циклов не очень большое, но если вы выполните внешнюю функцию JavaScript в цикле, который повторяется 4 000 000 раз, их влияние станет заметным. Мы создали тестовый модуль WebAssembly, который позволит сравнить степень влияния вызова функций в JavaScript и внутреннего вызова на производительность приложения. Он выполняет простое приращение 4 000 000 раз во внешней функции JavaScript и столько же раз в функции WebAssembly в том же модуле.

Поскольку код выполняет очень мало вычислений, основную разницу во времени выполнения можно отнести к накладным расходам, связанным с вызовом внешней функции JavaScript. В Chrome внутренние функции WebAssembly выполняются в 4–8 раз быстрее, чем внутренние вызовы WebAssembly, а в Firefox в 2–2,5 раза быстрее. В наших тестах при пересечении вызовами границы JavaScript/WebAssembly Chrome справился гораздо хуже, чем Firefox.

Примечание *Данные тесты дают лишь грубые приближенные результаты. Они проводились на ПК с Windows и не обязательно отражают общие различия в производительности. Различные версии браузеров также могут давать разные результаты.*

Давайте посмотрим, как устроен наш тест производительности. Сначала вам нужно создать новый файл WAT. Создайте пустой файл и назовите его `func_perform.wat`. Затем откройте его и создайте модуль с одной импортированной и одной глобальной переменными, как показано в листинге 3.15.

Листинг 3.15. Импорт функции JavaScript `external_call`

```
func_perform.wat (module
  (part 1 of 4)   ;; внешний вызов функции JavaScript
  ❶ (import "js" "external_call" (func $external_call (result i32)))
  ❷ (global $i (mut i32) (i32.const 0)) ;; глобальная переменная для внутренней
                                     ;; функции
  ...
```

Выражение `import` ❶ импортирует функцию, которую мы определим в JavaScript. Эта функция вернет значение модулю WebAssembly. Выражение `global` ❷ создает изменяемую глобальную переменную с начальным значением 0. В целом использование изменяемых глобальных переменных считается плохой практикой, и этот код предна-

значен только для проверки разницы в производительности вызова функции в WebAssembly и JavaScript.

После определения глобальной переменной необходимо определить функцию WebAssembly, которую, как показано в листинге 3.16, мы будем вызывать 4 000 000 раз.

Листинг 3.16. Внутренний вызов функции WebAssembly

```
func_perform.wat ...
(часть 2 из 4) ❶(func $internal_call (result i32) ;; возвращает i32 вызывающей функции
    global.get $i
    i32.const 1
    i32.add
    ❷global.set $i ;; первые 4 строки кода в функции увеличивают $i

    ❸global.get $i ;; затем $i возвращается вызывающей функции
)
...
```

Функция `$internal_call` ❶ возвращает вызывающей функции 32-битное значение, которое будет инкрементированным значением глобальной переменной `$i`. Все, что делает эта функция, – увеличивает на 1 значение `$i` ❷, а потом возвращает его в стек ❸, чтобы вернуть вызывающей функции.

Затем, используя выражение `export`, создадим функцию, которую сможет вызвать JavaScript. Она, в свою очередь, должна вызывать функцию `$internal_call` 4 000 000 раз. В листинге 3.17 показан код этой внешней функции.

Листинг 3.17. Вызов внутренней функции 4 000 000 раз

```
func_perform.wat ...
(часть 3 из 4) ❶(func (export "wasm_call") ;; функция wasm_call передана в JavaScript
    ❷(loop $again ;; цикл $again
        ❸call $internal_call ;; вызов WASM-функции $internal_call
        i32.const 4_000_000
        ❹i32.le_u ;; значение $i <= 4 000 000?
        ❺br_if $again ;; если да, повторить цикл
    )
)
...
```

Функция ❶ экспортируется для вызова из кода JavaScript. Эта функция имеет простой цикл с меткой `$again` ❷, который вызовет функцию `$internal_call` ❸ 4 000 000 раз. Цикл сравнивает значение, возвращаемое `$internal_call` (значение в глобальном `$i`), с `4_000_000`. Если значение `$i` меньше 4 000 000 (`i32.le_u`) ❹, выполняется возврат к началу цикла `$again` ❺.

Теперь, когда у нас есть функция, которая тестирует быстродействие внутреннего вызова функции WebAssembly, в листинге 3.18

мы создадим почти идентичную функцию, которая будет вызывать внешнюю функцию JavaScript.

Листинг 3.18. Вызов внутренней функции 4 000 000 раз

```
func_perform.wat    ...
(часть 4 из 4)    ❶ (func (export "js_call")
                  (loop $again
                    ❷ (call $external_call) ;; вызов импортированной функции $external_call
                    i32.const 4_000_000
                    i32.le_u    ;; значение, возвращаемое $external_call, <= 4 000 000?
                    br_if $again ;; если да, выполнить переход к началу цикла
                  )
                )
              ) ;; конец модуля ❸
```

Между этой функцией и той, что показана в листинге 3.19, есть только два различия. Первое – это имя экспортируемой функции "js_call" ❶, которая вызывает импортированную \$external_call ❷. Второе – для того чтобы закрыть выражение module ❸, в конце кода мы помещаем закрывающую скобку.

После того как вы закончите создание кода *func_perform.wat*, скомпилируйте его в модуль WebAssembly с помощью *wat2wasm*, используя следующую команду:

```
wat2wasm func_perform.wat
```

Затем создайте пустой файл JavaScript с именем *func_perform.js*. В нем JavaScript загрузит и вызовет наш модуль WebAssembly. Вставьте в файл код JavaScript из листинга 3.19.

Листинг 3.19. Функция *external_call*, определенная в JavaScript *importObject*

```
func_perform.js    const fs = require('fs');
(часть 1 из 2)    const bytes = fs.readFileSync(__dirname + '/func_perform.wasm');

                  ❶ let i = 0;
                  let importObject = {
                    js: {
                      ❷ external_call: function () { // импортированная функция JavaScript
                        i++;
                        ❸ return i; // увеличить переменную i и вернуть ее
                      }
                    }
                  };
                  ...
```

Мы объявляем переменную *i* ❶ и инициализируем ее нулем. Внутри *importObject* создаем функцию *external_call* ❷, которую мы ранее

импортировали в WebAssembly. Единственное, что делает эта функция, – увеличивает на 1 и затем возвращает значение в `i` ❸.

Затем нам нужно реализовать модуль `func_perform.wasm` и выполнить `wasm_call` и `js_call`, как показано в листинге 3.20.

Листинг 3.20. Определение асинхронного IIFE внутри JavaScript

```
func_perform.js
(часть 2 из 2)
...
(async () => {
  ❶ const obj = await WebAssembly.instantiate(new Uint8Array(bytes),importObject);
  // выполнить деструктуризацию wasm_call и js_call из obj.instance.exports
  ❷ ({ wasm_call, js_call } = obj.instance.exports);

  let start = Date.now();
  ❸ wasm_call(); // вызов wasm_call из модуля WebAssembly
  let time = Date.now()-start;
  ❹ console.log('wasm_call time=' + time); // время выполнения в миллисекундах

  start = Date.now();
  ❺ js_call(); // call js_call from WebAssembly module
  time = Date.now()-start;
  ❻ console.log('js_call time=' + time); // время выполнения в миллисекундах
})();
```

Как и во всех других приложениях в этой книге, необходимо создать экземпляр модуля `func_perform.wasm` ❶ в JavaScript. Для создания функций `wasm_call` и `js_call` мы используем синтаксис деструктуризации JavaScript ❷. Эта деструктуризация входит в синтаксис ECMAScript 2015, который удобен для извлечения нескольких переменных из объекта. В качестве альтернативы вы можете установить значения переменных `wasm_call` и `js_call` в `obj.instance.exports`.

После получения функций из модуля WebAssembly мы вызываем `wasm_call` ❸ и записываем в лог время, которое потребовалось для ее выполнения в миллисекундах ❹. Затем мы вызываем функцию `js_call` ❺ и записываем ❻ время, которое потребовалось для выполнения.

Запустите JavaScript с помощью node следующим образом:

```
node func_perform.js
```

В консоли вы должны увидеть что-то наподобие этих строк:

```
wasm_call time=7
js_call time=32
```

Итак, для вызова функции внутри WebAssembly 4 000 000 раз потребовалось 7 мс, а для вызова из JavaScript – 32 мс. Это может показаться большой разницей, но на самом деле разница в 25 миллисекунд на 4 000 000 вызовов довольно мала. Если эта функция вызывается

только один раз за фрейм, тогда разница не имеет значения. Однако если у вас есть цикл, который выполняется сотни или тысячи раз за рендеринг фрейма, возможно, из соображений быстродействия стоит организовать ваш код по-другому. В следующем разделе мы рассмотрим таблицы функций и их производительность.

Таблицы функций

JavaScript может в качестве значений присваивать переменным функции, позволяя приложению динамически заменять функции во время выполнения. WebAssembly так не умеет, но у него есть *таблицы*, которые на момент написания этой книги могут содержать только функции. По этой причине мы будем называть их *таблицами функций*, хотя в будущем планируется поддержка и других типов. Таблицы функций позволяют WebAssembly динамически заменять функции во время выполнения, благодаря чему компиляторы могут поддерживать указатели на функции и виртуальные функции ООП. Например, программы C/C++ используют таблицы WebAssembly для реализации указателей на функции. В настоящее время таблицы поддерживают только тип `anyfunc` (универсальный тип функции WebAssembly), но в будущем они, вероятно, смогут также поддерживать объекты JavaScript и элементы DOM (*Document Object Model – объектная модель документа*). В отличие от объектов импорта, JavaScript и WebAssembly могут динамически изменять таблицы во время выполнения. Вызов функции из таблицы, а не через импорт снижает производительность, поскольку запись в таблице функций должна вызываться косвенно. Давайте сравним производительность функций, вызываемых через таблицу и вызываемых напрямую.

Создание таблицы функций в WAT

В этом разделе мы создадим и экспортируем простую таблицу функций в WAT. Построим модуль с четырьмя функциями: две из них импортируются из JavaScript, а две определены внутри модуля WebAssembly. Эти функции будут очень простыми, потому что их назначение – сравнить производительность выполнения табличных функций с прямым импортом. Создайте новый файл с именем `table_export.wat` и введите код из листинга 3.21.

Листинг 3.21. Экспорт функций в таблицу

```
table_export.wat (module
    ;; функция инкремента javascript
    ❶ (import "js" "increment" (func $js_increment (result i32)))
    ;; функция декремента javascript
    ❷ (import "js" "decrement" (func $js_decrement (result i32)))
    ❸ (table $tbl (export "tbl") 4 anyfunc) ;; экспортированная таблица с 4 функциями
```

```

(global $i (mut i32) (i32.const 0))

❹(func $increment (export "increment") (result i32)
  ❺(global.set $i (i32.add (global.get $i) (i32.const 1))) ;; $i++
    global.get $i
  )

❻(func $decrement (export "decrement") (result i32)
  ❼(global.set $i (i32.sub (global.get $i) (i32.const 1))) ;; $i--
    global.get $i
  )

;; заполняем таблицу
❽(elem (i32.const 0) $js_increment $js_decrement $increment $decrement)
)

```

В этом модуле есть два импортируемых объекта: JavaScript-функции `increment` ❹ и `decrement` ❻. Однако добавить функцию JavaScript в таблицу функций изнутри JavaScript вы не можете. Существует набор `WebAssembly.Table`, который позволяет вам размещать функции в таблице – но только те, которые определены в модуле `WebAssembly`. Мы сможем обойти это ограничение, импортировав функцию JavaScript в модуль `WebAssembly` и добавив ее в таблицу.

Выражение `table` ❸ создает таблицу под названием `$tbl`, на которую мы можем ссылаться в коде WAT. Мы экспортируем `$tbl` и сообщаем выражению `table`, что в этой таблице есть четыре объекта типа `anyfunc` (в настоящее время поддерживается единственный тип объекта таблицы). Затем создаем две функции `WebAssembly`: функция `$increment` ❹ присваивает глобальной переменной `$i` значение `$i + 1` ❺, а функция `$decrement` ❻ присваивает глобальной переменной `$i` значение `$i - 1` ❼.

Завершая этот модуль, мы устанавливаем значения в таблице с помощью выражения `elem` ❽. В качестве своего первого параметра выражение `elem` принимает индекс первого элемента, который мы создали. Для создания всех четырех элементов мы используем `(i32.const 0)`, потому что первый параметр – это начальное значение индекса, которое мы хотим обновить. Далее перечисляем четыре функциональные переменные, которые нам нужны в таблице.

В качестве альтернативы, если бы мы хотели указать только первые два элемента в таблице, нам не нужно было бы передавать все четыре имени функции, и мы бы сделали следующее:

```

(elem (i32.const 0) $js_increment $js_decrement) ;; указать первые 2 функции
                                           ;; таблицы

```

Чтобы указать вторые два элемента, мы должны изменить значение первого параметра, чтобы выражение знало, что мы начинаем с третьего элемента в таблице, как показано ниже:

```
(elem (i32.const 2) $increment $decrement) ;; установить 3 и 4 содержимое  
;; таблицы
```

Первый параметр оператора `elem` в листинге 3.21 – это индекс 0, аналогичный массиву. Закончив набирать код, скомпилируйте `table_export.wat` в `table_export.wasm` с помощью команды `wat2wasm`.

Совместное использование таблицы разными модулями

Теперь, когда у нас есть `table_export.wasm`, давайте создадим второй файл WebAssembly, который использует ту же таблицу. Создайте новый WAT-файл с именем `table_test.wat`. Новый модуль WebAssembly мы начнем с ряда операторов `import` и выражения определения `type`, как показано в листинге 3.22.

Листинг 3.22. Импорт функций в модуле WebAssembly

```
table_test.wat (часть 1 из 3) (module  
  ❶(import "js" "tbl" (table $tbl 4 anyfunc))  
    ;; импорт функции increment  
  ❷(import "js" "increment" (func $increment (result i32)))  
    ;; импорт функции decrement  
  ❸(import "js" "decrement" (func $decrement (result i32)))  
  
    ;; импорт функции wasm_increment  
  ❹(import "js" "wasm_increment" (func $wasm_increment (result i32)))  
    ;; импорт функции wasm_decrement  
  ❺(import "js" "wasm_decrement" (func $wasm_decrement (result i32)))  
  
    ;; Табличные функции типа определяют все i32 и не принимают никаких параметров  
  ❻(type $returns_i32 (func (result i32)))  
  ...
```

Первый оператор `import` ❶ импортирует таблицу с четырьмя функциями `anyfunc` в файл `table_export.wat`. Затем мы импортируем функции JavaScript `increment` ❷ и `decrement` ❸, которые будем использовать для сравнения производительности импортированных функций JavaScript с функциями JavaScript, определенными в таблицах.

Далее импортируем функции `wasm_increment` ❹ и `wasm_decrement` ❺, определенные в таблице в файле `table_export.wat`. Так мы можем проверить производительность `call_indirect` для элемента таблицы с вызовом той же функции, импортированной напрямую с помощью оператора `import`.

Последнее – это выражение `type` ❻, которое определяет сигнатуру функций в таблице. В качестве статического параметра для `call_indirect` нужно было предоставить тип `$returns_i32`. Так как тип косвенно вызываемой функции должен динамически проверяться на соответствие этому предоставленному типу, можно ожидать, что `call_indirect` будет медленнее, чем `call`.

Теперь давайте воспользуемся кодом из листинга 3.23 для определения четырех глобальных переменных, которые мы будем использовать для индексации в таблице функций.

Листинг 3.23. Индексы таблицы функций глобальных переменных

```
table_test.wat ...
(часть 2 из 3) ❶(global $inc_ptr i32 (i32.const 0)) ;; Индекс таблицы JS функции increment
               ❷(global $dec_ptr i32 (i32.const 1)) ;; Индекс таблицы JS функции decrement

               ❸(global $wasm_inc_ptr i32 (i32.const 2)) ;; Индекс WASM функции increment
               ❹(global $wasm_dec_ptr i32 (i32.const 3)) ;; Индекс WASM функции decrement
               ...
```

Эти четыре глобальные переменные являются индексами в таблице функций и дают нам более простой способ отслеживать, каким функциям соответствуют индексы. Переменные `$inc_ptr` ❶ и `$dec_ptr` ❷ указывают на JavaScript функции `increment` и `decrement`, а `$wasm_inc_ptr` ❸ и `$wasm_dec_ptr` ❹ указывают на версии WebAssembly функций `increment` и `decrement` в таблице.

Определение тестовых функций

Определив импорты и глобальные переменные, надо определить четыре тестовые функции. Все эти функции будут выполнять одну и ту же задачу, используя разные методы: они вызовут из `loop` функцию `increment` 4 000 000 раз, а затем вызовут `decrement` столько же раз. Одна из них будет вызывать функции косвенно из импортированной таблицы; другая будет вызывать прямо из `import`; третья вызовет версию WebAssembly из таблицы; и последняя вызовет функции `increment` и `decrement` напрямую из `import`. В конце концов, мы сможем сравнить производительность вызова функций JavaScript через таблицу или напрямую, а также производительность вызова функции модуля WebAssembly напрямую или через таблицу.

В листинге 3.24 показаны четыре определения функций.

Листинг 3.24. Тестирование производительности

```
table_test.wat ;; Тестирование производительности косвенного табличного вызова функций JavaScript
(часть 3 из 3) ❶(func (export "js_table_test")
  (loop $inc_cycle
    ;; косвенный вызов JavaScript-функции increment
    (call_indirect (type $returns_i32) (global.get $inc_ptr))
    i32.const 4_000_000
    i32.le_u ;; значение, возвращаемое вызовом $inc_ptr, <= 4000000?
    br_if $inc_cycle ;; если да, зациклить
  )

  (loop $dec_cycle
    ;; косвенный вызов JavaScript-функции decrement
```

```

        (call_indirect (type $returns_i32) (global.get $dec_ptr))
        i32.const 4_000_000
        i32.le_u ;; значение, возвращаемое вызовом $dec_ptr, <= 4000000?
        br_if $dec_cycle ;; если да, заиклеть
    )
)

;; Тестирование производительности прямого вызова функций JavaScript
❷(func (export "js_import_test")
    (loop $inc_cycle
        call $increment ;; прямой вызов JavaScript-функции increment
        i32.const 4_000_000
        i32.le_u ;; значение, возвращаемое вызовом $increment, <= 4000000?
        br_if $inc_cycle ;; если да, заиклеть
    )

    (loop $dec_cycle
        call $decrement ;; прямой вызов JavaScript-функции decrement
        i32.const 4_000_000
        i32.le_u ;; значение, возвращаемое вызовом $decrement, <= 4000000?
        br_if $dec_cycle ;; если да, заиклеть
    )
)

;; Проверка производительности косвенного вызова таблицы функций WASM
❸(func (export "wasm_table_test")
    (loop $inc_cycle
        ;; косвенный вызов WASM-функции increment
        (call_indirect (type $returns_i32) (global.get $wasm_inc_ptr))
        i32.const 4_000_000
        i32.le_u ;; значение, возвращаемое вызовом $wasm_inc_ptr, <= 4000000?
        br_if $inc_cycle ;; если да, заиклеть
    )

    (loop $dec_cycle
        ;; косвенный вызов WASM-функции decrement
        (call_indirect (type $returns_i32) (global.get $wasm_dec_ptr))
        i32.const 4_000_000
        i32.le_u ;; значение, возвращаемое вызовом $wasm_dec_ptr, <= 4000000?
        br_if $dec_cycle ;; если да, заиклеть
    )
)

;; Тестирование производительности прямого вызова функций WASM
❹(func (export "wasm_import_test")
    (loop $inc_cycle
        call $wasm_increment ;; прямой вызов WASM-функции increment
        i32.const 4_000_000
        i32.le_u
        br_if $inc_cycle
    )

    (loop $dec_cycle
        call $wasm_decrement ;; прямой вызов WASM-функции decrement
        i32.const 4_000_000

```

```

        i32.le_u
        br_if $dec_cycle
    )
)
)

```

Первая функция, которую мы определяем для экспорта, – это `js_table_test` ❶. Она выполняет 8 000 000 косвенных вызовов простых функций JavaScript. Следующая – `js_import_test` ❷, которая вызывает те же функции, что и `js_table_test`. Однако она делает это напрямую из `import`. Это позволяет нам сравнивать производительность 8 000 000 запусков одной и той же функции с использованием таблицы и без нее. Две другие функции, `wasm_table_test` ❸ и `wasm_import_test` ❹, устроены так же, как и js-версии, но вызывают функции модуля WebAssembly вместо JavaScript.

Закончив работу с файлом `table_test.wasm`, создайте новый файл JavaScript с именем `table.js` и вставьте в него код из листинга 3.25.

Листинг 3.25. JavaScript функции *increment* и *decrement*

```

table.js  const fs = require('fs');
(часть 1 из 4) const export_bytes = fs.readFileSync(__dirname+'/table_export.wasm');
const test_bytes = fs.readFileSync(__dirname + '/table_test.wasm');

let i = 0;
let increment = () => {
    i++;
    return i;
}
let decrement = () => {
    i--;
    return i;
}
...

```

Мы протестируем эти функции, вызывая их напрямую с помощью `import`, и косвенно с помощью таблицы.

Создание WebAssembly `importObject` в JavaScript

Теперь нам нужен объект `importObject`, который мы будем использовать для обоих модулей WebAssembly. Этот объект определяет все функции, значения и таблицы, которые мы хотим передать из JavaScript в модуль WebAssembly. В листинге 3.27 мы также будем использовать его для передачи таблицы из одного модуля WebAssembly в другой. В первую очередь установим для объектов `tbl`, `wasm_decrement` и `wasm_increment` значение `null`, поскольку они еще не используются модулем `table_export.wasm`. Но они понадобятся, когда мы загрузим модуль `table_test.wasm`. Листинг 3.26 показывает `importObject` в действии.

Листинг 3.26. JavaScript `importObject`

```
table.js ...  
(часть 2 из 4) const importObject = {  
  js: {  
    ❶ tbl: null, // tbl изначально имеет значение null и предназначен для второго  
                // модуля WASM  
    ❷ increment: increment, // JavaScript-функция increment  
      decrement: decrement, // JavaScript-функция decrement  
    ❸ wasm_increment: null, // Изначально null, присваивается функции вторым модулем  
      wasm_decrement: null // Изначально null, присваивается функции вторым модулем  
  }  
};  
...
```

На этом этапе значение `tbl`❶, которое мы передаем через `importObject`, равно `null`, потому что оно создано в модуле `table_export.wasm`, и нам нужно будет инициализировать это значение с помощью таблицы, экспортированной из `table_export.wasm`. Функции `increment` и `decrement`❷ были определены ранее в JavaScript. Функции `wasm_increment` и `wasm_decrement`❸ мы еще не определили, потому что нам нужно будет поместить их в функцию `import` после того, как они будут созданы в `table_export.wasm`.

Реализация модулей WebAssembly

Теперь давайте посмотрим на листинг 3.27, чтобы понять, как реализовать два модуля WebAssembly.

Листинг 3.27. Реализация модуля WebAssembly в асинхронном IIFE

```
table.js ...  
(часть 3 из 4) ❶ (async () => {  
  // реализация модуля, который использует таблицу функций  
  ❷ let table_exp_obj = await WebAssembly.instantiate(  
    new Uint8Array(export_bytes), importObject);  
  
  // задать переменную tbl для экспортируемой таблицы  
  ❸ importObject.js.tbl = table_exp_obj.instance.exports.tbl;  
  
  ❹ importObject.js.wasm_increment =  
    table_exp_obj.instance.exports.increment;  
  ❺ importObject.js.wasm_decrement =  
    table_exp_obj.instance.exports.decrement;  
  ❻ let obj = await WebAssembly.instantiate(  
    new Uint8Array(test_bytes), importObject);  
  ...  
}
```

Как и раньше, для реализации модулей WebAssembly мы используем асинхронный IIFE ❶. Однако теперь, чтобы продемонстрировать,

как мы можем совместно использовать функции и таблицы функций между модулями WebAssembly, мы создаем два модуля WebAssembly вместо одного. При выполнении модуля *table_export.wasm* мы помещаем объект WebAssembly в переменную *table_exp_obj* ❷. До этого мы помещали все объекты модуля WebAssembly в переменную *obj*, но поскольку мы используем более одного модуля в этом приложении, нам нужны более конкретные названия.

Мы используем *table_exp_obj*, чтобы добавить переменную *tbl* ❸, определенную ранее в *importObject*, в таблицу функций, созданную в модуле *table_export.wasm*. Затем задаем значения переменных *wasm_increment* ❹ и *wasm_decrement* ❺ в *importObject* и создаем экземпляр модуля *table_test.wasm* ❻.

Последний блок кода в этом разделе в листинге 3.28 выполняет тест и измеряет время в миллисекундах, которое потребовалось для выполнения каждого теста.

Листинг 3.28. Вызов функций WebAssembly и запись времени выполнения

```

table.js ...
(часть 4 из 4) // для создания JS-функций используем деструктурирующий синтаксис
               // из экспорта
❶ ({ js_table_test, js_import_test,
    wasm_table_test, wasm_import_test } = obj.instance.exports);

    i = 0; // переменная i должна быть переинициализирована на 0
    let start = Date.now(); // получить начальную метку времени
❷ js_table_test(); // запуск функции, которая тестирует вызовы таблиц JS
    let time = Date.now()-start; // сколько времени потребовалось для запуска
    console.log('js_table_test time=' + time);

    i = 0; // i должна быть переинициализирована на 0
    start = Date.now(); // получить начальную метку времени
❸ js_import_test(); // запуск функции, которая проверяет вызовы прямого
                    // импорта JS
    time = Date.now()-start;
    console.log('js_import_test time=' + time);

    i = 0; // i должна быть переинициализирована на 0
    start = Date.now(); // получить начальную метку времени
❹ wasm_table_test(); // запуск функции, которая проверяет вызовы таблиц WASM
    time = Date.now()-start; // сколько времени потребовалось для запуска
    console.log('wasm_table_test time=' + time);

    i = 0; // i должна быть переинициализирована на 0
    start = Date.now(); // получить начальную метку времени
❺ wasm_import_test(); // запуск функции, которая проверяет вызовы прямого
                    // импорта WASM
    time = Date.now()-start; // сколько времени потребовалось для запуска
    console.log('wasm_import_test time=' + time);
  })();

```

Для создания четырех переменных ❶, {js_table_test, js_import_test, wasm_table_test, wasm_import_test} из атрибутов объекта obj.instance.exports мы используем синтаксис деструктуризации JavaScript. Это просто удобный способ создать все четыре функциональные переменные одновременно и определить для них одноименные функции, экспортируемые модулем WebAssembly.

Затем мы запускаем каждую из функций одну за другой, используя node, например:

```
node table.js
```

Вот четыре строки, которые вы должны увидеть в консоли, показывающие время выполнения {js_table_test❷, js_import_test❸, wasm_table_test❹, wasm_import_test} из атрибутов объекта obj.instance.exports❺:

```
js_table_test time=67
js_import_test time=60
wasm_table_test time=25
wasm_import_test time=20
```

Строка, отображающая среду выполнения js_import_test❸, показывает, что для этого запуска вызов JavaScript через импорт выполнялся примерно на 10 % быстрее, чем вызов той же функции с использованием таблицы. Это может показаться незначительной разницей в зависимости от потребностей вашего приложения.

Затем мы видим время в мс, необходимое для вызова аналогичных версий WebAssembly функций increment и decrement. Эта разница более значительна: вызов таблицы занимает примерно на 25 % больше времени, чем прямой импорт.

Примечание Если вы получаете противоречивые результаты тестирования, можно увеличить количество запусков с 4 000 000 до 20 000 000 или более. В данном случае значение 4 000 000 нам подошло, но результаты будут зависеть от аппаратного обеспечения вашего компьютера.

Итак, вы узнали, как совместно использовать функции и таблицы функций между различными модулями WebAssembly. Вы также узнали, в чем разница между вызовом функции непосредственно из import и косвенным вызовом через таблицу, каково влияние на производительность вызова функции с использованием таблицы. Понимание того, как функции вызываются, импортируются, экспортируются и используются в таблицах функций, является фундаментальной особенностью языка, которую необходимо понять, прежде чем освоить разработку с помощью WebAssembly.

Заключение

В этой главе мы научились вызывать функции WebAssembly из JavaScript и функции JavaScript из WebAssembly. Мы рассмотрели влияние каждого типа вызовов на производительность. Также мы создали приложение, которое проверяет простые числа, чтобы показать, как выглядит функция, которую имеет смысл создавать в модуле WebAssembly. Мы рассмотрели передачу параметров функциям, определенным в WebAssembly, и то, как создавать в WebAssembly функции, которые не доступны для JavaScript. Узнали, что нужно для создания функций, которые управляют строками в WebAssembly, и как получить доступ к этим строкам из JavaScript. Углубились в типы данных WebAssembly и то, как они преобразуются в типы данных JavaScript. Мы изучили таблицы и способы их использования для косвенного вызова функций WebAssembly и JavaScript, включая созданные во втором модуле функции WebAssembly. Мы также выделили время на изучение влияния использования таблиц и функций, созданных во втором модуле WebAssembly, на производительность. В следующей главе мы рассмотрим использование WAT для низкоуровневого программирования.

4

НИЗКОУРОВНЕВЫЕ БИТОВЫЕ ОПЕРАЦИИ



В этой главе обсуждаются некоторые методы битовых операций в приложениях WebAssembly, которые мы применим в проектах, представленных в следующих главах, для повышения производительности приложений. Эта тема может оказаться сложной для читателей, знакомых с программированием на языках низкого уровня, поэтому, если работа с двоичными данными вас не интересует, вы можете перейти к следующей главе, а к этой просто вернуться в случае необходимости.

Прежде чем исследовать методы битовых операций, мы рассмотрим несколько важных тем, которые включают три различных основания системы счисления: десятичное, шестнадцатеричное и двоичное; некоторые особенности арифметических операций над целыми числами и числами с плавающей запятой; числа в дополнительном двоичном коде (two's complement), а также байты с прямым и обратным порядками битов. Кроме того, разберем такие понятия, как биты старшего и младшего разряда, побитовое маскирование, битовый сдвиг и побитовое вращение.

WebAssembly позволяет максимально использовать возможности вашего «железа» непосредственно из веб-браузера. Если вы хотите написать приложение WebAssembly, которое будет выполняться с молниеносной скоростью, очень важно понимать, как работать с данными на уровне битов. Знание битовых операций также необходимо для понимания того, как выполняются различные операции с типами данных WebAssembly и какие у них ограничения. WAT может выполнять операции с данными на уровне битов аналогично ассемблерным языкам. Низкоуровневое программирование – сложный предмет, и если вы еще незнакомы ни с одной концепцией, которые описаны в этой главе, вам может быть непросто усвоить их все сразу. Хорошая новость в том, что необязательно знать все приемы низкоуровневого программирования для работы с WebAssembly, но их понимание может помочь вам написать быстрый и высокопроизводительный код для распространения в сети. Также очень часто программа, оптимизируемая компилятором, генерирует код, который выполняет битовые операции, поэтому знание того, как работают эти битовые операции, пригодится вам при дизассемблировании двоичного файла WebAssembly в код WAT.

Системы счисления: двоичная, десятичная и шестнадцатеричная

Шестнадцатеричная система счисления – это стандартная система счисления в компьютерном программировании, в которой используется система счисления по основанию 16 вместо десятичной системы счисления по основанию 10, которую вы использовали с двухлетнего возраста. Компьютерные программисты работают с шестнадцатеричной системой, потому что компьютеры изначально используют двоичный код, а шестнадцатеричные числа переводятся в двоичные более понятным способом, чем десятичные. Думаю, вам нет нужды учиться преобразовывать десятичное число в шестнадцатеричное вручную, потому что, скорее всего, вы либо знакомы с тем, как это делать, либо имеете доступ к калькулятору; большинство приложений-калькуляторов предлагают режим программиста, который выполняет это преобразование за вас, как показано на рис. 4.1.

Я специально разработал инструмент преобразования десятичных чисел в шестнадцатеричные (адрес: <https://wasmbok.com/hex>) и онлайн-калькулятор, написанный на WAT (адрес: <https://wasmbok.com/calculator.html>). Имейте в виду, что если вы хотите встроить числовые данные в строку WAT, вам нужно использовать двузначные шестнадцатеричные числа, а не десятичные.



Рис. 4.1. Приложение Microsoft Windows Calculator в режиме программиста

Арифметические операции над целыми числами и числами с плавающей запятой

Прежде чем начать обсуждение, как использовать битовые операции с различными типами данных, нам нужно познакомиться с типами, которые поддерживает WebAssembly. Два основных типа данных в WebAssembly – это целые числа и числа с плавающей запятой. Эти типы присваиваются всем локальным и глобальным переменным при их объявлении, и их нужно использовать для всех параметров в объявлениях функций. Целые числа могут быть как отрицательными, так и положительными. Однако использовать отрицательные целые числа немного сложнее, чем отрицательные числа с плавающей запятой.

Хранить целые числа и числа с плавающей запятой вы можете в переменных и в линейной памяти. Если вы уже знакомы с JavaScript, то заметите, что линейная память похожа на типизированный целочисленный массив без знака. Подробнее о линейной памяти мы поговорим в главе 6.

Примечание Если вы знакомы с ассемблерными языками низкого уровня, то можете рассматривать линейную память как кучу (неупорядоченный массив) памяти. Если же вы незнакомы с низкоуровневыми языками, просто проигнорируйте это примечание.

В этом разделе мы рассмотрим, как работают целочисленные переменные и переменные с плавающей запятой, их различные типы и как выполнять с ними некоторые базовые битовые операции. Давайте сначала бегло рассмотрим основные типы данных, которые поддерживает WebAssembly. Есть два целочисленных типа данных (`i32` и `i64`), а также два типа данных с плавающей запятой (`f32` и `f64`).

Целые числа

Большинство математических операций над целыми числами обычно выполняются быстрее, чем операции над числами с плавающей запятой. WebAssembly в настоящее время поддерживает 32-битные и 64-битные целые числа, но, к сожалению, не может совместно использовать 64-битные целые числа с JavaScript так же легко, как другие типы данных.

`i32` (32-битные целые числа)

Тип данных `i32` быстрый, небольшой и легко перемещается между WebAssembly и JavaScript. Он может представлять от 0 до 4 294 967 295 беззнаковых данных и от -2 147 483 648 до 2 147 483 647 для целых чисел со знаком. Если вы работаете с числами, которые имеют значение менее миллиарда, и вам не нужны дробные числа, использование `i32` – отличный выбор. Работа со знаковыми или беззнаковыми значениями больше говорит о функциях, которые вы выполняете с помощью данных, чем о самих данных в переменной. Тип данных `i32` представляет отрицательные числа с использованием дополнительного кода.

Дополнительный двоичный код

Дополнительный двоичный код (дополнение до двух) – это широко используемый способ представления отрицательных чисел в двоичном формате. Компьютеры работают только в двоичной системе, поэтому могут хранить в памяти лишь единицы и нули. Двоичное число довольно легко преобразовать в десятичное, а вот представить отрицательное число только с помощью единиц и нулей уже посложнее. Важно отметить, что числа с плавающей запятой имеют специальный знаковый бит, и поэтому для их представления не используют дополнительный код.

Чтобы понять, как работает дополнительный код, давайте проведем метафорическое сравнение. Представьте, что у вас есть кнопочный счетчик, и каждый раз, когда вы нажимаете кнопку, его циферблат смещается на единицу (рис. 4.2). В этом конкретном счетчике отображаются только десятичные цифры от 0 до 9, поэтому, когда вы нажимаете кнопку в десятый раз, он возвращается к 0. Для счетчика число 10 эквивалентно 0, поэтому он каждый раз возвращается в начало. Кстати, нажатие кнопки 9 раз функционально аналогично вычитанию 1 для всех чисел, кроме 0.

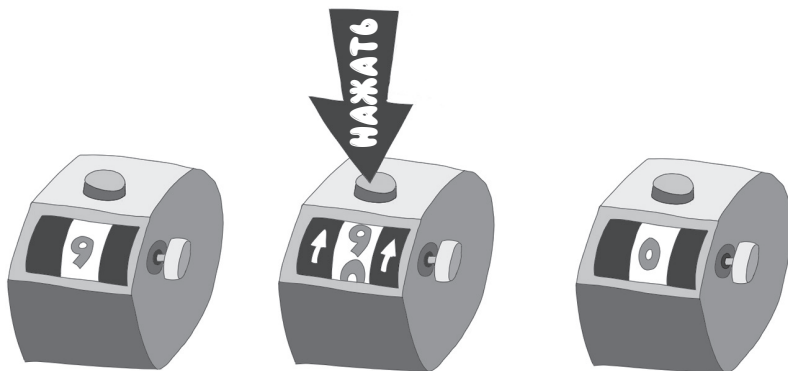


Рис. 4.2. Если прибавить 1, то 9 переходит в 0

Использование больших чисел в качестве отрицательных будет полезно только в том случае, если вы знаете, что ваши числа будут находиться вне диапазона выбранных отрицательных чисел. Если вы объявите, что 9 эквивалентно -1 , и вам понадобится сосчитать до 9 с помощью счетчика, то эта система счисления не работает. Основываясь на рассмотренном нами принципе переполнения, при помощи 8-битного целого числа со знаком мы можем представить числа от -128 до 127. 8-битное представление числа 255 без знака (восемь единиц) совпадает с 8-битным представлением числа -1 со знаком, потому что прибавление 255 к любому числу заставляет биты «проворачиваться», как на счетчике, и вычитать единицу. Самые большие числа становятся отрицательными, потому что они приводят к переворачиванию битов. Программа, выполняющая преобразование двоичных чисел в дополнительный код, использует двоичное ИСКЛЮЧАЮЩЕЕ ИЛИ (XOR), чтобы инвертировать все биты, а затем прибавляет к результату число 1, что дает нам отрицательное число. Вы увидите код этой функции, когда мы рассмотрим XOR и инверсию битов позже в данной главе.

i64 (64-битные целые числа)

Тип данных `i64` может представлять положительные целые числа от 0 до 18 446 744 073 709 551 615 для целых чисел без знака и от $-9\,223\,372\,036\,854\,775\,808$ до $9\,223\,372\,036\,854\,775\,807$ для целых чисел со знаком. При объявлении переменной тип данных `i64` в WAT не указывает, со знаком он или без знака. Вместо этого WAT должен выбирать операции для выполнения с этими данными в зависимости от того, хочет ли пользователь рассматривать число как со знаком или без знака. Однако не все операции заставляют пользователя делать такой выбор: например, `i64.add`, `i64.sub` и `i64.mul` работают одинаково, независимо от того, со знаком целые числа или нет, в отличие от функций деления, таких как `i64.div_s` и `i64.div_u`. Операция деления должна обрабатывать числа со знаком или без знака по-разному, поэтому к различным версиям операции должны быть добавлены суффиксы `_s` (signed, со знаком) и `_u` (unsigned, без знака).

Как упоминалось ранее, одна из проблем с типами данных `i64` заключается в том, что вы не можете напрямую перемещать 64-битные целые числа между `WebAssembly` и `JavaScript`. `JavaScript` использует только 64-битные числа с плавающей запятой, но в них могут уместиться как 32-битные целые числа, так и 32-битные числа с плавающей запятой. Короче говоря, перенос 64-битных целых чисел в `JavaScript`-часть приложения может доставить вам много проблем.

Примечание На момент написания этой книги поддержка `WebAssembly` для объектов `JavaScript BigInt` находится на завершающей стадии разработки. По завершении передавать 64-битные целые числа в `JavaScript` из `WebAssembly` и наоборот будет проще.

Числа с плавающей запятой

Число с плавающей запятой состоит из трех частей в двоичном формате: знаковый бит, за которым следует серия битов, представляющих показатель степени, а затем биты, представляющие значащие разряды, или, как их еще называют, значащую часть числа. Помните, что в двоичном формате нет десятичной запятой; программистам пришлось изобрести систему представления десятичных знаков внутри двоичного числа. Знаковый бит указывает, является ли число положительным или отрицательным. Показатель степени представляет, на сколько позиций нужно переместить десятичную запятую (влево или вправо), а значащие цифры – это просто набор цифр вашего числа с плавающей запятой. Давайте посмотрим, как можно представить десятичные числа с плавающей запятой, увеличивая число с помощью показателя степени: если вы возьмете число, например, 345 и умножите его на 10 в степени 2, эта операция добавит два нуля в конце вашего числа, как показано на рис. 4.3. Фактически это перемещение десятичного знака на две позиции вправо.



Рис. 4.3. Сдвиг десятичной запятой вправо с положительным показателем степени

Так как мы используем только десятичные числа с показателем степени 2, за которым следуют 3 десятичные цифры, вместо 2345 мы получим число 34 500 или 345×10^2 , как показано на рис. 4.4.

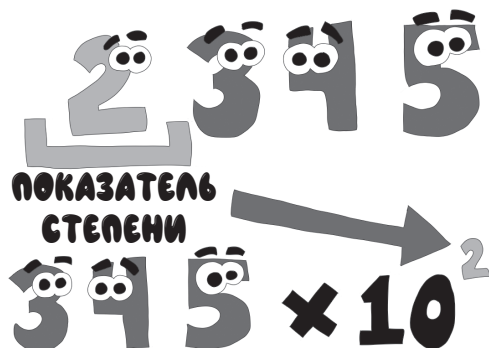


Рис. 4.4. Использование десятичного показателя степени

Чтобы получить дробное число, нам нужен отрицательный, а не положительный показатель, как показано на рис. 4.5: отрицательный показатель степени перемещает десятичную запятую влево.

СДВИГ ДЕСЯТИЧНОЙ ЗАПЯТОЙ ВЛЕВО

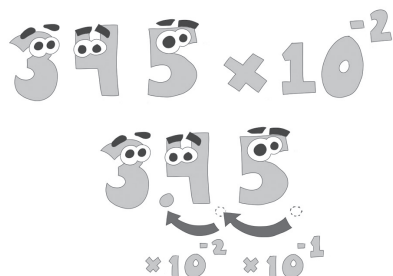


Рис 4.5. Сдвиг десятичной запятой влево с отрицательным показателем степени

Показатель степени отрицательной двойки превращает число 345 в 3,45, в результате чего получается дробное значение. Обратите внимание, что у этой системы есть проблема: до сих пор у нас нет метода для представления отрицательного первого бита показателя степени. Как уже было сказано, мы могли бы использовать запись 2345 для представления числа 345×10^2 без знака минус. Мы можем выбрать один из двух подходов: дополнительный код или смещенный показатель (смещенный порядок числа). В первом случае можно договориться, что старшие цифры обозначают отрицательные числа: например, представление 8345 может обозначать 345×10^{-2} , потому что $10 - 2 = 8$. Но разработчики чисел с плавающей запятой выбрали не

этот метод. Вместо этого они используют метод *смещенного порядка числа*, в котором, чтобы получить отрицательный показатель, просто вычитается конкретное выбранное значение из показателя степени. Например, если всегда вычитать 5 из показателя, то представление 3345 будет означать 345×10^{-2} , потому что $3 - 5 = -2$.

Настоящие числа с плавающей запятой являются двоичными, а не десятичными, но основные принципы те же. В двоичных числах с плавающей запятой старшим битом является знаковый бит, который равен 0, если число положительное, и 1, если оно отрицательное. Восемь бит, следующих за знаковым битом, представляют показатель степени. Мантисса (числовая часть) представляет собой дробное число от одного до двух. Крайний левый бит представляет значение 0,5, за которым следуют 0,25, 0,125, 0,0625 и т. д., то есть каждый раз уменьшаясь вдвое. Поскольку минимальное значение мантиссы равно 1, фактическое значение мантиссы всегда на единицу больше суммы всех этих дробных битов. Расположение этих битов показано на рис. 4.6.

Примечание В показателе степени числа с плавающей запятой, в отличие от целых чисел, не используется дополнительный код. Вместо этого из значения без знака, размещенного в этих восьми битах, вычитают 127, чтобы получить вещественный показатель, известный также как смещенная экспонента. Этот способ также известен как смещенный порядок.

32-БИТНОЕ С ПЛАВАЮЩЕЙ ЗАПЯТОЙ

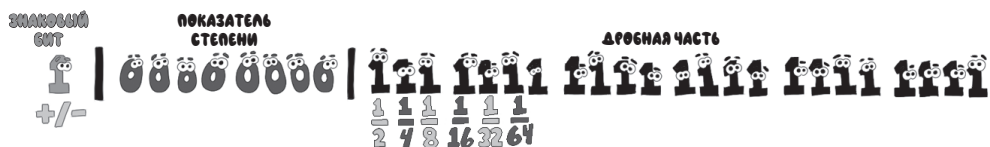


Рис. 4.6. 32-битные числа с плавающей запятой

Тот факт, что мантисса имеет минимальное значение 1, создает проблему для числа 0. Повышение ненулевого значения до любой степени никогда не сможет привести к значению 0. Чтобы компенсировать это, показатель степени имеет два специальных значения, которые позволяют представлять 0 и бесконечные значения в числе с плавающей запятой. Если все биты в показателе и мантиссе равны 0, то представленное число равно 0, даже если 0^0 равно 1. Интересно, что числа с плавающей запятой могут представлять 0 и -0 в зависимости от значения в знаковом бите. Infinity и $-Infinity$ – это значения, в которых все биты показателя являются единицами, а все биты мантиссы являются нулями. Если биты мантиссы не нулевые, то значение с плавающей запятой – это NaN (not a number – не число).

Примечание Если вы хотите увидеть, как числа с плавающей запятой выглядят в двоичном формате, у меня есть простое веб-приложение, которое позволяет вводить двоичные значения в знаковые разряды, мантиссы и показатели степени, чтобы увидеть полученное значение с плавающей запятой. Вы также можете ввести значение с плавающей запятой, чтобы увидеть соответствующие биты двоичного представления. Найти приложение можно по адресу https://wasmbok.com/binary_float.html.

Субнормальные числа

Субнормальные числа (иногда их называют денормализованными числами – *denormalized numbers*) – еще один пограничный случай с плавающей запятой в спецификации IEEE-754 для чисел с плавающей запятой. Субнормальные числа – это редкий случай, когда все биты показателя степени равны 0. В ситуации, когда все биты показателя равны 0, в мантиссе больше не прибавляют 1 к представленному значению. Благодаря этому можно представить еще меньшие значения с десятичной запятой. В этой книге мы не будем использовать субнормальные числа, но вы должны хотя бы знать об их существовании.

Формат с плавающей запятой f64

Тип данных f64 – это 64-битное число с плавающей запятой двойной точности, имеющее 52 бита, которые представляют значащие цифры, 11-битный показатель и знаковый бит. Такой тип данных обеспечивает высокую точность, но почти на каждом вычислительном оборудовании операции над этим типом данных выполняются медленнее, чем над целыми числами или над меньшими числами с плавающей запятой. Одним из его преимуществ является то, что такой же тип данных использует JavaScript, что делает его удобным для представления чисел, которые необходимо перемещать между JavaScript и WebAssembly.

Формат с плавающей запятой f32

Тип данных f32 меньше и обрабатывается быстрее, чем f64, но имеет гораздо меньшую точность, из-за чего будет доступно меньше значащих цифр для работы. f64 имеет примерно 16 десятичных значащих цифр, тогда как f32 – только около семи. Двоичные значащие цифры не совсем соответствуют десятичным цифрам. Количество значащих десятичных цифр является приблизительным, но дает представление об ограничениях каждого типа.

Биты старшего и младшего разрядов

В этом разделе мы рассмотрим значимость битов. Младший бит – это младший значащий бит двоичного числа. На рис. 4.7 в числе 128 цифра 1 соответствует наиболее значащему (старшему) разряду, а цифра 8 – наименее значащему (младшему).

САМЫЙ СТАРШИЙ РАЗРЯД



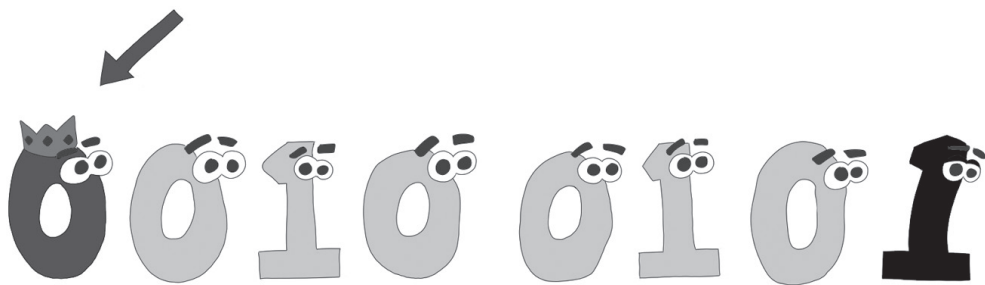
САМЫЙ МЛАДШИЙ РАЗРЯД

Рис. 4.7. Наиболее и наименее значащие разряды

Разряд, представляющий наибольшее числовое значение, является наиболее значащим. На рис. 4.7 разряд, в котором расположена цифра 1, – самый значащий, потому что он обозначает сотни, и цифр, расположенных на более высокой позиции, в этом числе нет. Грустная восьмерка – это наименее значащая цифра, потому что она представляет собой разряд единиц.

Двоичные числа также имеют старшие и младшие разряды. На компьютере один байт – это всегда восемь двоичных разрядов; даже если это число 00000001, оставшиеся семь двоичных разрядов все еще остаются в байте. Например, число 37 – это 100101 в двоичном формате, но в памяти компьютера значение байта равно 00100101. Старшим разрядом в этом байте является крайний левый 0, как показано на рис. 4.8.

БИТ СТАРШЕГО РАЗРЯДА



БИТ МЛАДШЕГО РАЗРЯДА

Рис. 4.8. Биты старшего и младшего разрядов

Бит младшего разряда в этом байте (1) – крайний справа, а бит старшего разряда (0) – крайний слева.

Глядя только на биты старшего и младшего разрядов переменной, можно получить некоторую информацию о байте. Младший бит целого числа указывает, является это число четным (0) или нечетным (1). Старший бит целого числа со знаком определяет, является ли это число положительным (0) или отрицательным (1).

Битовые операции

В WebAssembly операции с данными выполняются на низком уровне абстракции. Битовые операции позволяют повысить быстродействие кода. Поэтому понимание того, как работают эти операции, будет полезным даже для программирования на языках высокого уровня. Для написания приложений мы будем использовать большинство операций из этой главы, поэтому вам придется возвращаться к ней по мере необходимости. Эти операции универсальны, поэтому трудно привести однозначный пример того, когда нужно их использовать. Однако по мере возникновения ситуаций будет очевидно, какая операция для нее подходит.

Сдвиг и вращение битов

В этом разделе мы рассмотрим линейный сдвиг и циклический сдвиг битов. Это основные битовые операции, которые мы будем время от времени использовать в этой книге. Один байт данных состоит из восьми бит и может представлять число от 0 до 255 в десятичной системе счисления, что совпадает с числом от 0 до FF в шестнадцатеричной системе. Знаете ли вы, что четыре бита называются *полубайтом*? Одиночные шестнадцатеричные цифры состоят из одного полубайта (половина байта) и могут иметь значение от 0 до 15 в десятичном формате и от 0 до F в шестнадцатеричном.

Тот факт, что четыре бита могут хранить одну шестнадцатеричную цифру, позволяет относительно легко работать с шестнадцатеричными числами в WAT, особенно когда дело доходит до битового *сдвига*. Сдвиг – это наиболее распространенная операция, которая является фундаментом для оптимизации кода, о чем мы поговорим в следующих главах. Сдвиг напоминает столкновение битов с обрыва с подстановкой на освободившиеся места нулей (или единиц в некоторых случаях сдвигов вправо со знаком). Двоичные значения можно сдвигать на любое количество битов влево или вправо. Например, двоичное число 1110 1001 – это E9 в шестнадцатеричном формате (и 233 в десятичном). Если мы сдвинем это число на 4 бита вправо с помощью выражения (`i32.shr_u`), оно вернет 0000 1110 в двоичном формате, или 0E в шестнадцатеричном. На рис. 4.9 представлена драматичная иллюстрация четырехбитового сдвига числа E9.

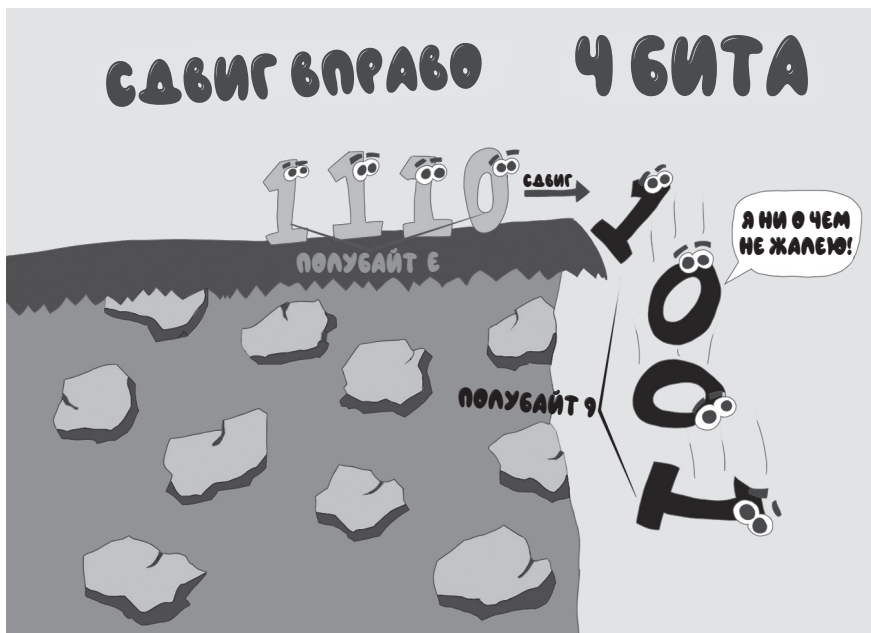


Рис. 4.9. В результате сдвига шестнадцатеричного E9 на четыре бита вправо получается 0E

Сдвиг данных вправо может быть полезным трюком. Каждый сдвиг на один бит вправо является функциональным эквивалентом деления на 2. Точно так же каждый сдвиг на один бит влево является функциональным эквивалентом умножения на 2. Вы можете использовать сдвиг в сочетании с маскированием, чтобы выделить нужные разряды двоичных данных.

В WebAssembly сдвиг влево не зависит от знака, однако для сдвига вправо существуют операции сдвига со знаком или без знака. Когда вы сдвигаете вправо или влево двоичное число, освободившиеся разряды на противоположной стороне целого числа обычно заполняются нулями. Однако в случае отрицательного числа есть одна тонкость. Если вы сдвинете знак вправо, освободившийся разряд с левой стороны придется заполнить единицей, чтобы сохранить знак целого числа. Сдвиг числа, представленного в дополнительном коде, сохраняет его знак (отрицательность).

Циклический сдвиг (вращение битов), в отличие от упомянутого ранее *линейного сдвига*, перебрасывает вытесняемые биты обратно на другую сторону числа. При циклическом сдвиге вправо младший бит переходит в начало числа и становится старшим битом, а все остальные биты сдвигаются вправо. WAT выполняет циклический сдвиг с помощью команд `rotl` (rotate to left, повернуть влево) или `rotr` (rotate to right, повернуть вправо), как показано на рис. 4.10.

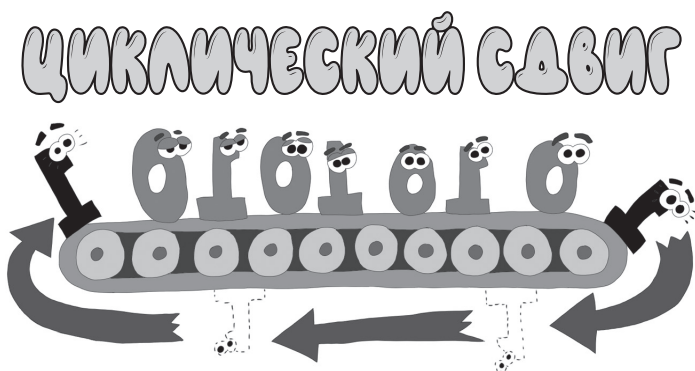


Рис. 4.10. Вращение бита вправо

Маскирование битов с помощью AND и OR

Побитовое маскирование – это метод, используемый для принудительной установки некоторых указанных битов целого числа в 1 или 0, в зависимости от типа маски, которую мы используем, чтобы выделить или переопределить их. Этот метод может быть очень полезен при написании высокопроизводительных приложений. Также важно знать, что в WebAssembly нет концепции логических значений (boolean values). Сравнения в WebAssembly обычно возвращают значение 1 для истинного результата и 0 для ложного. Для операций двоичной логики нам понадобятся выражения `i32.and` и `i32.or`. В этом разделе мы будем использовать `i32.and` и `i32.or`, чтобы выделить или переопределить битовые значения с помощью побитового маскирования.

Когда вы маскируете биты с помощью побитовой логической операции И, биты 0 в вашем целом числе побеждают в «битве битов». В маске они действуют как липкая лента для маскирования, которая перекрывает любые другие биты целого числа, заменяя их нулями. Биты 1 в маске позволяют сохранить исходное значение разряда числа. На рис. 4.11 показано, как выглядит маскирование двоичного кода 1011 1110 (190) с помощью маски 0000 0111 (7).

Как вы можете видеть на рис. 4.11, все единичные биты нашего исходного значения перекрыты соответствующими нулями в маске И. Когда побитовая операция И выполняется с двумя разными целыми числами, выигрывает бит 0 (рис. 4.12).

Маскирование с помощью ИЛИ (выражение `i32/i64.or`) даст противоположные результаты: биты 1 перекрывают любые биты. Для выделения битов вы не можете использовать маску ИЛИ, как это было с И. Вместо этого ее используют для установки определенных битов в 1. На рис. 4.13 показано маскирование с помощью ИЛИ.

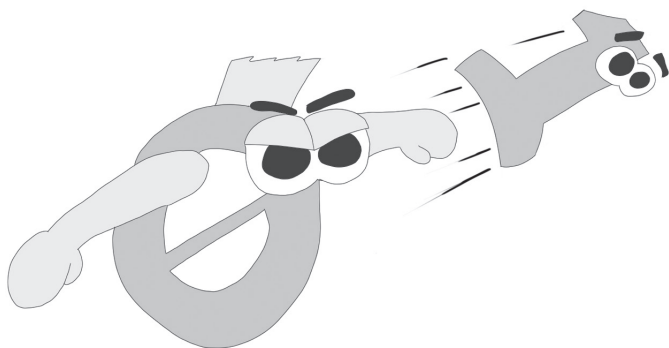
Как вы можете видеть на рис. 4.13, когда вы используете побитовое ИЛИ, биты 1 в вашей маске перекрывают любые соответствующие биты в вашем начальном целочисленном значении. Таким образом, в случае использования побитового ИЛИ бит 1 побеждает бит 0 (рис. 4.14).

МАСКА И



Рис. 4.11. Маскирование с помощью побитового И

ПОБИТОВОЕ И



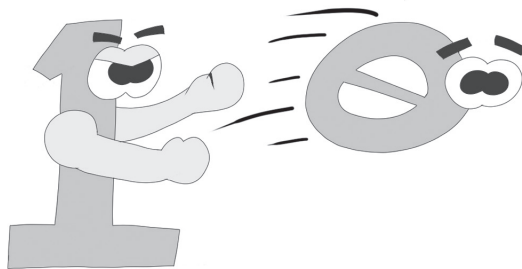
0 ПОБЕЖДАЕТ!

Рис. 4.12. Побитовое И: 0 побеждает в «битве битов»



Рис. 4.13. Маскирование с помощью побитового ИЛИ

ПОБИТОВОЕ ИЛИ



1 ПОБЕЖДАЕТ!

Рис. 4.14. В побитовом ИЛИ всегда побеждает бит 1

Инверсия битов с помощью XOR

Последняя бинарная операция, которую мы рассмотрим, – это XOR (ИСКЛЮЧАЮЩЕЕ ИЛИ). Операция XOR `i32/i64.xor` немного отличается от И и ИЛИ. Если 0 маскирует 0, 1 маскирует 0 или 0 маскирует 1, XOR работает так же, как типичный `i32/i64.or`. Вас может удивить то, что при наличии двух единиц XOR устанавливает полученный бит в 0. Эта функция удобна для *инвертирования битов* (замена единиц нулями, а нулей единицами), когда вы инвертируете каждый бит в его противоположность. Некоторые операции требуют замены каждого бита 1 целого числа на 0 и каждый бит 0 на 1. На рис. 4.15 показано, как инвертировать каждый бит в полубайте с помощью XOR.



Рис. 4.15. Инвертирование битов с помощью XOR

Во время обсуждения дополнительного кода мы отметили, что если вы хотите найти отрицательное значение любого числа, то можете сделать это, инвертировав каждый бит и прибавив число 1. Вы можете инвертировать все биты, выполнив операцию XOR между исходным числом и целым числом, у которого каждый бит установлен в 1. Давайте напишем короткое приложение, которое преобразует положительное целое число в отрицательное с помощью дополнительного кода и инвертирования битов. Создайте файл с именем `twos_complement.wat` и вставьте в него код из листинга 4.1.


```

twos_ (module
complement.wat  (func $twos_complement (export "twos_complement")
                  (param $number i32)
                  (result i32)
                    local.get $number
                    ❶ i32.const 0xffffffff ;; все единицы, чтобы инвертировать биты
                    ❷ i32.xor              ;; инвертировать все биты
                    i32.const 1
                    ❸ i32.add              ;; добавить единицу после инвертирования всех битов,
                                           ;; чтобы получить дополнительный код
                  )
        )

```

Это очень простой модуль, который принимает параметр `i32` с именем `$number`. Мы помещаем `i32` в стек, а следом за ним – 32-битное число, где все биты равны 1 ❶. После вызова `i32.xor` ❷ все биты исходного числа инвертируются. Каждый ноль превращается в единицу, а каждая единица в ноль. Затем мы вызываем `i32.add` ❸, чтобы прибавить к этому числу значение 1 и получить дополнительный код, что дает нам отрицательное число. Этот код хорошо работает как демонстрация вычисления дополнительного кода; однако было бы лучше, если бы мы просто вычли `$number` из 0, чтобы инвертировать его.

Обратный vs. прямой порядок байтов

Все числа, с которыми вы знакомы, представлены в формате обратного порядка разрядов (*big-endian*), то есть цифры самого старшего разряда находятся слева, а цифры самого младшего разряда – справа. В большинстве компьютерных устройств используется формат прямого порядка байтов (*little-endian*), где младший разряд находится слева, а старший разряд – справа, поэтому, например, 128 будет записано как 821. Имейте в виду, что порядок следования байтов связан с порядком именно байтов, а не разрядов, поэтому мой пример с дробным порядком байтов 821 является чрезмерным упрощением, которое нельзя напрямую преобразовать в двоичный формат. Вычислительные устройства с прямым порядком байтов, как правило, упорядочивают байты в порядке, обратном привычному нам. Число 168 496 141 в шестнадцатеричном формате с обратным порядком байтов (*big-endian*) – 0A0B0C0D. Старший байт – это 0A, а младший байт – 0D, потому что каждая шестнадцатеричная цифра представлена полубайтом, или половиной байта. Если мы разместим байты в прямом порядке (*little-endian*), они будут расположены как 0D0C0B0A, как показано на рис. 4.16.

Сегодня, из соображений производительности, большинство вычислительного оборудования располагает байты в прямом порядке. При этом WebAssembly использует прямой порядок байтов *независимо от аппаратного обеспечения*. Когда вы инициализируете данные

с помощью оператора (data) в WebAssembly, важно помнить о порядке байтов.

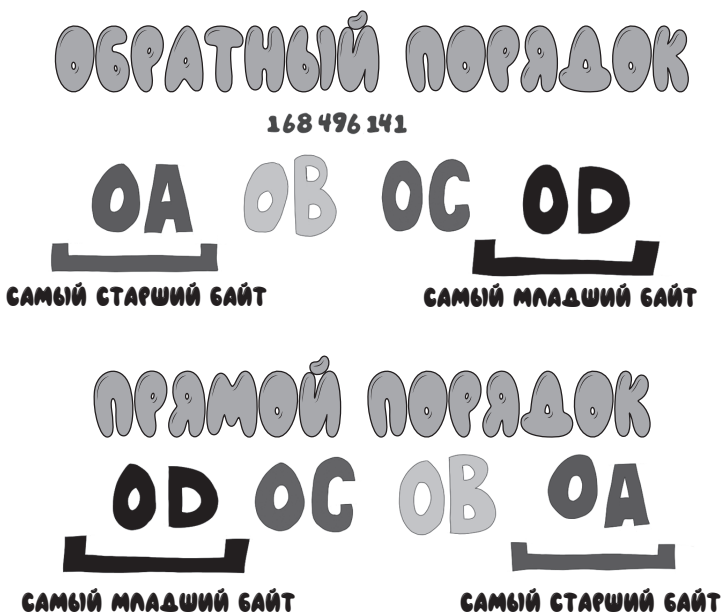


Рис. 4.16. Обратный и прямой порядок байтов

Заключение

В этой главе мы рассмотрели множество понятий низкоуровневого программирования. Мы разобрали различные системы счисления (десятичные, шестнадцатеричные и двоичные), используемые в низкоуровневом программировании. Изучили особенности арифметики целых чисел и чисел с плавающей запятой, бегло познакомились с дополнительным кодом, а также прямым и обратным порядком байтов. Мы говорили о битовых операциях, включая биты старшего и младшего разрядов, побитовую маскировку, а также линейный и циклический битовый сдвиг. Эти приемы низкоуровневого программирования, повышающие быстродействие за счет битовых операций, пригодятся нам в последующих главах.

В следующей главе вы познакомитесь с несколькими методами управления строками как структурами данных, включая строки с нулевым байтом в конце и строки с префиксом длины. Мы также рассмотрим копирование строк и преобразование числовых данных в строки в десятичном, шестнадцатеричном и двоичном форматах.

5

СТРОКИ В WEBASSEMBLY



Так как WebAssembly не имеет встроенного строкового типа данных, как языки высокого уровня, в этой главе мы обсудим, как обрабатывать строки в WAT. Для того чтобы представить строковые данные в WebAssembly, нужно разместить в линейной памяти значения символов ASCII или Unicode. Необходимо знать, где именно вы устанавливаете данные в линейной памяти и сколько байтов будет занимать строка.

Прежде чем изучать взаимосвязь между строковыми объектами и способы их сохранения в линейной памяти, мы рассмотрим символьные форматы ASCII и Unicode. Вы узнаете, как JavaScript извлекает строки из линейной памяти и выводит их в командную строку. После того как вы научитесь компилировать строковые данные из WebAssembly в JavaScript, мы рассмотрим плюсы и минусы двух популярных методов управления строками: строка с завершающим нулевым байтом и строки с префиксом длины. Вы узнаете, как скопировать строку из одного места в линейной памяти и перенести ее в другое, используя побайтовую копию и 64-битную копию. Затем вы преобразуете целочисленные данные в числовые строки в десятичном, шестнадцатеричном и двоичном форматах.

Примечание По большей части для хранения строк вам не нужно выбирать конкретный адрес в линейной памяти, если он находится в пределах количества выделенных вами страниц. Если не сказано иначе, мы будем предполагать, что адрес памяти был выбран произвольно.

ASCII и Unicode

При работе со строками в WebAssembly вам необходимо знать, какой набор символов вы используете, потому что разные наборы выглядят по-разному в линейной памяти. *Стандартный американский код обмена информацией (ASCII – American Standard Code for Information Interchange)* – это 7-битная система кодирования символов, которая поддерживает до 128 символов, где 8-й бит может использоваться для коррекции ошибок или быть равным 0. Если ваш код поддерживает только английский язык, набор символов ASCII подойдет отлично.

Формат *Unicode (UTF – Unicode Transformation Format)* имеет 7-битную, 8-битную, 16-битную и 32-битную версии, которые называются UTF-7, UTF-8, UTF-16 и UTF-32. UTF-7 и ASCII идентичны. UTF-8 включает UTF-7 и позволяет использовать некоторые дополнительные символы латиницы, а также символы средневековой и азиатской письменности, образуя формат переменной длины, который позволяет использовать дополнительные байты, когда начальный байт формата выходит за пределы набора символов ASCII. UTF-16 также представляет собой набор символов переменной длины, где большинство символов представлены двумя байтами. Поскольку символы некоторых кодов имеют больший объем (до четырех байт), UTF-16 поддерживает более 1,1 млн символов. UTF-32 – это фиксированный 32-битный набор символов, который поддерживает более 4 млрд символов. В этой книге мы будем работать исключительно с набором символов ASCII/UTF-7, потому что его легко читать и понимать.

Строки в линейной памяти

Единственный способ передать строку из WebAssembly в JavaScript – создать массив символьных данных внутри объекта памяти `buffer`, как мы сделали в главе 2 с приложением «hello world». Затем вы можете передать в JavaScript 32-битное целое число, которое представляет расположение этих символьных данных в буфере памяти. Единственная проблема этой схемы заключается в том, что она не сообщает JavaScript, где заканчиваются данные. Язык C решает эту проблему с помощью строки, завершающейся нулевым байтом: байт со значением 0 (не символом 0) сообщает программе, что строка заканчивается предыдущим байтом. Мы рассмотрим три способа передачи строк между WAT и JavaScript, включая завершение нулевым байтом, а затем посмотрим, как копировать строки.

Передача длины строки в JavaScript

Наиболее очевидный способ работы со строками – передать позицию строки и длину строки в JavaScript, чтобы JavaScript мог извлечь строку из линейной памяти и узнать, когда она заканчивается. Создайте новый WAT-файл с именем *strings.wat* и поместите код из листинга 5.1.

Листинг 5.1. Передача строк из WebAssembly в JavaScript

```
strings.wat (часть 1 из 11) (module
;; Импортированная функция JavaScript принимает позицию и длину
❶(import "env" "str_pos_len" (func $str_pos_len (param i32 i32)))
❷(import "env" "buffer" (memory 1))
;; Строка из 30 символов
❸(data (i32.const 256) "Know the length of this string")
;; 35 символов
❹(data (i32.const 384) "Also know the length of this string")

❺(func (export "main")
;; длина первой строки – 30 символов
❻(call $str_pos_len (i32.const 256) (i32.const 30))
;; длина второй строки – 35 символов
❼(call $str_pos_len (i32.const 384) (i32.const 35))
)
)
```

Этот модуль импортирует нашу функцию JavaScript под названием "str_pos_len" ❶, которая находит строку в буфере памяти, используя комбинацию положения строки и ее расположения в линейной памяти. Нам также необходимо импортировать буфер памяти, который мы объявим в JavaScript ❷.

Далее мы определяем в памяти две строки: "Know the length of this string" ❸ и "Also know the length of this string" ❹. Очевидно, что нам нужно знать длину этих строк, потому что они – всего лишь массивы символов в линейной памяти, а также нужно указать, где они начинаются и заканчиваются. Первая строка состоит из 30 символов, а вторая – из 35. Затем в функции "main" ❺ мы дважды вызываем \$str_pos_len. В первую очередь ❻, мы передаем позицию первой строки в памяти (i32.const 256), за которой следует длина этой строки (i32.const 30). Тем самым мы сообщаем JavaScript, что сейчас извлечем 30 байт в строку, начиная с позиции 256 в памяти, и отобразим ее на консоли. Во вторую очередь, когда мы вызываем \$str_pos_len ❼, мы передаем позицию второй строки в памяти (i32.const 384), за которой следует длина этой строки (i32.const 35). Затем JavaScript отображает вторую строку на консоли. Скомпилируйте модуль WebAssembly, используя команду из листинга 5.2.

Листинг 5.2. Компиляция strings.wat

```
wat2wasm strings.wat
```

После того как вы скомпилировали модуль WebAssembly, создайте файл JavaScript с именем *strings.js* и введите код, приведенный в листинге 5.3.

Листинг 5.3. Код JavaScript, который вызывает строковую функцию WebAssembly

```
strings.js (часть 1 из 3) const fs = require('fs');
const bytes = fs.readFileSync(__dirname + '/strings.wasm');

let memory = new WebAssembly.Memory( {initial: 1 } );
let importObject = {
  env: {
    buffer: memory,
    ❶ str_pos_len: function(str_pos, str_len) {
      ❷ const bytes = new Uint8Array(memory.buffer, str_pos, str_len);
      ❸ const log_string = new TextDecoder('utf8').decode(bytes);
      ❹ console.log(log_string);
    }
  }
};

( async () => {
  let obj = await WebAssembly.instantiate( new Uint8Array(bytes),
                                          importObject );

  let main = obj.instance.exports.main;

  main();
})();
```

Внутри `importObject` мы определяем `str_pos_len` ❶, который принимает позицию строки в памяти и ее длину. Он использует позицию длины для получения массива байтов ❷ предоставленной длины. Для преобразования этого байтового массива в строку мы используем `TextDecoder` ❸. Затем вызываем `console.log` ❹ для отображения строки. Когда вы запустите JavaScript, вы должны увидеть сообщение в листинге 5.4.

Листинг 5.4. Вывод длины строки

```
Know the length of this string
Also know the length of this string
```

Далее мы обсудим, что строки, завершающиеся нулем, представляют собой метод отслеживания длины строки, который используют такие языки, как C/C++.

Строки с завершающим нулем

Второй метод передачи строк – это *строки с завершающим нулем* (*null-terminated* или *zero-terminated string*). Завершение нулем – это

метод определения длины строки, используемый в языке программирования C. В качестве последнего символа в массиве строки с завершающим нулем вы помещаете значение 0. Преимущество строк с завершающим нулем состоит в том, что вам не нужно знать длину строки во время ее использования. Обратной стороной является то, что при обработке строк требуется больше вычислений, потому что вашей программе нужно время, чтобы найти завершающий нулевой байт. Откройте файл *strings.wat* и добавьте код из листинга 5.5 для строк с нулевым завершением. Для начала вам нужно добавить импорт функции `null_str`, которую мы определим позже в JavaScript.

Листинг 5.5. Продолжение strings.wat из листинга 5.1, импортирование функции null_str

```
strings.wat (module
(часть 2 из 11) ❶(import "env" "str_pos_len" (func $str_pos_len (param i32 i32)))
;; добавьте строку ниже
❷(import "env" "null_str" (func $null_str (param i32)))
...

```

Обратите внимание, что, в отличие от `str_pos_len` ❶, функции `null_str` ❷ требуется только один параметр `i32`, потому что код, работающий с ним, должен знать, только где начинается строка в линейной памяти. Этот код должен сам найти, где расположен нулевой байт.

Затем между оператором `import`, определяющим буфер, и выражением (`data`), определяющим предыдущие строки, в листинге 5.6 добавим еще два выражения данных, которые определяют строки с завершающим нулем.

Листинг 5.6. Продолжение strings.wat из листинга 5.1, добавление строковых данных с нулевым символом в конце

```
strings.wat ...
(часть 3 из 11) (import "env" "buffer" (memory 1))
;; добавьте две строки ниже
❶(data (i32.const 0) "null-terminating string\00")
❷(data (i32.const 128) "another null-terminating string\00")

(data (i32.const 256) "Know the length of this string")
...

```

Первые данные определяют строку `"null-terminating string\00"` ❶. Обратите внимание на последние три символа `\00`. Символ `\` — это escape-символ в WAT. Если вы помещаете за escape-символом две шестнадцатеричные цифры, он определяет числовой байт с указанным вами значением. Это означает, что `\00` представляет один байт со значением 0. Второе выражение данных создает строку `"another null-terminating string\00"` ❷, которая также завершается нулевым символом в конце, заканчиваясь на `\00`.

В листинге 5.7 мы добавляем две строки в начало функции `main` для вызова импортированной функции JavaScript `$null_str`, передавая ей конкретное местоположение строк с завершающим нулем в линейной памяти.

Листинг 5.7. Продолжение strings.wat из листинга 5.1, вызов функции `null_str`

```
strings.wat
(часть 4 из 11) ...
(func (export "main")
  ❶ (call $null_str (i32.const 0))    ;; Добавьте эту строку
  ❷ (call $null_str (i32.const 128)) ;; Добавьте эту строку

  (call $str_pos_len (i32.const 256) (i32.const 30))
  ...
)
```

Мы передаем значение `0` ❶, то есть то место в памяти, где мы определили строку `"null-terminating string\00"`. Затем передаем значение `128` ❷, где определили строку `"another null-terminating string\00"`.

После внесения этих изменений в файл WAT откройте `strings.js`, чтобы ввести код. Сначала добавьте новую функцию к объекту `env`, вложенному в `importObject`, как показано в листинге 5.8.

Листинг 5.8. Продолжение листинга 5.3, функция `null_str`, добавленная к `importObject` в `strings.js`

```
strings.js
(часть 2 из 3) ...
❶ const max_mem = 65535; // add this line

let importObject = {
  env: {
    buffer: memory,
    // добавьте функцию null_str в importObject здесь
    ❷ null_str: function(str_pos) { // добавьте эту функцию
      ❸ let bytes = new Uint8Array( memory.buffer, str_pos,
                                   max_mem - str_pos );

      ❹ let log_string = new TextDecoder('utf8').decode(bytes);
      ❺ log_string = log_string.split("\0")[0];
      ❻ console.log(log_string);
    }, // конец функции
    str_pos_len: function(str_pos, str_len) {
      ...
    }
  }
}
```

Код начинается с определения максимально возможной длины строки в переменной `max_mem` ❶. Чтобы найти строку, заканчивающуюся нулем, мы декодируем фрагмент линейной памяти с максимальной длиной строки в одну длинную строку, а затем используем функцию `split` JavaScript, чтобы получить строку с завершающим нулем. Внутри объекта `env` добавляем еще одну функцию с именем `null_str`

❷, которая принимает единственный параметр `str_pos`. Затем JavaScript должен извлечь массив байтов из буфера памяти, начиная с позиции, указанной параметром `str_pos`, переданным в функцию. Мы не сможем выполнить поиск в буфере памяти, пока не преобразуем его в строку. Но перед этим нам нужно преобразовать его в массив байтов ❸. Затем мы создаем объект `TextDecoder` для декодирования этих байтов в одну длинную строку ❹.

Используя нулевой байт в качестве разделителя `"\0"` ❺, мы разбиваем строку на массив. Это разделение создает массив строк, которые заканчиваются нулевым байтом. Фактически только первый элемент в массиве является строкой, которую мы определили. Мы используем разделение, так как это быстрый, но «грязный» способ извлечь строку из линейной памяти. Затем устанавливаем `log_string` на первую строку в массиве. Вызываем функцию JavaScript `console.log` ❻, передавая ей `log_string` для отображения этой строки в консоли. Поскольку мы вызываем его с двумя разными строками из WebAssembly, теперь у нас должны быть четыре сообщения из листинга 5.9, записанные в консоль.

Листинг 5.9. Вывод строк с завершающим нулем

```
null-terminating string
another null-terminating string
Know the length of this string
Also know the length of this string
```

Строки с префиксом длины

Третий способ записать строку в памяти – вписать длину строки в начало строковых данных. Строка, созданная с помощью этого метода, называется *строкой с префиксом длины* (*length-prefixed string*) и повышает производительность обработки. Используемый нами префикс ограничивает длину строк до 255, поскольку один байт данных может содержать только число от 0 до 255.

16-битная строка с префиксом длины

Вы можете закодировать длину строки в двух байтах данных, что позволит строкам использовать более 65 000 символов. Имейте в виду, что располагать байты необходимо в прямом порядке. Чтобы использовать 16-битное число для кодирования длины, нужно поместить младший байт перед старшим. Например, если вы хотите, чтобы ваша строка имела длину 1024 (0400 в шестнадцатеричном формате), вам нужно будет закодировать ее в прямом порядке. Так как в строке прямой порядок байтов будет перевернут, получится `\00\04`. Усложняет ситуацию то, что JavaScript не всегда применяет прямой порядок байтов. Поэтому вам нужно проверить, какой порядок байтов используется.

Давайте дополним файл *strings.wat*, добавив новую строку импорта для функции *len_prefix*, которую мы определим позже в JavaScript:

Листинг 5.10. Продолжение strings.wat из листинга 5.1, импорт функции len_prefix

```
strings.wat (module
(часть 5 из 11)  (import "env" "str_pos_len" (func $str_pos_len (param i32 i32)))
                 (import "env" "null_str" (func $null_str (param i32)))
                 ;; добавьте следующую строку
                 ❶(import "env" "len_prefix" (func $len_prefix (param i32)))
                 ...

```

Функция *len_prefix* ❶ будет определять длину первого байта строки.

Затем мы добавляем две новые строки, которые начинаются с шестнадцатеричного числа, указывающего их длину. Добавьте код из листинга 5.11 в *strings.wat*.

Листинг 5.11. Продолжение strings.wat из листинга 5.1, добавление строковых данных с префиксом длины

```
strings.wat ...
(часть 6 из 11)  (data (i32.const 384) "Also know the length of this string")

                 ;; добавьте следующие четыре строки. Два элемента данных и два комментария
                 ;; в десятичной системе счисления – длина 22, в шестнадцатеричной – 16
                 ❶(data (i32.const 512) "\16length-prefixed string")
                 ;; в десятичной системе – длина 30, что равно 1e в шестнадцатеричной
                 ❷(data (i32.const 640) "\1eanother length-prefixed string")
                 (func (export "main")
                 ...

```

Первая строка, *"\16length-prefixed string"*, состоит из 22 символов, значит, мы добавляем к ней префикс *\16*, потому что 22 в десятичной системе счисления равняется 16 в шестнадцатеричной ❶. Вторая строка, *"\1eanother length-prefixed string"*, имеет длину 30 символов, поэтому мы добавляем к ней шестнадцатеричный префикс *\1e* ❷.

Затем нам нужно добавить два вызова к импортированной функции *\$len_prefix* с двумя ячейками памяти, в которых мы только что создали строки. Теперь функция *"main"* должна выглядеть как код в листинге 5.12.

Листинг 5.12. Продолжение strings.wat из листинга 5.1, добавление вызовов функции \$len_prefix

```
strings.wat ...
(часть 7 из 11)  (func (export "main")
                 (call $null_str (i32.const 0))
                 (call $null_str (i32.const 128))

```

```

    (call $str_pos_len (i32.const 256) (i32.const 30))
    (call $str_pos_len (i32.const 384) (i32.const 35))

    ❶(call $len_prefix (i32.const 512))    ;; добавьте эту строку
    ❷(call $len_prefix (i32.const 640))    ;; добавьте эту строку

  )
...

```

Первый вызов `$len_prefix` ❶ передает ему местоположение строки данных `"\16length-prefixed string"` в ячейке памяти 512. Второй вызов ❷ передает местоположение второй строки с префиксом длины `"\1eanother lengthprefixed string"` в ячейке памяти 640.

Прежде чем запустить этот код, нам нужно добавить новую функцию в наш JavaScript `importObject`. Откройте `strings.js` и добавьте функцию `len_prefix` к `importObject`, как показано в листинге 5.13.

Листинг 5.13. Продолжение листинга 5.3, добавление функции `len_prefix` к `importObject` в `strings.js`

```

strings.js
(часть 3 из 3) ...
let importObject = {
  env: {
    buffer: memory,
    null_str: function (str_pos) {
      let bytes = new Uint8Array(memory.buffer, str_pos,
                                max_mem - str_pos);

      let log_string = new TextDecoder('utf8').decode(bytes);
      log_string = log_string.split("\0")[0];
      console.log(log_string);
    }, // end null_str function
    str_pos_len: function (str_pos, str_len) {
      const bytes = new Uint8Array(memory.buffer,
                                    str_pos, str_len);
      const log_string = new TextDecoder('utf8').decode(bytes);
      console.log(log_string);
    },
    ❶len_prefix: function (str_pos) {
      ❷const str_len = new Uint8Array(memory.buffer, str_pos, 1)[0];
      ❸const bytes = new Uint8Array(memory.buffer,
                                    str_pos + 1, str_len);
      ❹const log_string = new TextDecoder('utf8').decode(bytes);
      console.log(log_string);
    }
  }
};
...

```

Новая функция `len_prefix` ❶ переходит на позицию строки, а затем берет первый байт из позиции как число в константе `str_len` ❷. Зна-

чение в `str_len` используется для копирования нужного количества байтов ❸ из линейной памяти, чтобы была возможность декодировать в `log_string` ❹ то, что будет записано в консоль. Теперь, когда у нас есть блоки кода WAT и JavaScript, мы можем скомпилировать модуль WAT с помощью `wat2wasm`, как показано в листинге 5.14.

Листинг 5.14. Компиляция strings.wat

```
wat2wasm strings.wat
```

Далее мы можем запустить наш файл JavaScript с помощью `node`, как показано в листинге 5.15.

Листинг 5.15. Запуск strings.js с помощью node

```
node strings.js
```

Вы должны увидеть такой же вывод, как в листинге 5.16.

Листинг 5.16. Вывод из strings.js

```
null-terminating string
another null-terminating string
Know the length of this string
Also know the length of this string
length-prefixed string
another length-prefixed string
```

В следующем разделе вы узнаете, как копировать строки с помощью WAT.

Копирование строк

Самый простой способ скопировать строку из одного места в линейной памяти в другое – пройти в цикле по каждому байту данных, загрузить их, а затем сохранить в новом месте. Однако этот метод занимает больше времени. Более эффективный метод – копировать строки по восемь байт за раз, используя загрузку и хранение 64-битных целых чисел. Но, к сожалению, не все строки кратны восьми байтам. Эффективнее всего применять комбинацию методов побайтового и 64-битного копирования.

Побайтовое копирование

Сначала мы напишем функцию для более медленного побайтового копирования: она принимает в качестве параметров исходную ячейку памяти, целевую ячейку памяти и длину строки, которую мы хотим скопировать.

Примечание На некоторых платформах производительность такой функции может снизиться, если строки не имеют правильного выравнивания по границе байта. Некоторые аппаратные архитектуры требуют, чтобы вы загружали и сохраняли данные по адресам, кратным размеру шины. Например, иногда процессор с 32-битной шиной может читать только с адресов, кратных четырем (4 байта = 32 бита).

Давайте продолжим наш файл `strings.wat`. В листинге 5.17 мы добавляем функцию `$byte_copy` в файл `strings.wat`.

Листинг 5.17. Затратный по времени метод побайтового копирования строк, добавленных в `strings.wat` (листинг 5.1)

```
strings.wat ...
(часть 8 из 11) (func $byte_copy
  (param $source i32) (param $dest i32) (param $len i32)
  (local $last_source_byte i32)

  ❶ local.get $source
    local.get $len
  ❷ i32.add ;; $source + $len

  local.set $last_source_byte ;; $last_source_byte = $source + $len

  (loop $copy_loop (block $break
    ❸ local.get $dest ;; протолкнуть $dest в стек, чтобы использовать
      ;; для вызова i32.store8
    ❹ (i32.load8_u (local.get $source)) ;; загрузить один байт из $ source
    ❺ i32.store8 ;; записать один байт в $dest

    ❻ local.get $dest
      i32.const 1
      i32.add
    ❼ local.set $dest ;; $dest = $dest + 1

    local.get $source
      i32.const 1
      i32.add
    ❽ local.tee $source ;; $source = $source + 1

    local.get $last_source_byte
      i32.eq
      br_if $break
      br $copy_loop
  )) ;; конец $copy_loop
)
...
```

Функция `$byte_copy` копирует блок памяти из `$source` ❶ в `$source + $len` ❷ в ячейку памяти `$dest` ❸ в `$dest + len` по одному байту за раз. Этот цикл загружает байт из `$source` с помощью выражения `(i32.load8_u)` ❹. Затем он сохраняет этот байт в местоположении `$dest`

с помощью команды `i32.store8` ⑤. Далее увеличиваем целевое местоположение ⑥ в переменной `$dest` ⑦ и увеличиваем переменную `$source` ⑧, чтобы эти переменные указывали на следующие байты в памяти.

64-битное копирование

Побайтовое копирование строки происходит медленнее, чем нужно, тогда как 64-битное целое число имеет длину восемь байт, а копировать восемь байт за раз значительно быстрее, чем копировать по одному байту. Мы напишем другую функцию, похожую на `$byte_copy`, с помощью которой можно копировать данные быстрее – по восемь байт за раз. К сожалению, не все строки имеют длину, кратную восьми. Если длина строки составляет 43 байта, мы можем скопировать первые 40 байт, используя пять отдельных восьмибайтовых копий, но для последних трех байт нам нужно будет вернуться к методу побайтового копирования.

Важно отметить, что эти байтовые копии не препятствуют доступу к памяти за пределами ее границ. Код будет пытаться поместить данные туда, куда не должен помещать, или взять их оттуда, откуда не должен брать. Однако если вы попытаетесь получить доступ к данным за пределами линейной памяти, модуль безопасности `WebAssembly` вызовет сбой чтения или записи, остановив выполнение кода. Как было сказано ранее, эти функции предназначены не для обычного применения, а для демонстрации различных методов копирования строк.

Добавьте в ваш файл `strings.wat` 64-битную функцию копирования `$byte_copy_i64` из листинга 5.18.

Листинг 5.18. Более быстрый метод копирования строк, добавленных в `strings.wat` (листинг 5.2)

```
strings.wat ...
(часть 9 из 11) ;; добавьте этот блок кода в файл strings.wat
(func $byte_copy_i64
  (param $source i32) (param $dest i32) (param $len i32)
  (local $last_source_byte i32)

  local.get $source
  local.get $len
  i32.add

  local.set $last_source_byte

  (loop $copy_loop (block $break
    ① (i64.store (local.get $dest) (i64.load (local.get $source)))

    local.get $dest
    ② i32.const 8
    i32.add
    local.set $dest                ;; $dest = $dest + 8
```

```

    local.get $source
    ❸ i32.const 8
    i32.add
    local.tee $source          ;; $source = $source + 8

    local.get $last_source_byte
    i32.ge_u
    br_if $break
    br $copy_loop
  )) ;; конец $copy_loop
)
...

```

Функции загрузки (load) и хранения (store): (i64.load) и (i64.store) ❶, которые загружают и хранят 64 бита (8 байт) за раз. Этот метод работает в четыре-пять раз быстрее, чем загрузка и хранение одного байта за раз (в архитектуре x64). Другое существенное отличие состоит в том, что \$dest ❷ и \$source ❸ увеличиваются на 8 вместо 1.

Функция комбинированного копирования

Как упоминалось ранее, не все строки кратны восьми. Поэтому в листинге 5.19 мы определим новую улучшенную функцию, которая, в свою очередь, с помощью функции \$byte_copy_i64 копирует восемь байт за раз, а затем оставшиеся байты с помощью \$byte_copy (копирует по одному байту за раз).

Листинг 5.19. Если есть возможность, копируйте восемь байт за раз, а если нет, тогда по одному байту

```

strings.wat
...
(часть 10 из 11) (func $string_copy
  (param $source i32) (param $dest i32) (param $len i32)
  (local $start_source_byte i32)
  (local $start_dest_byte i32)
  (local $singles i32)
  (local $len_less_singles i32)

  local.get $len
  ❶ local.set $len_less_singles ;; значение без единиц

  local.get $len
  i32.const 7          ;; 7 = 0111 в двоичном формате
  ❷ i32.and
  local.tee $singles    ;; установить $singles на последние 3 бита длины

  ❸ if                  ;; если последние 3 бита $len не 000
    local.get $len
    local.get $singles
    i32.sub
    ❹ local.tee $len_less_singles ;; $len_less_singles = $len - $singles

    local.get $source

```

```

        i32.add
        ;; $start_source_byte=$source+$len_less_singles
        ❶ local.set $start_source_byte

        local.get $len_less_singles
        local.get $dest
        i32.add
        ❷ local.set $start_dest_byte ;; $start_dest_byte=$dest+$len_less_singles
        ❸ (call $byte_copy (local.get $start_source_byte)
            (local.get $start_dest_byte)(local.get $singles))
    end

    local.get $len
    ❹ i32.const 0xff_ff_ff_f8 ;; все биты равны 1, кроме последних трех (равны 0)
    ❺ i32.and ;; установить последние три бита длины на 0
    local.set $len
    ❻ (call $byte_copy_i64 (local.get $source) (local.get $dest) (local.get $len))
)
...

```

Мы уже выяснили, что для ускорения процесса копирования функция `$string_copy` должна комбинировать методы восьмибайтового и однобайтового копирования. Параметр `$len` – это полная длина строки в байтах. Локальная переменная `$len_less_singles` ❶ – это количество байтов, которое можно скопировать с помощью 64-битной копии. Получить это количество можно через маскирование последних трех битов. Переменная `$singles` – это оставшиеся три бита, которые не кратны восьми и устанавливаются путем выполнения побитового (`i32.and`) выражения ❷ (плюс для побитового маскирования) между `$len` и 7 (111 в двоичном). Последние три бита длины указывают количество оставшихся байтов после восьмибайтового копирования большинства байтов. Например, использование выражения `i32.and` для `$len` значений 190 и 7 выглядит как на рис. 5.1.

Как видите, вызывая `i32.and` и передавая значения 190 и 7, мы получаем двоичное значение 110, которое в десятичном виде равно 6. Выражение `i32.and` обнуляет все биты, кроме последних трех, нашего параметра `$len`.

Если количество `$single` не равно нулю ❸, код сначала копирует отдельные байты, которые нельзя скопировать с помощью 64-битной копии. Блок `if` устанавливает `$len_less_singles` ❹ в `$len-$singles`: это количество байтов, которые должны быть скопированы индивидуально. Локальная переменная `$start_source_byte` ❺ устанавливается в `$source+$len_less_singles` на начальный байт побайтовой копии ❷.

Затем переменная `$start_dest_byte` ❻ устанавливается равной `$dest + $len_less_singles`, что делает ее местом назначения для побайтовой копии. Далее оператор условного перехода вызывает `$byte_copy` для копирования оставшихся байтов.

После блока `if` код должен скопировать байты с помощью 64-битной функции копирования (вызов `$byte_copy_i64`) ❻. Количество байтов для копирования с помощью этой функции мы определяем, ис-

пользуя побитовое выражение длины (`i32.and`) ❶ в `$len` с 32-битным постоянным значением `0xff_ff_ff_f8` ❷. Значение `0xff_ff_ff_f8` в двоичном формате – это все единицы, кроме последних трех бит. Побитовое И обнуляет последние три бита длины, что делает длину кратной восьми.



Рис. 5.1. Маскирование всех битов, кроме последних трех, с помощью двоичного И

Теперь, когда у нас есть функция копирования строки, давайте отредактируем функцию `main` и протестируем ее, добавив код из листинга 5.20.

Листинг 5.20. Обновленная версия функции `main` для `strings.wat` (листинг 5.2)

```
strings.wat ...
(часть 11 из 11) (func (export "main")
  ❶(call $str_pos_len (i32.const 256) (i32.const 30))
  ❷(call $str_pos_len (i32.const 384) (i32.const 35))

  ❸(call $string_copy
    (i32.const 256) (i32.const 384) (i32.const 30))

  ❹(call $str_pos_len (i32.const 384) (i32.const 35))
  ❺(call $str_pos_len (i32.const 384) (i32.const 30))
)
```

Мы удалили код, который выводил на консоль строки с завершающим нулем и строки с префиксом длины. Оставили две строки, вы-

водящие в ячейке 256 линейной памяти ❶ "Know the length of this string" и в ячейке 384 памяти ❷ "Also know the length of this string". Если их оставить, на консоль будет выведено исходное значение строк перед копированием.

Вызов `$string_copy` ❸ копирует 30 байт из первой строки во вторую. Затем мы выводим позицию второй строки и ее исходную длину. Это выведет на консоль "Know the length of this stringtring" ❹, что является неправильным, потому что заканчивается словом `stringtring`. Причина, по которой последнее слово ошибочно не заканчивается на `string` и содержит пять дополнительных символов, заключается в том, что мы не изменили длину строки, из которой копировали. Если бы мы копировали строку с завершающим нулем или строку с префиксом длины, проблемы не возникло бы, потому что нулевой байт или префикс отслеживал бы длину за нас: но в данном случае нам нужно знать, что новая длина – 30.

Если мы вызовем `$str_pos_len`, передавая 384 в качестве индекса и 30 в качестве длины ❺, то получим правильный вывод на консоль "Know the length of this string". Мы можем перекомпилировать `strings.wat`, используя команду из листинга 5.21.

Листинг 5.21. Компиляция `strings.wat`

```
wat2wasm strings.wat
```

Запустив `strings.js` из командной строки, вы должны получить вывод, показанный в листинге 5.22.

Листинг 5.22. Вывод из `strings.js` после вызова `$string_copy`

```
Know the length of this string
Also know the length of this string
Know the length of this stringtring
Know the length of this string
```

В следующем разделе вы узнаете, как преобразовывать числа в строки.

Создание числовых строк

При работе со строками часто требуется преобразование числовых данных в строковые. В языках высокого уровня, таких как JavaScript, есть функции, которые могут сделать это за вас, а в WAT вам нужно будет создавать свои собственные функции. Давайте посмотрим, что нужно для создания строк из чисел в десятичном, шестнадцатеричном и двоичном форматах.

Создайте WAT-файл с именем `number_string.wat` и добавьте код из листинга 5.23 в начало файла.

```

number_string.wat (module
  (часть 1 из 7) ❶(import "env" "print_string" (func $print_string (param i32 i32)))
                  ❷(import "env" "buffer" (memory 1))

                  ❸(data (i32.const 128) "0123456789ABCDEF")

                  ❹(data (i32.const 256) "                                0")
                  ❺(global $dec_string_len i32 (i32.const 16))
  ...

```

В начале этого модуля из JavaScript импортируются функция `$print_string` ❶ и одна страница линейной памяти ❷. Далее мы определяем элемент `data` ❸ с массивом символов, который содержит каждый шестнадцатеричный символ. Затем определяем элемент данных, который будет содержать наши строковые данные ❹, за которыми следует длина этой `data`-строки ❺.

В следующих нескольких листингах мы определим три функции, которые создают числовые строки в трех разных форматах. Сначала мы используем функцию `$set_dec_string` для определения линейной области памяти `$dec_string`.

Листинг 5.24 содержит код, который преобразовывает целое число в десятичную строку. На первый взгляд, этот код может быть немного сложным для понимания, поэтому сначала я его объясню вам. На высоком уровне, когда мы собираем строку из числа, нам нужно смотреть на число по одной цифре за раз и добавлять символьную форму этой цифры к нашей строке. Допустим, рассмотрим число 9876. В разряде единиц находится цифра 6. Мы можем найти эту цифру, разделив полное число на 10 и используя остаток (*modulo* – остаток от деления). В WAT это тип кода `i32.rem_u` (`rem` для остатка). Затем вы используете символьную форму числа 6 и добавляете ее к своей строке. Остальные цифры должны двигаться, как по конвейерной ленте (как показано на рис. 5.2). В коде это делается делением на 10. Поскольку выполняется целочисленное деление, дробное число в результате не получается, а 6 просто отбрасывается. Затем вы используете остаток, чтобы получить следующую цифру (7) и добавить ее к строке. Продолжаем это делать, пока цифры не закончатся. В листинге 5.24 показан исходный код функции `$set_dec_string`.

Листинг 5.24. Функция WebAssembly, которая создает десятичную строку из целого числа

```

number_string.wat ...
  (часть 2 из 7) (func $set_dec_string (param $num i32) (param $string_len i32)
                (local $index      i32)
                (local $digit_char i32)
                (local $digit_val  i32)

                local.get $string_len

```

```

❶ local.set $index    ;; установить $index в длину строки

local.get $num
i32.eqz              ;; равно ли $num нулю
if                   ;; если число равно 0, мне не нужны все пробелы
    local.get $index
    i32.const 1
    i32.sub
    local.set $index ;; $index--

    ;; сохранить ascii '0' в ячейку памяти 256 + $index
    (i32.store8 offset=256 (local.get $index) (i32.const 48))
end

(loop $digit_loop (block $break ;; цикл преобразует число в строку
    local.get $index ;; установить $index в конец строки, уменьшить до 0
    i32.eqz          ;; $index равен нулю?
    br_if $break     ;; если да, выйти из цикла

    local.get $num
    i32.const 10
    ❷ i32.rem_u       ;; десятичная цифра - остаток от деления на 10

    ❸ local.set $digit_val ;; заменяет вызов выше
    local.get $num
    i32.eqz          ;; проверка, равен ли теперь $num нулю
    if
        i32.const 32      ;; 32 - знак пробела ascii
        local.set $digit_char ;; если $num равно нулю, нужен пробел слева
    else
        ❹ (i32.load8_u offset=128 (local.get $digit_val))
        local.set $digit_char ;; установить $digit_char на цифру ascii
    end

    local.get $index
    i32.const 1
    i32.sub
    local.set $index
    ;; сохранить цифру ascii в 256 + $index
    ❺ (i32.store8 offset=256
        (local.get $index) (local.get $digit_char))
    local.get $num
    i32.const 10
    i32.div_u
    ❻ local.set $num    ;; убрать последнюю десятичную цифру, разделив на 10

    br $digit_loop    ;; цикл
)) ;; конец $block и $loop
)

```

Мы начинаем функцию с \$index ❶ – переменной, которая указывает на последний байт в \$dec_string. Устанавливаем значения этой строки справа налево, поэтому переменную \$index нужно уменьшать при каждом переходе в цикле. При каждом проходе цикла числовое

значение, установленное в `$dec_string`, является последней цифрой по основанию 10. Чтобы получить это значение, мы делим значение `$num` ❶ на 10 и получаем остаток ❷ с остатком от деления на 10. Это значение сохраняется в локальной переменной `$digit_val` ❸, поэтому позже мы сможем использовать его для установки символа ASCII в данных `$dec_string`. Мы используем `$digit_val` в качестве смещения в строке `$digit_char` для загрузки символа с `i32.load8_u` ❹. Затем этот символ записывается по адресу `$dec_string+$index` с помощью `i32.store8` ❺. Рисунок 5.2 иллюстрирует этот процесс.

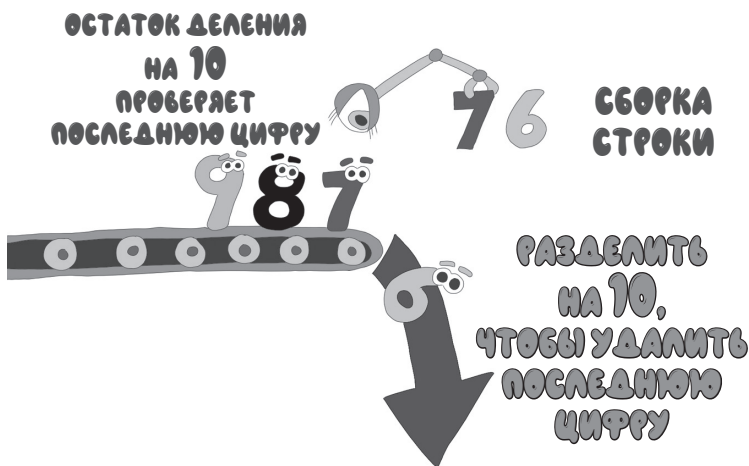


Рис. 5.2. Перебираем цифры по очереди и добавляем символы по одному

Теперь, когда у нас есть функция, которая выполняет большую часть работы в нашем модуле `WebAssembly`, давайте добавим функцию для экспорта в JavaScript, как показано в листинге 5.25.

Листинг 5.25. Функция `to_string`

```
number_string.wat ...
(часть 3 из 7) (func (export "to_string") (param $num i32)
  ❶(call $set_dec_string
    (local.get $num) (global.get $dec_string_len))
  ❷(call $print_string
    (i32.const 256) (global.get $dec_string_len))
  )
)
```

Функция очень проста. Она вызывает `$set_dec_string` ❶, передавая число, которое мы хотим преобразовать в строку, и длину строки, которая нам нужна, включая внутренний отступ слева. Затем она вызывает функцию JavaScript `print_string` ❷, передавая местоположение созданной нами строки в линейной памяти (`i32.const 256`) и ее длину.

Теперь, когда мы завершили работу с файлом *number_string.wat*, мы можем скомпилировать его с помощью *wat2wasm*, как в листинге 5.26.

Листинг 5.26. Компиляция number_string.wat

```
wat2wasm number_string.wat
```

Затем нам нужно написать код JavaScript, который будет запускать наш модуль WebAssembly. Создайте файл с именем *number_string.js* и сохраните в него код из листинга 5.27.

Листинг 5.27. JavaScript вызывает модуль WebAssembly для преобразования числа в строку

```
number_string.js  const fs = require('fs');
                  const bytes = fs.readFileSync(__dirname + '/number_string.wasm');
                  ❶ const value = parseInt(process.argv[2]);
                  let memory = new WebAssembly.Memory({ initial: 1 });

                  (async () => {
                    const obj =
                      await WebAssembly.instantiate(new Uint8Array(bytes), {
                        env: {
                          buffer: memory,
                          ❷ print_string: function (str_pos, str_len) {
                            const bytes = new Uint8Array(memory.buffer,
                              str_pos, str_len);
                            const log_string = new TextDecoder('utf8').decode(bytes);
                            // слева от log_string должен быть отступ.
                            ❸ console.log(`>${log_string}!`);
                          }
                        }
                      });
                    obj.instance.exports.to_string(value);
                  })();
```

Код JavaScript загружает модуль WebAssembly и принимает дополнительный аргумент ❶, который мы преобразуем в числовую строку, заполняет строку левыми отступами до 16 символов. Модуль WebAssembly вызовет функцию JavaScript *print_string* ❷, которая записывает строку в консоль, добавляя символ > в начало строки и символ ! в конец. Эти дополнительные символы помещаем в вывод *console.log* ❸, чтобы показать, где начинается и заканчивается строка из модуля WebAssembly. С помощью команды *node* запустите код JavaScript, как показано в листинге 5.28.

Листинг 5.28. Использование node для вызова файла number_string.js, чтобы передать значение 1229

```
node number_string.js 1229
```

В результате число 1229 преобразуется в строку, а вывод в листинге 5.29 запишется в консоль.

Листинг 5.29. Перед числом 1229 левый отступ 16 символов, начинается с > и заканчивается на !

```
>          1229!
```

В следующем разделе мы воспользуемся аналогичными методами для создания шестнадцатеричной строки.

Создание шестнадцатеричной строки

Преобразование целого числа в шестнадцатеричную строку напоминает преобразование целого числа в десятичное, как мы делали в предыдущем разделе. Мы используем битовую маску для конкретных шестнадцатеричных цифр и сдвиг, чтобы убрать цифру, аналогично нашему десятичному сдвигу (см. рис. 5.2).

Вспомните главу 4, где мы говорили, что четыре бита данных называются полубайтом, а шестнадцатеричная цифра соответствует одному полубайту данных. На высоком уровне код должен рассматривать целое число по одному полубайту за раз как одну шестнадцатеричную цифру. Мы выбираем полубайт самого младшего разряда, а также шестнадцатеричную цифру самого младшего разряда, а затем добавляем эту цифру в строку. Вместо того чтобы находить остаток, используем маску, чтобы смотреть только на последнюю цифру. Вместо деления на 10 мы удаляем последнюю шестнадцатеричную цифру, сдвигая четыре бита (один полубайт).

В шестнадцатеричном формате каждая цифра представляет собой число от 0 до 15 вместо 0 до 9, поэтому каждая шестнадцатеричная цифра должна быть одной из следующих: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, где значения A = 10, B = 11, C = 12, D = 13, E = 14 и F = 15. Мы часто используем шестнадцатеричные числа в качестве альтернативы двоичным числам, которые могут быть очень длинными для представления; например, число 233 в десятичной системе счисления равно двоичному 11101001. Мы можем сократить число 233 в двоичном виде до шестнадцатеричного E9, потому что 1110 – это 14 (E в шестнадцатеричном формате), а 1001 – это 9 как в десятичном, так и в шестнадцатеричном формате.

Мы маскируем E9 (двоичный формат 1110 1001) с помощью (i32.and) и двоичного значения 0000 1111 (0F), чтобы найти наименее значимый полубайт. Использование i32.and таким образом приводит к тому, что E9 замаскирован в 09, как показано на рис. 5.3.

Для преобразования целочисленных данных в шестнадцатеричную строку используется комбинация битового сдвига и логической операции И с маской. Мы создаем шестнадцатеричную версию функции \$set_dec_string, которая называется \$set_hex_string. Эта функция устанавливает шестнадцатеричную строку на основе переданного

в нее числа. Мы можем сделать этот цикл проще, чем цикл в `$set_dec_string`, потому что можем использовать простые битовые операции, чтобы найти смещение в `$digits`. В конце функции добавляются дополнительные символы ASCII 0x, чтобы указать, что отображаемая строка находится в шестнадцатеричном формате. В листинге 5.30 показано, как выглядит функция `$set_hex_string`.

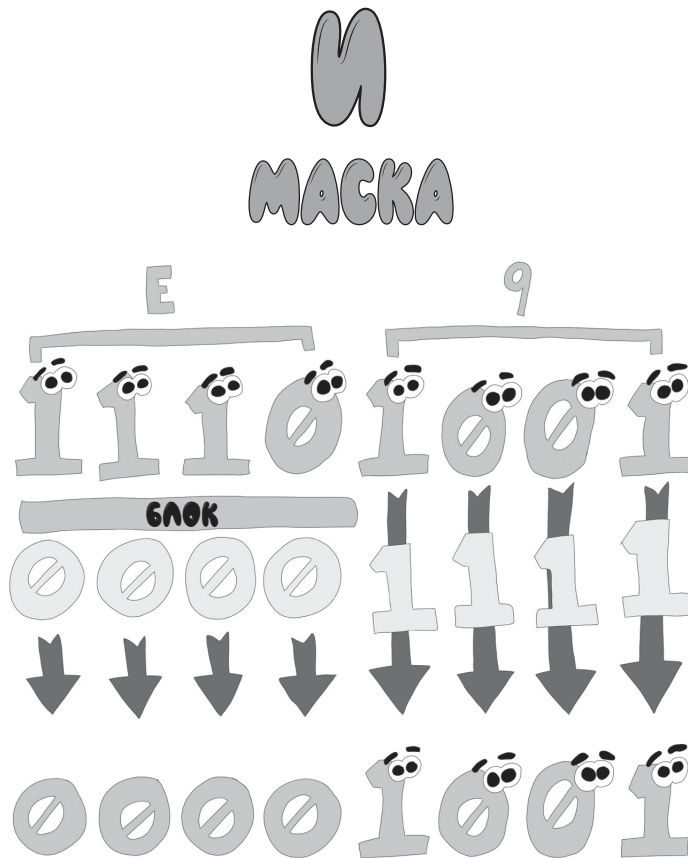


Рис. 5.3. Байт E9 замаскирован в 09 маской 0F (0000 1111)

Листинг 5.30. Создание шестнадцатеричной строки из целого числа.
Добавьте этот код перед функцией `$set_dec_string`

```
number_string.wat ...
(часть 4 из 7) ;; добавьте этот код перед функцией $ set_dec_string
                (global $hex_string_len i32 (i32.const 16)) ;; количество шестнадцатеричных
                                                         ;; символов
                (data (i32.const 384) "                0x0") ;; шестнадцатеричные строковые
                                                         ;; данные

                (func $set_hex_string (param $num i32) (param $string_len i32)
                  (local $index i32)
```



```

(local $digit_char i32)
(local $digit_val i32)
(local $x_pos i32)

global.get $hex_string_len
local.set $index ;; Присвоить индексу количество шестнадцатеричных символов

(loop $digit_loop (block $break
  local.get $index
  i32.eqz
  br_if $break

  local.get $num
  i32.const 0xf ;; последние 4 бита равны 1
  ❶ i32.and      ;; смещение в $digits находится в последних 4 битах числа

  ❷ local.set $digit_val ;; цифровое значение - это последние 4 бита
  local.get $num
  i32.eqz

  ❸ if          ;; if $num == 0
    local.get $x_pos
    i32.eqz
    if
      local.get $index
      ❹ local.set $x_pos ;; позиция 'x' в шестнадцатеричном префиксе "0x"
    else
      i32.const 32      ;; 32 - это код пробела в ASCII
      local.set $digit_char
    end
  else
    ;; загрузить символ из 128 + $digit_val
    ❺ (i32.load8_u offset=128 (local.get $digit_val))
    local.set $digit_char
  end

  local.get $index
  i32.const 1
  i32.sub

  ❻ local.tee $index      ;; $index = $index - 1
  local.get $digit_char

  ;; сохранить $digit_char по адресу 384 + $index
  ❼ i32.store8 offset=384
  local.get $num
  i32.const 4

  ❽ i32.shr_u          ;; сдвигает 1 шестнадцатеричную цифру с $num
  local.set $num

  br $digit_loop
))

local.get $x_pos
i32.const 1
i32.sub

i32.const 120          ;; ascii x

```

```

❶ i32.store8 offset=384    ;; сохранить 'x' в строке

    local.get $x_pos
    i32.const 2
    i32.sub

    i32.const 48           ;; ascii '0'
❷ i32.store8 offset=384    ;; сохранить "0x" перед строкой
) ;; конец $set_hex_string
...

```

В функции `$set_dec_string` мы используем остаток от деления на 10, чтобы найти каждую цифру, а затем сдвигаем разряды, разделив число на 10. Вместо того чтобы находить остаток, функция `$set_hex_string` может использовать `i32.and` ❶ для маскирования всех битов, кроме последних четырех битов `$num`. Значение этого полубайта является одной шестнадцатеричной цифрой и используется как значение `$digit_val` ❷.

Если все оставшиеся цифры равны 0 ❸, мы задаем позицию для помещения префикса шестнадцатеричной строки `0x` в локальную переменную `$x_pos` ❹. В противном случае, если какие-либо оставшиеся цифры равны 1 или больше, мы используем значение в `$digit_val`, чтобы загрузить ❺ значение ASCII для этой шестнадцатеричной цифры из строки `$digits` и сохранить его в `$digit_char`. Затем мы уменьшаем `$offset` ❻ и используем это значение для сохранения символа из `$digit_char` в `$hex_string` ❼.

Потом цикл сдвигает одну шестнадцатеричную цифру (четыре бита) с помощью `i32.shr_u` ❽, который сдвигает биты вправо. Последняя задача, которую выполняет эта функция, – это добавление префикса `0x` к строке с помощью значения, которое мы установили ранее в `$x_pos`, в качестве смещения и сохранение символа ASCII `x` в этой позиции ❾. Затем она уменьшает позицию `$x_pos` и сохраняет ASCII `0` ❿. Данный процесс выглядит примерно как на рис. 5.4.

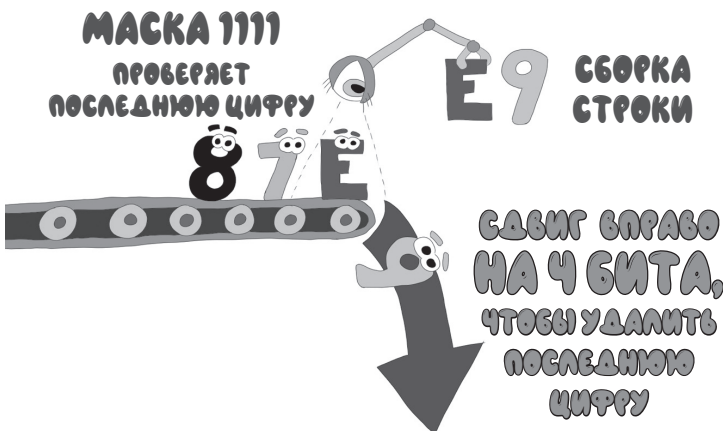


Рис. 5.4. Создание шестнадцатеричной строки из целого числа

После добавления функции `$set_hex_string` нам нужно обновить функцию `to_string`, чтобы она вызывала `$set_hex_string` и выводила полученную строку в консоль. Обновите функцию `to_string`, чтобы она выглядела как код в листинге 5.31.

Листинг 5.31. Новая версия функции `to_string` для вызова `$set_hex_string` и `$print_string`

```
number_string.wat ...
(часть 5 из 7)  (func (export "to_string") (param $num i32)
                (call $set_dec_string
                    (local.get $num) (global.get $dec_string_len))
                (call $print_string (i32.const 256) (global.get $dec_string_len))

                ❶(call $set_hex_string (local.get $num)
                    (global.get $hex_string_len))
                ❷(call $print_string (i32.const 384) (global.get $hex_string_len))
                )
            )
```

Эти два новых оператора вызывают `$set_hex_string` ❶ для размещения шестнадцатеричной строки в линейной памяти. Затем мы вызываем `$print_string` ❷, передавая в память шестнадцатеричную строку (`i32.const 384`) и длину строки. Никаких изменений в файле JavaScript не требуется. Все, что нам нужно сделать, – это перекомпилировать наш WAT-файл, как показано в листинге 5.32.

Листинг 5.32. Компиляция `number_string.wat` с помощью `wat2wasm`

```
wat2wasm number_string.wat
```

После того как вы перекомпилировали модуль WebAssembly, вы можете запустить файл `number_string.js` с помощью `node`, как показано в листинге 5.33.

Листинг 5.33. Запуск `number_string.js` с передачей значения 2049

```
node number_string.js 2049
```

В листинге 5.34 показан вывод:

Листинг 5.34. Вторая строка – это результат преобразования шестнадцатеричной строки

```
>          2049!
>          0x801!
```

В следующем разделе мы добавим функцию для генерации строки двоичных цифр из 32-битного целого числа.

Создание двоичной строки

Последний формат, который мы рассмотрим, – это преобразование целого числа в строку, представляющую двоичные данные. Когда вы работаете с низкоуровневым кодом, лучше всего иметь интуитивное представление о двоичных числах. В некоторых случаях правильное концептуальное представление о числах может помочь вам повысить производительность вашего кода за счет использования битовых операций в качестве альтернативы десятичной математике. Даже если ваш код в десятичном формате, компьютеры работают с двоичным кодом. Понимание того, как компьютер работает с кодом, часто бывает полезным для повышения производительности.

Мы создадим функцию `$set_bin_string`, которая использует двойной цикл для разделения каждого 4-битного полубайта пробелом, чтобы сделать его более удобочитаемым. Мы будем применять логическую операцию (`i32.and`) с числом 1, чтобы увидеть, равен ли последний бит 1 или 0, а затем, при каждом проходе по внутреннему циклу функции, будем сдвигать один бит из числа. В листинге 5.35 показано, как должен выглядеть код.

Листинг 5.35. Создание двоичной строки из целого числа. Добавьте этот код перед функцией `$set_hex_string`

```
number_string.wat ...
(часть 6 из 7) ;; добавьте этот код перед функцией $set_hex_string
(global $bin_string_len i32 (i32.const 40))
(data (i32.const 512) " 0000 0000 0000 0000 0000 0000 0000 0000")

(func $set_bin_string (param $num i32) (param $string_len i32)
  (local $index i32)
  (local $loops_remaining i32)
  (local $nibble_bits i32)

  global.get $bin_string_len
  local.set $index
  ❶ i32.const 8                ;; в 32 битах 8 полубайт (32/4 = 8)
  local.set $loops_remaining  ;; внешний цикл разделяет полубайты

  ❷ (loop $bin_loop (block $outer_break ;; внешний цикл для пробелов
    local.get $index
    i32.eqz
    br_if $outer_break      ;; прекратить цикл, когда $index будет равен 0

    i32.const 4
    ❸ local.set $nibble_bits  ;; 4 бита в каждом полубайте

    ❹ (loop $nibble_loop (block $nibble_break ;; внутренний цикл для цифр
      local.get $index
      i32.const 1
      i32.sub
      local.set $index      ;; увеличить $index
```

```

    local.get $num
    i32.const 1
    ❶ i32.and          ;; i32.and 1 равен 1, если последний бит равен 1, иначе 0
    if                ;; если последний бит равен 1
        local.get $index
        i32.const 49      ;; ascii '1' is 49
        ❷ i32.store8 offset=512 ;; сохранить '1' в 512 + $index

    else              ;; else выполняется, если последний бит был равен 0
        local.get $index
        i32.const 48      ;; ascii '0' равен 48
        ❸ i32.store8 offset=512 ;; сохранить '0' в 512 + $index
    end

    local.get $num
    i32.const 1
    ❹ i32.shr_u          ;; $num сдвинут вправо на 1 бит
    local.set $num       ;; сдвиг последнего бита $num

    local.get $nibble_bits
    i32.const 1
    i32.sub
    local.tee $nibble_bits ;; увеличить $nibble_bits
    i32.eqz              ;; $nibble_bits == 0
    ❺ br_if $nibble_break ;; прервать цикл, если $nibble_bits == 0

    br $nibble_loop
)) ;; конец $nibble_loop

local.get $index
i32.const 1
i32.sub
local.tee $index      ;; увеличить $index
i32.const 32          ;; пробел ascii
10 i32.store8 offset=512 ;; сохранить пробел ascii в 512+$index

br $bin_loop
)) ;; конец $bin_loop
)
...

```

В функции `$set_bin_string` есть два цикла. Внешний цикл ставит пробел между каждым полубайтом. Поскольку в этом коде мы работаем с 32-битными числами, нам нужно пройти в цикле восемь полубайт ❶. Мы помечаем внешний цикл `$bin_loop` ❷, а блок, из которого мы прерываем цикл, называется `$outer_break` ❸.

Перед внутренним циклом нам нужно присвоить локальной переменной `$nibble_bits` значение 4 ❹. Код перебирает четыре бита для каждого полубайта во внутреннем цикле. Во внутреннем цикле `$nibble_loop` ❺ мы помещаем блок `$nibble_block`, который может прервать выполнение этого цикла ❻. Внутри `$nibble_loop` мы используем выражение `(i32.and)` ❼ вместе с оператором `if/else`, чтобы определить, будет последний бит в переменной `$num` равен 1 или 0. Если он

равен 1, мы используем оператор `i32.store8` ⑥ для сохранения ASCII 1 в адресе линейной памяти `$index+512`. Если он не равен 1, мы сохраняем ASCII 0 ⑦.

Затем нам нужно сдвинуть этот бит для следующего прохода цикла. Для этого сдвига, как и в функции `$set_hex_string`, мы используем выражение `(i32.shr_u)` ⑧, но на этот раз мы сдвигаем один бит вместо четырех. После четырехкратного прохождения цикла `$nibble` мы выходим из него ⑨ и сохраняем пробел ASCII в линейной позиции памяти `$offset+$bin_string` ⑩.

Теперь можем вызвать `$set_bin_string`, чтобы вывести на печать значение нашей строки из функции `to_string`. Обновите функцию `to_string` кодом из листинга 5.36.

Листинг 5.36. Прибавление `$set_bin_string` к функции `to_string`

```
number_string.wat ...
(часть 7 из 7) (func (export "to_string") (param $num i32)
  (call $set_dec_string
    (local.get $num) (global.get $dec_string_len))
  (call $print_string (i32.const 256) (global.get $dec_string_len))
  (call $set_hex_string
    (local.get $num) (global.get $hex_string_len))
  (call $print_string (i32.const 384) (global.get $hex_string_len))
  ❶(call $set_bin_string
    (local.get $num) (global.get $bin_string_len))
  ❷(call $print_string (i32.const 512) (global.get $bin_string_len))
  )
)
```

Первый из двух только что добавленных `call`-операторов (`call $set_bin_string` ❶) устанавливает двоичную строку, используя функцию, определенную в листинге 5.35. Второй `call`-оператор (`call $print_string` ❷) выводит двоичную строку на консоль. Теперь давайте перекомпилируем модуль `WebAssembly`, как показано в листинге 5.37.

Листинг 5.37. Перекомпиляция `number_string.wat` с помощью функции двоичной строки

```
wat2wasm number_string.wat
```

Теперь мы можем запустить файл `number_string.js` через команду `node`, как в листинге 5.38.

Листинг 5.38. Запуск `number_string.js` с модулем `WebAssembly` двоичной строки

```
node number_string.js 4103
```

Записанный в консоль вывод должен выглядеть как в листинге 5.39.

Листинг 5.39. Двоичная строка, выведенная в консоль

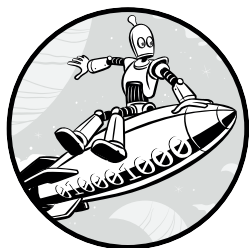
```
>          4103!  
>          0x1007!  
> 0000 0000 0000 0000 0001 0000 0000 0111!
```

Заключение

Эту главу мы посвятили работе со строками в WAT. Вы узнали о форматах символов ASCII и Unicode, как хранить строковые объекты в линейной памяти и как использовать JavaScript для извлечения строк и вывода их в консоль. Мы рассмотрели, как передавать строковые данные из WebAssembly в JavaScript, и изучили два популярных метода управления строками: строки с завершающим нулем и строки с префиксом длины. Вы скопировали строку из одного места в линейной памяти в другое, используя побайтовое копирование и 64-битную копию. Затем мы рассмотрели, как преобразовать целочисленные данные в числовые строки в десятичном, шестнадцатеричном и двоичном форматах. В следующей главе мы сосредоточимся на использовании линейной памяти в WAT.

6

ЛИНЕЙНАЯ ПАМЯТЬ



В этой главе мы рассмотрим, что такое линейная память, как использовать ее для обмена данными между кодами JavaScript и WebAssembly и как ее создать внутри JavaScript. Мы также обновим линейную память из WebAssembly, а затем будем использовать эти обновления внутри кода JavaScript.

Одной из распространенных задач в разработке компьютерных игр является *обнаружение столкновений*, т. е. обнаружение соприкосновения двух объектов и соответствующая реакция на него. Количество требуемых вычислений растет экспоненциально по мере добавления объектов. Код для обнаружения столкновений лучше всего писать на WebAssembly. В этой главе мы создадим список кругов, определенных случайным образом в JavaScript, а потом добавим их данные в линейную память WebAssembly. Затем мы будем использовать эти данные, чтобы определить, сталкиваются ли какие-либо из этих кругов.

Достоинства WebAssembly лучше всего проявляются при работе с огромным количеством данных, требующих значительной обработки. Модуль WebAssembly может выполнять математические вычисления быстрее, чем JavaScript. Однако каждое взаимодействие между

JavaScript и модулем WebAssembly несет определенные затраты. Линейную память можно использовать для загрузки значительных объемов данных в JavaScript, чтобы затем обработать их внутри модуля WebAssembly.

Линейная память в WebAssembly

Линейная память действует как один гигантский массив данных, который могут совместно использовать WebAssembly и JavaScript. Если вы знакомы с низкоуровневым программированием, для вас линейная память будет похожа на динамическую память в исходных приложениях. Если вы знакомы с JavaScript, представьте ее как один гигантский объект `ArrayBuffer`. Такие языки, как C и C++, создают локальные переменные, выделяя память в стеке компьютера. Локальные переменные, распределенные стеком, освобождаются из памяти, как только выполнится функция. Этот эффективный процесс означает, что выделение и освобождение данных в стеке так же просто, как увеличение и уменьшение указателя стека. Ваше приложение просто увеличивает указатель стека, и вуаля, у вас есть новая распределенная переменная, как показано на рис. 6.1.



Рис. 6.1. Указатель стека

Стек отлично работает с локальными переменными. Однако одним из ограничений в WAT является то, что локальные переменные, использующие стек, могут быть только одного из четырех типов, и все они числовые. Иногда вам могут потребоваться более сложные структуры данных, такие как строки, структуры, массивы и объекты.

Команды распределения, такие как `malloc` в C и `new` в C++ и JavaScript, выделяются в динамической памяти, а библиотеки управления памятью, включенные в эти языки, должны искать достаточно большой свободный раздел динамической памяти, чтобы вместить туда требуемый блок памяти. Со временем это может привести к фрагментации памяти – когда выделенные сегменты памяти перемежаются с незанятой памятью, как показано на рис. 6.2.

Линейная память WebAssembly выделяется большими блоками, называемыми *страницами*, которые после выделения не могут быть освобождены. Память WebAssembly немного напоминает управление памятью на языке ассемблера: после того как вы выделили опреде-

ленное количество страниц для вашего модуля WebAssembly, вы как программист должны отслеживать, для чего вы используете память и где она находится. В следующих нескольких разделах мы более подробно рассмотрим, как использовать линейную память, исследуя страницы памяти.

ЛИНЕЙНАЯ/ДИНАМИЧЕСКАЯ ПАМЯТЬ

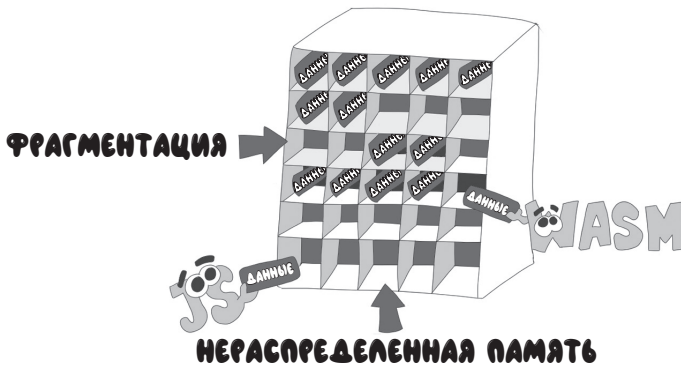


Рис. 6.2. Линейная память передала данные из JavaScript и WebAssembly

Страницы

Страницы – это наименьший фрагмент данных, который может быть выделен для модуля WebAssembly. На момент написания этой книги все страницы в WebAssembly имели размер 64 КБ. В текущей версии WebAssembly 1.0 вы не можете изменить этот размер, хотя WebAssembly Community Group постоянно предлагает добавить возможность изменять размер страницы в зависимости от потребностей приложения. На момент написания этой книги максимальное количество страниц, которое приложение может выделить, составляет 32 767, а общий максимальный объем памяти – 2 ГБ. Этого объема достаточно для веб-приложений, но не для серверных приложений. Увеличение размера страницы может позволить серверным приложениям увеличить максимальный объем линейной памяти, которую они могут выделить. Для встроенных приложений WebAssembly размер страниц 64 КБ может быть слишком большим; например, ATmega328 имеет только 32 КБ флеш-памяти. Возможно, что к тому моменту, когда вы прочтете это, в обновленной версии WebAssembly могут снять данное ограничение.

Примечание В настоящее время WebAssembly Community Group работает над увеличением максимального количества страниц до 65 535, что позволит выделять до 4 ГБ памяти. Вполне вероятно, что к моменту публикации этой книги максимальный размер линейной памяти составит 4 ГБ, а не 2 ГБ.

Вы можете создать то количество страниц, которое ваше приложение будет использовать либо внутри модуля WebAssembly, либо в среде встраивания для импорта.

Выделение страниц в вашем коде

Чтобы выделить страницу линейной памяти в WAT, используйте простое выражение (`memory`), как в листинге 6.1.

Листинг 6.1. Объявление страницы памяти в WAT

```
(memory 1)
```

Передача значения 1 в выражение памяти указывает модулю выделить одну страницу линейной памяти. Чтобы выделить максимальный объем памяти в модуле WebAssembly во время выполнения, используйте выражение из листинга 6.2.

Листинг 6.2. Объявление максимального количества страниц памяти

```
(memory 32_767)
```

Попытка передачи значения 32_767 в выражение (`memory`) приведет к ошибке компиляции. Память, созданная с помощью выражения (`memory`), недоступна для среды встраивания без выражения (`export`).

Создание памяти в среде встраивания

Другой способ создать линейную память – внутри среды встраивания. Если среда встраивания – JavaScript, то для создания памяти используется код `new WebAssembly.Memory`, как в листинге 6.3.

Листинг 6.3. Создание объекта WebAssembly Memory в JavaScript

```
const memory = new WebAssembly.Memory({initial: 1});
```

Затем, используя выражение (`import`), вы можете получить к нему доступ из модуля WebAssembly, как в листинге 6.4.

Листинг 6.4. Импорт страницы памяти, выделенной в JavaScript

```
(import "js" "mem" (memory 1))
```

Чтобы использовать `import`, необходимо создать в JavaScript объект `Memory` с помощью класса `WebAssembly.Memory`, а затем передать его в модуль WebAssembly при его инициализации с помощью объекта `import`. Создайте файл с именем *pointer.js* и добавьте код JavaScript из листинга 6.5, который создает объект `WebAssembly.Memory`.

Листинг 6.5. Инициализируйте линейную память WebAssembly и передайте ее модулю WebAssembly

```
pointer.js const fs = require('fs');
const bytes = fs.readFileSync(__dirname + '/pointer.wasm');
const memory = new WebAssembly.Memory({❶initial: 1, ❷maximum: 4});

const importObject = {
  env: {
    ❸mem: memory,
  }
};

( async () => {
  let obj = await WebAssembly.instantiate(new Uint8Array(bytes),
    ❹importObject);

  let pointer_value = obj.instance.exports.get_ptr();
  console.log(`pointer_value=${pointer_value}`);
})();
```

При создании этот код передает объект с двумя значениями инициализации. Требуется аргумент `initial` **❶**, и, передав ему значение 1, мы указываем движку JavaScript выделить одну страницу линейной памяти (64 КБ). Второе (необязательное) значение `maximum` **❷** сообщает браузеру, что, скорее всего, позже мы увеличим размер линейной памяти и не будем увеличивать объем памяти до более чем четырех страниц. Вы можете увеличить размер линейной памяти, вызвав метод `memory.grow`. Устанавливать максимальное значение для увеличения объема памяти не нужно, но передача максимального значения указывает браузеру выделить больше памяти, чем начальное значение, потому что есть вероятность вызова для увеличения памяти. Если вы попытаетесь увеличить объем линейной памяти до значения, превышающего максимальное, приложение выдаст ошибку. После создания объекта памяти мы передаем его модулю WebAssembly через `importObject` в `env.mem` **❸**. JavaScript передает `importObject` в модуль при запуске **❹**.

Указатели

Указатель – это переменная, которая ссылается на конкретное место в памяти. В информационных технологиях указатели имеют множество применений, но в данном контексте мы будем использовать их для указания на структуры данных в линейной памяти. На рис. 6.3 представлен указатель, ссылающийся на ячейку памяти 5, которая имеет значение 99.

Указатели WebAssembly ведут себя иначе, чем те, с которыми вы, возможно, знакомы из языков C или C++, где они могут указывать на локальные переменные или переменные в неупорядоченном массиве данных. В листинге 6.6 код на языке C создает указатель `ptr`, который указывает на адрес локальной переменной `x`.

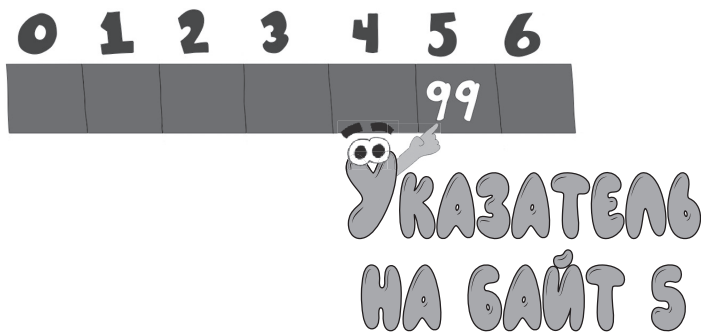


Рис. 6.3. Указатель на пятый байт в памяти

Листинг 6.6. Пример установки значений указателя в языке C

```
int x = 25;
int *ptr = &x;
```

Указатель `ptr` помещается в адрес локальной переменной `x` в стеке. WebAssembly не имеет отдельного типа указателя, такого как, например, целочисленный тип указателя в языке C `int*`. Линейная память WebAssembly – это большой массив данных. Когда вы представляете указатель в WAT, вы должны поместить данные в линейную память; тогда указатель будет являться для этих данных индексом `i32`. В листинге 6.6 при компиляции программы в WebAssembly переменная `x` получает адрес в линейной памяти. В отличие от языка C, WAT не может создавать указатель для локальной или глобальной переменной. Чтобы в WebAssembly получить функциональность указателя в языке C, вы можете поместить глобальную переменную в определенное место в линейной памяти и использовать ее для установки или получения значения, хранящегося в линейной памяти WebAssembly, как показано в листинге 6.7.

Листинг 6.7. Имитация указателей в WAT

```
pointer.wat (module
  ❶ (memory 1)
  ❷ (global $pointer i32 (i32.const 128))
  ❸ (func $init
    ❹ (i32.store
      ❺ (global.get $pointer) ;; сохранить по адресу $pointer
      ❻ (i32.const 99)      ;; значение сохранено
    )
  )
  ❼ (func (export "get_ptr") (result i32)
    ❶ (i32.load (global.get $pointer)) ;; вернуть значение по адресу $pointer
  )
  ❶ (start $init)
)
```

Этот модуль создает одну страницу линейной памяти ❶ и глобальный указатель `$pointer` ❷, который указывает на ячейку памяти 128. Мы создаем функцию `$init` ❸ (на которую указывает `$pointer`), устанавливающую с помощью выражения `(i32.store)` ❹ значение ячейки памяти равным 99. Первый параметр, переданный в `(i32.store)` ❺, – место в памяти, где хранится значение, а второй параметр ❻ – это значение, которое вы хотите сохранить. Чтобы получить значение из этого местоположения указателя, используется выражение `i32.load` ❼, передаваемое в ячейку памяти, которую вы хотите получить. Чтобы получить это значение, мы создаем функцию `"get_ptr"` ❽. Затем оператор `(start $init)` ❾ вызывает функцию `$init` ❿. Оператор `start` объявляет ее функцией инициализации модуля. Она будет автоматически выполняться при запуске модуля.

После того как вы скомпилировали файл `pointer.wasm` и выполнили его с помощью `node`, вы должны увидеть следующее:

```
pointer_value=99
```

Объект памяти JavaScript

Теперь, когда у нас есть некоторое представление о том, как работает линейная память, мы создадим объект памяти `WebAssembly`, инициализируем данные внутри модуля `WebAssembly`, а затем получим доступ к этим данным из `JavaScript`. При работе с линейной памятью есть большая вероятность, что вы захотите получить к ней доступ из `WebAssembly` и среды встраивания. Так как в нашем случае средой встраивания является `JavaScript`, мы определим в ней линейную память, чтобы иметь к ней доступ до инициализации модуля `WebAssembly`. Этот модуль `WAT` должен выглядеть как в листинге 6.7, а импортировать линейную память будет из `JavaScript`.

Создание объекта памяти `WebAssembly`

Создайте файл с названием `store_data.wat` и добавьте туда код из листинга 6.8.

Листинг 6.8. Создание объекта линейной памяти в `WebAssembly`

```
store_data.wat (module
  ❶ (import "env" "mem" (memory 1))
  ❷ (global $data_addr (import "env" "data_addr") i32)
  ❸ (global $data_count (import "env" "data_count") i32)

  ❹ (func $store_data (param $index i32) (param $value i32)
    (i32.store
      (i32.add
        (global.get $data_addr) ;; добавьте $data_addr в $index*4 (i32=4 байтов)
```

```

        (i32.mul (i32.const 4) (local.get $index)) ;; умножить $index на 4)
    )
    (local.get $value) ;; значение сохранено
)
)

❶(func $init
    (local $index i32)

    ❷(loop $data_loop
        local.get $index

        local.get $index
        i32.const 5
        i32.mul

        ❸call $store_data ;; вызывается с параметрами $index и $index * 5

        local.get $index
        i32.const 1
        i32.add          ;; $index++

        local.tee $index
    ❹global.get $data_count
        i32.lt_u
        br_if $data_loop
    )

    ❺(call $store_data (i32.const 0) (i32.const 1))

)

❻(start $init)
)

```

В листинге 6.8 модуль импортирует свою линейную память ❶ из среды встраивания JavaScript, которую мы определим далее. Из JavaScript модуль импортирует адрес данных, которые мы загружаем в глобальную переменную \$data_addr ❷. Он также импортирует \$data_count ❸, содержащий некоторое количество целых чисел i32, которые мы сохраним при запуске модуля. Функция \$store_data ❹ берет индекс и значение и устанавливает местоположение данных (\$data_addr + \$index * 4) в \$value (мы умножаем на 4, потому что тип данных i32 содержит четыре байта). Использование импортированной глобальной переменной \$data_addr позволяет JavaScript выбирать место в модуле памяти для хранения этих значений.

Как и в листинге 6.6, функция \$init ❶ выполняется при инициализации модуля ввиду наличия оператора (start \$init) ❻. В отличие от предыдущей функции \$init, эта функция инициализирует данные в цикле ❷. Цикл может быть полезным способом инициализировать данные в определенных частях линейной памяти одним и тем же значением или некоторым значением, которое может быть вычислено

в цикле. Этот цикл устанавливает несколько 32-битных целых чисел на основе глобальной переменной `$data_count` ❸, которую модуль импортирует из JavaScript. Когда этот цикл вызывает `$store_data` ❹, он передает индекс, который представляет собой количество завершений `loop`, и значение, равное `$index * 5`. Так как я выбрал значение `$index * 5`, во время отображения данных вы увидите, как значения данных увеличиваются 5 раз.

После `loop` мы добавляем к `$store_data` еще один вызов ❺, чтобы установить первое значение данных в массиве равным 1. Если мы не инициализируем его значением, буфер памяти начинается со всеми данными, установленными в 0. Поскольку `loop` устанавливает первое значение данных равным 0, при просмотре данных в JavaScript будет не понятно, где начинаются данные набора. Установка равным 1 делает начало набора данных более очевидным, когда мы отображаем его из JavaScript в следующем разделе.

После того как вы закончили создание файла `store_data.wat`, скомпилируйте его с помощью `wat2wasm` в файл `store_data.wasm`.

Запись в консоль в цвете

Перед тем как написать `store_data.js`, давайте кратко рассмотрим `node`-модуль под названием `colors`, который позволяет вам выводить строки в консоль, используя выбранные вами цвета. В следующих разделах мы будем использовать этот пакет для упрощения просмотра различных результатов в наших выходных данных. Чтобы установить цвета, используйте команду `npm`, как показано в листинге 6.9.

Листинг 6.9. Применение `npm` для установки модуля `colors`

```
npm i colors
```

Теперь мы можем использовать в нашем приложении этот модуль, что позволяет нам изменить тип строки в JavaScript для элементов, которые задают цвета, полужирный текст и некоторые другие функции. Создайте файл с именем `colors.js` и добавьте туда код из листинга 6.10.

Листинг 6.10. Запись в консоль в цвете

```
colors.js colors = require('colors');

console.log('RED COLOR'.red.bold); // задает жирный красный цвет текста
console.log('blue color'.blue);    // задает синий цвет текста
```

После запуска `colors.js` с помощью `node`, как показано в листинге 6.11, записанный вывод будет отображаться в указанных нами цветах.

Листинг 6.11. Запуск `color.js` и вывод цветного текста в консоль

```
node colors.js
```

Теперь вы должны увидеть вывод как в листинге 6.12, где первая строка выделена красным, а вторая – синим.

Листинг 6.12. Результат работы модуля `colors`

```
RED COLOR
blue color
```

В будущих приложениях мы будем использовать этот модуль, чтобы улучшить внешний вид вывода в консоль. В книге красный вывод будет черным, но выделен жирным шрифтом. Давайте продолжим, создав файл `store_data.js`, как показано в следующем разделе.

Создание JavaScript в `store_data.js`

Теперь нам нужен JavaScript-файл `store_data.js` для выполнения модуля `store_data.wasm`. Введите код JavaScript, как в листинге 6.13.

Листинг 6.13. Буфер линейной памяти `WebAssembly` и `importObject` с глобальным импортом

```
store_data.js (часть 1 из 2)  const colors = require('colors'); // вывод цветного текста в консоль
                           const fs = require('fs');
                           const bytes = fs.readFileSync(__dirname + '/store_data.wasm');

                           // выделить блок памяти 64 КБ
                           ❶const memory = new WebAssembly.Memory({initial: 1 });
                           // 32-битное представление данных буфера памяти
                           ❷const mem_i32 = new Uint32Array(memory.buffer);

                           ❸const data_addr = 32; // адрес первого байта наших данных

                           // 32-битный индекс начала наших данных
                           ❹const data_i32_index = data_addr / 4;
                           ❺const data_count = 16; // количество 32-битных целых чисел
                           ❻const importObject = { // Объекты, импортируемые WASM из JavaScript
                               env: {
                                   mem: memory,
                                   data_addr: data_addr,
                                   data_count: data_count
                               }
                           };
                           ...
```

Мы создаем три константы, первая из которых создает новый объект `WebAssembly.Мемору` ❶, который мы будем использовать при иници-

циализации модуля WebAssembly. Константа `mem_i32` ❷ обеспечивает 32-битное целочисленное представление в буфере памяти. Крайне важно помнить, что это не копия данных в буфере, а особый способ взгляда на этот буфер как на массив 32-битных целых чисел без знака. Когда мы меняем значения в буфере памяти внутри модуля WebAssembly, мы можем использовать `mem_i32`, чтобы увидеть изменения в этих значениях. Константа `data_addr` ❸ – это расположение байта данных, которые мы установили в модуле WebAssembly. Это место является *байтовым индексом m (byte index)*, а не 32-битным целочисленным номером массива.

Поскольку 32-битное целое число состоит из четырех байт, нам нужен начальный индекс данных, который представляет собой константу `data_addr`, деленную на 4. Мы устанавливаем это значение в `data_i32_index` ❹. Затем у нас есть определенное количество 32-битных целочисленных значений, установленных в модуле, определенном как `const data_count` ❺. Последней `const` в этой части кода является `importObject` ❻. `importObject` содержит три импортированных объекта данных для модуля WebAssembly.

Последняя часть JavaScript в листинге 6.14 использует IIFE для реализации модуля WebAssembly и вывода значений из линейной памяти на консоль.

Листинг 6.14. Вывод значений данных внутри линейной памяти, после того как IIFE реализует модуль WebAssembly

```
store_data.js
(часть 2 из 2)  ...
                ( async () => {
❶ let obj = await WebAssembly.instantiate(new Uint8Array(bytes),
                                                importObject );

❷ for( let i = 0; i < data_i32_index + data_count + 4; i++ ) {
    let data = mem_i32[i];
    if (data !== 0) {
❸ console.log(`data[${i}]=${data}`.red.bold);
    }
    else {
❹ console.log(`data[${i}]=${data}`);
    }
  }
})();
```

Эта последняя часть JavaScript создает объект модуля `store_data.wasm` ❶, передавая `importObject`, который мы создали в листинге 6.13. После инициализации модуля WebAssembly данные в буфере памяти изменятся, потому что функция `$init` в коде WAT выполняется во время инициализации. Затем мы заикливаем ❷ по массиву `mem_i32`, начиная с первого адреса в буфере памяти и отображения четырех целых чисел после установки данных. Этот цикл отображает в браузере

значение `mem_i32`, записывая его в консоль красным цветом ❸, если значение не равно 0, а если равно – цветом консоли по умолчанию ❹.

Используйте команду `node` для запуска `store_data.js`; вы должны увидеть записанный в консоли вывод из листинга 6.15.

Листинг 6.15. Вывод данных

```
data[0]=0
data[1]=0
data[2]=0
data[3]=0
data[4]=0
data[5]=0
data[6]=0
data[7]=0
data[8]=1
data[9]=5
data[10]=10
data[11]=15
data[12]=20
data[13]=25
data[14]=30
data[15]=35
data[16]=40
data[17]=45
data[18]=50
data[19]=55
data[20]=60
data[21]=65
data[22]=70
data[23]=75
data[24]=0
data[25]=0
data[26]=0
data[27]=0
```

Первым элементом данных, который был установлен модулем `WebAssembly`, является `data[8]`, с которого начинается вывод красного цвета. Значение 8 – это значение в константе `data_i32_index`, которая составляет одну четвертую значения в `data_addr`. В коде задано 16 целых чисел, потому что мы установили для `const data_count` значение 16. В листинге 6.15 все нулевые элементы данных не были установлены в модуле `WebAssembly`. Вы можете видеть, что первые восемь чисел, а также последние четыре равны 0, и все они отображаются в цвете консоли по умолчанию.

Обнаружение столкновений

Ранее мы создавали объект буфера памяти внутри JavaScript, а инициализировали его из модуля `WebAssembly`. На этот раз мы инициа-

лизируем буфер памяти внутри JavaScript с помощью значений, сгенерированных в JavaScript. Мы также создадим более интересные структуры данных, которые будут обрабатывать наши данные об обнаружении столкновений. При изменении данных в буфере памяти WebAssembly нам следует сгруппировать данные в структуры, чтобы сделать их управляемыми. Мы создадим набор случайных определений окружностей в JavaScript, определяемых координатами x , y и радиусом. Затем JavaScript установит эти значения в буфер памяти WebAssembly. Чтобы организовать объекты в линейной памяти, нужно использовать комбинацию начального адреса, шага и сдвига.

Начальный адрес, шаг и сдвиг

При работе с линейной памятью внутри нашего модуля WebAssembly нам необходимо понимать структуры данных на низком уровне. В нашем JavaScript мы работаем с данными в линейной памяти как с типизированным массивом JavaScript. Внутри модуля WebAssembly линейная память больше похожа на кучу или большой массив байтов. Когда мы хотим создать массив структур данных, нам нужно знать *начальный адрес* (base address) этого массива, *шаг* (stride) – расстояние в байтах между каждой структурой и *сдвиг* (offset) любых элементов структуры (насколько глубоко в структуре мы сможем найти наш элемент).

Мы будем работать со структурой в нашей линейной памяти, которая имеет четыре элемента: координаты x и y , радиус и метку hit. Мы установим *единичный шаг* (unit stride), как показано на рис. 6.4. Это означает, что расстояние между каждой структурой в нашем массиве соответствует размеру структуры.

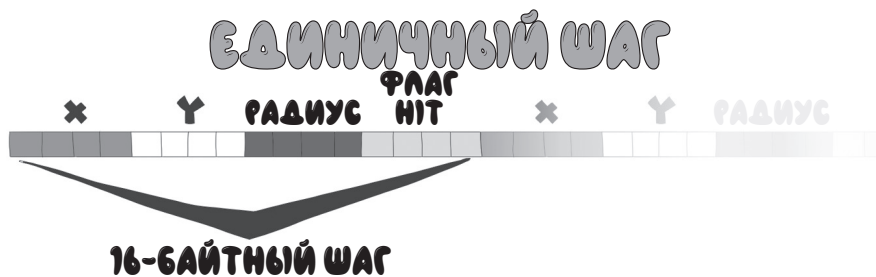


Рис. 6.4. Установка единичного шага

Чтобы получить адрес памяти конкретной структуры данных, к которой мы хотим получить доступ, мы умножаем индекс структуры на шаг и добавляем базовый адрес. *Начальный адрес* – это первоначальный адрес нашего массива структур.

В качестве альтернативы единичному шагу вы можете использовать отступ. Если разработчик решит выравнивать свои адреса структур по степени двойки, он может добавить неиспользуемые байты (называемые *заполнением* – padding) в конец своих структур. Например, если мы хотим, чтобы наша структура была выровнена по 16-байто-

вым адресам, мы могли бы добавить четыре байта заполнения в конец структуры, за счет чего мы получим шаг в 16 байт, как показано на рис. 6.5.



Рис. 6.5. Заполнение шага

Однако в этом примере нам не нужно заполнение. Каждый элемент нашей структуры данных имеет сдвиг. Например, предположим, что у нас есть два 32-битных целочисленных элемента *x* и *y*, которые являются первыми двумя элементами в структуре данных. Первый элемент *x* находится в начале структуры данных и поэтому имеет нулевой сдвиг 0. Поскольку элемент *x* является 32-битным целым числом, он занимает первые четыре байта структуры данных. Это означает, что сдвиг на 4 байта по *y* начинается с пятого байта (байты 0, 1, 2 и 3). Используя базовый адрес (начальный адрес нашей структуры данных), шаг и сдвиг для каждого элемента, мы можем построить массив структур данных.

Примечание Синтаксис для загрузки (*load*) и хранения (*store*) имеет модификатор сдвига (*offset*), который позволяет вам сдвигать адресный параметр на некоторое постоянное целочисленное значение. Это не совсем то же самое, что сдвиг элемента, о котором мы поговорим позже.

Загрузка структур данных из JavaScript

Начнем мы с создания файла JavaScript с именем *data_structures.js* для нового примера приложения. В этом приложении мы создадим структуры данных, которые представляют в памяти круги. Позже проведем проверку обнаружения столкновений между этими кругами. Добавьте код из листинга 6.16 в *data_structures.js*.

Листинг 6.16. Настройка констант для определения структуры программы обнаружения столкновений

```
data_structures.js (часть 1 из 3)
const colors = require('colors'); // включить цветной вывод в консоль
const fs = require('fs');
const bytes = fs.readFileSync(__dirname + '/data_structures.wasm');
// выделить в памяти блок 64 Кб
```

```

❶ const memory = new WebAssembly.Memory({initial: 1});

// 32-битовый блок буферной памяти
const mem_i32 = new Uint32Array(memory.buffer);

const obj_base_addr = 0; // адрес первого байта наших данных
❷ const obj_count = 32; // количество структур
❸ const obj_stride = 16; // 16-байтовый шаг
// сдвиги элементов структуры
❹ const x_offset = 0;
❺ const y_offset = 4;
const radius_offset = 8;
const collision_offset = 12;
// 32-битные целочисленные индексы
const obj_i32_base_index = obj_base_addr / 4; // 32-битный индекс данных
const obj_i32_stride = obj_stride / 4; // 32-битный шаг
// сдвиги в 32-битном целочисленном массиве
❻ const x_offset_i32 = x_offset / 4;
const y_offset_i32 = y_offset / 4;
const radius_offset_i32 = radius_offset / 4;
const collision_offset_i32 = collision_offset / 4;

❼ const importObject = { // Объекты, импортируемые WASM из JavaScript
  env: {
    mem: memory,
    obj_base_addr: obj_base_addr,
    obj_count: obj_count,
    obj_stride: obj_stride,
    x_offset: x_offset,
    y_offset: y_offset,
    radius_offset: radius_offset,
    collision_offset: collision_offset,
  }
};
...

```

Сначала мы создаем серию значений `const`, используемых для создания структур в буфере памяти `WebAssembly`. Как и в листинге 6.16, этот код создает одну страницу памяти `WebAssembly` размером 64 КБ ❶ и 32-битное целочисленное представление без знака в эти данные. Константа `obj_base_addr` устанавливает базовый адрес структур данных на 0, самый первый байт памяти на странице.

Мы делаем `obj_count` ❷ `const` равным количеству структур, установленных в этом коде. Константа `obj_stride` ❸ содержит количество байтов в структуре. Мы установили это значение равным 16, потому что в данной структуре есть четыре 32-битных целых числа, что составляет 16 байт. Следующая группа объявлений `const` содержит сдвиги элементов.

`x_offset` ❹ – это сдвиг `x` от начала структуры, как и количество байтов в каждой структуре в местоположении значения `x`. `y_offset` ❺ – это количество байтов структуры в местоположении значения `y`, равное 4, потому что значение `x` является 32-битным целым числом,

помещая значение `u` в пятый байт в структуре. Затем мы устанавливаем сдвиг для элемента `radius` ⑥ и элемента `collision` ⑦.

Мы вычисляем целочисленный индекс и шаг путем деления байтового адреса и шага на 4, потому что байтовый адрес и шаг – это количество байтов, а целочисленный индекс – это 32-битные целые числа (4 байта). Нам также нужно найти индексы в целочисленном массиве, который мы вычисляем, разделив байтовые индексы на 4 ⑥. `ImportObject` ⑦ был изменен и теперь включает новые значения `const`, которые мы добавили.

Определив константы, мы создадим несколько кругов произвольного размера для использования в программе. Ранее мы отметили, что круг определяется координатами `x`, `y` и радиусом. Мы случайным образом определим координаты `x` и `y` кругов со значениями от 0 до 99 и радиусом от 1 до 11. Код в листинге 6.17 перебирает объект памяти, устанавливая случайные значения в буфере памяти для каждой из структур.

Листинг 6.17. Инициализация кругов со случайными координатами `x`, `y` и радиусом

```
data_structures.js ...
(часть 2 из 3)  for( let i = 0; i < obj_count; i++ ) {
                ❶ let index = obj_i32_stride * i + obj_i32_base_index;
                ❷ let x = Math.floor( Math.random() * 100 );
                  let y = Math.floor( Math.random() * 100 );
                  let r = Math.ceil( Math.random() * 10 );

                ❸ mem_i32[index + x_offset_i32] = x;
                  mem_i32[index + y_offset_i32] = y;
                  mem_i32[index + radius_offset_i32] = r;
                }
                ...
```

Цикл получает индекс каждой из структур ❶ для обнаружения столкновений кругов. Значения `x`, `y` и радиуса `r` ❷ установлены как случайные. Эти случайные значения затем используются для задания значений памяти ❸ на основе индекса объекта и сдвига элементов.

Отображение результатов

Затем нам нужно создать объект модуля `data_structures.wasm`, который запускает функцию `$init`, выполняющую обнаружение столкновений между каждым из кругов, которые мы сгенерировали случайным образом. Дополните файл `data_structures.js` кодом из листинга 6.18.

Листинг 6.18. После запуска `WebAssembly` код перебирает линейную память в поисках столкновений кругов

```
data_structures.js ...
(часть 3 из 3)  ( async () => {
```

```

❶ let obj = await WebAssembly.instantiate(new Uint8Array(bytes),
                                         importObject );

❷ for( let i = 0; i < obj_count; i++ ) {
    ❸ let index = obj_i32_stride * i + obj_i32_base_index;

    ❹ let x = mem_i32[index+x_offset_i32].toString().padStart(2, ' ');
      let y = mem_i32[index+y_offset_i32].toString().padStart(2, ' ');
      let r = mem_i32[index+radius_offset_i32].toString()
        .padStart(2, ' ');
      let i_str = i.toString().padStart(2, '0');
      let c = !!mem_i32[index + collision_offset_i32];

      if (c) {
        ❺ console.log(`obj[${i_str}] x=${x} y=${y} r=${r} collision=${c}`
                      .red.bold);
      }
      else {
        ❻ console.log(`obj[${i_str}] x=${x} y=${y} r=${r} collision=${c}`
                      .green);
      }
    }
  }
})();

```

Функция IIFE реализует модуль WebAssembly ❶, а затем перебирает объекты в массиве mem_i32 ❷. Этот цикл получает индекс для структуры, используя шаг, индекс и значение начального индекса ❸. Затем мы используем индекс, который вычислили для получения значений x, y, радиуса и столкновения из массива mem_i32 ❹. Если столкновение произошло, эти значения записываются в консоль красным цветом ❺, а если нет, то зеленым цветом ❻.

Теперь у нас есть код JavaScript, загружающий серию структур, которые определяют круги с координатами x и y, со случайно выбранными значениями от 0 до 100. Каждый круг также имеет радиус со значением, случайным образом выбираемым от 1 до 10. Буфер памяти WebAssembly инициализируется с этими значениями. JavaScript установит соответствующие значения сдвига и шага в ImportObject. В дополнение к x, y и радиусу в каждой структуре есть четыре байта, предназначенных для хранения значения столкновения. Если происходит столкновение, это значение равно 1, а если нет, тогда равно 0. Вычисляет столкновения функция инициализации модуля WebAssembly (start). После инициализации модуля WebAssembly консоль отображает результаты этой проверки на столкновения. На данный момент мы еще не определили модуль WebAssembly. Давайте это сделаем.

Функция обнаружения столкновений

В предыдущих разделах, перед тем как передать код в JavaScript, мы определили его в WAT. В этом разделе мы сначала напишем JavaScript, так как он инициализирует значения, которые определяют круги

в массиве структур. Для обнаружения столкновения между двумя кругами нужно использовать теорему Пифагора, посредством которой можно определить, превышает ли сумма радиусов кругов расстояние между центрами этих кругов. Код WAT пройдет по каждому из кругов, которые мы определили в памяти WebAssembly, сравнивая их друг с другом, благодаря чему увидит, сталкиваются ли они. Здесь мы не будем вдаваться в подробности обнаружения столкновений. Мы используем это для того, чтобы показать, как можно разделить данные на структуры и использовать эти данные для выполнения вычислений с вашим кодом WAT.

Первая часть кода WAT определяет импорт из JavaScript. В листинге 6.19 показано начало модуля WAT.

Листинг 6.19. Импорт глобальных переменных, определяющих структуру данных

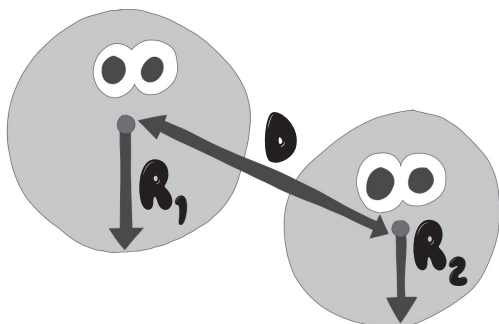
```
data_structures.wat (часть 1 из 6)
(module
  (import "env" "mem" (memory 1))
  ❶(global $obj_base_addr (import "env" "obj_base_addr") i32)
  ❷(global $obj_count (import "env" "obj_count") i32)
  ❸(global $obj_stride (import "env" "obj_stride") i32)

  ;; местоположения сдвига элементов
  ❹(global $x_offset (import "env" "x_offset") i32)
  ❺(global $y_offset (import "env" "y_offset") i32)
  ❻(global $radius_offset (import "env" "radius_offset") i32)
  ❼(global $collision_offset (import "env" "collision_offset") i32)
  ...
)
```

Глобальные переменные, передаваемые в модуль WebAssembly, определяют формат линейной памяти и структур данных в ней. Глобальная переменная `$obj_base_addr` ❶ – это место в памяти, где определены структуры круга. Глобальная переменная `$obj_count` ❷ – это количество кругов, определенных в линейной памяти. Глобальная переменная `$obj_stride` ❸ – это количество байтов между каждым определением круга. Затем мы импортируем значения для каждого из элементов. `$x_offset` ❹, `$y_offset` ❺, `$radius_offset` ❻ и `$collision_offset` ❼ – это количество байтов между началом значений x , y , радиуса и метки столкновения объекта. Они должны быть заданы внутри этого модуля.

Затем мы определим функцию `$collision_check`. Подробная информация о том, как работает эта функция, имеет ценность только в том случае, если вам интересно, как работает обнаружение столкновения кругов в деталях. Вкратце: функция использует теорему Пифагора, чтобы определить, меньше ли расстояние между двумя кругами, чем сумма радиусов круга. Давайте обозначим радиус первой окружности R_1 , радиус второй окружности R_2 и расстояние между окружностями D , как проиллюстрировано на рис. 6.6. Столкновения не происходит, если $R_1 + R_2$ меньше, чем D .

R_1 = РАДИУС КРУГА 1
 R_2 = РАДИУС КРУГА 2
 D = РАССТОЯНИЕ

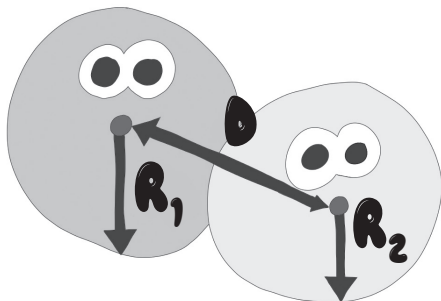


СТОЛКНОВЕНИЙ НЕТ

Рис. 6.6. Столкновения не происходит, если сумма $R_1 + R_2$ меньше, чем расстояние между кругами

Если расстояние меньше суммы $R_1 + R_2$, мы получим столкновение, как показано на рис. 6.7.

R_1 = РАДИУС КРУГА 1
 R_2 = РАДИУС КРУГА 2
 D = РАССТОЯНИЕ



СТОЛКНОВЕНИЕ ЕСТЬ

Рис. 6.7. Сумма $R_1 + R_2$ больше, чем расстояние между кругами

В листинге 6.20 приведен код функции `$collision_check`.

Листинг 6.20. Функция обнаружения столкновений WebAssembly

```
data_structures.wat ...
(часть 2 из 6) ❶(func $collision_check
    (param $x1 i32) (param $y1 i32) (param $r1 i32)
    (param $x2 i32) (param $y2 i32) (param $r2 i32)
    (result i32)

    (local $x_diff_sq i32)
    (local $y_diff_sq i32)
    (local $r_sum_sq i32)

    local.get $x1
    local.get $x2
    i32.sub
    local.tee $x_diff_sq
    local.get $x_diff_sq
    i32.mul
    ❷local.set $x_diff_sq ;; ($x1 - $x2) * ($x1 - $x2)

    local.get $y1
    local.get $y2
    i32.sub
    local.tee $y_diff_sq
    local.get $y_diff_sq
    i32.mul
    ❸local.set $y_diff_sq ;; ($y1 - $y2) * ($y1 - $y2)

    local.get $r1
    local.get $r2
    i32.add
    local.tee $r_sum_sq
    local.get $r_sum_sq
    i32.mul
    ❹local.tee $r_sum_sq ;; ($r1 + $r2) * ($r1 + $r2)

    local.get $x_diff_sq
    local.get $y_diff_sq
    ❺i32.add ;; Теорема Пифагора A в квадрате + B в квадрате = C в квадрате

    ❻i32.gt_u ;; если расстояние меньше суммы радиусов, возвращаем true
)
...
```

Эта функция получает элементы x , y и радиус двух кругов ❶, а затем возвращает 1, если они накладываются друг на друга, и 0, если нет. Сначала она находит расстояние x между двумя кругами, вычитая x_2 из x_1 , возводит это значение в квадрат и сохраняет его в `$x_diff_sq` ❷; затем она находит расстояние y между двумя окружностями, вычитая y_2 из y_1 . Результат вычитания возводится в квадрат и сохра-

няется в `$y_diff_sq` ❸. С помощью теоремы Пифагора мы можем построить уравнение $A^2 + B^2 = C^2$ ❹. В этом сценарии `$x_diff_sq` равно A^2 , а `$y_diff_sq` равно B^2 . Сумма этих двух значений равна C^2 , которая сравнивается с суммой квадратов радиусов ❺. Если квадрат радиуса больше, чем C^2 , круги накладываются друг на друга, и функция возвращает 1; в противном случае функция возвращает 0. Функция принимает это решение с помощью выражения `i32.gt_u` ❻.

После функции `$collision_check` нам понадобится несколько вспомогательных функций. Вспомогательная функция `$get_attr` принимает параметр начального адреса объекта и параметр сдвига элемента и возвращает значение в линейную память на место этого адреса. Эта функция представлена в листинге 6.21.

Листинг 6.21. Получение элемента объекта из линейной памяти

```
data_structures.wat ...
(часть 3 из 6) ❶ (func $get_attr (param $obj_base i32) (param $attr_offset i32)
    (result i32)
    local.get $obj_base
    local.get $attr_offset
    ❷ i32.add          ;; добавьте в начальный адрес сдвиг элемента
    ❸ i32.load         ;; загрузите адрес и верните его
    )
    ...
```

В определении функции ❶ параметр `$obj_base` – это начальный адрес объекта, а `$attr_offset` – сдвиг конкретного элемента, который мы хотим получить. Функция складывает эти значения ❷. Затем она загружает значение с этого адреса ❸, чтобы вернуть его в качестве результата.

Следующая вспомогательная функция – это `$set_collision`, которая устанавливает флаг столкновения для двух круговых объектов в значение `true`. Функция написана в листинге 6.22.

Листинг 6.22. Установка элемента столкновения для данного объекта

```
data_structures.wat ...
(часть 4 из 6) ❶ (func $set_collision
    (param $obj_base_1 i32) (param $obj_base_2 i32)
    local.get $obj_base_1
    global.get $collision_offset
    ❷ i32.add      ;; адрес = $obj_base_1 + $collision_offset
    i32.const 1
    ❸ i32.store   ;; сохранить 1 как истину в элементе столкновения для объекта

    local.get $obj_base_2
    global.get $collision_offset
    ❹ i32.add      ;; адрес = $obj_base_2 + $collision_offset
    i32.const 1
```

```

    ⑤ i32.store ;; сохранить 1 как истину в элементе столкновения для объекта
)
...

```

Эта функция принимает два базовых параметра объекта ①, чтобы установить метки столкновения для этих объектов в памяти. Это делается путем прибавления \$obj_base_1 к \$collision_offset ②, а затем устанавливается в этом месте линейной памяти значение 1 ③. Далее прибавляется \$obj_base_2 к \$collision_offset ④ и устанавливается значение в этом месте равным 1 ⑤.

Теперь, когда все функции определены, мы можем добавить функцию \$init в код WAT, как показано в листинге 6.23.

Листинг 6.23. Двойной цикл, который проверяет все объекты в линейной памяти на наличие столкновений

```

data_structures.wat ...
(часть 5 из 6) (func $init
  ① (local $i i32)      ;; счетчик внешнего цикла
    (local $i_obj i32) ;; адрес объекта i
    (local $xi i32)(local $yi i32)(local $ri i32) ;; x,y,r для объекта i

  ② (local $j i32)      ;; счетчик внутреннего цикла
    (local $j_obj i32) ;; адрес для объекта j
    (local $xj i32)(local $yj i32)(local $rj i32) ;; x,y,r для объекта j

    (loop $outer_loop
      (local.set $j (i32.const 0)) ;; $j = 0

      (loop $inner_loop
        (block $inner_continue
          ;; продолжить, если $i == $j
          ③ (br_if $inner_continue (i32.eq (local.get $i) (local.get $j) ) )
          ;; $i_obj = $obj_base_addr + $i * $obj_stride
          (i32.add (global.get $obj_base_addr)
            ④ (i32.mul (local.get $i) (global.get $obj_stride) ) )

          ;; load $i_obj + $x_offset and store in $xi
          ⑤ (call $get_attr (local.tee $i_obj) (global.get $x_offset) )
            local.set $xi

          ;; загрузить $i_obj + $y_offset и сохранить в $yi
          (call $get_attr (local.get $i_obj) (global.get $y_offset) )
            local.set $yi

          ;; загрузить $i_obj + $radius_offset и сохранить в $ri
          (call $get_attr (local.get $i_obj) (global.get $radius_offset) )
            local.set $ri

          ;; $j_obj = $obj_base_addr + $j * $obj_stride
          ⑥ (i32.add (global.get $obj_base_addr)
            (i32.mul (local.get $j)(global.get $obj_stride)))

```

```

;; загрузить $j_obj + $x_offset и сохранить в $xj
(call $get_attr (local.tee $j_obj) (global.get $x_offset) )
local.set $xj

;; загрузить $j_obj + $y_offset и сохранить в $yj
(call $get_attr (local.get $j_obj) (global.get $y_offset) )
local.set $yj

;; загрузить $j_obj + $radius_offset и сохранить в $rj
(call $get_attr (local.get $j_obj) (global.get $radius_offset) )
local.set $rj

;; проверить наличие столкновений между объектами i и j
❶(call $collision_check
  (local.get $xi)(local.get $yi)(local.get $ri)
  (local.get $xj)(local.get $yj)(local.get $rj))

  if ;; если столкновение произошло
  ❷(call $set_collision (local.get $i_obj) (local.get $j_obj))
  end
)

❸(i32.add (local.get $j) (i32.const 1)) ;; $j++

;; если $j < цикла $obj_count
(br_if $inner_loop
  (i32.lt_u (local.tee $j) (global.get $obj_count)))
)

❹(i32.add (local.get $i) (i32.const 1)) ;; $i++

;; если $i < цикла $obj_count
(br_if $outer_loop
  (i32.lt_u (local.tee $i) (global.get $obj_count) ) )
)
)
...

```

Функция начинается с двух групп локальных переменных. Одна группа содержит в себе счетчик, адрес объектов и локальные переменные *x*, *y* и *r* для использования во внешнем loop ❶. Вторая группа локальных переменных предназначена для использования во внутреннем loop ❷. Эта функция представляет собой двойной цикл, который сравнивает каждый круг с каждым другим кругом в линейной памяти в поисках столкновений между ними. В начале внутреннего цикла проверяется, совпадает ли значение *\$i* с *\$j* ❸. Если да, код пропускает проверку этого конкретного объекта *\$j*, потому что иначе получится, что каждый круг столкнулся сам с собой.

Следующая строка кода вычисляет в линейной памяти адрес объекта *i* ❹ посредством вычисления *\$obj_base_addr + \$i * \$obj_stride*. Затем мы задаем значение *\$i_obj* с помощью *local.tee* в выражении *(call \$get_attr)* ❺ в следующей строке. Вызов *\$getattr* ❺ извлекает значение *x* из объекта *i*, а затем устанавливает *\$xi*.

Следующие четыре строки загружают значения в `$yi` и `$ri` аналогичным образом. Затем `$xj`, `$yj` и `$rj` ⑥ также задаются с помощью вызова `$get_attr`. Эти значения передаются в вызов `$collision_check` ⑦, который возвращает 1, если круги `$i` и `$j` сталкиваются, и 0, если нет. Если произошло столкновение, приведенный ниже оператор `if` выполняет вызов `$set_collision` ⑧, который затем устанавливает метки столкновения на этих двух объектах равными 1. В конце цикла выполняется увеличение `$j` ⑨ и переход обратно к началу внутреннего цикла, если `$j` меньше `$obj_count`. В конце внешнего цикла происходит увеличение `$i` ⑩ и переход по условию обратно в начало внешнего цикла, если `$i` меньше `$obj_count`.

Последнее, что мы выполняем в этом модуле, – это оператор (`start $init`), как показано в листинге 6.24, где при инициализации модуля выполняется функция `$init`.

Листинг 6.24. `start` указывает на функцию, которая будет выполняться при инициализации модуля

```
data_structures.wat ...  
  (start $init)  
)
```

Теперь, когда мы подготовили весь код в файле `data_structures.wat`, мы можем скомпилировать файл WebAssembly с помощью `wat2wasm`, как показано в листинге 6.25.

Листинг 6.25. Компиляция файла `data_structures.wat`

```
wat2wasm data_structures.wat
```

Далее скомпилированный файл `data_structures.wasm` мы запустим как `data_structures.js` с помощью `node`, как показано в листинге 6.6.

Листинг 6.26. Запуск `data_structures.js`

```
node data_structures.js
```

Вы должны увидеть следующий вывод:

Листинг 6.27. Вывод из `data_structures.js`

```
obj[00] x=48 y=65 r= 4 collision=true  
obj[01] x=46 y=71 r= 6 collision=true  
obj[02] x=12 y=75 r= 3 collision=true  
obj[03] x=54 y=43 r= 2 collision=false  
obj[04] x=16 y= 6 r= 1 collision=false  
obj[05] x= 5 y=21 r= 9 collision=true  
obj[06] x=71 y=50 r= 5 collision=false
```

```
obj[07] x=11 y=13 r= 5 collision=true
obj[08] x=43 y=70 r= 7 collision=true
obj[09] x=88 y=60 r= 9 collision=false
obj[10] x=96 y=21 r= 9 collision=true
obj[11] x= 5 y=87 r= 2 collision=true
obj[12] x=64 y=39 r= 3 collision=false
obj[13] x=75 y=74 r= 6 collision=true
obj[14] x= 2 y=74 r= 8 collision=true
obj[15] x=12 y=85 r= 7 collision=true
obj[16] x=60 y=27 r= 5 collision=false
obj[17] x=43 y=67 r= 2 collision=true
obj[18] x=38 y=53 r= 3 collision=false
obj[19] x=34 y=39 r= 5 collision=false
obj[20] x=42 y=62 r= 2 collision=true
obj[21] x=72 y=93 r= 7 collision=false
obj[22] x=78 y=79 r= 8 collision=true
obj[23] x=50 y=96 r= 7 collision=false
obj[24] x=34 y=18 r=10 collision=true
obj[25] x=19 y=44 r= 8 collision=false
obj[26] x=92 y=82 r= 7 collision=true
obj[27] x=59 y=56 r= 3 collision=false
obj[28] x=41 y=75 r= 9 collision=true
obj[29] x=28 y=29 r= 6 collision=true
obj[30] x=32 y=10 r= 1 collision=true
obj[31] x=83 y=15 r= 6 collision=true
```

В этом выводе строка, в которой происходит столкновение любых кругов, должна быть выделена красным цветом, а строки, в которых не обнаружено столкновение, должны быть зеленого цвета. Теперь у нас есть приложение, которое использует JavaScript для загрузки случайно сгенерированных данных для кругов в линейную память WebAssembly. Затем функция инициализации в модуле WebAssembly перебирает все эти данные и обновляет линейную память всякий раз, когда один из этих кругов сталкивается с другим кругом. Обнаружение столкновений – отличный вариант использования WebAssembly, поскольку оно позволяет загружать большой объем данных в линейную память и дает возможность вашему модулю WebAssembly работать быстро и эффективно.

Заключение

В этой главе вы узнали, что такое линейная память WebAssembly и как ее создать из модуля WebAssembly или JavaScript. Мы инициализировали линейные данные из модуля WebAssembly и получили к ним доступ из JavaScript. Затем создали структуры данных в линейной памяти, используя смещения начального адреса, шага и элементов, и инициализировали эти структуры данных из JavaScript, используя случайные данные.

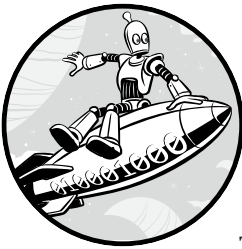
Последнее, что мы сделали, – создали массив структур данных кругов с координатами x , y и радиусом. Они были переданы в модуль Web-

Assembly, который использовал двойной цикл для обхода структур данных круга в поисках накладывающихся друг на друга кругов. Обнаружив такие круги, модуль WebAssembly устанавливал метку столкновения внутри линейной памяти для обоих кругов. Затем JavaScript перебирал все эти круги, отображая их координаты x и y , их радиус, и проверял, столкнулись ли они с какими-либо другими кругами.

На этом этапе вы уже должны понимать, как использовать линейную память из WAT и JavaScript. Вы также должны уметь использовать линейную память в своих приложениях для создания структур данных и обработки больших объемов данных в WebAssembly, которые затем можно отображать в JavaScript. В следующей главе мы рассмотрим, как управлять объектной моделью документа (DOM – Document Object Model) из WebAssembly.

7

ВЕБ-ПРИЛОЖЕНИЯ



Эта глава поможет вам понять, как WebAssembly взаимодействует с DOM через JavaScript. Хотя это может показаться утомительным занятием, детальное изучение WebAssembly, его сильных и слабых сторон – неизбежный труд. Если вы используете набор инструментов WebAssembly, вам необходимо знать, сколько дополнительного кода этот набор сгенерирует в виде связанного кода JavaScript. С этого момента большинство примеров будут запускаться с веб-страницы вместо использования `node` из командной строки.

Мы начнем с создания простого статического веб-сервера с использованием Node.js. Веб-приложения WebAssembly нельзя загружать в браузер непосредственно из файловой системы; вместо этого они требуют, чтобы вы запустили веб-сервер. Node.js предоставляет все инструменты, необходимые для создания веб-сервера. Затем мы создадим наше первое веб-приложение WebAssembly.

Второе веб-приложение, которое мы разработаем, повторно использует функции, представленные в главе 5, чтобы получать число из элемента ввода HTML и передавать его в WebAssembly, который преобразует число в десятичную, шестнадцатеричную и двоичную строки.

К концу этой главы вы изучите разработку веб-приложений, которые загружают и запускают модуль WebAssembly, а затем вызывают функции из этого модуля. Приложения также будут записывать данные из этих модулей в элементы DOM. Примеры в этой главе не являются законченными приложениями, которые вы обычно пишете с помощью WebAssembly. Они только демонстрируют, как веб-страница может загружаться, запускаться и взаимодействовать с модулями WebAssembly.

DOM

Современные веб-приложения настолько сложны, что легко забыть, что HTML-страница по своей сути представляет собой простой документ. Изначально сеть задумывалась как средство обмена документами и информацией, но вскоре стало очевидно, что нам нужен стандартный метод для динамического обновления этих документов с использованием такого языка, как JavaScript или Java. DOM был разработан как не зависящий от языка интерфейс для работы с документами HTML и XML. Поскольку документ HTML представляет собой древовидную структуру, DOM представляет документ как логическое дерево. DOM – это объектная модель документа, при помощи которой JavaScript и другие языки изменяют HTML в веб-приложении.

В WebAssembly версии 1.0 нет средств прямого управления DOM, поэтому JavaScript должен вносить все изменения в HTML-документе. Если вы используете набор инструментов, такой как Rust или Emscripten, операции с DOM обычно выполняются из связанного кода JavaScript. Как правило, в веб-приложении WebAssembly должен быть сосредоточен на работе с числовыми данными, но применительно к DOM большая часть обработки данных, вероятно, будет относиться к операциям со строками. Производительность обработки строк в WebAssembly полностью зависит от библиотеки, которую вы используете для задачи. По этой причине тяжелую работу с DOM лучше выполнять в части JavaScript приложения.

Создание и настройка простого сервера Node

Чтобы создать и настроить статический веб-сервер с помощью Node.js, создайте папку для своего проекта и откройте ее в VS Code или в IDE. Нам нужно установить два пакета с помощью npm. Установите первый пакет connect, используя команду из листинга 7.1.

Листинг 7.1. Установка пакета подключения с помощью npm

```
npm install connect --save-dev
```

Установите второй пакет serve-static с помощью команды из листинга 7.2.

Листинг 7.2. Установка *serve-static* с помощью *npm*

```
npm install serve-static --save-dev
```

После установки пакетов создайте файл с именем *server.js* и для определения статического веб-сервера введите код, приведенный в листинге 7.3.

Листинг 7.3. Код HTTP-сервера *Node.js*

```
server.js var connect = require('connect');
var serveStatic = require('serve-static');
connect().use(serveStatic(__dirname + "/")).listen(8080, function(){
  console.log('localhost:8080');
});
```

Мы создали статический сервер, который предоставляет файлы из текущего каталога, но пока что у нас нет таких файлов. Воспользуйтесь редактором VS Code, чтобы создать файл с именем *index.html*, и введите код HTML-страницы, как в листинге 7.4.

Листинг 7.4. Простая веб-страница

```
index.html <html>
  <head></head>
  <body>
    <h1>OUR SERVER WORKS!</h1>
  </body>
</html>
```

Теперь вы можете запустить свой веб-сервер *Node.js* при помощи следующей команды:

```
node server.js
```

Веб-сервер запускается через порт 8080. Чтобы проверить его работоспособность, введите *localhost: 8080* в адресную строку своего браузера; вы должны увидеть страницу, показанную на рис. 7.1.

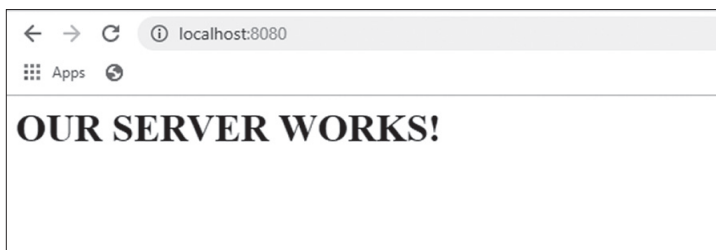


Рис. 7.1. Тестирование простого статического сервера

Теперь, когда у нас есть работающий веб-сервер Node.js, давайте создадим первое веб-приложение WebAssembly.

Первое веб-приложение WebAssembly

Мы начнем с простого веб-приложения, которое получает два числа, складывает их, а затем отображает все эти значения. Готовая версия этого приложения доступна по адресу https://wasmbok.com/add_message.html.

Это приложение демонстрирует, как WebAssembly взаимодействует с DOM. Можно заметить, что мы не меняем способ работы модуля WebAssembly, а вместо этого меняем среду встраивания, в то время как WebAssembly не изменяется.

Чтобы создать веб-приложение, мы должны запустить веб-сервер, написать для HTML-страницы код JavaScript, который будет взаимодействовать с DOM, и загрузить модуль WebAssembly с помощью функции `instantiateStreaming` (вместо `instantiate`, как мы делали в предыдущих главах). Мы определим модуль WebAssembly, который складывает два целых числа вместе, и файл HTML, который загружает и запускает этот модуль WebAssembly. В листинге 1.8 JavaScript запускал функцию `AddInt` с помощью Node.js для загрузки и выполнения модуля WebAssembly. В этом приложении код JavaScript будет находиться в HTML-файле, и для запуска приложения потребуется браузер.

В листинге 7.5 показан WAT-модуль, реализующий функцию сложения чисел. Создайте файл `add_message.wat` и добавьте код из листинга 7.5.

Листинг 7.5. Добавление двух чисел в файл `add_message.wat` и вызов JavaScript-функции записи в консоль

```
add_message.wat (module
  ❶(import "env" "log_add_message"
    (func $log_add_message (param i32 i32 i32)))

  ❷(func (export "add_message")
    ❸(param $a i32) (param $b i32)
      (local $sum i32)

      local.get $a
      local.get $b
      ❹i32.add
      local.set $sum

    ❺(call $log_add_message
      ❻(local.get $a) (local.get $b) (local.get $sum))
  )
)
```

Сейчас этот модуль WAT должен показаться вам очень знакомым. Он импортирует `log_add_message` ❶ из JavaScript и определяет функ-

цию `add_message` ❷, которая будет экспортирована в среду встраивания. Он также принимает два параметра `i32` ❸. Эти два параметра складываются, и результат сохраняется в локальной переменной `$sum`. Затем модуль вызывает функцию JavaScript `log_add_message` ❹, передавая параметры `$a` и `$b`, а также `$sum` ❺ (сумму этих двух параметров).

Наверное, вам интересно, как WebAssembly взаимодействует с DOM? К сожалению, правда в том, что WebAssembly 1.0 не взаимодействует напрямую с DOM. Чтобы взаимодействовать с DOM, он должен работать со средой встраивания (JavaScript). Все различия между вызовом модуля WebAssembly из Node.js и веб-страницы будут заключаться в среде встраивания. Модуль WebAssembly может выполнять только вызовы функций среды встраивания. Мы создадим функции JavaScript внутри HTML-страницы. Модуль WebAssembly вызовет эти функции JavaScript, которые, в свою очередь, обновят DOM. Скомпилируйте файл `add_message.wat` с помощью `wat2wasm`.

Определение HTML-заголовка

Теперь мы создадим HTML-страницу. Ранее мы использовали Node.js в качестве среды встраивания, работая непосредственно в JavaScript, но для статического веб-сайта вам понадобится HTML-страница. Веб-браузер не выполняет JavaScript напрямую так же, как Node.js. Веб-браузеры загружают HTML-страницы, в которых JavaScript встроен через теги `<script>`. Я предполагаю, что вы знакомы с основами HTML, но если нет, этот код в любом случае будет простым. Создайте новый файл `add_message.html` и вставьте в него код из листинга 7.6.

Листинг 7.6. Заголовок страницы для приложения `add_message` в основном совпадает со стандартным шаблоном HTML

```
add_message.html <!DOCTYPE html>
(часть 1 из 3)  <html lang="en">
                <head>
                  <meta charset="UTF-8">
                  <meta name="viewport"
                    content="width=device-width, initial-scale=1.0">
                  <title>Add Message</title>
                ...
```

Здесь вы видите открывающий HTML-тег и информацию в заголовке. Он просто устанавливает конфигурацию шрифта и отображает имя приложения Add Message в качестве заголовка страницы.

JavaScript

Перед завершением элемента `<head>` мы ставим тег `<script>`, за которым следует код JavaScript. Подобно тому, как мы использовали

Node.js, для создания и выполнения функций в модуле WebAssembly требуется код JavaScript. HTML-страница использует тег `<script>` для хранения этого кода, как показано в листинге 7.7.

Листинг 7.7. JavaScript, загружающий модуль WebAssembly внутри тега `<script>`

```
add_message.html ...
(часть 2 из 3)
<script>
  ❶ //const sleep = m => new Promise(r => setTimeout(r, m));
    var output = null;
    var add_message_function;

  ❷ var log_add_message = (a, b, sum) => {
    if (output == null) {
      console.log("page load not complete: log_add_message");
      return;
    }
    ❸ output.innerHTML += `${a} + ${b} = ${sum}<br>`;
  };

  let importObject = {
    env: {
      ❹ log_add_message: log_add_message,
    }
  };

  (async () => {
    ❺ // await sleep(5000);
    let obj = await
    ❻ WebAssembly.instantiateStreaming(fetch('add_message.wasm'),
                                     importObject);
    add_message_function = obj.instance.exports.add_message;
    ❼ let btn = document.getElementById("add_message_button");
    btn.style.display = "block";
  })();

  ❽ function onPageLoad() {
    //(async () => {
    ❾ //await sleep(5000);
    ❿ output = document.getElementById("output");
    //})();
  })
</script>
...
```

При создании веб-страницы нам необходимо знать, когда все элементы веб-страницы завершили загрузку, и время, необходимое для потоковой передачи и запуска модуля WebAssembly.

Это приложение записывает сообщения в вывод тега абзаца `output`. При выполнении JavaScript тег абзаца еще не загружен, потому что

он находится ниже по HTML-странице. Модуль WebAssembly будет транслироваться и загружаться асинхронно, поэтому вы не можете быть уверены, в какой момент модуль WebAssembly будет готов – до или после завершения загрузки страницы.

Чтобы проверить, будет ли эта функция работать независимо от того, в каком порядке происходят события, сначала мы создаем функцию `sleep` ❶, чтобы заставить JavaScript ждать. Эта функция закомментирована в коде. Чтобы проверить порядок загрузки, раскомментируйте `sleep` как внутри IIFE, так и в функции `onPageLoad`.

Мы создаем переменную `add_message_function` в качестве заполнителя, который будет указывать на функцию `add_message` внутри нашего модуля WebAssembly при его запуске.

Затем мы определяем `log_add_message` ❷, содержащую стрелочную функцию, которая проверяет, установлено ли для `output` значение, отличное от `null`. Значение для `output` по умолчанию – `null`, но как только страница загружается, `output` устанавливается в элемент абзаца с идентификатором вывода (`output id`); поэтому если функция запускается до завершения загрузки страницы, она будет записывать сообщение в лог. Функция `log_add_message` ❸ импортируется и вызывается из модуля WebAssembly, который передает в `log_add_message` два числа для сложения и сумму этих чисел. Затем функция записывает эти значения в `output` ❹ HTML-тег абзаца из листинга 7.8.

В IIFE функция `sleep` ❺ закомментирована, но вы можете раскомментировать ее для тестирования. Однако для извлечения модуля при загрузке модуля WebAssembly с веб-страницы нужно использовать `WebAssembly.instantiateStreaming` ❻ в сочетании с вызовом `fetch`. После создания экземпляра модуля элемент `add_message_button` ❼ извлекается из DOM и отображается, когда мы устанавливаем для элемента `block` значение атрибута `style.display`. Теперь пользователь сможет нажать эту кнопку, чтобы запустить функцию `WebAssembly`.

Примечание *К сожалению, в Safari 14.0 не реализован `instantiateStreaming`, поэтому вам нужно будет заполнить `WebAssembly.instantiateStreaming` с помощью `WebAssembly.instantiate`.*

Кроме того, мы определяем функцию `onPageLoad` ❹, которая выполняется после завершения загрузки блока HTML `body`. Эта функция присваивает переменной `output` ❽, определенной в начале листинга 7.7, идентификатор тега абзаца `output`. Перед загрузкой страницы эта переменная имеет значение `null`. Если функция, для которой требуется тег `output`, выполняется до того, как страница завершит загрузку, она будет проверять значение `null` перед ее использованием. Это предотвращает попытку кода использовать тег абзаца до его загрузки. Мы включили дополнительную функцию `sleep` ❾, которую можно использовать для задержки установки переменной `output`. Это позволяет нам смоделировать ситуацию, когда страница загружается дольше, чем ожидается.

HTML-тег <body>

HTML-тег <body> содержит элементы DOM, которые будут отображаться на нашей веб-странице. Добавьте код из листинга 7.8 в `add_message.html` после тега `script`.

Листинг 7.8. Элементы DOM в HTML-теге <body>

```
add_message.html ...  
(часть 3 из 3) </head>  
❶ <body onload="onPageLoad()" style="font-family: 'Courier New', Courier, monospace;">  
❷ <input type="number" id="a_val" value="0"><br><br>  
❸ <input type="number" id="b_val" value="0"><br><br>  
❹ <button id="add_message_button" type="button" style="display:none"  
❺ onclick="add_message_function(  
    document.getElementById('a_val').value,  
    document.getElementById('b_val').value )">  
    Add Values  
</button>  
<br>  
❻ <p id="output" style="float:left; width:200px; min-height:300px;">  
    </p>  
</body>  
</html>
```

Тег <body> ❶ содержит элемент `onload`, который вызывает функцию JavaScript `onPageLoad`. Это гарантирует, что переменная `output` в нашем JavaScript не будет установлена до тех пор, пока не будет создан тег абзаца `output`.

Еще у нас есть два элемента `input` с идентификаторами `a_val` ❷ и `b_val` ❸. Значения этих входных данных передаются в `WebAssembly`, когда нажимается элемент `button` ❹. Обработчик нажатия кнопки `onclick` ❺ настроен на вызов функции `add_message_function`, которая вызывает функцию `add_message` в модуле `WebAssembly` после его запуска. Функция `add_message` передает значения, находящиеся в двух полях ввода (`a_val` и `b_val`) над кнопкой. Кроме того, у нас есть тег абзаца с идентификатором `output` ❻, который мы заполним значениями из модуля `WebAssembly`.

Готовое веб-приложение

Теперь мы можем запустить наше веб-приложение. Как упоминалось ранее, браузер должен получить страницу с веб-сервера, поэтому с помощью команды в листинге 7.9 сначала убедитесь, что веб-сервер, созданный в листинге 7.3, запущен.

Листинг 7.9. Запуск простого веб-сервера

```
node server.js
```

Если вы получили сообщение об ошибке, как в листинге 7.10, ваш веб-сервер уже работает на этом порту.

Листинг 7.10. Ошибка веб-сервера, если порт уже используется

```
Error: listen EADDRINUSE: address already in use :::8080
```

Эта ошибка, скорее всего, означает, что вы уже запустили `server.js` другой командой в терминале командной строки. Когда веб-сервер заработает, откройте в браузере следующий URL-адрес: `http://localhost:8080/add_message.html`.

Вы должны увидеть веб-страницу, изображенную на рис. 7.2.

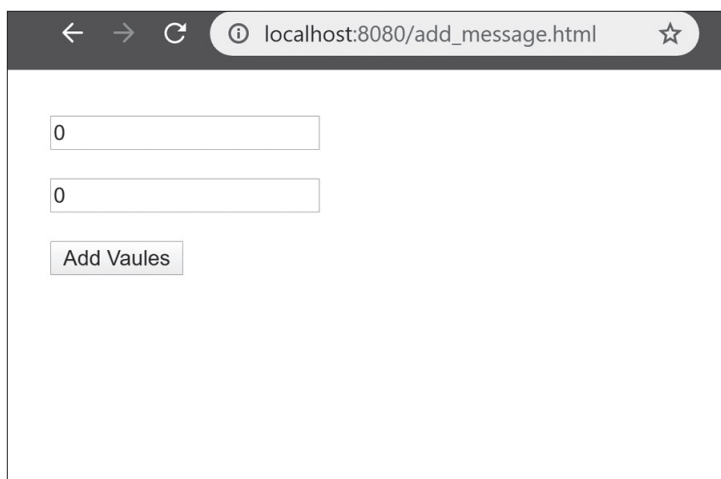


Рис. 7.2. Веб-приложение add_message.html

Задайте значения в двух числовых полях и нажмите **Add Values** (Сложить значения), чтобы увидеть результаты (рис. 7.3).

Обратите внимание, что, как в других главах, модуль WebAssembly вызывает функции JavaScript. В этой главе вам не нужно было изучать какие-либо новые команды WAT. Поскольку работать с DOM напрямую из Wasm 1.0 невозможно, мы внесли все изменения в DOM при помощи JavaScript. Несмотря на то что это был первый раз, когда мы использовали HTML-страницу, это не повлияло на работу WebAssembly. WebAssembly 1.0 довольно ограничен в выборе функциональных возможностей и наиболее полезен для повышения производительности приложений, требующих большого количества математических вычислений. В следующих версиях WebAssembly будут добавлены новые функции. Но пока вам нужно помнить об этих ограничениях при выборе приложения, лучше всего подходящего для этой новой технологии.

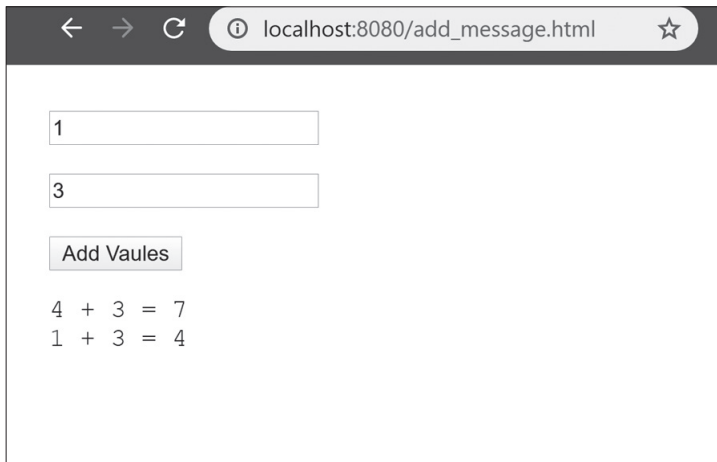


Рис. 7.3. Два дополнительных сообщения, добавленных в приложение

Шестнадцатеричные и двоичные строки

Опираясь на предыдущий пример, мы создадим второе приложение, которое будет использовать наши функции из главы 5 для преобразования числовых данных в десятичные, шестнадцатеричные и двоичные строки и отображения их на веб-странице. Окончательную версию приложения вы можете увидеть по адресу https://wasmbbook.com/hex_and_binary.html.

HTML

HTML почти такой же, как в листинге 7.6, но с другим содержанием `title`. Создайте файл с именем `hex_and_binary.html` и вставьте в него код из листинга 7.11.

Листинг 7.11. Начало файла `hex_and_binary.html`

```
hex_and
_binary.html <!DOCTYPE html>
(часть 1 из 3) <html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title> ❶ Hex and Binary</title>
...

```

Теперь название страницы (тег `title`) другое – Hex and Binary ❶. Затем, как показано в листинге 7.12, мы добавляем тег `script` и код JavaScript, который будет создавать экземпляр и вызывать модуль WebAssembly.

```

hex_and
_binary.html ...
(часть 2 из 3) <script>
    // выделить блок памяти 64 КБ
    const memory = new WebAssembly.Memory({ initial: 1 });
    var output = null;

    // функция изменится при запуске модуля WebAssembly
    ❶ var setOutput = (number) => {
        // это сообщение появится, если вы запустите функцию
        // до того, как будет создан экземпляр WebAssembly.
        ❷ console.log("function not available");
        return 0;
    };

    // Эта функция будет вызываться нажатием кнопки и запускать
    // функцию setOutput в модуле WebAssembly
    ❸ function setNumbers(number) {
        ❹ if (output == null) {
            // если страница загружена не полностью, вернуться
            return;
        }

        // вызов функции WebAssembly setOutput генерирует строку HTML
        // и помещает ее в линейную память, возвращая ее длину
        ❺ let len = setOutput(number);

        // мы знаем позицию и длину HTML-строки в линейной памяти,
        // поэтому можем извлечь ее из буфера памяти
        ❻ let bytes = new Uint8Array(memory.buffer, 1024, len);

        // преобразовать байты, взятые из линейной памяти, в
        // строку JavaScript и использовать их в выводе HTML
        ❼ output.innerHTML = new TextDecoder('utf8').decode(bytes);
    }

    ❽ function onPageLoad() {
        // по завершении загрузки страницы присвойте переменной output
        // значение элемента с id "output"
        ❾ output = document.getElementById("output");
        var message_num = 0;
    }

    let importObject = {
        env: {
            buffer: memory
        }
    };

    (async () => {
        // использовать WebAssembly.instantiateStreaming вместе с
        // fetch вместо WebAssembly.instantiate и fs.readFileSync
        let obj = await WebAssembly.instantiateStreaming(

```

```

        fetch('hex_and_binary.wasm'),
        importObject);
    // сбросить переменную setOutput до значения функции setOutput
    // из модуля WASM
    ⑩ setOutput = obj.instance.exports.setOutput;
        let btn = document.getElementById("set_numbers_button");
        btn.style.display = "block";
    })();

</script>
</head>
...

```

Внутри тега `script` мы создаем переменную `setOutput` ① и устанавливаем для нее стрелочную функцию, которая выводит в консоль текст "function not available" ②. Это сообщение отобразится, если пользователь нажмет кнопку **Set Numbers** до того, как модуль WebAssembly завершит загрузку.

Затем мы определяем функцию `setNumbers` ③, которая будет вызываться, когда пользователь нажимает кнопку **Set Numbers**. Если загрузка страницы не завершена, при нажатии кнопки выходное значение останется равным `null` ④, и мы вернемся из этой функции. Функция `setNumbers` вызывает `setOutput` ⑤ в модуле WebAssembly, который создает HTML-строку из переданного числа и возвращает длину этой строки, которую мы будем использовать для извлечения из линейной памяти. Мы берем байты, которые будут использоваться для создания отображаемой строки из линейной памяти `buffer` ⑥.

Затем атрибут тега `output` `innerHTML` ⑦ настраивается на отображение строки, сгенерированной из этих байтов `bytes` с помощью объекта `TextDecoder`, который отображает строку на веб-странице.

Мы определяем функцию `onPageLoad` ⑧, которую тег `body` выполняет после завершения загрузки. Эта функция задает значение переменной `output` ⑨, используемой для отображения строки вывода из модуля WebAssembly. Он также запускает модуль WebAssembly и присваивает переменной `setOutput` ⑩ значение функции `setOutput` в модуле WebAssembly, поэтому мы можем вызвать ее из JavaScript.

Наконец, нам понадобится тег `body`, который содержит тег `output` для отображения вывода вызываемой функции WebAssembly, числового поля `input`, которое будет заполнять пользователь, и кнопки `button`, на которую нужно нажать, чтобы вызвать функцию `setNumbers`. В листинге 7.13 показан этот код.

Листинг 7.13. Элементы пользовательского интерфейса HTML-страницы

```

hex_and
_binary.html
(часть 3 из 3) ① ...
    <!-- Тег body вызывает onPageLoad, когда загрузка body завершена -->
    <body onload="onPageLoad()"
        style="font-family: 'Courier New', Courier, monospace;">

```

```

❷<div id="output"><!-- отображает вывод из WebAssembly -->
    <h1>0</h1>
    <h4>0x0</h4>
    <h4> 0000 0000 0000 0000 0000 0000 0000 0000</h4>
</div>
<br>
<!--пользователь вводит число для преобразования в шестнадцатеричный
и двоичный формат-->
❸<input type="number" id="val" value="0"><br><br>
<!--когда пользователь нажимает кнопку, запускается функция WASM -->
❹<button id="set_numbers_button" type="button" style="display:none"
    ❺onclick="setNumbers( document.getElementById('val').value )">
    Set Numbers
</button>
</body>
</html>

```

Элемент `onload` ❶ просит браузер выполнить `onPageLoad` после завершения загрузки `body`. Тег ❷ `<div id = "output">` – это место, где будет отображаться вывод модуля `WebAssembly`. Тег числа `input` ❸, `<input type = "number" id = "val" value = "0">` – это то место, где пользователь вводит число для преобразования в шестнадцатеричное и двоичное. `button` ❹ вызывает модуль `WebAssembly` при нажатии с помощью элемента `onclick` ❺. Теперь, когда у нас готова HTML-страница, мы можем создать WAT-файл для этого приложения.

WAT

В этом приложении много WAT-кода, поэтому мы разделим его на четыре части. Кроме того, вам нужно будет скопировать несколько функций из главы 5. Создайте файл с именем `hex_and_binary.wat` и добавьте туда код из листинга 7.14.

Листинг 7.14. Определения строковых данных в начале модуля

```

hex_and (module
_binary.wat (import "env" "buffer" (memory 1))
(часть 1 из 4)
    ;; шестнадцатеричные цифры
    ❶ (global $digit_ptr i32 (i32.const 128))
        (data (i32.const 128) "0123456789ABCDEF")
        ;; указатель десятичной строки, раздел длины и данных
    ❷ (global $dec_string_ptr i32 (i32.const 256))
        (global $dec_string_len i32 (i32.const 16))
        (data (i32.const 256) " 0")

        ;; указатель шестнадцатеричной строки, раздел длины и данных
    ❸ (global $hex_string_ptr i32 (i32.const 384))
        (global $hex_string_len i32 (i32.const 16))
        (data (i32.const 384) " 0x0")

        ;; указатель двоичной строки, раздел длины и данных

```

```

❶ (global $bin_string_ptr i32 (i32.const 512))
(global $bin_string_len i32 (i32.const 40))
(data (i32.const 512) " 0000 0000 0000 0000 0000 0000 0000 0000")

;; указатель строки открывающего тега h1, раздел длины и данных
❷ (global $h1_open_ptr i32 (i32.const 640))
(global $h1_open_len i32 (i32.const 4))
(data (i32.const 640) "<H1>")

;; указатель строки закрывающего тега h1, раздел длины и данных
❸ (global $h1_close_ptr i32 (i32.const 656))
(global $h1_close_len i32 (i32.const 5))
(data (i32.const 656) "</H1>")

;; указатель строки открывающего тега h4, раздел длины и данных
❹ (global $h4_open_ptr i32 (i32.const 672))
(global $h4_open_len i32 (i32.const 4))
(data (i32.const 672) "<H4>")

;; указатель строки закрывающего тега h4, раздел длины и данных
❺ (global $h4_close_ptr i32 (i32.const 688))
(global $h4_close_len i32 (i32.const 5))
(data (i32.const 688) "</H4>")

;; длина выводимой строки и раздел данных
❻ (global $out_str_ptr i32 (i32.const 1024))
(global $out_str_len (mut i32) (i32.const 0))

...

```

Мы определяем последовательности разделов данных, указателей и длин данных, которые будут использоваться для сборки десятичных, шестнадцатеричных и двоичных строк из целочисленных данных. Глобальная переменная `$digit_ptr` ❶ является указателем на сегмент данных, который содержит 16 шестнадцатеричных цифр от 0 до F, определенных в ячейке линейной памяти 128. Эти данные используются для всех трех преобразований из целого числа в строку. У нас также есть глобальная переменная длины и указателя и сегмент данных для наших десятичных ❷, шестнадцатеричных ❸ и двоичных ❹ строк. Большая часть кода, который мы будем использовать, взята из главы 5.

Далее у нас есть несколько строк, представляющих HTML-теги. Есть открывающие ❺ и закрывающие ❻ указатели тегов H1, длина и сегменты данных, а также открывающие ❼ и закрывающие ❸ теги H4. Эти строки будут использоваться для сборки выводимой HTML-строки, которая будет сохранена в позиции 1024 ❽ линейной памяти (я выбрал ее потому, что она свободна).

Поскольку мы копируем строковые данные в выводимую строку, нам нужно будет отслеживать новую длину этой строки и передавать это значение в JavaScript; для отслеживания длины мы используем глобальную переменную `$out_str_len`. Вместо того чтобы снова писать здесь код исходных функций из главы 5, я просто напишу мно-

готовиче (...) и комментариев, указывающий номер листинга с нужным кодом, который необходимо скопировать. Скопируйте и вставьте в наш файл код функций из исходных листингов для всех шести функций в листинге 7.15.

Листинг 7.15. Повторное использование функций из главы 5

```
hex_and
_binary.wat ① (func $set_bin_string (param $num i32) (param $string_len i32)
(часть 2 из 4)   ;; $set_bin_string определен в листинге 5.35
...
)
...
② (func $set_hex_string (param $num i32) (param $string_len i32)
   ;; $set_hex_string определен в листинге 5.30
   ...
   ) ;; конец $set_hex_string
...
③ (func $set_dec_string (param $num i32) (param $string_len i32)
   ;; $set_dec_string определен в листинге 5.24
   ...
   )
...
④ (func $byte_copy
   (param $source i32) (param $dest i32) (param $len i32)
   ;; $byte_copy определен в листинге 5.17
   ...
   )
...
⑤ (func $byte_copy_i64
   (param $source i32) (param $dest i32) (param $len i32)
   ;; $byte_copy_i64 определен в листинге 5.18
   ...
   )
...
⑥ (func $string_copy
   (param $source i32) (param $dest i32) (param $len i32)
   ;; $string_copy определен в листинге 5.19
   ...
   )
...
...
```

Первые функции предназначены для преобразования числа в строку. Функция `$set_bin_string` ① преобразует число в двоичную строку. В качестве параметров требуется `i32 $num` для преобразования в двоичную строку и `$string_len` в качестве длины выводимой строки, которая включает полубайтовое заполнение пробелами (листинг 5.35). Далее идет `$set_hex_string` ②, которая преобразует число и длину в шестнадцатеричную строку с префиксом `0x`, чтобы указать, что строка представляет собой шестнадцатеричное число (листинг 5.30). Затем `$set_dec_string` ③ преобразует число в десятичную строку (листинг 5.24).

Далее следуют три функции копирования, которые копируют байт за раз, восемь байт за раз и строки. Каждая из них получает три па-

параметра: параметр `$source` – это строка, из которой мы копируем, параметр `$dest` – это строка, в которую мы копируем, а `$len` – длина строки. Первая из них – функция `$byte_copy` ❶, которая копирует данные по одному байту за раз (листинг 5.7). Функция `$byte_copy_i64` ❷ копирует восемь байт за раз (листинг 5.18). Функция `$string_copy` ❸ копирует восемь байт за раз с помощью `$byte_copy_i64`, пока не останется менее восьми байт, а затем поочередно копирует оставшиеся байты, используя `$byte_copy` (листинг 5.17).

В листинге 7.15 нет одной последней команды копирования. Это функция `$append_out`, которая всегда будет добавлять заданную исходную строку к выводимой строке, копируя ее в конец текущей выводимой строки. Добавьте код из листинга 7.16 в файл `hex_and_binary.wat`.

Листинг 7.16. Добавление функции `$append_out` в конец выводимой строки

```
hex_and
_binary.wat
(часть 3 из 4) ❶ ...
                ;; добавить исходную строку в конец выводимой строки
(func $append_out (param $source i32) (param $len i32)
  ❷ (call $string_copy
    (local.get $source)
    (i32.add
      (global.get $out_str_ptr)
      (global.get $out_str_len)
    )
    (local.get $len)
  )

  ;; увеличить длину выводимой строки
  global.get $out_str_len
  local.get $len
  i32.add
  ❸ global.set $out_str_len
)
...
```

С помощью `$string_copy` ❷ функция `$append_out` ❶ переносит исходную строку в конец выводимой строки, а затем прибавляет длину только что добавленной строки к переменной `$out_str_len` ❸, которая хранит длину выводимой строки.

Последняя функция в этом модуле – `setOutput`, которая создает строку, используемую для установки тега `output div`. Она экспортированная, поэтому ее можно вызывать из JavaScript. Добавьте код из листинга 7.17 в конец WAT-файла.

Листинг 7.17. Экспортированная функция `setOutput` для вызова из JavaScript

```
hex_and
_binary.wat
(часть 4 из 4) ...
(func (export "setOutput") (param $num i32) (result i32)
  ;; создать десятичную строку из значения $num
```

```

❶ (call $set_dec_string
    (local.get $num) (global.get $dec_string_len))
    ;; создать шестнадцатеричную строку из значения $num
❷ (call $set_hex_string
    (local.get $num) (global.get $hex_string_len))
    ;; создать двоичную строку из значения $num
❸ (call $set_bin_string
    (local.get $num) (global.get $bin_string_len))

    i32.const 0
❹ global.set $out_str_len ;; обнулить $out_str_len

    ;; добавить <h1>${decimal_string}</h1> в конец выводимой строки
❺ (call $append_out
    (global.get $h1_open_ptr) (global.get $h1_open_len))
    (call $append_out
    (global.get $dec_string_ptr) (global.get $dec_string_len))
    (call $append_out
    (global.get $h1_close_ptr) (global.get $h1_close_len))

    ;; добавить <h4>${hexadecimal_string}</h4> в конец выводимой строки
❻ (call $append_out
    (global.get $h4_open_ptr) (global.get $h4_open_len))
    (call $append_out
    (global.get $hex_string_ptr) (global.get $hex_string_len))
    (call $append_out
    (global.get $h4_close_ptr) (global.get $h4_close_len))

    ;; добавить <h4>${binary_string}</h4> в конец выводимой строки
❼ (call $append_out
    (global.get $h4_open_ptr) (global.get $h4_open_len))
    (call $append_out
    (global.get $bin_string_ptr) (global.get $bin_string_len))
    (call $append_out
    (global.get $h4_close_ptr) (global.get $h4_close_len))

    ;; вернуть длину выводимой строки
❽ global.get $out_str_len
)
)

```

В листинге 7.17 первые три вызова, которые делает функция `set_output`, – это `$set_dec_string` ❶, `$set_hex_string` ❷ и `$set_bin_string` ❸. Эти функции получают число, переданное в `setOutput`, и преобразуют его в десятичную, шестнадцатеричную и двоичную строки в линейной памяти. Как только эти строки сохранены, глобальная переменная `$out_str_len` ❹ устанавливается в 0, что сбрасывает указатель выводимой строки, поэтому добавление к выводимой строке происходит поверх строки, находящейся в настоящее время в памяти. После того как мы сбросили значение, мы можем начать добавлять новые выходные значения к выводимой строке.

Далее идут девять вызовов `$append_out`, сгруппированных в три блока. Первые три вызова добавляют открывающий и закрывающий

тег H1 со строкой десятичного значения 5 внутри него. Так получается HTML-строка для отображения десятичного числового значения на нашей веб-странице. Следующий блок добавляет шестнадцатеричную строку 6 в тег H4, а затем в тег H4 добавляется двоичная строка 7. Наконец, длина выводимой строки `$out_str_len` загружается в стек с помощью вызова `global.get` 8, который возвращает ее вызывающему JavaScript.

Компиляция и запуск

Модуль WAT готов, теперь используйте `wat2wasm` для компиляции в файл `hex_and_binary.wasm`, как показано в листинге 7.18.

Листинг 7.18. Компиляция `hex_and_binary.wat` с помощью `wat2wasm`

```
wat2wasm hex_and_binary.wat
```

Убедитесь, что у вас запущен `server.js`, и откройте `hex_and_binary.html` в браузере по адресу `http://localhost:8080/hex_and_binary.html`.

Вы должны увидеть страницу, показанную на рис. 7.4.

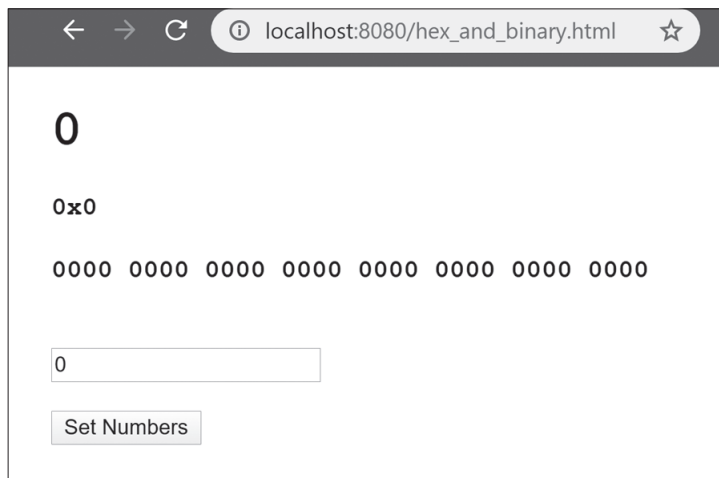


Рис. 7.4. Преобразование десятичного числа в шестнадцатеричное и двоичное

Введите число и дождитесь результата. Например, на рис. 7.5 я ввел число 1025 и нажал кнопку **Set Numbers**.

Для преобразования десятичного числа в шестнадцатеричные и двоичные строки это приложение использовало несколько функций WebAssembly, которые мы создали в главе 5. Мы добавили некоторые дополнительные функции, которые создали HTML-теги в модуле WebAssembly, чтобы могли передавать HTML в JavaScript и отображать его на веб-странице. Как видите, работа со строками и управление DOM из WAT занимает довольно много времени. Если вы работаете с на-

бором инструментов, большая часть этой тяжелой работы выполняется за вас. Некоторые из этих функций могут быть скомпилированы в связующий код JavaScript.

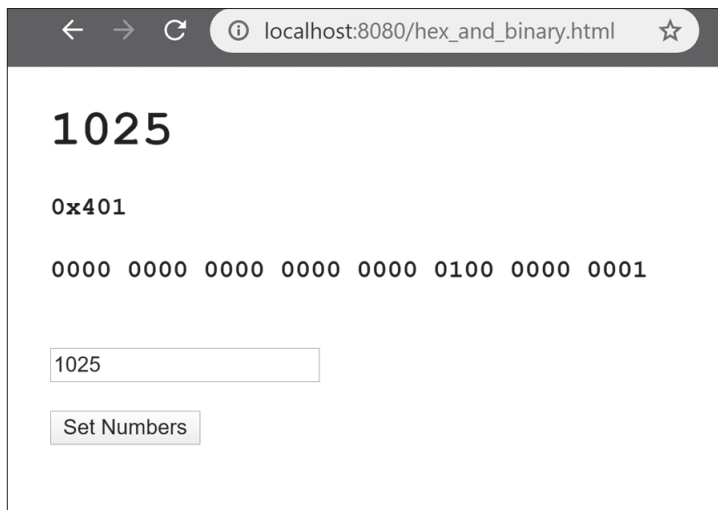


Рис. 7.5. Преобразование числа 1025 в шестнадцатеричное и двоичное

Заключение

WebAssembly 1.0 не работает напрямую с пользовательскими интерфейсами. Его сильная сторона – приложения с большим объемом вычислений. При взаимодействии с DOM из веб-приложения, созданного поверх WebAssembly, управление DOM – это в первую очередь задача для JavaScript. Работа со строками из WebAssembly полностью зависит от реализации. WebAssembly по-прежнему является отличным выбором для многих веб-приложений, особенно графических приложений, таких как игры. Но в текущем состоянии он не предназначен для работы напрямую с DOM.

Мы начали эту главу с создания простого веб-сервера JavaScript для запуска в Node.js. Так как нельзя загрузить веб-приложение WebAssembly из файловой системы, вместо этого вы должны получить вашу страницу с помощью веб-сервера. Мы написали наше первое веб-приложение WebAssembly, которое складывает два числа и затем записывает эти числа в DOM в тег абзаца, называющийся output.

Основное различие между веб-приложениями и приложениями Node.js заключается в среде встраивания. Приложения командной строки Node.js полностью написаны на JavaScript, при этом веб-приложение имеет свой JavaScript внутри веб-страницы HTML. Node.js может запускать модуль WebAssembly непосредственно из файловой системы, тогда как веб-приложение использует реализацию Streaming и fetch для создания экземпляра модуля WebAssembly, передан-

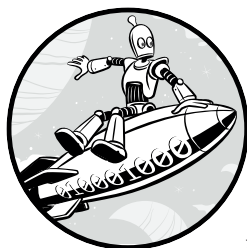
ного с веб-сервера. Приложение Node.js записывает свой вывод в консоль, тогда как HTML-страница обновляет `innerHTML` элемента DOM.

Второе приложение, которое мы создали, отображало десятичное, шестнадцатеричное и двоичное представления числа, переданного в модуль WebAssembly. Это было сделано путем сборки строки, содержащей элементы HTML, которые будут отображаться в приложении. Это приложение повторно использовало несколько функций, созданных в главе 5 для обработки строк. JavaScript в данном приложении записал строку в `innerHTML` тега `div` на нашей веб-странице.

Ни одно из созданных нами приложений не является лучшим вариантом использования WebAssembly. Моей целью в этой главе была демонстрация процесса разработки первых двух веб-приложений WebAssembly, и не обязательно тех, которые можно было бы создавать только с помощью WebAssembly. В следующей главе мы выполним рендеринг на HTML-холсте и исследуем обнаружение столкновений между большим количеством объектов на этом холсте. Эти задачи, обычно встречающиеся в веб-играх, лучше демонстрируют, что WebAssembly 1.0 может сделать для повышения производительности вашего веб-приложения.

8

РАБОТА С CANVAS



В этой главе вы узнаете, как использовать элемент HTML canvas в WebAssembly для создания быстрой и эффективной анимации в веб-приложении. Мы научимся управлять данными элементов изображения (пикселей) внутри линейной памяти WebAssembly, а затем перенесем их из линейной памяти прямо на HTML-холст. Продолжим код обнаружения случайных столкновений объектов (листинг 6.16), создав объекты в линейной памяти JavaScript, а затем через WebAssembly будем перемещать эти объекты, обнаруживать столкновения и визуализировать их. Графическая визуализация обнаружения столкновений является отличным способом протестировать возможности WebAssembly, поскольку количество возможных столкновений растет экспоненциально с увеличением количества объектов. К концу этой главы у нас будет приложение, которое может находить столкновения между тысячами различных объектов десятки раз в секунду. Наши объекты будут в форме квадрата. Если столкновения нет, они будут отображаться зеленым цветом, и красным, если столкновение происходит.

Ранее мы говорили, что веб-браузеры изначально были разработаны для отображения простых онлайн-документов, а это означает, что любые изменения положения какого-либо элемента документа часто

приводили к перерисовке всей страницы. Как следствие все приложения, требующие графических эффектов с высокой частотой кадров (например, игры), были крайне непроизводительны. С тех пор браузеры превратились в сложные среды хостинга приложений, что потребовало разработки более сложной модели рендеринга: *canvas*. Элемент *canvas* был представлен компанией Apple в 2004 году для своего веб-браузера Safari, а в 2006 году был признан как часть стандарта HTML. Благодаря элементу *canvas* веб-разработчики могут отображать 2D-изображения и анимацию со значительно большей производительностью, чем посредством управления DOM, как это делалось ранее. Использование *canvas* с WebAssembly поможет нам визуализировать анимацию в браузере с молниеносной скоростью.

Рендеринг HTML-страницы на холсте

Существует множество книг о API HTML *canvas*, но мы рассмотрим только те функции, которые продемонстрируют способности WebAssembly. Как и в случае с DOM, WebAssembly не может напрямую взаимодействовать с *canvas*. Вместо этого мы должны визуализировать данные элементов изображения непосредственно из линейной памяти в элемент *canvas*. Это позволяет нам писать приложение для работы с *canvas* с минимальным количеством кода JavaScript. Перед тем как написать код WebAssembly, мы напишем коды HTML и JavaScript. Увидеть готовое приложение можно по адресу <https://wasmbook.com/collide.html>.

Определение холста в HTML

Как обычно, мы разбиваем HTML-файл на части и будем обсуждать их поочередно. Первая часть определяет *холст*, то есть область на веб-странице, где отображается анимация. Создайте файл с именем *collide.html* и сохраните в нем код из листинга 8.1.

Листинг 8.1. Код HTML для определения холста

```
collide.html <!DOCTYPE html>
(часть 1 из 5) <html lang="en">
                <head>
                  <meta charset="UTF-8">
                  <meta name="viewport" content="width=device-width, initial-scale=1.0">
                  <title>Collision detection</title>
                </head>
                <body>
                  ❶<canvas ❷id="cnvs" ❸width="512" ❹height="512"></canvas>
                  ...
                </body>
              </html>
```

Здесь вам нужно обратить внимание на элемент *canvas* ❶. Мы задаем элементу *canvas* идентификатор *cnvs* ❷, чтобы позже для его полу-

чения можно было использовать вызов `document.getElementById`. Мы устанавливаем `width` ❸ и `height` ❹ равными 512, так как 512 – это 2⁹, или 0x200 в шестнадцатеричной форме. Благодаря такому значению нам будет проще работать с шириной (`width`) и высотой (`height`), используя двоичную логику, что может увеличить быстродействие приложения, если мы правильно разработаем наш код.

Определение констант JavaScript в HTML

В начале нашей JavaScript-части мы добавим постоянные значения для конфигурации некоторых высокоуровневых параметров в модуле `WebAssembly`. Значения будут совместно использоваться JavaScript и `WebAssembly`. Определение этих значений непосредственно внутри JavaScript упрощает обновление конфигурации. Мы начинаем код с некоторых констант, связанных с холстом, которые задают параметры для взаимодействия между `WebAssembly` и HTML-элементом `canvas`. У нас также есть набор констант, связанных с организацией данных в линейной памяти, которые определяют наш базовый адрес, шаг и сдвиг для визуализируемых нами объектов. Кроме того, мы должны определить новый объект `ImageData`, выделяющий часть буфера линейной памяти как объект, который приложение может рисовать прямо на холсте. Добавьте код из листинга 8.2 в свой HTML-файл.

Листинг 8.2. Настройка данных изображения в JavaScript

```
collide.html ...
(часть 2 из 5) <script>
❶ const cnvs_size = 512; // квадратный холст, ширина и высота которого = 512

❷ const no_hit_color = 0xff_00_ff_00; // цвет без столкновения (зеленый)
  const hit_color = 0xff_00_00_ff; // цвет при столкновении (красный)

  // количество пикселей - canvas_size x canvas_size, так как это квадратный холст
❸ const pixel_count = cnvs_size * cnvs_size;

❹ const canvas = document.getElementById("cnvs");
❺ const ctx = canvas.getContext("2d");
  ctx.clearRect(0, 0, 512, 512);

  // количество байтов, необходимых для данных пикселей, равно количеству
  // пикселей * 4
❻ const obj_start = pixel_count * 4; // 4 байта в каждом пикселе.
  const obj_start_32 = pixel_count; // 32-битный сдвиг к исходному объекту
  const obj_size = 4; // количество пикселей в квадратном объекте
  const obj_cnt = 3000; // 3000 объектов
  const stride_bytes = 16; // 16 байт в каждом сдвиге

  const x_offset = 0; // компонент x в байтах 0-3
  const y_offset = 4; // компонент y в байтах 4-7
  const xv_offset = 8; // компонент скорости x в байтах 8-11
  const yv_offset = 12; // компонент скорости y в байтах 12-15
```



```

❷ const memory = new WebAssembly.Memory({initial: 80});
    const mem_i8 = new Uint8Array(memory.buffer);           // 8-битное представление
    const mem_i32 = new Uint32Array(memory.buffer);         // 32-битное представление

❸ const importObject = {
  env: {
    buffer: memory,

    cnvs_size: cnvs_size,
    no_hit_color: no_hit_color,
    hit_color: hit_color,
    obj_start: obj_start,
    obj_cnt: obj_cnt,
    obj_size: obj_size,

    x_offset: x_offset,
    y_offset: y_offset,
    xv_offset: xv_offset,
    yv_offset: yv_offset
  }
};

// Объект ImageData может быть перенесен на холст
const image_data =
❹ new ImageData( new Uint8ClampedArray(memory.buffer, 0, obj_start),
                  cnvs_size,
                  cnvs_size );

...

```

У нас есть единственная константа `cnvs_size` ❶, которая содержит высоту и ширину элемента `canvas`, потому что они одинаковые. Затем у нас есть две константы, которые определяют шестнадцатеричные значения цвета. Первая, `no_hit_color` ❷, определяет цвет, которым будет окрашен объект, когда он не сталкивается с другим объектом. Вторая, `hit_color`, определяет цвет, который будет у объекта при столкновении с другим объектом. Значение этих шестнадцатеричных чисел более подробно рассматривается в разделе «Данные растрового изображения». Затем мы определяем `pixel_count` ❸, который получаем путем возведения в квадрат размера `cnvs_size`, потому что у нас квадратный холст.

Потом работаем с интерфейсом API Canvas, *контекстом рисования*, который позволяет JavaScript взаимодействовать с `canvas`. Существует несколько вариантов работы с `canvas` на HTML-странице. Мы будем работать с контекстом "2d" холста, потому что он относительно прост. Мы создаем константу элемента `canvas` ❹ с вызовом `document.getElementById` для извлечения контекста из HTML-холста. Затем вызываем функцию `getContext` ❺ для этой константы `canvas`, чтобы создать константу, содержащую контекстный интерфейс, который мы назвали `ctx`. Мы будем использовать этот объект `ctx` для рендеринга растрового изображения, созданного в WebAssembly, в элемент `canvas`.

Примечание *WebAssembly также хорошо работает с контекстами холста `webgl` и `webgl2`, которые отображают 3D-модели на холсте. К сожалению, WebGL не входит в рамки этой книги.*

За константами, относящимися к холсту, следует группа констант, связанных с объектами линейной памяти. Эти константы начинаются с константы `obj_start` ❹ и следуют формату базового адреса, шага и сдвига, который мы обсуждали в главе 6. В `obj_start` должен быть указан тот базовый адрес, который следует за всеми нашими данными пикселей в начале линейной памяти. Мы устанавливаем `obj_start` в `pixel_count * 4`, потому что каждый пиксель занимает четыре байта данных, а данные объекта сразу же следуют за разделом этого размера. В этой части мы используем некоторые константы для определения размера шага и сдвига для каждого из элементов объекта. Мы определяем линейную память ❺ с начальным размером в 80 страниц, чего достаточно для размещения всех необходимых объектов и данных пикселей. Затем создаем 8-битное и 32-битное представления этого объекта данных. Все константы, которые мы создали до сих пор, необходимо передать в модуль WebAssembly с помощью `importObject` ❻.

Наконец, мы создаем новый объект `ImageData` ❼, который представляет собой интерфейс JavaScript и используется для доступа к базовым данным пикселей в нашем элементе `canvas`. Объект `Memory` ❸, который мы создали в листинге 8.2, имеет элемент с именем `buffer`, который представляет собой типизированный массив, содержащий данные в линейной памяти. Элемент `buffer` – это буфер данных, который может представлять данные пикселей, отображаемые на холсте. Чтобы создать новый объект `ImageData`, объект `memory.buffer` должен быть передан в `ImageData` как `Uint8ClampedArray` вместе со значениями ширины и высоты холста.

Создание случайных объектов

Далее мы создадим случайные объекты, аналогично тому, как делали ранее в книге. Мы продолжаем использовать случайные данные, потому что это позволяет нам сосредоточиться на WebAssembly, а не на самих данных. Однако в WebAssembly нет функции генерации случайных чисел, поэтому создавать наши случайные объекты внутри JavaScript намного проще. У объектов есть четыре атрибута: координаты `x` и `y` (положение), а также компоненты скорости по осям `x` и `y` (движение). Для представления значения каждого из них мы используем 32-битное целое число. В листинге 8.3 показан зацикленный код, создающий данные для нескольких объектов, представленных константой `obj_cnt`, которую мы определили ранее.

Листинг 8.3. Подготовка данных в линейной памяти

```
collide.html ...
(часть 3 из 5) ❶ const stride_i32 = stride_bytes/4;
                ❷ for( let i = 0; i < obj_cnt * stride_i32; i += stride_i32 ) {
```

```

    // значение меньше, чем canvas_size
    ❸ let temp = Math.floor(Math.random() * cnvs_size);

    // присвоить атрибуту x случайное значение
    ❹ mem_i32[obj_start_32 + i] = temp;

    // случайное значение меньше, чем canvas_size
    ❺ temp = Math.floor(Math.random()*cnvs_size);

    // присвоить атрибуту y случайное значение
    ❻ mem_i32[obj_start_32 + i + 1] = temp;

    // случайное значение между -2 и 2
    ❼ temp = (Math.round(Math.random() * 4) - 2);

    // присвоить атрибуту скорости x случайное значение
    ❸ mem_i32[obj_start_32 + i + 2] = temp;

    // случайное значение между -2 и 2
    ❹ temp = (Math.round(Math.random() * 4) - 2);

    // присвоить атрибуту скорости y случайное значение
    ❺ mem_i32[obj_start_32 + i + 3] = temp;
  }
  ...

```

Код в этом цикле обращается к данным в линейной памяти через 32-битное целочисленное представление в `mem_i32`. Поскольку цикл работает с 32-битными числами, мы создаем 32-битную версию `stride_bytes`, которую называем `stride_i32` ❶. Мы присваиваем ей значение `stride_bytes/4`, потому что на каждый `i32` приходится четыре байта. Цикл `for` повторяется до тех пор, пока индекс `i` не станет равным количеству объектов, заданных в `obj_count`, умноженному на количество 32-битных целых чисел в нашем шаге, определенном в `stride_i32` ❷. Это создает в линейной памяти круговую структуру данных.

Внутри цикла мы задаем четыре 32-битных случайных целых числа, которые будут представлять положение и скорость каждого объекта. В первую очередь установим атрибуты положения. Мы получаем случайное число от 0 до значения ширины холста ❸, хранящееся в `cnvs_size`, и сохраняем его в местоположении атрибута положения `x` ❹ в линейной памяти. Затем генерируется случайное число от 0 до значения высоты холста ❺, которое сохраняется в ячейке атрибута `y` ❻ в линейной памяти. Потом мы устанавливаем атрибуты скорости, генерируя число от -2 до 2 ❼, сохраняя его в местоположении атрибута скорости `x` ❸, и делаем то же самое ❹ для атрибута скорости `y` ❺.

Данные растрового изображения

Мы можем визуализировать данные растрового изображения непосредственно в элемент HTML `canvas` с помощью функции `putImageData`, передавая объект `ImageData`, который определили ранее. HTML-

холст представляет собой сетку пикселей; каждый из них может быть представлен тремя байтами, причем каждый байт содержит какой-либо из трех цветов: красный, зеленый и синий. В растровом формате пиксель представлен одним 32-битным целым числом, где каждый байт целого числа представляет один из цветов. Четвертый байт целого числа представляет *альфа-значение*, которое устанавливает насыщенность цвета пикселя. Когда альфа-байт равен 0, пиксель прозрачен, а когда он равен 255, он насыщенного цвета. В линейной памяти WebAssembly мы создадим массив 32-битных целых чисел, представляющий массив данных пикселей. Этот тип массива делает WebAssembly очень удобным инструментом для управления данными, которые нужно отобразить на HTML-холсте.

В `tere script` мы сохраним функцию модуля WebAssembly, которая генерирует эти растровые данные в переменной `animation_wasm`. Нам также нужна функция JavaScript, вызывающая нужную нам функцию WebAssembly. Затем мы вызываем `ctx.putImageData` для рендеринга данных изображения в элемент `canvas`. Листинг 8.4 содержит фрагмент кода JavaScript, который необходимо добавить в файл HTML.

Листинг 8.4. Функция JavaScript *animate* визуализирует кадр анимации

```
collide.html ...  
(часть 4 из 5) ❶ var animation_wasm; //функция WebAssembly, которую будем вызывать каждый кадр  
  
❷ function animate() {  
    ❸ animation_wasm();  
    ❹ ctx.putImageData(image_data, 0, 0); // рендер данных пикселя  
    ❺ requestAnimationFrame(animate);  
}  
...
```

Переменная `animation_wasm` ❶ ссылается на функцию WebAssembly, которая генерирует данные изображения. Следующая функция `animate` ❷ вызывает функцию `animation_wasm` ❸ модуля WebAssembly, которая генерирует `image_data` для следующего кадра анимации. Затем объект `image_data` передается в вызов `ctx.putImageData` ❹, который визуализирует изображение, сгенерированное WebAssembly, в элементе `canvas`. Последняя функция `requestAnimationFrame` ❺ несколько сложнее, поэтому мы рассмотрим ее более подробно в следующем разделе.

Функция *requestAnimationFrame*

Анимация – это оптическая иллюзия: серия статических изображений, отображаемых в быстрой последовательности, заставляющая глаз поверить в движение. Так работают все телевизионные экраны, мониторы компьютеров и фильмы, которые вы когда-либо смотрели. JavaScript предоставляет удобную функцию `requestAnimationFrame`:

когда вы ее вызываете, функция, переданная в `requestAnimationFrame`, вызывается при следующей визуализации кадра. В `requestAnimationFrame` мы передаем функцию, которую хотим вызвать в следующий раз, когда наш компьютер будет готов визуализировать кадр анимации.

Мы вызываем эту функцию в конце кода JavaScript, передавая функцию `animate`, которую определили в листинге 8.4. Мы вызываем `requestAnimationFrame` второй по окончании функции `animate`, чтобы задать функцию в качестве обратного вызова для последующего рендеринга кадра. Этот второй вызов необходимо сделать, потому что функция `requestAnimationFrame` не задает функцию, которая будет вызываться каждый раз при визуализации кадра; она задается только для рендеринга следующего кадра. Функция `animate` должна вызвать модуль `WebAssembly`, который выполняет вычисления обнаружения столкновений и перемещения объекта. `WebAssembly` вычисляет данные изображения, помещенные на холст. Однако напрямую отобразить эти данные на холсте невозможно. Чтобы отобразить данные пикселей на холсте, мы должны вызвать `putImageData` из нашей JavaScript функции `animate`. Вызов `putImageData` перемещает фрагмент линейной памяти, который мы выделили для представления данных пикселей в элемент `canvas`.

В первый раз мы вызываем `requestAnimationFrame` сразу после запуска модуля `WebAssembly` в последней строке кода. В листинге 8.5 показана последняя часть HTML-кода.

Листинг 8.5. Создание экземпляра модуля WebAssembly и вызов requestAnimationFrame

```
collide.html ...  
(часть 5 из 5) (async () => {  
  let obj = await  
    ❶ WebAssembly.instantiateStreaming( fetch('collide.wasm'), importObject );  
    ❷ animation_wasm = obj.instance.exports.main;  
    ❸ requestAnimationFrame(❹animate);  
})();  
</script>  
</body>  
</html>
```

Внутри асинхронного IIFE мы начинаем с вызова функции `instantiateStreaming` ❶. Мы присваиваем переменной `animation_wasm` ❷, которую уже определили в листинге 8.4, экспортируемую функцию в модуле `WebAssembly` с именем `main`. Ранее мы вызвали функцию `animation_wasm` из функции `animate`. И наконец, вызов `requestAnimationFrame` ❸ передает функцию `animate` ❹, определенную ранее. Поскольку `animate` также вызывает `requestAnimationFrame` для себя, браузер вызывает `animate` при каждом обновлении.

Модуль WAT

Теперь, когда мы определили HTML, нам нужно создать модуль Web-Assembly в WAT, который будет управлять перемещением объекта, обнаружением столкновений и данными растрового изображения. Создайте файл с именем *collide.wat*, в который мы поместим простой код обнаружения столкновений и код рендеринга на холсте. Мы используем множество функций, и некоторые из них могут не обладать идеальной производительностью. В следующей главе мы займемся оптимизацией этого кода. А в этой главе сосредоточимся на разработке кода, а не на высокой производительности. Модуль будет определять глобальные переменные, которые импортируют значения из JavaScript. Нам нужно будет определить ряд функций, которые очищают холст, вычисляют абсолютное значение целого числа, устанавливают отдельные пиксели и рисуют объекты, подвергаемые столкновениям. Затем нужно будет определить функцию `main`, которая будет использовать двойной цикл для перемещения каждого объекта и обнаруживать, сталкивается ли он с другим объектом.

Импортируемые значения

Начало модуля в листинге 8.6 импортирует константы, переданные в модуль через `importObject`, который мы определили в нашем JavaScript. Эти значения включают в себя наш буфер памяти, размер холста, цвета объектов, а также значения базового адреса, сдвига и шага, которые мы можем использовать для доступа к объектам в линейной памяти.

Листинг 8.6. Объявление импортированных глобальных переменных и буфера памяти

```
collide.wat (module
(часть 1 из 12) ❶(global $cnvs_size      (import "env" "cnvs_size")    i32)

                ❷(global $no_hit_color (import "env" "no_hit_color") i32)
                (global $hit_color    (import "env" "hit_color")    i32)
                ❸(global $obj_start   (import "env" "obj_start")    i32)
                ❹(global $obj_size    (import "env" "obj_size")    i32)
                ❺(global $obj_cnt     (import "env" "obj_cnt")      i32)

                ❻(global $x_offset    (import "env" "x_offset")     i32) ;; байты 00-03
                (global $y_offset    (import "env" "y_offset")     i32) ;; байты 04-07
                ❼(global $xv_offset   (import "env" "xv_offset")    i32) ;; байты 08-11
                (global $yv_offset   (import "env" "yv_offset")    i32) ;; байты 12-15
                ❽(import "env" "buffer" (memory 80))                ;; буфер холста
                ...
)
```

Сначала мы импортируем глобальную переменную `$cnvs_size` ❶, получившую значение 512 в JavaScript, которая представляет ширину

и высоту холста. Далее следуют два значения цвета: `$no_hit_color` ❷, представляющий 32-битный цвет объекта при отсутствии столкновения, и `$hit_color`, представляющий цвет объекта при столкновении. Помните, что мы определили их как шестнадцатеричное значение для зеленого и красного цветов.

У нас есть переменная `$obj_start` ❸, которая содержит базовое местоположение для данных объекта. Переменная `$obj_size` ❹ – это ширина и высота квадратных объектов в пикселях. Переменная `$obj_cnt` ❺ содержит количество объектов, которые приложение будет отображать, и проверяет на наличие столкновений. Далее идет сдвиг для двух координат ❻ `$x_offset` и `$y_offset` и двух атрибутов для значений скорости ❼ `$xv_offset` и `$yv_offset`. Последний оператор `import` ❽ в этом блоке кода импортирует модуль `buffer`, который мы определили в JavaScript.

Очистка холста

Далее мы определим функцию, которая очищает весь буфер растрового изображения. Если холст не очищается каждый раз при визуализации кадра, старые «воспоминания» от каждого объекта останутся в памяти, и за объектами по экрану будет тянуться шлейф. Функция `$clear_canvas` устанавливает для каждого цвета значение `0xff_00_00_00`, означающее насыщенный черный цвет. В листинге 8.7 показан код функции `$clear_canvas`.

Листинг 8.7. Определение функции `$clear_canvas`

```
collide.wat ...
(часть 2 из 12) ;; очистить весь холст
(func $clear_canvas
  (local $i i32)
  (local $pixel_bytes i32)

  global.get $cnvs_size
  global.get $cnvs_size
  i32.mul ;; умножить $width на $height

  i32.const 4
  i32.mul ;; 4 байта на пиксель

  ❶ local.set $pixel_bytes ;; $pixel_bytes = $width * $height * 4

  ❷ (loop $pixel_loop
    ❸ (i32.store (local.get $i) (i32.const 0xff_00_00_00))

    (i32.add (local.get $i) (i32.const 4))
    ❹ local.set $i ;; $i += 4 (байт на пиксель)

    ;; если $i < $pixel_bytes
    ❺ (i32.lt_u (local.get $i) (local.get $pixel_bytes))
```

```

    ❸br_if $pixel_loop ;; завершить цикл, если все пиксели обработаны
  )
)
...

```

Функция `$clear_canvas` вычисляет количество байтов пикселей, возводя в квадрат размер холста (потому что мы выбрали квадратный холст) и умножая его на 4, потому что для каждого пикселя используется четыре байта. Затем мы сохраняем в локальной переменной `$pixel_bytes` ❶ это значение, которое представляет собой количество байтов, выделенных для пикселей в памяти. Далее функция ❷ перебирает каждый пиксель, записывая в него шестнадцатеричное значение `0xff_00_00_00` ❸, где все цвета пикселей равны 0, кроме альфа-значения, которое равно `0xff` (полная насыщенность). Функция увеличивает индекс, хранящийся в `$i`, на 4 ❹, поскольку целое число `i32` состоит из четырех байт. Код проверяет, меньше ли индекс `$i`, чем количество байтов пикселей ❺, и если это так, переходит обратно к началу `loop` ❻, потому что если `$i` меньше количества пикселей, это означает, что остались объекты, которые нужно удалить.

Функция вычисления абсолютного значения

В этом приложении мы будем использовать стратегию обнаружения столкновений квадратных объектов, а не кругов, потому что наши объекты имеют квадратную форму. Нам нужно будет перейти на алгоритм обнаружения столкновений квадратов, который требует, чтобы код находил абсолютное значение целого числа со знаком. В листинге 8.8 мы напомним небольшую функцию `$abs`, которая принимает целое число со знаком и проверяет, является ли переданный параметр отрицательным, и если да, делает его положительным числом, чтобы получить абсолютное значение.

Листинг 8.8. Функция абсолютного значения `$abs`

collide.wat
(часть 3 из 12)

```

...
;;эта функция возвращает абсолютное значение полученного числа
(func $abs
  (param $value      i32)
  (result            i32)

  ❶(i32.lt_s (local.get $value) (i32.const 0)) ;; $value отрицательное?
  ❷if ;; если $value отрицательное, вычитите его из 0, чтобы получить
    ;; положительное значение
    i32.const 0
    local.get $value
    ❸i32.sub
    ❹return
  end

```



```

❻ local.get $value ;; вернуть начальное значение
)
...

```

Функция \$abs в первую очередь просматривает полученное значение и проверяет, меньше ли нуля значение этого целого числа со знаком ❶. Если оно меньше нуля ❷, функция вычитает ❸ это число из нуля, делая его отрицательным, и возвращает положительное число ❹. Если число не было отрицательным, функция возвращает начальное число ❺.

Примечание *Когда я впервые писал функцию \$abs, то использовал алгоритм дополнительного кода, чтобы сделать число отрицательным. Как оказалось, вычитание числа из 0 в WebAssembly происходит гораздо быстрее.*

Установка цвета пикселя

Чтобы нарисовать объект на холсте, необходимо задать цвет пикселя в линейной памяти с учетом координат x , y и значения цвета. Этой функции потребуется проверка границ, потому что мы пишем в область линейной памяти, отведенную для представления области холста. Без этой проверки, если мы попытаемся записать в область памяти, которой нет на холсте, функция будет записывать в ту область линейной памяти, которую мы могли бы использовать для других целей.

Функция проверяет координаты относительно границ холста и сообщает, если эти координаты выходят за границы. Она определяет, где в линейной памяти необходимо обновить данные пикселей. Прежде чем заняться кодом, давайте бегло рассмотрим, как координаты на холсте переводятся в линейную память.

Холст представляет собой 2D-поверхность со строками и столбцами. На рис. 8.1 показан простой холст из четырех пикселей в высоту и четырех пикселей в ширину.

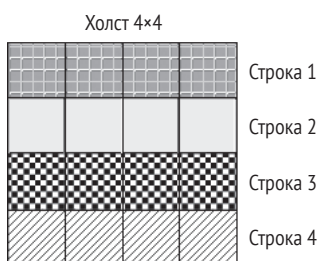


Рис. 8.1. Холст 4×4

Каждая строка на холсте по-разному текстурирована – вскоре вы поймете, почему. Холст имеет координаты x и y , где координата x первого столбца равна 0 и увеличивается слева направо; координата y

также начинается с 0 и увеличивается сверху вниз. На рис. 8.2 показан холст 4×4 с координатами x и y .

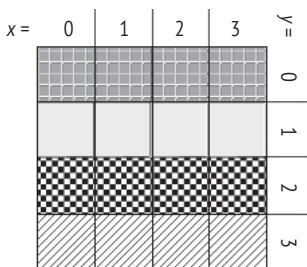


Рис. 8.2. Холст 4×4 с координатами x и y

Именно так устроен холст на мониторе компьютера, но память компьютера не организована по строкам и столбцам. Память является одномерной структурой с адресом, представляющим каждый пиксель. По этой причине наши пиксельные данные расположены в памяти, как показано на рис. 8.3.

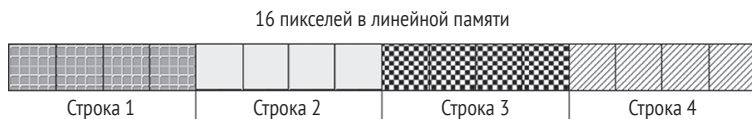


Рис. 8.3. 16 пикселей холста в линейной памяти

Строки расположены последовательно в массиве данных из 16 пикселей. Если посмотреть, как линейная память упорядочила пиксели с точки зрения координат x и y , это будет выглядеть как на рис. 8.4.

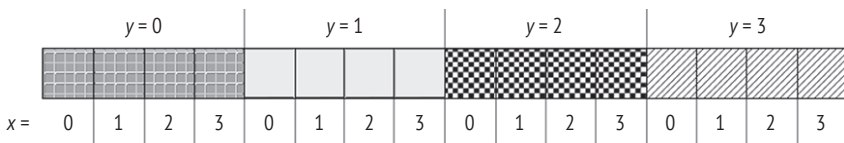


Рис. 8.4. Координаты x и y в линейной памяти

Наша функция `$set_pixel` уже имеет координаты x и y , осталось найти адрес памяти. Уравнение $y * 4 + x$ дает нам значения линейной памяти, показанные на рис. 8.5.

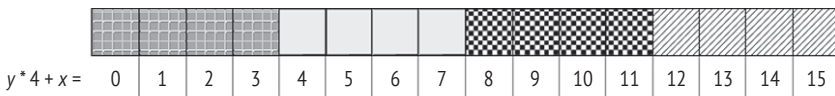


Рис. 8.5. Формула для преобразования координат x и y в линейную память

Вычислив адрес в памяти, мы можем обновить линейную память с помощью `i32.store`, чтобы установить значение по этому адресу

равным значению цвета в параметре \$c. В листинге 8.9 показан исходный код.

Листинг 8.9. Функция, которая задает каждому пикселю цвет

```
collide.wat    ;; эта функция устанавливает пиксель с координатами $x, $y в цвет $c
(часть 4 из 12) (func $set_pixel
    (param $x      i32)    ;; координата x
    (param $y      i32)    ;; координата y
    (param $c      i32)    ;; значение цвета

    ;; $x > $cnvs_size
    ❶ (i32.ge_u (local.get $x) (global.get $cnvs_size))
    if    ;; $x за границами холста
        return
    end

    ❷ (i32.ge_u (local.get $y) (global.get $cnvs_size)) ;; $y > $cnvs_size?
    if    ;; $y находится за пределами холста
        return
    end

    local.get $y
    global.get $cnvs_size
    ❸ i32.mul

    local.get $x
    ❹ i32.add      ;; $x + $y * $cnvs_size (получить пиксели в линейную память)

    i32.const 4
    ❺ i32.mul      ;; умножить на 4, так как каждый пиксель = 4 байтам

    local.get $c    ;; загрузить значение цвета

    ❻ i32.store    ;; сохранить цвет в линейной памяти
)
...
```

Эта функция сначала выполняет проверку границ, чтобы пользователь не мог попытаться установить цвет пикселя, которого нет на нашем холсте. Дабы убедиться, что координата x находится в границах холста, мы проверяем, больше ли x , чем $cnvs_size$ ❶, и, если это так, возвращаем функцию без обновления памяти. То же самое делаем с координатой y ❷.

После проверки границ нам нужно получить местоположение целевого пикселя в целых числах. Для этого нужно умножить y на $cnvs_size$ ❸ и получить количество пикселей в памяти, которые находятся в строках перед нужным пикселем, затем прибавить к этому значению x ❹. Поскольку значение местоположения выражается в 32-битных целых числах (четыре байта на пиксель), нам нужно умножить это число на 4 ❺, тем самым мы получим байтовое местопо-

ложение нашего пикселя в линейной памяти. Это место в памяти, где мы сохраняем `$c`, используя вызов оператора `i32.store` ⑥.

Рисуем объект

Объекты, которые не сталкиваются, представлены зелеными квадратами, а столкнувшиеся – красными. В разделе констант кода JavaScript мы установили размер этих квадратов равным 4, поэтому каждый из них имеет ширину и высоту 4 пикселя. Эти пиксели рисуются с помощью цикла, который увеличивает значение `x`, пока не достигнет положения начала объекта плюс его ширина. Так рисуется первая строка пикселей. Как только значение координаты `x` превышает максимальное значение `x`, код увеличивает значение координаты `y` на единицу. Затем мы рисуем вторую строку пикселей и повторяем до тех пор, пока она не превысит максимальное значение `y` пикселя в этом объекте. Затем код выходит из цикла. В итоге мы получаем объект размером 4×4 пикселя. Давайте добавим код этой функции в наш WAT-файл, как показано в листинге 8.10.

Листинг 8.10. Функция `$draw_obj` рисует квадрат пикселей через функцию `$set_pixel`

```
collide.wat ;; рисуем многопиксельный объект в виде квадрата с заданными координатами $x, $y
(часть 5 из 12) и цветом $c
(func $draw_obj
  (param $x i32)    ;; положение объекта x
  (param $y i32)    ;; положение объекта y
  (param $c i32)    ;; цвет объекта

  (local $max_x      i32)
  (local $max_y      i32)

  (local $xi         i32)
  (local $yi         i32)

  local.get $x
  ① local.tee $xi
  global.get $obj_size
  i32.add
  ② local.set $max_x      ;; $max_x = $x + $obj_size

  local.get $y
  local.tee $yi
  global.get $obj_size
  i32.add
  ③ local.set $max_y      ;; $max_y = $y + $obj_size

  (block $break (loop $draw_loop

    local.get $xi
    local.get $yi
    local.get $c
```

```

❹ call $set_pixel      ;; задать пиксель в $xi, $yi в цвет $c

    local.get $xi
    i32.const 1
    i32.add
❺ local.tee $xi        ;; $xi++

    local.get $max_x
❻ i32.ge_u            ;; $xi >= $max_x?

    if
        local.get $x
        ❼ local.set $xi    ;; сбросить $xi до $x

        local.get $yi
        i32.const 1
        i32.add
        ❸ local.tee $yi    ;; $yi++

        local.get $max_y
        ❶ i32.ge_u        ;; $yi >= $max_y?
        br_if $break

    end
    br $draw_loop
))
)
...

```

Функция `$draw_obj` принимает в качестве параметров координаты x , y и цвет в виде переменных `ragam i32 $x`, `$y` и `$c`. Начиная с позиции `$x` для координаты x и `$y` для координаты y рисуются пиксели. Необходимо перебирать каждый пиксель, пока функция не достигнет позиций `$max_x` и `$max_y` для координат x и y . Функция начинается с использования `local.tee` ❶ для присвоения `$xi` значения, переданного функции как `$x`. Затем к нему прибавляют размер объекта (`$obj_size`), чтобы найти значение `$max_x` ❷. После этого функция таким же образом находит `$max_y` ❸.

Мы находим начальную и конечную координаты x , а затем выполняем ту же задачу для оси y ❹. Я выбрал значение 512 в качестве ширины и высоты холста, потому что предполагал, что такая маска будет обеспечивать лучшую производительность, чем использование `i32.get_u` для проверки границ холста. В главе 9 мы проверим эту гипотезу, чтобы увидеть, было ли это верным предположением или преждевременной оптимизацией. В главе 4 мы подробно рассматривали, как работает побитовое маскирование.

Имея минимальные и максимальные значения x и y , мы входим в цикл, который рисует каждый пиксель с помощью выражения `call $set_pixel` ❺. Цикл увеличивает `$xi` ❻ и сравнивает его с `$max_x`, сбрасывает `$xi` до `$x` и увеличивает `$yi` ❼, если `$xi` больше или равно `$max_x` ❼. Затем, когда `$yi` превышает `$max_y`, объект полностью прорисовывается, и код выходит из цикла.

Установка и получение атрибутов объекта

Давайте создадим несколько вспомогательных функций для назначения и получения значений атрибутов объекта внутри линейной памяти. Эти функции принимают номер объекта и сдвиг элемента и возвращают значение из линейной памяти. В случае `$set_obj_attr` функция также принимает значение и присваивает его атрибуту объекта. Также функция возвращает значение в линейной памяти для этого объекта и его атрибута. Добавьте код из листинга 8.11 для `$set_obj_attr` в свой модуль WAT.

Листинг 8.11. Размещение объекта в памяти исходя из значения шага 16 байт

```
collide.wat ...
(часть 6 из 12) ;; задать атрибут объекта в линейной памяти, используя номер объекта,
;; сдвиг атрибутов и значение для установки атрибута
(func $set_obj_attr
  (param $obj_number i32)
  (param $attr_offset i32)
  (param $value i32)

  local.get $obj_number
  i32.const 16
  ❶ i32.mul ;; 16-байтовый шаг, умноженный на кол-во объектов

  global.get $obj_start ;; добавить начальный байт для объектов (базовый)
  ❷ i32.add ;; ($obj_number*16) + $obj_start

  local.get $attr_offset ;; добавить элемент сдвига в адрес
  ❸ i32.add ;; ($obj_number*16) + $obj_start + $attr_offset

  local.get $value

  ;; сохранить $value в ($obj_number*16)+$obj_start+$attr_offset
  ❹ i32.store
)
...
```

Большая часть кода в этой функции вычисляет адрес в линейной памяти, где хранится атрибут для конкретного объекта. Вспомните из главы 6, что мы вычислили расположение нашего элемента в памяти с помощью базового адреса (`$obj_start` в этой функции), шага в 16 байт, номера объекта (`$obj_number`) и сдвига атрибута (`$attr_offset`). Эта функция использует формулу `$obj_number * 16` ❶ + `$obj_start` ❷ + `$attr_offset` ❸ для определения местоположения атрибута в памяти, который мы хотим изменить. Затем она вызывает оператор `i32.store` ❹ для сохранения данного значения в памяти.

Далее мы создадим соответствующий `$get_obj_attr`, который должен вычислить адрес значения атрибута. Возможно, вы знакомы с принципом разработки программного кода *Don't Repeat Yourself*

(*DRY, не повторяйся*). DRY-код – отличный способ написать поддерживаемый код, который легко читать и обновлять другим разработчикам. К сожалению, иногда DRY-код может снижать производительность. В следующем примере мы повторим некоторые вычисления, которые сделали в предыдущей функции. Перейдя к главе об оптимизации производительности, мы сделаем наш код еще менее DRY (иногда такой код называют *WET (Write Everything Twice – пиши всё дважды)*)¹, чем он был в этой главе. Некоторые методы, формирующие DRY-код, могут добавлять уровни абстракции, требующие дополнительных вычислительных циклов. Например, вызов функции требует дополнительных циклов для проталкивания значений в стек и перехода к новым местам в коде. Оптимизирующие компиляторы часто смягчают влияние абстракций, используемых в коде. Понимание, как они это делают и почему DRY-код не всегда наиболее эффективен во время выполнения, может быть полезным.

В других ассемблерных языках макросы – отличный способ поддерживать производительность, следуя при этом принципам разработки DRY-кода. К сожалению, в настоящее время `wat2wasm` не поддерживает макросы. В листинге 8.12 показан код функции `$get_obj_attr`.

Листинг 8.12. Получение объекта в памяти на основе значения шага 16 байт

```

collide.wat    ...
(часть 7 из 12) ;; получить атрибут объекта в линейной памяти с помощью объекта
                ;; число и сдвиг элементов
(func $get_obj_attr
  (param $obj_number i32)
  (param $attr_offset i32)
  (result          i32)

  local.get $obj_number
  i32.const 16
  ❶ i32.mul                ;; $obj_number * 16

  global.get $obj_start
  ❷ i32.add                ;; ($obj_number*16) + $obj_start

  local.get $attr_offset
  ❸ i32.add                ;; ($obj_number*16) + $obj_start + $attr_offset

  ❹ i32.load              ;; загрузить указатель выше
  ;; возвращает элемент
)
...

```

Чтобы получить байты сдвига для объекта, мы начинаем с умножения `$obj_number` на значение шага 16 байт ❶. Затем мы прибав-

¹ Игра слов. На английском языке *dry* – сухой, *wet* – мокрый. – Прим. ред.

ляем ❷ базовый адрес, который хранится в глобальной переменной \$obj_start. Далее следует прибавление значения сдвига, хранящегося в \$attr_offset ❸. На этом этапе верхняя часть стека имеет местоположение в памяти атрибута, который мы хотим получить, поэтому вызов i32.load ❹ проталкивает это значение в стек.

Функция \$main

Функция \$main будет вызываться из кода JavaScript при каждой отрисовке кадра. Ее задача – перемещать объекты в зависимости от их скорости, обнаруживать столкновение между ними и отображать объект красным цветом, если он сталкивается с другим объектом, и зеленым, если столкновения нет. Функция \$main очень длинная, поэтому мы разделим ее на несколько частей.

Определение локальных переменных

Первая часть функции \$main, показанная в листинге 8.13, определяет все локальные переменные и вызывает функцию \$clear_canvas.

Листинг 8.13. Начало функции \$main объявляет локальные переменные и очищает холст

*collide.wat
(часть 8 из 12)*

```
...  
;; перемещение и обнаружение столкновения между всеми объектами в нашем  
;; приложении  
(func $main (export "main")  
  ❶ (local $i          i32) ;; индекс внешнего цикла  
    (local $j          i32) ;; индекс внутреннего цикла  
  ❷ (local $outer_ptr   i32) ;; указатель на объект внешнего цикла  
    (local $inner_ptr   i32) ;; указатель на объект внутреннего цикла  
  
  ❸ (local $x1          i32) ;; координата x объекта внешнего цикла  
  
    (local $x2          i32) ;; координата x объекта внутреннего цикла  
    (local $y1          i32) ;; координата y объекта внешнего цикла  
    (local $y2          i32) ;; координата y объекта внутреннего цикла  
  
  ❹ (local $xdist       i32) ;; расстояние между объектами по оси x  
    (local $ydist       i32) ;; расстояние между объектами по оси y  
  
  ❺ (local $i_hit       i32) ;; булев флаг i столкновения объектов  
  ❻ (local $xv          i32) ;; скорость x  
    (local $yv          i32) ;; скорость y  
  
  ❼ (call $clear_canvas) ;; очистить холст до черного цвета  
  ...  
)
```

Эта функция имеет двойной цикл, который сравнивает каждый объект со всеми другими объектами. Для этого мы определяем две переменные цикла \$i ❶ и \$j, которые будут счетчиками цикла для

наших внешних и внутренних циклов соответственно. Нам нужно перебрать каждый объект, используя переменную \$i, и сравнить его с каждым другим объектом с помощью переменных \$j. Затем мы используем две переменные-указателя \$outer_ptr ❷ и \$inner_ptr, которые указывают на ячейку линейной памяти для двух столкнувшихся объектов.

Следующие четыре локальные переменные – это координаты x и y ❸ для объектов внутреннего и внешнего циклов. Расстояние между координатами x и y двух объектов сохраняется в локальных переменных \$xdist ❹ и \$ydist. Локальная переменная \$i_hit ❺ – это булев (логический) флаг, который имеет значение 1, если объект \$i сталкивается с другим объектом, и 0, если не сталкивается. Две переменные \$xv ❻ и \$yv хранят скорость объекта \$i. После объявления локальных переменных функция выполняет первое действие: очищает все пиксели холста до черного цвета с помощью оператора (call \$clear_canvas) ❻.

\$move_loop

Следующая часть функции \$main определяет цикл, который в каждом кадре перемещает все объекты в линейной памяти. Эта часть функции получит атрибуты \$x, \$y, \$xv и \$yv для объекта \$i. Переменные \$xv и \$yv – это переменные скорости x и y, они используются для перемещения объекта путем изменения значений координат \$x и \$y. Код также следит за тем, чтобы значения x и y оставались в границах холста. Сразу после цикла перемещения переменная \$i обнуляется. Добавьте код из листинга 8.14 в модуль WAT.

Листинг 8.14. Цикл для перемещения каждого объекта

```
collide.wat    ...
(часть 9 из 12) ❶(loop $move_loop
    ;; получить атрибут x
    (call $get_obj_attr (local.get $i) (global.get $x_offset))
    local.set $x1

    ;; получить атрибут y
    (call $get_obj_attr (local.get $i) (global.get $y_offset))
    local.set $y1

    ;; получить атрибут скорости x
    (call $get_obj_attr (local.get $i) (global.get $xv_offset))
    local.set $xv

    ;; получить атрибут скорости y
    (call $get_obj_attr (local.get $i) (global.get $yv_offset))
    local.set $yv

    ;; добавить скорость к x и заставить не выходить за границы холста
    ❷(i32.add (local.get $xv) (local.get $x1))
    i32.const 0x1fff ;; 511 в десятичной системе
```

```

❸ i32.and          ;; очистить старшие 23 бита
   local.set $x1

   ;; добавить скорость к у и заставить не выходить за границы холста
   (i32.add (local.get $yv) (local.get $y1))
   i32.const 0x1ff ;; 511 в десятичной системе
   i32.and      ;; очистить старшие 23 бита
❹ local.set $y1

   ;; задать атрибут x в линейной памяти
❺ (call $set_obj_attr
   (local.get $i)
   (global.get $x_offset)
   (local.get $x1)
  )

   ;; задать атрибут y в линейной памяти
   (call $set_obj_attr
   (local.get $i)
   (global.get $y_offset)
   (local.get $y1)
  )

   local.get $i
   i32.const 1
❻ i32.add
❼ local.tee $i      ;; увеличить $i

   global.get $obj_cnt
   i32.lt_u        ;; $i < $obj_cnt

❽ if ;; если $i < $obj_count, вернуться назад в начало $move_loop
   br $move_loop
end

)

   i32.const 0
❾ local.set $i
...

```

Код в листинге 8.14 проходит по всем нашим объектам, изменяя координаты *x* и *y* в зависимости от скоростей *x* и *y*. Первые несколько строк цикла ❶ вызывают `$get_obj_attr` для атрибутов позиции *x*, позиции *y*, скорости *x* и скорости *y*, передавая индекс цикла и сдвига вычисляемого атрибута. При этом значение атрибута проталкивается в стек. Затем выражение `local.set` используется для присвоения значения локальной переменной, которую мы будем использовать в `loop`.

Цикл прибавляет переменные скорости (`$xv` и `$yv`) к переменным положения (`$x1` и `$y1`) с вызовом `i32.add` ❷. Перед тем как `$x1` будет установлен в новую позицию, `i32.and` ❸ применяется для маскирования последних девяти разрядов позиции *x*. Переменная содержит

значение координаты x между 0 и 511, при этом значение обнуляется, если оно превышает 511. Затем то же самое делается для y ④, чтобы задать положение y . После присвоения новых значений x и y вызывается функция `$set_obj_attr` ⑤, чтобы записать их в линейную память. Счетчик цикла i ⑥ увеличивается с вызовом `i32.and` и `local.tee` ⑦. Если значение в i меньше, чем количество объектов (`$obj_count` ⑧), код возвращается к началу цикла. В ином случае вызывается `local.set` ⑨ для обнуления i .

Квадраты в листинге 8.14 находятся на холсте, и когда какой-то из них выходит за пределы экрана вправо, он снова появляется в левой части экрана, как в олдскульных аркадных играх. Такие игры, как Asteroids от Atari и Pac-Man от Namco, не нуждались в дополнительном коде, чтобы реализовать подобный эффект; вместо этого их экраны имели ширину 256 пикселей и использовали 8-битное число для хранения координаты x . Таким образом, если объект в игре имеет x -координату 255 и перемещается на один пиксель вправо, однобайтовое значение возвращается к нулю, и игровой объект снова появляется в левой части экрана, как показано на рис. 8.6.

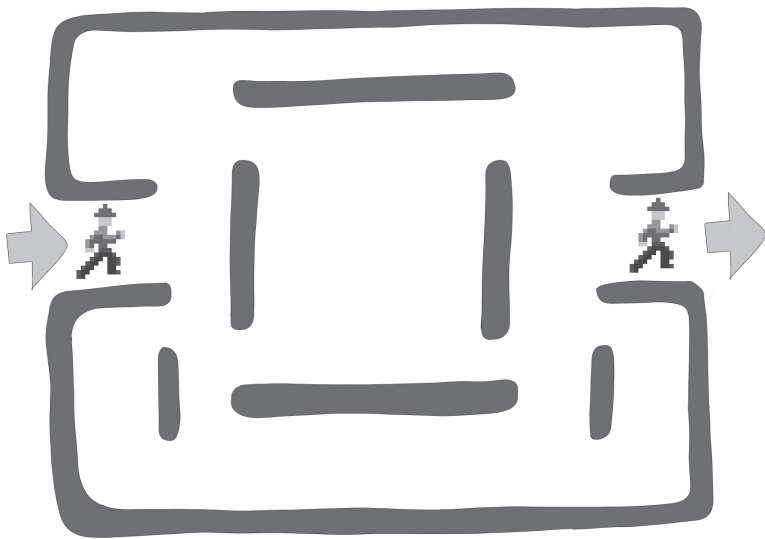


Рис. 8.6. Персонаж уходит с экрана справа и появляется слева

Мы можем добиться этого эффекта с помощью `i32.and`, чтобы замаскировать все разряды числа, кроме младших девяти. Чтобы получить атрибуты для x , y , xv и yv ①, код вызывает `$get_obj_attr`. Новое значение x вычисляется путем прибавления xv ② и последующего применения логической операции `i32.and` ③ к константе `0x1ff`. В двоичном формате `0x1ff` имеет девять младших битов, установленных в 1, а все старшие биты установлены в 0. В главе 4 мы узнали, как использовать `i32.and` для установки определенных битов в 0, как показано на рис. 8.7.

другим объектом. Это продолжается до тех пор, пока внешний цикл не проверит каждый объект. Добавьте код из листинга 8.15 в функцию \$main.

Листинг 8.15. Внешний цикл двойного цикла обнаружения столкновений

```
collide.wat
(часть 10 из 12) ...
(loop $outer_loop (block $outer_break
  i32.const 0
  ❶ local.tee $j      ;; задаем j равным 0

  ;; $i_hit - логическое значение. 0 - ложь, 1 - истина
  ❷ local.set $i_hit  ;; задаем i_hit равным 0

  ;; получить атрибут x для объекта $i
  (call $get_obj_attr (local.get $i) (global.get $x_offset))
  ❸ local.set $x1
  ;; получить атрибут y для объекта $i
  (call $get_obj_attr (local.get $i) (global.get $y_offset))
  ❹ local.set $y1
  ...
)
```

В начале цикла \$j ❶ и \$i_hit ❷ сбрасываются в 0. Затем код вызывает функцию \$get_obj_attr, чтобы найти значения для \$x1 ❸ и \$y1 ❹.

Внутренний цикл

Следующий раздел функции \$main – это внутренний цикл, который нужен для обнаружения столкновения между двумя квадратами. Обнаруживать столкновения квадратов очень просто: вы сравниваете координаты x и размер, чтобы увидеть, накладываются ли друг на друга объекты по оси x. Если по оси x наложения нет, а по оси y – есть, это выглядит как на рис. 8.8, и столкновение в таком случае не происходит.

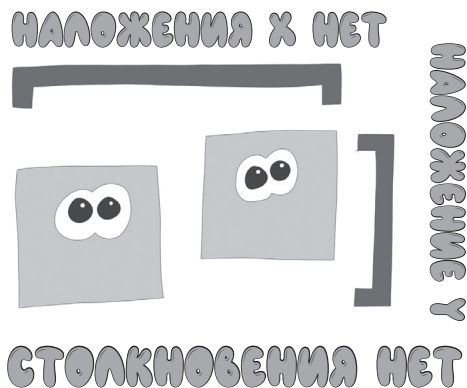


Рис. 8.8. По оси x наложения нет, а по оси y – есть

Если по оси x наложение есть, а по оси y – нет, столкновение между объектами тоже не происходит, как на рис. 8.9.

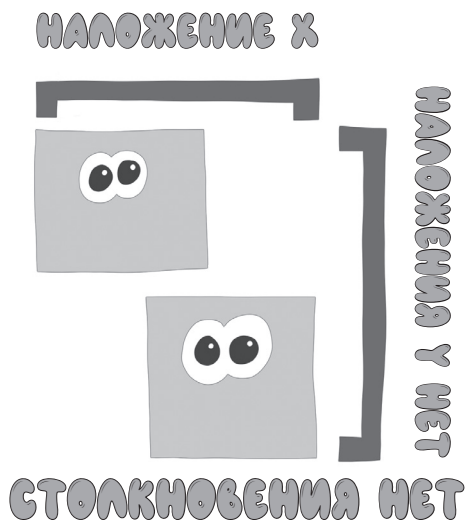


Рис. 8.9. По оси x наложение есть, а по оси y – нет

Единственный сценарий столкновения – это одновременное наложение объектов по осям x и y , как показано на рис. 8.10.



Рис. 8.10. Происходит наложение по осям x и y , поэтому возникает столкновение

Внутренний цикл выполняет эту проверку для каждого объекта в линейной памяти. В листинге 8.16 показан соответствующий код.

Листинг 8.16. Внутренний цикл двойного цикла обнаружения столкновений

```

collide.wat
...
(часть 11 из 12) ❶ (loop $inner_loop (block $inner_break
    local.get $i
    local.get $j
    i32.eq
    ❷ if      ;; если $i == $j, увеличить $j на 1
        local.get $j

```

```

        i32.const 1
        i32.add
        local.set $j
    end

    local.get $j
    global.get $obj_cnt
    i32.ge_u
    ③ if      ;; если $j >= $obj_count, завершить внутренний цикл
        br $inner_break
    end

    ;; получить атрибут x
    ④ (call $get_obj_attr (local.get $j)(global.get $x_offset))
    local.set $x2 ;; установить атрибут x для объекта внутреннего цикла

    ;; расстояние между $x1 и $x2
    (i32.sub (local.get $x1) (local.get $x2))

    call $abs ;; расстояние не отрицательно, получить абсолютное значение
    ⑤ local.tee $xdist ;; $xdist = абсолютное значение ($x1 - $x2)

    global.get $obj_size
    i32.ge_u

    ⑥ if ;; если $xdist >= $obj_size объект не сталкивается с другим объектом
        local.get $j
        i32.const 1
        i32.add
        local.set $j

        br $inner_loop ;; увеличить $j и перейти в начало внутреннего цикла
    end

    ;; получить атрибут y
    (call $get_obj_attr (local.get $j)(global.get $y_offset))
    local.set $y2

    (i32.sub (local.get $y1) (local.get $y2))
    call $abs
    ⑦ local.tee $ydist

    global.get $obj_size
    i32.ge_u

    ⑧ if
        local.get $j
        i32.const 1
        i32.add
        local.set $j

        br $inner_loop
    end

    i32.const 1
    ⑨ local.set $i_hit

```

```

        ;; выйти из цикла, если есть столкновение
    )) ;; завершить внутренний цикл
...

```

Внутренний цикл `loop` ❶ сравнивает текущий объект внешнего цикла с каждым другим объектом, который еще не был проверен. Но прежде ему нужно убедиться, что он не сравнивает объект с самим собой. Объект всегда сталкивается сам с собой, поэтому если `$i` равно `$j` ❷, мы игнорируем этот факт столкновения и увеличиваем `$j`. Затем мы сравниваем `$j` с `$obj_cnt` ❸, чтобы увидеть, все ли объекты в коде уже проверены. Если да, код выходит из внутреннего цикла.

Если код не проверил все объекты, атрибут `x` загружается в переменную `$x2` путем вызова `$get_obj_attr` ❹. Затем мы получаем расстояние между `$x1` и `$x2`, вычитая `$x2` из `$x1`, получая абсолютное значение, и устанавливаем переменную `$xdist` ❺. Мы сравниваем расстояние по оси `x` между двумя объектами с размером объекта, чтобы увидеть, накладываются ли два объекта по оси `x`. Если они не накладываются, потому что `$xdist` больше, чем `$obj_size` ❻, код увеличивает `$j` и возвращается к началу цикла.

Таким же образом вычисляется расстояние `y` и сохраняется в переменной `$ydist` ❼, а код проверяет, больше ли переменная `$ydist`, чем `$obj_size` ❸. Если это так, эти объекты не сталкиваются, поэтому мы увеличиваем `$j` и возвращаемся к началу цикла. Если мы в этот момент не перешли в начало цикла, мы знаем, что оси `x` и `y` объектов накладываются, что указывает на столкновение, поэтому мы устанавливаем `$i_hit` ❽ равным 1 и выходим из внутреннего цикла.

Перерисовка объектов

Если код вышел из внутреннего цикла, то случилось одно из двух: либо было обнаружено столкновение, либо его не было. Код проверяет переменную столкновения (`$i_hit`) и вызывает функцию `$draw_obj` с цветом отсутствия столкновений (зеленый), если столкновения не было, и с `$hit_color` (красный), если столкновение произошло. Затем `$i` увеличивается, и если `$i` меньше количества объектов, код возвращается к началу внешнего цикла. В листинге 8.17 показана последняя часть кода, которую нужно добавить в WAT-файл.

Листинг 8.17. Рисование объекта во внутреннем цикле

```

collide.wat ...
(часть 12 из 12)
    local.get $i_hit
    i32.const 0
    i32.eq
    ❶ if      ;; если $i_hit == 0 (нет столкновения)
        (call $draw_obj
            (local.get $x1) (local.get $y1) (global.get $no_hit_color))

```



```

❷ else    ;; если $i_hit == 1 (есть столкновение)
    (call $draw_obj
      (local.get $x1) (local.get $y1) (global.get $hit_color))
    end

    local.get $i
    i32.const 1
    i32.add
❸ local.tee $i      ;; увеличить $i

    global.get $obj_cnt
    i32.lt_u
❹ if          ;; если $i < $obj_cnt, перейти в начало внешнего цикла
    br $outer_loop
end
❺ )) ;; завершить внешний цикл
) ;; конец функции
) ;; конец модуля

```

Сразу после завершения внутреннего цикла внешний цикл проверяет, установлено ли для переменной `$i_hit` значение 0 ❶, что указывает на отсутствие столкновения. Если это так, мы вызываем `$draw_obj`, передавая глобальную переменную `$no_hit_color` в качестве последнего параметра, и функция рисует квадрат зеленого цвета. Если переменная `$i_hit` установлена в 1 ❷ (истина), мы передаем функции `$draw_obj` значение `$hit_color` (красный). На этом этапе значение `$i` ❸ увеличивается, и если оно меньше `$obj_cnt` ❹, т. е. рисование наших объектов не завершено, код возвращается к началу цикла. Если нет ❺, код выходит из цикла, и эта функция завершается.

Компиляция и запуск приложения

Прежде чем мы запустим приложение для обнаружения столкновений, нам нужно скомпилировать наш WAT в модуль WebAssembly. Используйте следующую команду `wat2wasm` для компиляции `collide.wat` в `collide.wasm`:

```
wat2wasm collide.wat
```

Когда вы запускаете веб-сервер и открываете файл `collide.html` в веб-браузере с `localhost`, на вашем экране должно появиться изображение, аналогичное рис. 8.11.

Квадраты должны перемещаться по холсту, появляясь на левой стороне холста после ухода за правую границу, и вверх, когда они выходят за границу холста вниз. Блоки, которые сталкиваются друг с другом, отображаются красным цветом, а блоки, которые не сталкиваются с другими блоками, отображаются зеленым.

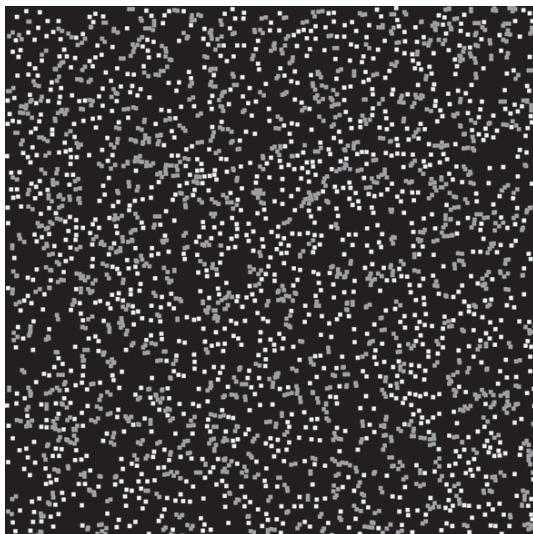


Рис. 8.11. Приложение столкновений

Заключение

В этой главе мы исследовали, как WebAssembly и HTML-холст могут работать вместе для создания фантастической анимации на веб-странице. Теперь вы должны понимать, когда лучше всего использовать WebAssembly, а когда JavaScript. Рендеринг на холсте из WebAssembly можно выполнить быстро и эффективно, изменив местоположения в памяти, которые представляют данные растрового изображения. Созданное нами приложение столкновений использует множество констант JavaScript для определения деталей. Это позволило нам манипулировать числами в приложении без перекомпиляции модуля WebAssembly. JavaScript может легко генерировать случайные числа, которые мы можем использовать в приложении. Генерация случайных чисел из WebAssembly намного сложнее. Поскольку случайные числа нужно генерировать только один раз, использование JavaScript не сильно снижает производительность.

Вы узнали, что такое данные растрового изображения и как использовать WebAssembly для создания этих данных в линейной памяти. Мы использовали функцию `requestAnimationFrame` из JavaScript для вызова модуля WebAssembly один раз для каждого кадра, применяя WebAssembly для генерации данных растрового изображения, которое будет отрисовано на холсте. Эти данные изображения перемещаются в HTML-холст с помощью функции `putImageData` в контексте холста.

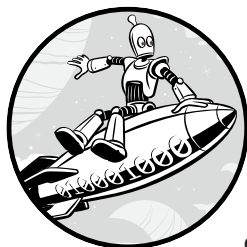
В коде модуля WAT мы настроили и изменили область памяти, выделенную для данных изображения, и создали функцию очистки холста. Мы нарисовали на холсте пиксели определенными цветами; затем рисовали объекты размером больше одного пикселя и застав-

ляли эти объекты появляться на противоположной стороне холста, когда они выходили за его пределы. Наконец, использовали обнаружение столкновений квадратов, чтобы определять и изменять цвет наших объектов, если между ними были столкновения.

При всем этом мы не потратили много времени и усилий, пытаясь сделать приложение столкновений быстрым. В следующей главе мы заставим это приложение работать как можно быстрее путем небольшой оптимизации производительности.

9

ОПТИМИЗАЦИЯ ПРОИЗВОДИТЕЛЬНОСТИ



Эта глава предназначена непосредственно для разработчиков, у которых есть возможность уделить много времени оптимизации производительности приложения. Сначала мы обсудим инструменты профилировщика для оценки производительности модуля WebAssembly и исследуем, как сравнить производительность кода WebAssembly с аналогичным кодом JavaScript. Мы потратим некоторое время на изучение стратегий повышения производительности кода WebAssembly, включая встраивание функций, замену умножения и деления на битовые сдвиги, комбинирование констант и удаление кода с помощью DCE (Dead Code Elimination, исключение неработающего кода).

Мы также рассмотрим другие методы определения производительности модуля: будем использовать `console.log` и `Date.now` для замера производительности нашего приложения и применять набор инструментов тестирования *benchmark.js* для сбора подробных данных о производительности приложения. Затем, просто для удовольствия, выведем в консоль байт-код промежуточного представления (IR – Intermediate Representation) движка Chrome JavaScript V8 для функции

JavaScript. Байт-код IR JavaScript может дать вам представление о работе, которую выполняет функция JavaScript, что полезно для выбора, где писать функцию – в WebAssembly или JavaScript.

Использование профилировщика

Профилировщики – это инструменты, которые анализируют различные аспекты производительности приложения, включая использование памяти приложением и время выполнения. Они помогут вам принять решение о том, где и для чего выполнять оптимизацию. Вам часто придется искать компромисс между различными типами оптимизации. Например, вам нужно будет решить, следует ли сосредоточиться на сокращении *времени достижения интерактивности* (TTI – time to interactive), чтобы пользователи могли начать использовать приложение как можно скорее, или же сосредоточиться на максимальной производительности уже запущенного приложения. Если вы разрабатываете игру, лучше иметь долгую загрузку и дальнейшую плавную работу приложения, чем наоборот. Однако разработчики, например, интернет-магазина предпочитают, чтобы покупатель мог как можно быстрее начать взаимодействовать с веб-сайтом. В большинстве случаев вам будет необходимо искать баланс между этими двумя аспектами, с чем и поможет профилировщик.

Профилировщики также эффективны при поиске узких мест в вашем коде, позволяя вам сосредоточить свое время и усилия на этих местах. Мы рассмотрим профилировщики Chrome и Firefox, потому что в настоящее время они лучше всего поддерживают WebAssembly. Будем профилировать приложение для обнаружения столкновений из главы 8.

Профилировщик Chrome

Давайте откроем новое окно браузера Chrome в режиме инкогнито с профилировщиком. Окна в режиме инкогнито не загружают кешы веб-сайтов, файлы cookie или подключаемые модули Chrome, что вызывает проблемы при профилировании, поскольку они запускают дополнительный код JavaScript и влияют на производительность сайта, который вы хотите профилировать. Кешы и файлы cookie обычно менее проблематичны, но могут засорять вашу среду данными, не имеющими отношения к профилируемому коду. Вы можете открыть окно в режиме инкогнито из меню в правом верхнем углу браузера, щелкнув **New incognito window** (Новое окно в режиме инкогнито), как показано на рис. 9.1.

После открытия окна в режиме инкогнито убедитесь, что ваш веб-сервер запущен с помощью команды `node server.js` из командной строки, и введите **localhost:8080/collide.html** в свой веб-браузер. Выберите **More Tools** (Дополнительные инструменты), затем **Developer tools** (Инструменты разработчика) в меню в правом верхнем углу, как показано на рис. 9.2.

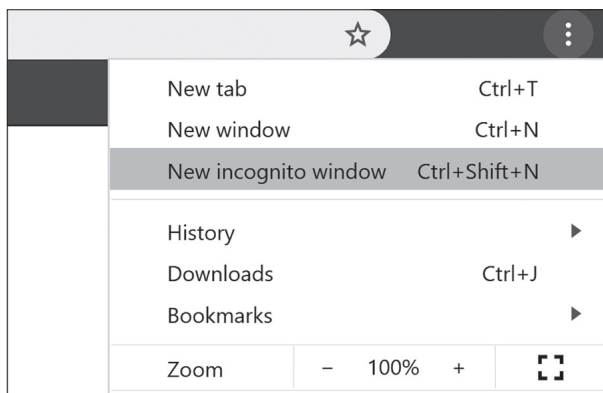


Рис. 9.1. Открыть New incognito window в Chrome

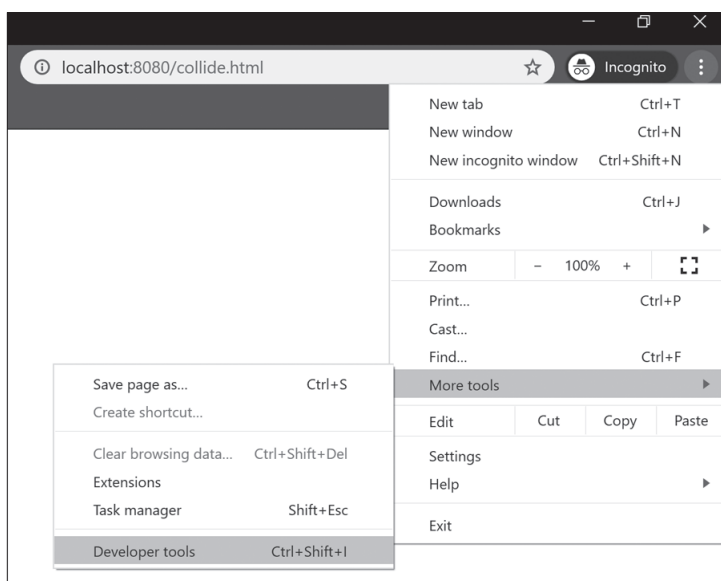


Рис. 9.2. Окно **Developer tools** в Chrome

Вы увидите несколько вкладок в верхней части инструментов разработчика. Чтобы увидеть профилировщик, нажмите **Performance** (Производительность), как показано на рис. 9.3.

Вкладка **Performance** при первом открытии предлагает два варианта: **Record** (Запись) и **Reload** (Начать профилирование и обновить страницу). Кнопка **Record** начинает запись профиля без перезагрузки приложения. Этот вид профилирования наиболее полезен, когда вам не так важно время запуска приложения, нежели максимальная производительность. Прежде чем приступить к профилированию, убедитесь, что установлен флажок **Memory** (Память) в верхней части вкладки **Performance**. Если этого не сделать, большое количество па-

мяти не будет профилировано. Если вы хотите профилировать свое приложение с момента инициализации, нужно нажать кнопку **Reload**. Нажмите **Record**, чтобы продолжить наши действия. Через пять секунд записи нажмите **Stop** (Стоп), как показано на рис. 9.4.

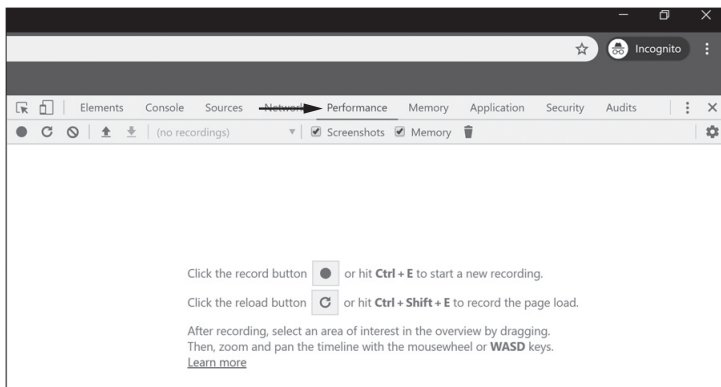


Рис. 9.3. Окно вкладки **Performance** в Chrome

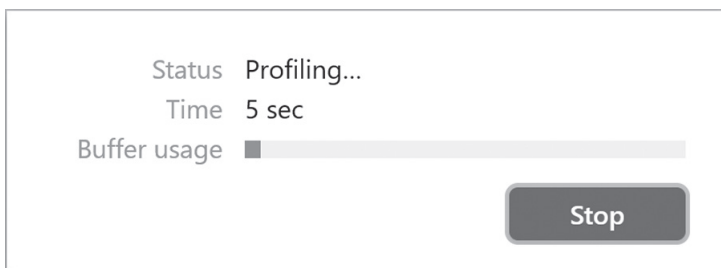


Рис. 9.4. Запись профиля в Chrome

Когда запись остановится, откроется профилировщик и покажет запись каждого фрейма, отображаемого вашим приложением. В середине вкладки **Summary** (Сводка) показано, что большая часть времени выполнения этого приложения относится к разделу **Scripting** (рис. 9.5), который отражает время выполнения скриптов JavaScript и WebAssembly.

На главной странице **Performance** круговая диаграмма показывает время, затраченное на обработку сценариев, рендеринга, отрисовки, системы и бездействия. Над круговой диаграммой находится ряд вкладок, включая **Summary**, **Bottom-Up** (Снизу вверх), **Call Tree** (Дерево вызовов) и **Event Log** (Журнал событий), которые мы рассмотрим в этой главе. В разделе над этими вкладками показана динамическая память JS (JS Heap), а выше – информация об обработанных фреймах, ЦП и FPS. Давайте кратко рассмотрим динамическую память JavaScript в следующем разделе.

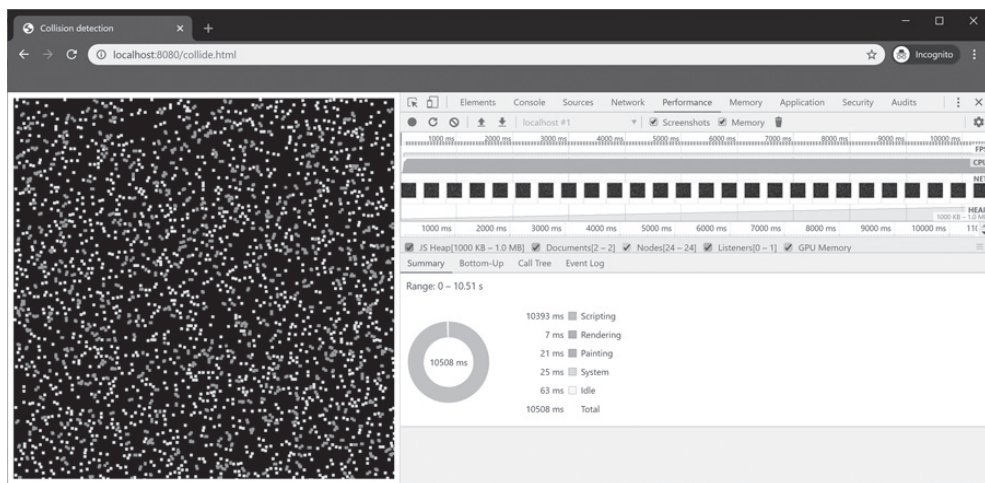


Рис. 9.5. Вкладка **Performance** в Chrome после записи профиля

Динамическая память JavaScript

В профиле на рис. 9.5 показано, что объем динамической памяти увеличился. Выясним, почему это произошло. Во-первых, нужно проверить, сколько памяти выделено, прежде чем она будет автоматически очищена. Некоторые разработчики считают, что поскольку JavaScript – это язык с автоматической очисткой динамической памяти, им не нужно беспокоиться об этом. К сожалению, это не так; код по-прежнему может создавать объекты быстрее, чем они будут автоматически удалены. Также ссылки на объекты могут удерживаться дольше, чем необходимо, в результате чего JavaScript может не понять, следует ли удалять их. Если приложение увеличивает объем памяти так же быстро, как поступают данные, имеет смысл посмотреть, сколько памяти выделяется перед ее автоматической очисткой. Затем нужно понять, где приложение выделяет память. Сейчас объем динамической памяти составляет около 1 МБ.

После некоторого дополнительного профилирования мы видим, что объем JS Heap увеличивается до 2,2 МБ, а после запуска автоматической очистки объем памяти снова уменьшается до 1,2 МБ. До первого запуска автоматической очистки может пройти несколько минут, так что проявите терпение. На рис. 9.6 показан профиль JS Heap во время автоматической очистки. Как вы можете видеть, в правой части графика объем памяти резко уменьшился до 1 МБ.

Нужно точно определить, где происходит это выделение памяти, потому что если замедлить рост объема динамической памяти, это потенциально снизит нагрузку на автоматическую очистку.

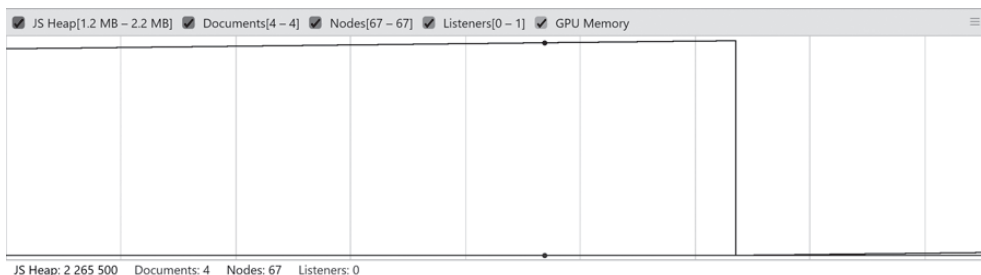


Рис. 9.6. Уменьшение объема памяти после очистки

Выделение памяти

Поскольку рост динамической памяти происходит постоянно, мы можем предположить, что, скорее всего, выделение памяти происходит при каждой визуализации кадра. Большая часть работы этого приложения выполняется в модуле WebAssembly, поэтому сначала мы закомментируем вызов WebAssembly, чтобы увидеть, продолжает ли память демонстрировать тот же профиль роста JS Heap. Откройте файл *collide.html* и закомментируйте вызов `animation_wasm()` внутри функции `animate`, как показано в листинге 9.1.

Листинг 9.1. Закомментированный вызов функции `animation_wasm`

```
collide.html ...
function animate() {
  //   animation_wasm();
  ctx.putImageData(image_data, 0, 0); // рендер данных пикселей
  requestAnimationFrame(animate);
}
...
```

Теперь перезагрузите страницу и заново сделайте запись профиля. На рис. 9.7 показан новый профиль JS Heap без вызова функции `animation_wasm`.

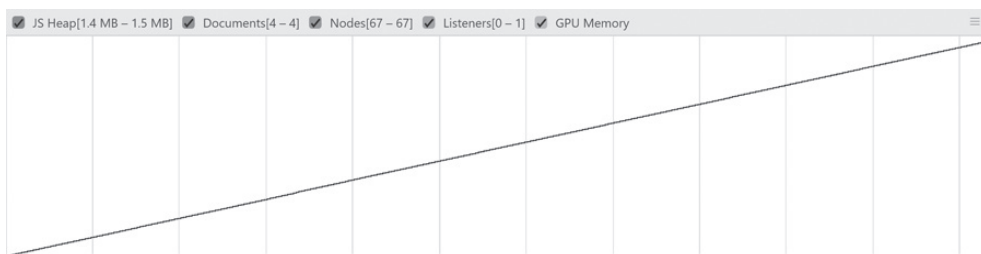


Рис. 9.7. График выделения динамической памяти после удаления функции `animation_wasm`

Без вызова модуля WebAssembly приложение больше не работает должным образом. Однако вы по-прежнему можете видеть тот же профиль роста JS Heap, значит, увеличение объема памяти происходит не из модуля WebAssembly. Раскомментируйте вызов модуля WebAssembly; затем закомментируйте вызов `ctx.putImageData` и создайте новый профиль, как показано в листинге 9.2.

Листинг 9.2. Функция `animation_wasm` возвращена; вызов `putImageData` удален

```
collide.html    function animate() {  
                animation_wasm();  
                // ctx.putImageData(image_data, 0, 0); // рендер данных пикселей  
                requestAnimationFrame(animate);  
            }  
    
```

Закомментировав вызов `ctx.putImageData`, мы можем создать новый профиль для проверки увеличения объема памяти (рис. 9.8).

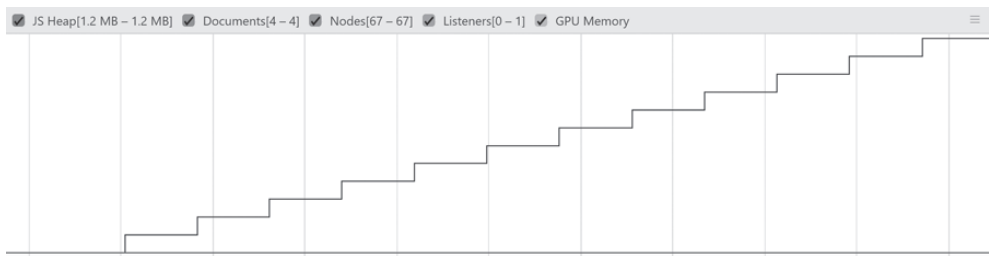


Рис. 9.8. Если удалить вызов `putImageData`, рост памяти замедляется

Без вызова `ctx.putImageData` рост памяти сильно замедлился. Она все еще растет, но на диаграмме мы видим ступеньки, а не прямую линию вверх. Похоже, что вызов `ctx.putImageData` создает большие объекты, которые сборщик мусора в конечном итоге должен удалить. Теперь мы знаем, как выделяется эта память. Поскольку `ctx.putImageData` – встроенная функция, мы ничего не можем сделать для ее оптимизации. Если бы проблема заключалась в выделении памяти, нам нужно было бы найти альтернативные средства для отрисовки изображений на холсте.

Кадры

В окне профилировщика есть область над динамической памятью, которая предоставляет дополнительную информацию о производительности, в том числе количество отображаемых кадров в секунду (frames per second, FPS). Она также показывает график, отображающий нагрузку на ЦП. Еще там показаны маленькие эскизы каждого отображенного кадра. Когда вы наводите указатель мыши на эти

кадры, вы можете наблюдать, как ваше приложение визуализирует свою анимацию (рис. 9.9), что может быть очень полезно, если ваше приложение не работает должным образом.

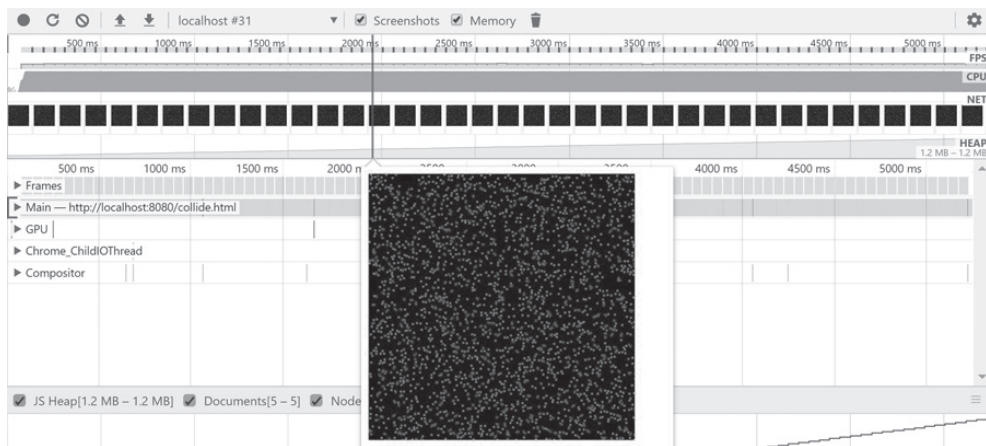


Рис 9.9. Рендеринг отдельного кадра в профилировщике

Вы можете навести указатель мыши на зеленые поля **Frames** (Кадры), чтобы увидеть значение FPS в любой точке профиля (рис. 9.10).

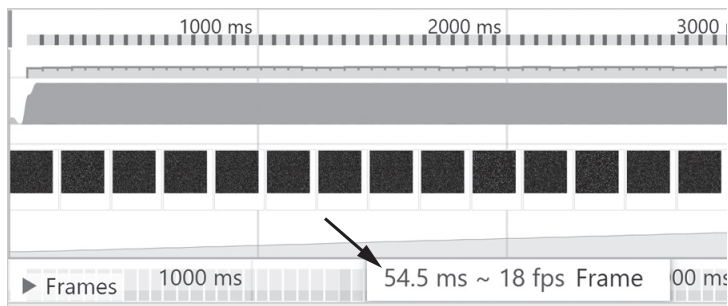


Рис. 9.10. Просмотр значения FPS в профилировщике

Как видите, частота кадров на этом этапе выполнения приложения составляет 18 fps. Когда мы прокручиваем кадры, число колеблется между 17 и 20. Кадры в секунду являются основным показателем производительности приложения для обнаружения столкновений, поэтому нам нужно запомнить этот результат – примерно 18 fps, чтобы сравнить его с более поздними результатами. Имейте в виду, что запуск профилировщика, по-видимому, вредит производительности приложения, поэтому хотя результаты профилировщика полезно сравнивать между собой, они могут быть не совсем точными в отношении того, как приложение работает в реальных условиях.

Bottom-Up

На вкладке **Bottom-Up** показаны функции, вызываемые в приложении, общее время их выполнения и время одного действия, которое представляет собой количество времени, в течение которого функция выполнялась, за исключением времени, затраченного на функции, которые они вызывают. Окно **Self Time** (Время своего действия) очень полезно, потому что функции, вызывающие другие функции, выполнение которых занимает много времени, всегда будут показывать более длительное время **Total Time** (Общее время выполнения), как вы можете видеть на рис. 9.11.

Summary Bottom-Up Call Tree Event Log			
Filter		No Grouping	
Self Time	Total Time	Activity	
4771.8 ms 92.5 %	4771.8 ms 92.5 %	▶ <wasm-unnamed>	
347.1 ms 6.7 %	5134.4 ms 99.5 %	▶ animate	
13.9 ms 0.3 %	13.9 ms 0.3 %	▶ putImageData	
11.0 ms 0.2 %	11.0 ms 0.2 %	▶ Composite Layers	
6.5 ms 0.1 %	5146.0 ms 99.7 %	▶ Animation Frame Fired	
5.1 ms 0.1 %	5139.5 ms 99.6 %	▶ Function Call	
3.6 ms 0.1 %	3.6 ms 0.1 %	▶ Update Layer Tree	
1.6 ms 0.0 %	1.6 ms 0.0 %	▶ requestAnimationFrame	

Рис. 9.11. Окно вкладки **Bottom-Up** в Chrome

Время своего действия `<wasm-unnamed>` пока что является самым длинным. Общее время выполнения больше для нескольких функций, таких как `animate`, потому что эти функции вызывают модуль WebAssembly. Немного разочаровывает то, что Chrome не указывает, какую функцию он вызывает внутри модуля WebAssembly, но мы можем сразу определить, что приложение тратит более 90 % времени обработки на выполнение кода WebAssembly.

Профилировщик Firefox

Использование профилировщика Firefox – еще один отличный способ собрать данные о производительности вашего приложения. Я рекомендую открывать его в окне инкогнито. Сделайте это, открыв меню в правом верхнем углу браузера и щелкнув **New Private Window** (Новое приватное окно) (рис. 9.12).

Откройте профилировщик в **Web Developer**, далее **Performance** (рис. 9.13).

В меню **Performance** нажмите кнопку **Start Recording Performance** (Начать запись производительности), чтобы записать данные. Через несколько секунд остановите запись. На рис. 9.14 показан пример вкладки **Performance**. На вкладке **Waterfall** (Ниспадающее изображение), которая появляется по умолчанию после записи, показаны вызовы функций верхнего уровня и время их выполнения.

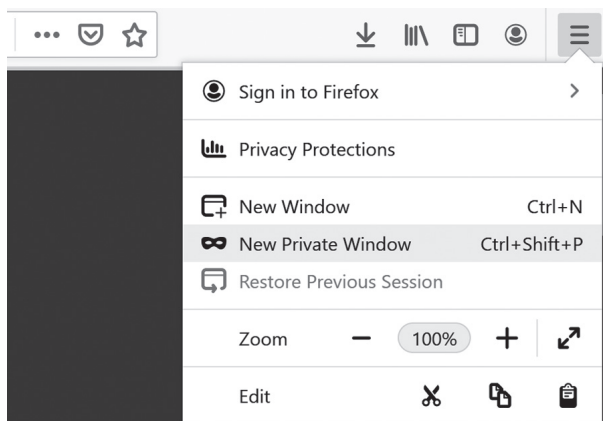


Рис. 9.12. Новое приватное окно в Firefox

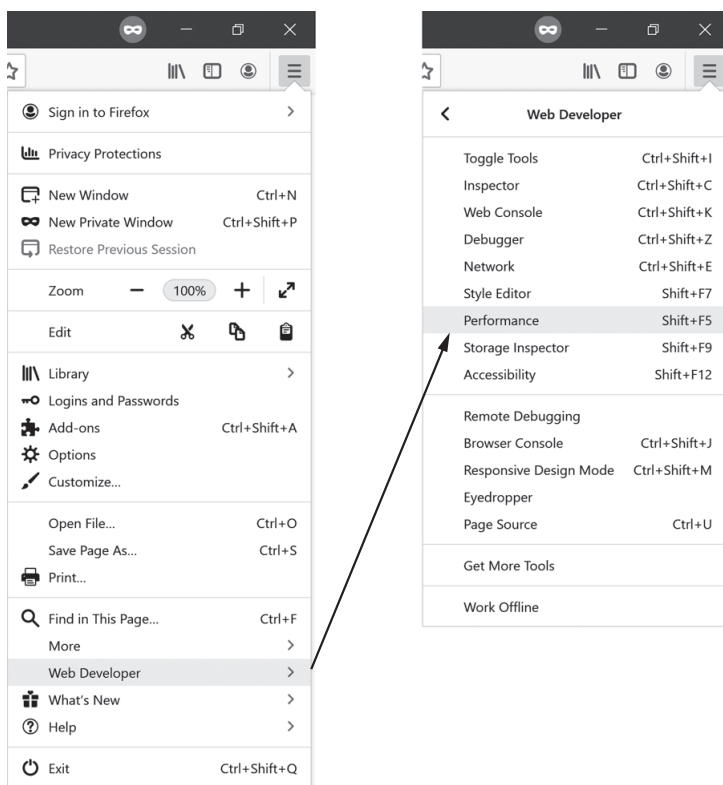


Рис. 9.13. Нажмите **Web Developer**, далее **Performance** в меню Firefox

Прокрутите вниз, чтобы увидеть, где происходит автоматический сбор мусора и сколько времени требуется для его выполнения. Этот момент не очень интересен для нашего приложения, которое в основ-

ном выполняет вызов `requestAnimationFrame`. Три вкладки в верхней части окна предоставляют дополнительную информацию. Вкладка **Waterfall** дает вам общее представление о том, какие задачи выполняются слишком долго. Мы не будем вдаваться в подробности о вкладке **Waterfall**, потому что это скорее *краткий обзор времени выполнения*. Вместо этого рассмотрим вкладки **Call Tree** и **JS Flame Chart**.

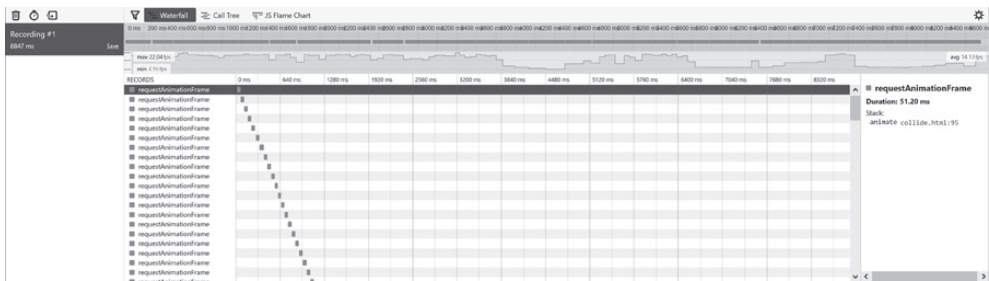


Рис. 9.14. Вкладка **Waterfall** в окне **Performance** в Firefox

Call Tree

На вкладке **Call Tree** показаны вызовы функций, на которые приложение тратит большую часть своего времени. Интерфейс позволяет вам детально изучить каждую из функций и увидеть, какие вызовы они совершают. На рис. 9.15 показан снимок экрана вкладки **Call Tree**.

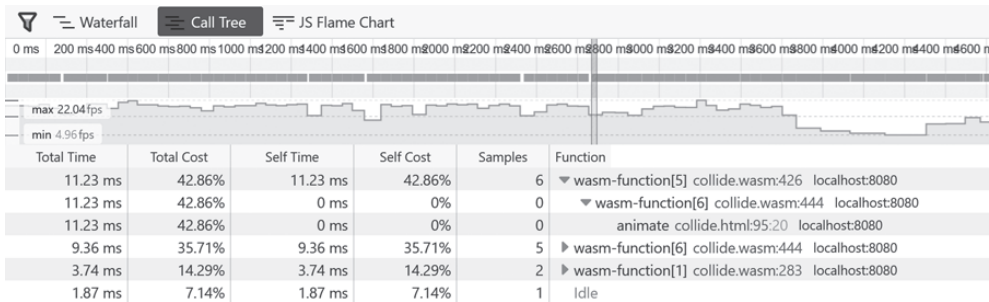


Рис. 9.15. Вкладка **Call Tree** в Firefox

Удобно то, что вы можете щелкнуть имя своего файла WebAssembly, и ссылка приведет вас к нужной функции в вашем коде WebAssembly. Имена функций потеряны, но индекс, показывающий номер функции в WAT, следует за меткой `wasm-function`. Это немного упрощает определение того, что именно вызывает функция.

JS Flame Chart

Вкладка **JS Flame Chart** содержит почти ту же информацию, которую вы видите на вкладке **Call Tree**, но она организована по временной

шкале, а не в виде сводки. Вы можете увеличить определенную часть диаграммы, чтобы увидеть, какие функции выполняются в этой точке профиля (рис. 9.16).

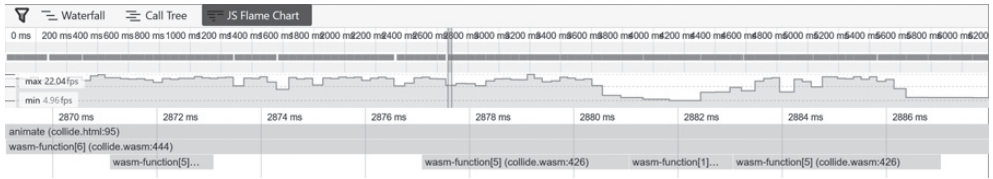


Рис. 9.16. Вкладка **JS Flame Chart** в Firefox

Мы видим вызов функции JavaScript `animate`. Функция `animate` большую часть времени выполняет `wasm-function[6]`, которая является седьмой функцией в нашем WAT-коде и называется `$main`. Функция `$main` вызывает `wasm-function[5]`, которая является шестой функцией (`$get_obj_attr`), и `wasm-function[1]` (`$abs`).

Каждая из этих вкладок показывает минимальный и максимальный FPS с левой стороны и средний FPS с правой стороны. Левая часть профилировщика выглядит примерно так, как показано на рис. 9.17.

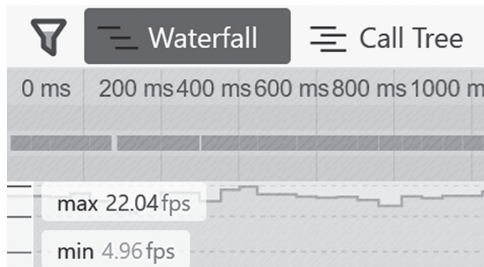


Рис. 9.17. Максимальное и минимальное значения FPS в Firefox

Как видите, максимальный FPS чуть больше 22, а минимальный – чуть меньше 5. Ранее я упомянул, что запуск профилировщика может влиять на уровень FPS. Среднее значение FPS указано в правой части профилировщика (рис. 9.18).

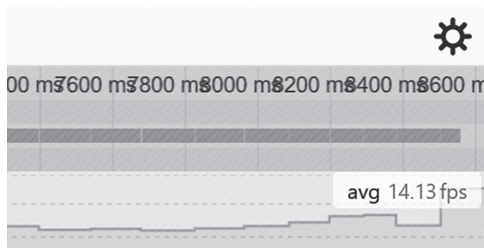


Рис. 9.18. Среднее значение FPS в Firefox

Среднее значение FPS для этого профиля примерно равно 14. В следующем разделе мы рассмотрим, как повысить производительность приложения с помощью `wasm-opt`.

wasm-opt

Мы используем инструмент командной строки `wasm-opt` для оптимизации производительности файла WebAssembly. Он поставляется с `wat-wasm` и `Binaryen.js`. Если вы уже установили `wat-wasm` для использования с инструментом `wat2wasm`, у вас есть нужная версия `wasm-opt`, поэтому вы можете пропустить следующий раздел. Если нет, установите `Binaryen.js`, который является версией JavaScript инструмента `Binaryen WebAssembly` для преобразования промежуточного представления (IR) в код WebAssembly. В нем есть несколько полезных опций для оптимизации кода WebAssembly.

Установка Binaryen

Есть несколько вариантов установки `Binaryen`. Я рекомендую использовать `Binaryen.js`, который вы можете установить с помощью `npm`:

```
npm install binaryen -g
```

Пользователи, желающие собрать приложение из исходного кода, могут получить его на GitHub по адресу <https://github.com/WebAssembly/binaryen>. Существует также пакет `npm` под названием `wasm-opt`, который для конкретной платформы устанавливает двоичные файлы для `Binaryen`, но вместо этого я бы рекомендовал установить `wat-wasm` или `binaryen.js` с помощью `npm`.

Запуск wasm-opt

У инструмента `wasm-opt` есть несколько флагов, которые вы можете использовать для минимизации скорости загрузки и оптимизации времени выполнения вашего модуля WebAssembly. Эти флаги используются, чтобы указать оптимизатору, на чем ему сосредоточиться – на производительности или скорости загрузки. Если можно внести изменения для уменьшения размера файла без ущерба для производительности, эти изменения будут сделаны в любом случае. То же самое произойдет, если можно внести изменения для повышения производительности, не влияя на скорость загрузки. Эти флаги сообщают компилятору, какую оптимизацию следует предпочесть при необходимости учитывать компромисс.

Мы запустим `wasm-opt` для нашего файла `collide.wasm` с обоими типами флагов, начиная с настройки оптимизации размера, а затем снова скомпилируем код с настройкой производительности. Эти флаги будут одинаковыми для любого набора инструментов, использу-

ющего Binaryen, например таких как Emscripten или AssemblyScript. Первые два флага, которые мы рассмотрим, оптимизируют размер файла WAT.

Оптимизация под размер загрузки

У команды `wasm-opt` есть два флага, которые оптимизируют размер файла WebAssembly для загрузки: `-Oz` и `-Os`. Обратите внимание, что `O` – это заглавная буква, а не ноль. Флаг `-Oz` создает файл WebAssembly меньшего размера, но для уменьшения размера файла требуется больше времени. Файл `-Os` создает файл WebAssembly немного большего размера, но для его создания потребуется уже меньше времени. Наше приложение небольшое, поэтому любая оптимизация будет выполнена очень быстро. Если вы создаете большой проект в Emscripten, компиляция которого займет много времени, можно использовать флаг `-Os`. Для наших целей нам не нужно использовать `-Os`. В листинге 9.3 показано, как оптимизировать наш файл `collide.wasm` для уменьшения его размера с помощью флага `-Oz`.

Листинг 9.3. Запуск `wasm-opt` для оптимизации файла `collide.wasm` под размер загрузки

```
wasm-opt collide.wasm -Oz -o collide-z.wasm
```

Когда вы запускаете эту оптимизацию, размер файла WebAssembly уменьшается с 709 до 666 байт. Это всего лишь примерно 6 %, зато нам это не составило труда. Для того чтобы получить эффективное сжатие, используйте этот флаг с набором инструментов.

Оптимизация времени выполнения

При разработке игр нас больше интересует FPS, чем время скачивания. Есть три параметра оптимизации: `-O1`, `-O2` и `-O3`. Помните, что `O` – это буква, а не ноль. Флаг `-O3` обеспечивает наивысший уровень оптимизации, но требует больше всего времени для обработки. Флаг `-O1` обеспечивает наиболее быстрый процесс, но наименьшую оптимизацию. Флаг `-O2` находится где-то посередине. Поскольку наше приложение очень маленькое, нет существенной разницы между временем выполнения `-O1` и `-O3`. В листинге 9.4 мы используем флаг `-O3`, чтобы получить максимальную отдачу от нашей оптимизации файла `collide.wasm`.

Листинг 9.4. Использование `wasm-opt` для оптимизации производительности файла `collide.wasm`

```
wasm-opt collide.wasm -O3 -o collide-3.wasm
```

После получения новой версии файла `collide.wasm` измените файл `collide.html`, чтобы запустить оптимизированную версию. Теперь,

когда мы запускаем его через профилировщик, мы можем получить представление о повышении производительности. Профилирование с помощью Chrome показывает, что приложение сейчас работает со скоростью 35 fps (рис. 9.19).

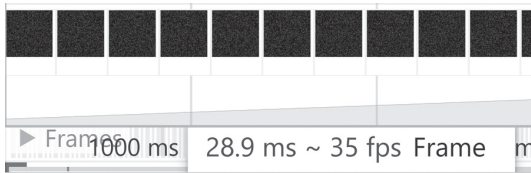


Рис. 9.19. Новое значение FPS в Chrome для оптимизированного файла `collide-3.wasm`

На рис. 9.10 мы видели, что исходная частота кадров составляла 18 fps. Простой запуск `wasm-opt` может удвоить значение частоты кадров вашего приложения в Chrome. Давайте посмотрим, что произойдет, когда мы запустим наш профилировщик в Firefox (рис. 9.20).

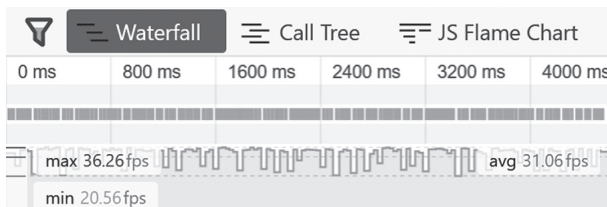


Рис. 9.20. Новое значение FPS в Firefox для оптимизированного файла `collide-3.wasm`

Если вернуться на рис. 9.18, мы увидим, что при первом запуске приложение работало со средней скоростью 14 fps, поэтому частота кадров в Firefox увеличилась более чем вдвое. В следующем разделе мы рассмотрим дизассемблированный оптимизированный WAT-код, чтобы увидеть, какие виды оптимизации были сделаны с помощью `wasm-opt`.

Взглянем на оптимизированный код WAT

У вас уже должно быть установлено расширение WebAssembly для VS Code (мы сделали это в главе 1). В Visual Studio вы можете щелкнуть правой кнопкой мыши на файл WebAssembly и выбрать **Show WebAssembly** (Показать WebAssembly), чтобы просмотреть WAT для данного файла. В листинге 9.5 мы используем `wasm2wat` в командной строке для преобразования оптимизированного файла `collide-3.wasm` в файл WAT.

Листинг 9.5. Запустите `wasm2wat`, чтобы дизассемблировать `collide-3.wasm` в WAT

```
wasm2wat collide-3.wasm -f -o collide-3.wat
```

В VS Code откройте файл *collide.wat* рядом с *collide-3.wat* и посмотрите, какие обновления *wasm-opt* внес в файл WebAssembly, как показано на рис. 9.21.

В оптимизированном коде отсутствуют все имена функций и переменных. Я добавил несколько комментариев, чтобы помочь отслеживать мои действия. Можно легко заметить, что оптимизация сократила количество функций с семи до трех за счет расширения небольших функций во встроенный код. В одной из оставшихся функций оптимизация удалила переменную. Чтобы упростить чтение кода, вы можете создать две разные переменные, даже если технически вам нужна только одна. Однако оптимизатор может заметить это и уменьшить количество переменных. Также обратите внимание, что оптимизатор заменяет умножение на число в степени двойки сдвигом влево. Например, в коде на рис. 9.21 оптимизатор заменил умножение на 4 сдвигом влево на 2 разряда. В следующем разделе мы более подробно рассмотрим, как некоторые из этих приемов повышают производительность.

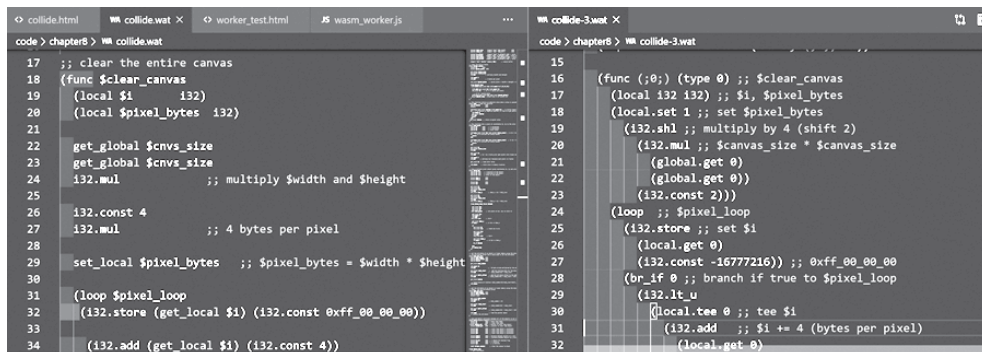


Рис. 9.21. Сравнение оптимизированной и неоптимизированной версий *collide.wat*

Приемы повышения производительности

Теперь мы рассмотрим некоторые приемы, которые вы можете использовать для повышения производительности вашего приложения WebAssembly. Оптимизатор использует некоторые из этих приемов, поэтому вы можете закодировать свое приложение таким образом, чтобы облегчить работу оптимизатора. Чтобы понять, как улучшить ваш код, нужно изучить WAT-код, сгенерированный оптимизатором. Давайте рассмотрим несколько распространенных методов оптимизации, которые можно использовать с WAT.

Встраивание функций

Вызов функции приводит к небольшим накладным расходам. Эти накладные расходы обычно не представляют большой проблемы, если

функция не вызывается тысячи или миллионы раз в секунду. Встраивание функции – это процесс замены вызова функции на встроенную линейную копию того же кода. Это устраняет дополнительные накладные расходы на обработку вызова функции, но увеличивает размер модуля WebAssembly, поскольку он дублирует код везде, где вызывалась функция. Когда мы запускали оптимизатор в модуле *col-lide.wasm*, он встроил четыре из семи функций. Давайте посмотрим на простой пример встраивания функции. Следующий код WAT не является частью приложения; это просто демонстрация. В листинге 9.6 мы создадим функцию, которая складывает три числа, а затем другую функцию для вызова `$add_three` несколько раз.

Листинг 9.6. Код для демонстрации встраивания функций

```
(func $add_three ;; функция складывает три числа
  (param $a i32)
  (param $b i32)
  (param $c i32)
  (result i32)
  local.get $a
  local.get $b
  local.get $c
  i32.add
  i32.add
)
(func (export "inline_test") ;; Я встраиваю функции в inline_test
  (param $p1 i32)
  (param $p2 i32)
  (param $p3 i32)
  (result i32)
  (call $add_three (local.get $p1) (i32.const 2) (local.get $p2))
  (call $add_three (local.get $p3) (local.get $p1) (i32.const 13))
  ;; сложить результаты и вернуть значение
  i32.add
)
```

В этом разделе мы сосредоточимся на встраивании функций с целью оптимизации. Чтобы встроить эти функции, мы вставляем содержимое функции в каждое место, где она должна быть вызвана. В листинге 9.7 выделенный серым код является исходным вызовом функции, а следом за ним идет код встроенной функции.

Листинг 9.7. Пример кода, встроенного вручную

```
(func (export "inline_test");; Я встрою функции в inline_test
  (param $p1 i32)
  (param $p2 i32)
  (param $p3 i32)
  (result i32)
  ;; (call $add_three (local.get $p1) (i32.const 2) (local.get $p2))
```

```

;; приведенная выше функция встроена в код ниже
    local.get $p1
    i32.const 2
    local.get $p2
    i32.add
    i32.add
;; (call $add_three (local.get $p3) (local.get $p1) (i32.const 13))
;; приведенная выше функция встроена в код ниже
    local.get $p3
    local.get $p1
    i32.const 13
    i32.add
    i32.add
    i32.add
)

```

Встраивание вызовов функций может открыть нам и другие методы оптимизации. Например, как вы можете видеть, мы прибавляем 2, а позже прибавляем 13. Поскольку оба этих значения являются константами, код будет работать лучше, если мы просто прибавим 15.

Давайте напишем небольшой модуль, в котором потенциально можно применить встраивание, скомпилируем и оптимизируем его, а затем посмотрим на код, сгенерированный *wasm-opt*. Мы создадим модуль с тремя функциями: *\$add_three*, *\$square* и *\$inline_test*. Создайте WAT-файл с именем *inline.wat* и добавьте туда код из листинга 9.8.

*Листинг 9.8. Для встраивания этого кода мы используем *wasm-opt**

```

inline.wat (module
  (func $add_three
    (param $a i32)
    (param $b i32)
    (param $c i32)
    (result i32)
    local.get $a
    local.get $b
    local.get $c
    i32.add
    i32.add
  )
  (func $square
    (param $a i32)
    (result i32)
    local.get $a
    local.get $a
    i32.mul
  )
  (func $inline_test (export "inline_test")
    (param $p1 i32)
    (param $p2 i32)

```

```

(param $p3 i32)
(result i32)
(call $add_three (local.get $p1) (i32.const 2) (local.get $p2))
(call $add_three (local.get $p3) (local.get $p1) (i32.const 13))
call $square
i32.add
call $square
)
)

```

Функция `$add_three` – это та же функция, которую мы встроили вручную в листинге 9.7. В верхней части стека функция `$square` возводит полученное значение в квадрат, а функция `$inline_test` является вызывающей. Давайте скомпилируем функцию `$inline_test` с помощью `wat2wasm`:

```
wat2wasm inline.wat
```

Теперь мы можем оптимизировать код с помощью `wasm-opt`:

```
wasm-opt inline.wasm -O3 -o inline-opt.wasm
```

Наконец, снова преобразуем код в WAT с помощью `wasm2wat`:

```
wasm2wat inline-opt.wasm
```

Теперь мы можем открыть `inline-opt.wat` и посмотреть, как выглядит наш оптимизированный код (листинг 9.9).

Листинг 9.9. Оптимизированная версия `inline.wat` и `inline-opt.wat`, в которые встроены обе функции

```

inline-opt.wat (module
  (type (;0;) (func (param i32 i32 i32) (result i32)))
  (func (;0;) (type 0) (param i32 i32 i32)
    (result i32) ;; функция $inline_test
    local.get 0
    local.get 1
    i32.const 2
    i32.add
    i32.add
    local.get 2
    local.get 0
    i32.const 13
    i32.add
    i32.add
    local.tee 0
    local.get 0
    i32.mul
  )
)

```

```

i32.add
local.tee 0
local.get 0
i32.mul)
(export "inline_test" (func 0)))

```

Оптимизатор удалил две функции `$add_three` и `$square` и поместил этот код в функцию `inline_test`.

Умножение и деление vs. сдвиг

В главе 8 вы узнали, что сдвигать целые биты вправо – это более быстрый способ, чем умножение на число, представляющее собой степень двойки. Например, сдвиг влево на 3 байта аналогичен умножению на 2^3 , т. е. на 8. То же самое со сдвигом целого числа вправо. Например, сдвиг вправо на 4 – это то же самое, что и деление на 2^4 , то есть на 16. Давайте посмотрим, как `wasm-opt` работает с умножением и делением на число, равное степени двойки. Создайте новый WAT-файл с именем `pow2_mul.wat` и добавьте из листинга 9.10 код, который создает модуль для умножения и деления на такое число.

Листинг 9.10. Функция умножения и деления на степень 2

```

pow2_mul.wat (module
  (func (export "pow2_mul")
    (param $p1 i32)
    (param $p2 i32)
    (result i32)
    local.get $p1
    i32.const 16
    i32.mul ;; умножение на 16, что равно 24
    local.get $p2
    i32.const 8
    i32.div_u ;; деление на 8, что равно 23
    i32.add
  )
)

```

Скомпилируйте этот код посредством `wat2wasm`, а `wasm-opt` используйте для оптимизации файла WebAssembly, затем дизассемблируйте файл WebAssembly обратно в WAT с помощью `wasm2wat`. Далее откройте оптимизированную версию `pow2_mul.wat` в VS Code, как показано в листинге 9.11.

Листинг 9.11. Оптимизированная версия функции `pow2_mul` из листинга 9.10

```

pow2_mul
_optimized.wat (module
  (type (;0;) (func (param i32 i32) (result i32)))
  (func (;0;) (type 0) (param i32 i32) (result i32)

```

```

local.get 1
i32.const 8
i32.div_u
local.get 0
i32.const 4
i32.shl
i32.add)
(export "pow2_mul" (func 0)))

```

Обратите внимание, что оптимизированный код сначала выполняет деление второго параметра, а потом умножение первого параметра. Когда вы умножаете на константу степени двойки, `wasm-opt` преобразует это действие в сдвиг влево. Однако `wasm-opt` не всегда заменяет деление со степенью двойки сдвигом вправо. Позже в этой главе мы выделим некоторое время на запуск различных версий этого кода с помощью файла *benchmark.js*, чтобы увидеть, как они работают. Мы сравним оптимизированный код, созданный с помощью `wasm-opt`, с кодом, который оптимизируем вручную, благодаря чему мы поймем, можно ли еще больше его улучшить.

Примечание *Этот код был создан с помощью `wat-wasm` версии 1.0.11. Если вы используете более позднюю версию, то оптимизация с помощью `wat2wasm` будет другой.*

Объединение констант

Часто для повышения производительности оптимизации объединяют константы. Например, предположим, что у вас есть два постоянных сдвига, которые вам нужно сложить. Ваш исходный код имеет вид $x = 3 + 8$, но для лучшей работы кода вначале можно просто выполнить присвоение $x = 11$. Подобные случаи не всегда очевидны для человеческого глаза, но `wasm-opt` эффективно следит за такими ситуациями за вас. В качестве наглядного примера создайте WAT-файл с именем *comb_constants.wat* и добавьте код из листинга 9.12, который просто объединяет три константы.

Листинг 9.12. Функция, которая объединяет три константы

```

combine (module
_constants.wat (func $combine_constants (export "combine_constants")
    (result i32)
    i32.const 10
    i32.const 20
    i32.add
    i32.const 55
    i32.add
  )
)

```

Обратите внимание, что значение, возвращаемое `$comb_constants`, всегда будет 85. Инструмент `wasm-opt` достаточно умен, чтобы это заметить. Последовательно применив к коду `wat2wasm`, `wasm-opt`, а затем `wasm2wat`, вы увидите код из листинга 9.13.

Листинг 9.13. Объединение трех констант в одну

```
combine (module
_constants (type (;0;) (func (result i32)))
_optimized.wat (func (;0;) (type 0) (result i32)
❶ i32.const 85)
  (export "combine_constants" (func 0)))
```

Функция в листинге 9.13 возвращает значение 85 **❶** и не пытается выполнить два сложения.

DCE

Удаление неиспользуемого кода (DCE – Dead Code Elimination) – это метод оптимизации, удаляющий любой код, который не вызывается или не экспортируется вашим модулем. Это простая оптимизация, которая не сокращает время выполнения, зато сокращает время загрузки. DCE происходит независимо от того, какой флаг оптимизации вы используете. Давайте разберем небольшой пример. Откройте новый файл с именем `dce_test.wat` и добавьте код из листинга 9.14, который создает модуль с двумя неиспользуемыми функциями.

Листинг 9.14. Две неиспользуемые функции в модуле

```
dce_test.wat (module
❶ (func $dead_code_1
  (param $a i32)
  (param $b i32)
  (param $c i32)
  (result i32)
  local.get $a
  local.get $b
  local.get $c
  i32.add
  i32.add
)
❷ (func $dead_code_2
  (param $a i32)
  (result i32)
  local.get $a
  local.get $a
  i32.mul
)
  (func $dce_test (export "dce_test")
    (param $p1 i32)
```

```

    (param $p2 i32)
    (result i32)
    local.get $p1
    local.get $p2
    i32.add
  )
)

```

Первые две функции `$dead_code_1` ❶ и `$dead_code_2` ❷ не вызываются и не экспортируются. Любая оптимизация, которую мы проводим, удалит эти функции. Запустите `wat2wasm`, чтобы сгенерировать код, и `wasm-opt` с флагом `-O3`, чтобы оптимизировать полученный код, затем `wasm2wat`, чтобы преобразовать его обратно в файл WAT. Откройте этот новый файл, чтобы посмотреть на оптимизированный код, как показано в листинге 9.15.

Листинг 9.15. DCE удалил две функции

```

dce_test (module
_optimized.wat (type (;0;)) (func (param i32 i32) (result i32)))
               (func (;0;) (type 0) (param i32 i32) (result i32)
                 local.get 0
                 local.get 1
                 i32.add)
               (export "dce_test" (func 0)))

```

Единственная оставшаяся функция – это `"dce_test"`. Благодаря DCE мы уменьшили размер модуля с 79 до 46 байт.

Сравнение приложения обнаружения столкновений с JavaScript

Мы уже знаем, как работает наше приложение WebAssembly для обнаружения столкновений. Давайте напомним тот же код на JavaScript и сравним его быстродействие с версией WebAssembly. Создайте новую веб-страницу с именем `collidejs.html`. Начните с добавления HTML-заголовка и элемента `canvas` на страницу `collide.html` и повторно сохраните его как `collidejs.html`, как показано в листинге 9.16.

Листинг 9.16. HTML-заголовок и элемент canvas в collidejs.html

```

collidejs.html <!DOCTYPE html>
(часть 1 из 2) <html lang="en">
               <head>
                 <meta charset="UTF-8">
                 <meta name="viewport" content="width=device-width, initial-scale=1.0">
                 <title>Collide JS</title>
               </head>

```

```
<body>
  <canvas id="canvas" width="1024" height="1024"></canvas>
  ...
```

Этот код похож на версию приложения WebAssembly. Основное отличие будет заключаться в теге `<script>`, показанном в листинге 9.17. Добавьте следующий код JavaScript в тег `<script>`.

Листинг 9.17. Версия JavaScript нашего приложения обнаружения столкновений

collidejs.html
(часть 2 из 2)

```
...
<script type="text/javascript">
  // версия javascript
  var animate_callback;
  const out_tag = document.getElementById('out');
  const cnvs_size = 1024 | 0;

  const noh_color = 0xff00ff00 | 0;
  const hit_color = 0xff0000ff | 0;

  const obj_start = cnvs_size * cnvs_size * 4;
  const obj_size = 8 | 0;
  const obj_cnt = 3000 | 0;

  const canvas = document.getElementById("canvas");
  const ctx = canvas.getContext("2d");

  class Collider {
    constructor() {
      this.x = Math.random() * cnvs_size;
      this.y = Math.random() * cnvs_size;
      this.xv = (Math.round(Math.random() * 4) - 2);
      this.yv = (Math.round(Math.random() * 4) - 2);
      this.color = "green";
    }

    move = () => {
      this.x += this.xv;
      this.y += this.yv;
      this.x %= 1024;
      this.y %= 1024;
    }

    draw = () => {
      ctx.beginPath();
      ctx.fillStyle = this.color;
      ctx.fillRect(this.x, this.y, obj_size, obj_size);
      ctx.stroke();
    }

    hitTest = (c2) => {
      let x_dist = this.x - c2.x;
      let y_dist = this.y - c2.y;
```

```

        if (Math.abs(x_dist) <= obj_size &&
            Math.abs(y_dist) <= obj_size) {
            this.color = "red";
            return true;
        }
        else {
            this.color = "green";
        }
        return false;
    }
}
let collider_array = new Array();
for (let i = 0; i < obj_cnt; i++) {
    collider_array.push(new Collider());
}

let animate_count = 0;

function animate() {
    // очистить
    ctx.clearRect(0, 0, canvas.width, canvas.height);
    for (let i = 0; i < collider_array.length; i++) {
        collider_array[i].move();
    }

    // цикл и рендер
    for (i = 0; i < collider_array.length; i++) {
        for (let j = 0; j < collider_array.length; j++) {
            if (i === j) {
                continue;
            }
            if (collider_array[i].hitTest(collider_array[j])) {
                break;
            }
        }
        collider_array[i].draw();
    }
    requestAnimationFrame(animate);
}
requestAnimationFrame(animate);
</script>
</body>
</html>

```

Я не буду вдаваться в подробности кода в листинге 9.17, потому что его цель – просто обеспечить сравнение быстродействия с кодом WebAssembly из главы 8. Теперь мы можем запустить *collidejs.html* в профилировщиках Chrome и Firefox и посмотреть, как они работают. На рис. 9.22 показано значение FPS для файла *collidejs.html* внутри профилировщика Chrome.

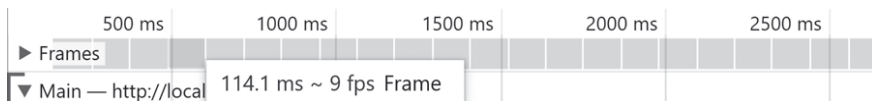


Рис. 9.22. Значение FPS нашего приложения версии JavaScript, запущенного в профилировщике Chrome

Chrome выполняет JavaScript версию приложения со скоростью около 9 fps, что медленнее, чем даже неоптимизированная версия WebAssembly, которая работала со скоростью около 18 fps в Chrome, и оптимизированная версия, которая работала со скоростью 35 fps. Оптимизированная версия кода WebAssembly была почти в четыре раза быстрее в Chrome.

Теперь давайте посмотрим, как наш JavaScript работает в Firefox (рис. 9.23).

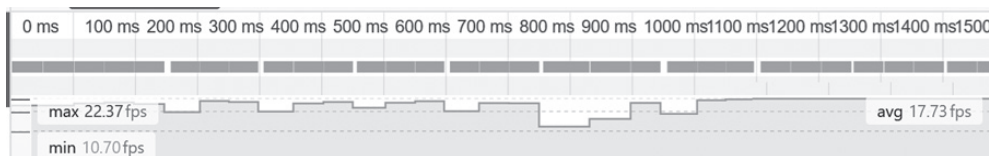


Рис. 9.23. Значение fps нашего приложения версии JavaScript, запущенного в профилировщике Firefox

Firefox в работе с приложением показал себя лучше, чем Chrome (почти вдвое быстрее). Ему даже удалось превзойти неоптимизированную версию приложения WebAssembly, которая работала со скоростью около 14 fps. Это было лишь немногим более чем вдвое быстрее оптимизированной версии WebAssembly в Firefox, которая работала со скоростью около 31 fps.

В этом разделе вы узнали, как сравнить свой код WebAssembly с аналогичным кодом JavaScript с помощью профилировщиков Firefox и Chrome. Теперь вы можете использовать эти знания для сравнения разных версий приложения в разных браузерах, за счет чего будет проще понять, какой код лучше всего делать в WebAssembly, а какой в JavaScript.

Оптимизация WAT вручную

Я потратил некоторое время на оптимизацию своего приложения обнаружения столкновений WebAssembly и смог еще больше повысить значение частоты кадров. Я не смогу описать все свои действия в этой книге, потому что их очень много. Тем не менее я хочу указать, какого прироста производительности мы можем достичь, если захотим уделить время на оптимизацию вручную. Мне удалось заставить приложение обнаружения столкновений работать со скоростью до 36 fps в профилировщике Chrome (рис. 9.24).

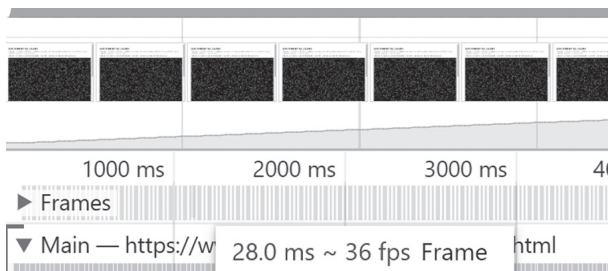


Рис. 9.24. Оптимизированное вручную приложение обнаружения столкновений, работающее в Chrome

В Firefox был еще более высокий показатель – 52 fps (рис. 9.25).



Рис. 9.25. Оптимизированное вручную приложение обнаружения столкновений, работающее в Firefox

Вы можете увидеть результат моих усилий по оптимизации вручную по адресу <https://wasmbbook.com/collide.html> и WAT-код на <https://wasmbbook.com/collide.wat>. Я запустил оптимизатор Binaryen на своем точно настроенном коде, и оказалось, что он даже замедлил код на несколько кадров в секунду. Binaryen постоянно совершенствует свою оптимизацию. Поэтому к тому времени, как вы это прочтете, результаты могут отличаться.

Запись производительности в лог

Один из простейших способов записи в лог производительности приложения JavaScript – использование класса `Date` и функции `console.log`. WebAssembly не может выводить данные в консоль без JavaScript. По этой причине нам нужно будет использовать JavaScript для записи производительности нашего WebAssembly и JavaScript-кода в консоль.

Давайте посмотрим на накладные расходы, связанные с выполнением большого количества вызовов модуля WebAssembly из нашего JavaScript. Мы создадим модуль WebAssembly с несколькими небольшими функциями, которые можем многократно вызывать из JavaScript. Создайте файл с именем `mod_and.wat` и добавьте код из листинга 9.18.

Листинг 9.18. Сравнение производительности функции нахождения остатка и битовой операции И

```
mod_and.wat (module
  (func $mod (export "mod")
```

```

    (param $p0 i32)
    (result i32)
    local.get $p0
    i32.const 1000
    i32.rem_u
  )
(func $and (export "and")
  (param $p0 i32)
  (result i32)
  local.get $p0
  i32.const 0x3ff
  i32.and
)
)

```

В этом модуле есть две функции: функция `$mod`, которая находит остаток от деления на 1000, и функция `$and`, которая использует побитовую маску AND. Скомпилируйте файл `mod_and.wat` с помощью `wat2wasm` и оптимизируйте его с помощью `wasm-opt`.

Далее нам нужно создать функцию JavaScript для запуска модуля WAT и сравнить показатели с эквивалентным кодом JavaScript. Создайте новый файл с именем `mod_and.js` и добавьте туда код из листинга 9.19.

Листинг 9.19. Запись времени выполнения с помощью `Date.now` и `console.log`

```

mod_and.js const fs = require('fs');
const bytes = fs.readFileSync('./mod_and.wasm');

(async () => {
  const obj =
    await WebAssembly.instantiate(new Uint8Array(bytes));
  let mod = obj.instance.exports.mod;
  let and = obj.instance.exports.and;
  let start_time = Date.now(); // сброс start_time
  // Синтаксис '| 0' - это подсказка движку JavaScript
  // использовать целые числа вместо чисел с плавающей запятой, что может
  // повысить производительность в некоторых случаях
  for (let i = 0 | 0; i < 4_000_000; i++) {
    mod(i); // вызов функции mod 4 млн раз
  }
  // подсчитать время, которое потребовалось для выполнения 4 млн вызовов mod
  console.log(`mod: ${Date.now() - start_time}`);
  start_time = Date.now(); // сброс start_time

  for (let i = 0 | 0; i < 4_000_000; i++) {
    and(i); // вызов функции and 4 млн раз
  }
  // подсчитать время, которое потребовалось для выполнения 4 млн вызовов and
  console.log(`and: ${Date.now() - start_time}`);

```

```
start_time = Date.now(); // сброс start_time
for (let i = 0 | 0; i < 4_000_000; i++) {
  Math.floor(i % 1000);
}
// подсчитать время, которое потребовалось для выполнения 4 млн вызовов modulo
console.log(`js mod: ${Date.now() - start_time}`);
})();
```

Перед запуском каждого блока кода мы устанавливаем для переменной `start_time` значение `Date.now()`. При этом в переменной `start_time` устанавливается текущее время в миллисекундах. Когда код завершается, нужно записать `Date.now() - start_time`, что даст нам время выполнения теста в миллисекундах. Мы сделаем это для нашего модуля WebAssembly и JavaScript-кода, чтобы сравнить их результаты.

Примечание | 0 – это подсказка движку JavaScript, чтобы он использовал целые числа вместо чисел с плавающей запятой. В некоторых случаях это может повысить производительность.

Теперь, когда у нас есть функция `mod_and.js`, мы можем запустить ее, используя следующую команду node:

```
node mod_and.js
```

В листинге 9.20 показан вывод после запуска `mod_and.js`.

Листинг 9.20. Вывод из `mod_and.js`

```
mod: 29
and: 23
js mod: 4
```

Выполнение функции `mod` 4 млн раз заняло 29 мс. Выполнение функции `and` 4 млн раз заняло 23 мс. Версии JavaScript потребовалось всего 4 мс для выполнения функции 4 млн раз. Если WebAssembly работает так быстро, почему на выполнение этих функций уходит в 5–7 раз больше времени? Проблема в том, что вызовы между JavaScript и WebAssembly сопровождаются некоторыми накладными расходами. Вызов небольшой функции 4 млн раз влечет за собой 4 млн таких расходов. Давайте перепишем наш код так, чтобы функции выполнялись несколько миллионов раз из WebAssembly, а не из цикла в JavaScript.

Сначала мы перепишем наш модуль WebAssembly, чтобы включить цикл внутри модуля, а не внутри JavaScript. Создайте новый WAT-файл с именем `mod_and_loop.wat` и добавьте код из листинга 9.21.


```
mod_and_loop.wat (module
  (func (export "mod_loop")
    (result i32)
    (local $i i32)
    (local $total i32)
    i32.const 100_000_000 ;; запуск цикла 100 млн раз
    local.set $i

    (loop $loop
      local.get $i
      i32.const 0x3ff
      i32.rem_u
      local.set $total

      local.get $i
      i32.const 1
      i32.sub
      local.tee $i ;; i--

      br_if $loop
    )
    local.get $total
  )

  (func (export "and_loop")
    (result i32)
    (local $total i32)
    (local $i i32)
    i32.const 100_000_000 ;; запуск цикла 100 млн раз
    local.set $i

    (loop $loop
      local.get $i
      i32.const 0x3ff
      i32.and
      local.set $total

      local.get $i
      i32.const 1
      i32.sub
      local.tee $i ;; i--

      br_if $loop
    )
    local.get $total
  )
)
```

Эти функции выполняют те же задачи, что и функции в оригинальной версии, только программа выполняет их 100 млн раз. Нам нужно

будет изменить файл JavaScript, чтобы эти функции вызывались один раз, а JavaScript – 100 млн раз. Таким образом, мы сможем сравнить производительность с модулем WebAssembly, который ранее изменили для выполнения нашей функции 100 млн раз. Создайте новую функцию с именем *mod_and_loop.js* и добавьте код из листинга 9.22.

Листинг 9.22. JavaScript, который запускает and_loop, mod_loop и аналогичный JavaScript

```
mod_and_loop.js const fs = require('fs');
const bytes = fs.readFileSync('./mod_and_loop.wasm');

(async () => {
  const obj =
    await WebAssembly.instantiate(new Uint8Array(bytes));
  let mod_loop = obj.instance.exports.mod_loop;
  let and_loop = obj.instance.exports.and_loop;

  let start_time = Date.now(); // задать start_time
  and_loop();
  console.log(`and_loop: ${Date.now() - start_time}`);

  start_time = Date.now(); // сброс start_time
  mod_loop();
  console.log(`mod_loop: ${Date.now() - start_time}`);
  start_time = Date.now(); // сброс start_time
  let x = 0;
  for (let i = 0; i < 100_000_000; i++) {
    x = i % 1000;
  }
  console.log(`js mod: ${Date.now() - start_time}`);
})();
```

Мы вызываем функции *mod_loop* и *and_loop*, записывая время выполнения каждого цикла. Затем запускаем наш цикл, в котором выполняем остаток от деления 100 млн раз и записываем, сколько времени это заняло. Если мы скомпилируем и оптимизируем наш модуль WebAssembly, а затем запустим *mod_and_loop.js* с помощью *node*, то должны увидеть вывод, как в листинге 9.23.

Листинг 9.23. Вывод из mod_and_loop.js

```
and_loop: 31
mod_loop: 32
js mod: 52
```

Теперь наш код WebAssembly работает на 67 % быстрее, чем тот же код JavaScript. Было несколько разочаровывающим то, что операция побитового И не оказалась эффективным методом в сравнении с вычислением остатка от деления, как я ожидал. Однако теперь мы знаем,

как провести простейший тест производительности с помощью `console.log` в сочетании с `Date.now()`.

Более сложное тестирование с помощью `benchmark.js`

Если вы хотите усложнить тестирование, не ограничиваясь простым использованием логов и `Date.now`, то можете установить модуль тестирования производительности `benchmark.js`. Чтобы увидеть, как Binaryen оптимизирует код, в листинге 9.10 мы создали функцию `WebAssembly`, которая умножает число на 16 и делит на 8, а затем пропустили ее через оптимизатор `wasn-opt`. Оптимизатор заменил в коде операции умножения на сдвиги, но не стал заменять сдвигом операцию деления. Он также изменил порядок деления и умножения.

Давайте протестируем несколько версий модуля `WebAssembly`, включая исходную версию и созданную оптимизатором, чтобы понять, можно ли, приложив усилия, превзойти оптимизатор. Для проверки производительности всех этих функций мы используем `benchmark.js`. Создайте новый WAT-файл с именем `pow2_test.wat` и добавьте код из листинга 9.24.

Листинг 9.24. Начало модуля с исходной функцией

```
pow2_test.wat (часть 1 из 5) (module
;; это исходная функция, которую мы написали
(func (export "pow2")
  (param $p1 i32)
  (param $p2 i32)
  (result i32)
  local.get $p1
  i32.const 16
  i32.mul
  local.get $p2
  i32.const 8
  i32.div_u
  i32.add
)
...

```

В листинге 9.24 показана исходная версия теста с использованием чисел, представляющих степени двойки, где мы сначала умножаем на 16, а потом делим на 8.

Следующая функция в листинге 9.25 выполняет сначала деление, потом умножение. Я сделал это, потому что `wasn-opt` поменял местами функции умножения и деления, и мне было любопытно узнать, положительно ли это повлияло на производительность функции.

Листинг 9.25. Меняем местами деление и умножение

```
pow2_test.wat ...  
(часть 2 из 5) ;; wasm-opt поместил div перед mul, теперь давайте узнаем, что из этого выйдет  
                (func (export "pow2_reverse")  
                  (param $p1 i32)  
                  (param $p2 i32)  
                  (result i32)  
                  local.get $p2  
                  i32.const 8  
                  i32.div_u  
                  local.get $p1  
                  i32.const 16  
                  i32.mul  
                  i32.add  
                )  
                ...
```

Следующая функция в листинге 9.26 использует сдвиг вместо умножения и деления на числа, представляющие собой степени двойки. Мы также используем порядок, установленный оптимизатором, где деление происходит до умножения.

Листинг 9.26. Заменяем выражения умножения и деления на двоичные сдвиги

```
pow2_test.wat ...  
(часть 3 из 5) ;; заменяем умножение и деление на сдвиги  
                (func (export "pow2_mul_div_shift")  
                  (param $p1 i32)  
                  (param $p2 i32)  
                  (result i32)  
                  local.get $p2  
                  i32.const 3  
                  i32.shr_u  
                  local.get $p1  
                  i32.const 4  
                  i32.shl  
                  i32.add  
                )  
                ...
```

Далее в листинге 9.27 мы используем сдвиги вместо деления и умножения, но на этот раз мы не меняем порядок деления и умножения по сравнению с исходным кодом.

Листинг 9.27. Исходный порядок с умножением перед делением

```
pow2_test.wat ...  
(часть 4 из 5) ;; вернуться к исходному порядку умножения и деления  
                (func (export "pow2_mul_div_nor")
```

```

    (param $p1 i32)
    (param $p2 i32)
    (result i32)
    local.get $p1
    i32.const 4
    i32.shl
    local.get $p2
    i32.const 3
    i32.shr_u
    i32.add
  )
  ...

```

Следующая функция в листинге 9.28 является версией кода, созданного `wasm-opt` с флагом `-O3`.

Листинг 9.28. Оптимизированная `wasm-opt` версия функции

```

pow2_test.wat  ...
(часть 5 из 5) ;; вот что было сгенерировано wasm-opt
                (func (export "pow2_opt") (param i32 i32) (result i32)
                  local.get 1
                  i32.const 8
                  i32.div_u
                  local.get 0
                  i32.const 4
                  i32.shl
                  i32.add
                )
              )

```

Теперь мы можем скомпилировать этот модуль с помощью `wat2wasm`. При этом нам *не* нужно выполнять оптимизацию, так как мы хотим протестировать WAT-код без изменений со стороны оптимизатора. Нам нужно создать код `benchmark.js`. Для этого установите модуль `benchmark.js` с помощью `npm`:

```
npm i --save-dev benchmark
```

Теперь мы можем написать программу на JavaScript для тестирования функций WebAssembly с помощью `benchmark.js`. Давайте разделим эту программу на несколько частей и рассмотрим каждую из них. Добавьте код из листинга 9.29 в файл `benchmark_test.js`.

Листинг 9.29. Первая часть JavaScript файла `benchmark_test.js`

```

benchmark_test.js // импорт benchmark.js
  ❶ var Benchmark = require('benchmark');
  ❷ var suite = new Benchmark.Suite();

  // используйте fs для чтения модуля pow2_test.wasm в байтовом массиве

```

```

❸ const fs = require('fs');
const bytes = fs.readFileSync('./pow2_test.wasm');
const colors = require('colors'); // разрешить цветные логи консоли

// переменные для функций WebAssembly
var pow2;
var pow2_reverse;
var pow2_mul_shift;
var pow2_mul_div_shift;
var pow2_mul_div_nor;

console.log(`
===== RUNNING BENCHMARK =====
`.rainbow ❹);
...

```

Для начала нам потребуется модуль `benchmark`❶, а затем из этого модуля мы создадим новый объект `suite`❷. Для загрузки модуля `WebAssembly` в массив байтов нам нужен модуль `fs`❸. Затем мы определяем серию переменных для хранения функций в модуле `WebAssembly`. Чтобы было легче определить, где начинается тест, когда мы прокручиваем нашу статистику назад, мы выводим логи с использованием цветового разделителя `rainbow`❹, который отображает строку `RUNNING BENCHMARK` (выполняется бенчмарк). Вы можете просто отредактировать имеющийся модуль для тестирования, как это сделал я, но в таком случае лучше как-то обозначить место, где начинается тестирование.

В листинге 9.30 мы добавим функцию, которую можем вызвать для инициализации и запуска бенчмарка. Добавьте следующую функцию в `benchmark_test.js`.

Листинг 9.30. Функция `init_benchmark`

```

benchmark_test.js ...
❶ function init_benchmark() {
  // добавляет обратные вызовы в тесты производительности
❷ suite.add('#1 '.yellow + 'Исходная функция', pow2);
  suite.add('#2 '.yellow + 'Обратный порядок Div/Mult', pow2_reverse);
  suite.add('#3 '.yellow + 'Замена умножения сдвигом', pow2_mul_shift);
  suite.add('#4 '.yellow + 'Замена умножения и деления сдвигом',
    pow2_mul_div_shift);
  suite.add('#5 '.yellow + 'Версия, оптимизированная wasm-opt', pow2_opt);
  suite.add('#6 '.yellow + 'Использование сдвига в исходном порядке',
    pow2_mul_div_nor);
  // прибавить прослушатели
❸ suite.on('cycle', function (event) {
    console.log(String(event.target));
  });

❹ suite.on('complete', function () {
    // после завершения теста записывать самые быстрые и самые медленные функции

```

```

let fast_string = ('Самый быстрый ' +
  ❶ this.filter('fastest').map('name'));
let slow_string = ('Самый медленный ' +
  this.filter('slowest').map('name'));
❷ console.log(`
-----
${fast_string.green}
${slow_string.red}
-----
`);

// создать массив всех успешных запусков, отсортировать от быстрого
// к медленному
❸ var arr = this.filter('successful');
❹ arr.sort(function (a, b) {
  return a.stats.mean - b.stats.mean;
});

console.log(`
`);
console.log("===== FASTEST =====".green);
❺ while (obj = arr.shift()) {
  let extension = '';
  let count = Math.ceil(1 / obj.stats.mean);

  if (count > 1000) {
    count /= 1000;
    extension = 'K'.green.bold;
  }

  if (count > 1000) {
    count /= 1000;
    extension = 'M'.green.bold;
  }

  count = Math.ceil(count);
  let count_string = count.toString().yellow + extension;
  console.log(
    `${obj.name.padEnd(45, ' ')} ${count_string} exec/sec`
  );
}
console.log("===== SLOWEST =====".red);
});
// запуск async
❻ suite.run({ 'async': false });
}
...

```

Мы определяем функцию `init_benchmark()` ❶, которая из модуля тестирования вызывает `suite.add` ❷ для каждой функции в нашем модуле `WebAssembly`. Функция `suite.add` сообщает бенчмарку, что нужно протестировать эту функцию и записать в журнал результаты

со строкой, переданной в качестве второго параметра. Также `suite.on` устанавливает обратный вызов для различных событий, которые происходят во время теста производительности. Первый вызов `suite.on` ❶ устанавливает обратный вызов каждого цикла, который выводит статистику протестированной функции. Следующий вызов `suite.on` ❷ устанавливает обратный вызов для завершения теста, который будет использовать метод `filter` ❸ для `log` ❹ (записи в журнал) самых быстрых и самых медленных функций. Затем мы фильтруем 'successful' ❺, чтобы получить массив всех успешно выполненных функций. При помощи функции `sort` ❻ мы сортируем полученный массив по среднему времени (`mean`) выполнения этого цикла. Она сортирует циклы от самого быстрого к самому медленному времени выполнения. Затем мы можем пройти ❼ по каждому из этих циклов, печатая их от самого быстрого к самому медленному. В конце этой функции мы запускаем бенчмарк `suite` при помощи метода `run` ❽.

Определив функцию `init_benchmark` в листинге 9.31, мы создаем асинхронный IIFE, чтобы получить экземпляр нашего модуля `WebAssembly`, и вызываем `init_benchmark`.

Листинг 9.31. Асинхронный IIFE создает экземпляр `WebAssembly` и запускает `benchmark.js`

```
benchmark_test.js ...
(async () => {
  ❶ const obj = await WebAssembly.instantiate(new Uint8Array(bytes));
  ❷ pow2 = obj.instance.exports.pow2;
    pow2_reverse = obj.instance.exports.pow2_reverse;
    pow2_mul_shift = obj.instance.exports.pow2_mul_shift;
    pow2_mul_div_shift = obj.instance.exports.pow2_mul_div_shift;
    pow2_opt = obj.instance.exports.pow2_opt;
    pow2_mul_div_nor = obj.instance.exports.pow2_mul_div_nor;
  ❸ init_benchmark();
})();
```

Здесь мы создаем экземпляр (`instantiate` ❶) нашего модуля `WebAssembly` и устанавливаем все функции ❷, которые будем вызывать из `benchmark.js`. Затем запускаем `benchmark.js`, вызывая `init_benchmark()` ❸. Теперь можем запустить наше приложение в `node` с помощью следующей команды:

```
node benchmark_test.js
```

На рис. 9.26 показан вывод.

Интересно, что самой медленной из этих функций была версия, оптимизированная для `wasm-opt`: исходная и оптимизированная для `wasm-opt` версии выполнялись примерно в одно и то же время. Быстрее всех выполнялся код, в котором мы заменили операции `i32.mul` и `i32.div_u` на сдвиги и переупорядочили вызовы инструментом

wasm-opt. Мы можем сделать вывод, что нельзя всегда рассчитывать, что wasm-opt (или любой другой программный оптимизатор) каждый раз будет давать наиболее производительный код. Рекомендуется всегда проводить тесты производительности вашего приложения.

Примечание Эти тесты проводились с помощью wat-wasm версии 1.0.11. Более поздние версии могут улучшить оптимизацию wasm-opt.

```
$ node benchmark_test.js

===== RUNNING BENCHMARK =====

#1 Original Function x 44,641,095 ops/sec ±0.69% (91 runs sampled)
#2 Reversed Div/Mult order x 45,393,937 ops/sec ±0.52% (95 runs sampled)
#3 Replace mult with shift x 45,304,429 ops/sec ±0.39% (94 runs sampled)
#4 Replace mult & div with shift x 50,317,467 ops/sec ±0.47% (97 runs sampled)
#5 wasm-opt optimized version x 44,139,754 ops/sec ±0.48% (92 runs sampled)
#6 Use shifts with OG order x 50,117,233 ops/sec ±0.48% (93 runs sampled)

-----
Fastest is #4 Replace mult & div with shift,#6 Use shifts with OG order
Slowest is #5 wasm-opt optimized version
-----

===== FASTEST =====
#4 Replace mult & div with shift      51M execs/sec
#6 Use shifts with OG order          51M execs/sec
#2 Reversed Div/Mult order           46M execs/sec
#3 Replace mult with shift            46M execs/sec
#1 Original Function                  45M execs/sec
#5 wasm-opt optimized version         45M execs/sec
===== SLOWEST =====
```

Рис. 9.26. Вывод из benchmark_test.js

Сравнение WebAssembly и JavaScript с флагом --print-bytecode

В этом разделе мы познакомимся с низкоуровневым байт-кодом. Очень интересно посмотреть, что сгенерирует JavaScript JIT. Также интересно сравнивать его результаты с WebAssembly, и не менее увлекательно выбирать способы повышения производительности. Если эта тема вас не интересует, переходите к следующему разделу.

Давайте кратко рассмотрим, как лучше сравнить коды WebAssembly и JavaScript. V8 компилирует JavaScript в байт-код IR, который очень похож на ассемблерный язык или WAT. IR использует регистры и сумматор (накапливающий регистр), но не зависит от виртуальной машины. Мы можем применять IR для сравнения кода JavaScript после его прогона через JIT-компилятор с нашим кодом WebAssembly. Поскольку оба языка представляют собой низкоуровневые байт-коды,

это облегчает нам сравнение. Но имейте в виду, что код JavaScript анализируется и компилируется в байт-код во время выполнения, тогда как WebAssembly компилируется заранее.

Создадим небольшую программу на JavaScript и воспользуемся флагом `node --print-bytecode`, чтобы просмотреть байт-код, сгенерированный из исходного кода JavaScript. Создайте файл JavaScript с именем `print_bytecode.js` и добавьте код из листинга 9.32.

Листинг 9.32. Выполняем функцию `bytecode_test` с флагом `--print-bytecode`

```
print_bytecode.js ❶function bytecode_test() {  
    let x = 0;  
    ❷for (let i = 0; i < 100_000_000; i++) {  
        ❸x = i % 1000;  
    }  
    ❹return 99;  
}  
  
// если мы не вызываем это, функция удаляется при проверке dce  
❺bytecode_test();
```

Функция `bytecode_test` похожа на код, который мы тестировали на производительность в листинге 9.22. Это простой цикл `for`, который получает остаток от деления счетчика `i`, сохраняет его в `x`, а затем возвращает значение 99. На самом деле эта функция не делает ничего полезного, мне просто хотелось поработать с функцией, которую легко понять и скомпилировать в байт-код.

Затем мы вызываем функцию в дополнение к ее определению; в противном случае V8 удалит ее как часть DCE. Мы можем запустить команду `node` из листинга 9.33, чтобы вывести байт-код в консоль.

Листинг 9.33. Запуск `node` с флагом `--print-bytecode`

```
node --print-bytecode --print-bytecode-filter=bytecode_test print_bytecode.js
```

Здесь мы передаем `node` флаг `--print-bytecode`, чтобы он напечатал байт-код. Мы также передаем флаг `--print-bytecode-filter`, устанавливая его как имя нашей функции для печати только ее байт-кода. Если мы не включим флаг фильтра, в выводе будет намного больше байт-кода, чем нам нужно. Наконец, мы передаем `node` имя файла JavaScript. Запустите из листинга 9.33 `print_bytecode.js` с флагами, чтобы получить результат как в листинге 9.34.

Листинг 9.34. Вывод байт-кода из `print_bytecode.js`

```
[сгенерированный байт-код для функции: bytecode_test]  
Parameter count 1  
Register count 2
```

```

Frame size 16
22 E> 0000009536F965F6 @    0 : a5          StackCheck
38 S> 0000009536F965F7 @    1 : 0b          LdaZero
      0000009536F965F8 @    2 : 26 fb          Star r0
57 S> 0000009536F965FA @    4 : 0b          LdaZero
      0000009536F965FB @    5 : 26 fa          Star r1
62 S> 0000009536F965FD @    7 : 01 0c 00 e1 f5 05 LdaSmi.ExtraWide [100000000]
62 E> 0000009536F96603 @   13 : 69 fa 00      TestLessThan r1, [0]
      0000009536F96606 @   16 : 99 16          JumpIfFalse [22]
(0000009536F9661C @ 38)
44 E> 0000009536F96608 @   18 : a5          StackCheck
89 S> 0000009536F96609 @   19 : 25 fa          Ldar r1
95 E> 0000009536F9660B @   21 : 00 44 e8 03 01 00 ModSmi.Wide [1000], [1]
      0000009536F96611 @   27 : 26 fb          Star r0
78 S> 0000009536F96613 @   29 : 25 fa          Ldar r1
      0000009536F96615 @   31 : 4c 02          Inc [2]
      0000009536F96617 @   33 : 26 fa          Star r1
      0000009536F96619 @   35 : 8a 1c 00      JumpLoop [28], [0]
(0000009536F965FD @ 7)
111 S> 0000009536F9661C @   38 : 0c 63          LdaSmi [99]
121 S> 0000009536F9661E @   40 : a9          Return
Constant pool (size = 0)
Handler Table (size = 0)

```

Правая часть вывода в листинге 9.34 содержит коды операций для IR. Здесь я перечислил эти коды операций и добавил комментарии в стиле WAT с правой стороны. Вместо стековой машины байт-код, сгенерированный механизмом V8, предназначен для виртуальной регистровой машины с регистром сумматора. *Сумматор* – это место, где виртуальная машина выполняет свои вычисления. Взгляните на код в листинге 9.35, который сгенерировал V8.

Листинг 9.35. Коды операций с пояснением после символов ;;

```

;; A = Сумматор R0 = Регистр 0, R1 = Регистр 1
StackCheck
LdaZero          ;; A = 0
Star r0          ;; R0 = A
LdaZero          ;; A = 0
Star r1          ;; R1 = A
;; ЭТО НАЧАЛО ЦИКЛА
LdaSmi.ExtraWide [100000000]      ;; A=100_000_000
TestLessThan r1, [0]             ;; R1 < A
JumpIfFalse [22] (0000006847C9661C @ 38) ;; ЕСЛИ R1 >= A GO 22 БАЙТА
                                   ;; ВЫХОД [КОНЕЦ ЦИКЛА]

StackCheck
Ldar r1          ;; A = R1
ModSmi.Wide [1000], [1]          ;; A %= 1_000
Star r0          ;; R0 = A
Ldar r1          ;; A = R1
Inc [2]          ;; A++
Star r1          ;; R1 = A

```

```
JumpLoop [28], [0] (0000006847C965FD @ 7) ;; НАЗАД НА 28 БАЙТ [НАЧАЛО ЦИКЛА]
LdaSmi [99] ;; A = 99 | КОНЕЦ ЦИКЛА
Return ;; ВЕРНУТЬ A
```

IR для V8 использует сумматор. У суммирующих машин есть один регистр общего назначения, на котором сумматор выполняет все математические операции. Коды операций с буквой *a* обычно относятся к сумматору, а с буквой *r* – к регистру. Например, первый код операции после `StackCheck` – это `LdaZero`, который загружает (*Ld*) в сумматор (*a*) ноль (*Zero*). Затем строка `Star r0` сохраняет (*St*) значение сумматора (*a*) в регистре (*r*), а потом передает значение *r0* для определения этого регистра. Это происходит потому, что IR не может напрямую установить для `Register0` значение 0; вместо этого ему необходимо загрузить это значение в сумматор, а затем переместить из него значение в `Register0`. Далее в коде вы видите `LdaSmi.ExtraWide`. Эта операция загружает (*Ld*) в сумматор (*a*) небольшое целое число (*Smi*), которое использует все 32 бита (*ExtraWide*). Если вы загрузите число, которое состоит всего из 16 бит, эта операция будет отображена как *Wide* вместо *ExtraWide*, а 8 бит вообще не дадут никакого эффекта после выполнения `LdaSmi`. Код операции `TestLessThan` сравнивает значение в указанном регистре (*r1*) со значением в сумматоре. Строка `JumpIf-False [22]` проверяет, привело ли `TestLessThan` к ложному результату, и если да – переходит на 22 байта вперед.

Флаг `--print-bytecode` может быть полезным инструментом для повышения производительности вашего JavaScript. Если вы знакомы с WAT или ассемблером, вам будет несложно понять принцип его работы. Этот метод также может быть полезен при сравнении вашего WAT-кода с JavaScript для настройки производительности в обеих частях приложения `WebAssembly`.

Заключение

В этой главе мы обсудили несколько инструментов для оценки производительности нашего WAT-кода. Мы также сравнили производительность кода `WebAssembly` с производительностью эквивалентного JavaScript. Затем изучили несколько стратегий повышения производительности нашего модуля `WebAssembly`.

Мы рассмотрели профилировщик в веб-браузере Chrome и обсудили его страницу **Summary** и раздел **JS Heap Memory**, в котором содержится информация об увеличении объема памяти и автоматической очистке. Также рассмотрели применение показателя частоты кадров, отображаемое в профилировщике, что является отличным способом определить уровень производительности игры или приложения с тяжелым пользовательским интерфейсом.

Мы использовали профилировщик Firefox для исследования нашего приложения обнаружения столкновений. Профилировщик Firefox предлагает несколько дополнительных инструментов, вклю-

чая Call Tree и JS Flame Chart. Мы отследили функцию WAT, которая была вызвана с помощью функции `wasmp[index]`, указанной в профилировщике.

Затем установили *Binaryen.js* и использовали инструмент `wasmp-opt`, чтобы оптимизировать наш модуль WebAssembly либо для быстрой загрузки, либо для максимальной производительности. Мы также преобразовали его обратно в WAT-код, чтобы увидеть изменения, сделанные оптимизатором.

Далее разобрали различные стратегии повышения максимальной производительности нашего приложения, включая встраивание функций, замену умножения и деления на битовые сдвиги и объединение констант. Мы обсудили удаление неиспользуемого кода, которое оптимизатор всегда выполняет для удаления любых неиспользуемых функций из нашего модуля.

Мы создали версию нашего приложения на JavaScript, чтобы сравнить производительность JavaScript с производительностью модуля WebAssembly.

После использования профилировщика на протяжении большей части этой главы мы рассмотрели другие методы определения производительности модуля. Как мы выяснили, использование `console.log` и `Date.now` – это самый простой метод измерения производительности в приложении, а набор инструментов тестирования *benchmark.js* предоставляет более подробную информацию для замера производительности различных функций. Ради интереса мы напечатали байт-код V8 IR для дальнейшей оценки кода JavaScript и сравнили его с WebAssembly. В следующей главе вы узнаете об отладке модулей WebAssembly.

10

ОТЛАДКА WEBASSEMBLY



В этой главе вы познакомитесь с несколькими методами отладки кода WAT. Мы обсудим протоколирование в консоли и использование предупреждений об ошибке, а также то, как записывать трассировку стека в консоль. Изучим отладчики в Firefox и Chrome, различия между ними и их ограничения.

Карта исходного кода (source map) сопоставляет код, выполняемый в браузере, с оригинальным предварительно скомпилированным исходным кодом. Она позволяет разработчику, разрабатывающему на таких языках, как TypeScript, или фреймворках, таких как React, пройти по исходному коду для его отладки. Наборы инструментов WebAssembly, например Emscripten, сопоставляют сгенерированный двоичный файл WebAssembly с оригинальным исходным кодом C++. На момент написания этой книги `wat2wasm` не генерирует сопоставление для кода WAT, преобразованного в двоичный формат WebAssembly. Это не делает отладку кода WAT бесполезной, однако любые имена локальных или глобальных переменных теряются при преобразовании в двоичный формат. Следовательно, код, который вы пишете в WAT, не будет точно таким же, как в отладчике.

Вы должны вручную сопоставить конкретные имена, которые вы самостоятельно даете переменным, с обобщенными именами, назначенными отладчиком вашего браузера. Позже в этой главе вы узнаете, как понять такое отображение. Как только вы научитесь отлаживать свой код WebAssembly, у вас будут инструменты для пошагового выполнения любого кода WebAssembly, который вы найдете в интернете, даже если у вас нет исходного кода.

Отладка из консоли

Самый простой способ начать отладку кода WebAssembly – вывести сообщения в консоль браузера. Для этого, как вы узнали ранее, WebAssembly должен взаимодействовать с JavaScript. В этой главе мы будем использовать функцию JavaScript для создания файла записи в лог событий отладки. Давайте создадим простую функцию WebAssembly для вычисления расстояния между двумя точками с помощью теоремы Пифагора. Мы намеренно сделаем ошибку в коде, чтобы потом найти ее в отладчике. Создайте новый файл с именем *pythagoras.wat* и добавьте код из листинга 10.1.

Листинг 10.1. Вычисление расстояния между двумя точками с помощью теоремы Пифагора

```
pythagoras.wat (module
  (import "js" "log_f64" (func $log_f64(param i32 f64)))

  (func $distance (export "distance")
    (param $x1 f64) (param $y1 f64) (param $x2 f64) (param $y2 f64)
    (result f64)
    (local $x_dist f64)
    (local $y_dist f64)

    local.get $x1
    local.get $x2
    f64.sub          ;; $x1 - $x2
    local.tee $x_dist ;; $x_dist = $x1 - $x2
    local.get $x_dist
    f64.mul          ;; $x_dist * $x_dist в стеке

    local.get $y1
    local.get $y2
    ❶ f64.add        ;; должно быть $y1 - $y2
    local.tee $y_dist ;; $y_dist = $y1 - $y2
    local.get $y_dist
    f64.mul          ;; $y_dist * $y_dist в стеке
    f64.add          ;; $x_dist * $x_dist + $y_dist * $y_dist в стеке

    f64.sqrt        ;; извлечь квадратный корень из x2 плюс y2
  )
)
```

Чтобы использовать теорему Пифагора, мы создаем прямоугольный треугольник по оси x и оси y между двумя точками. Длина по оси x – это расстояние между двумя значениями x . Таким же образом мы можем найти расстояние по оси y . Чтобы найти расстояние между двумя точками, нужно возвести эти два значения в квадрат, сложить их, а затем извлечь квадратный корень (рис. 10.1).

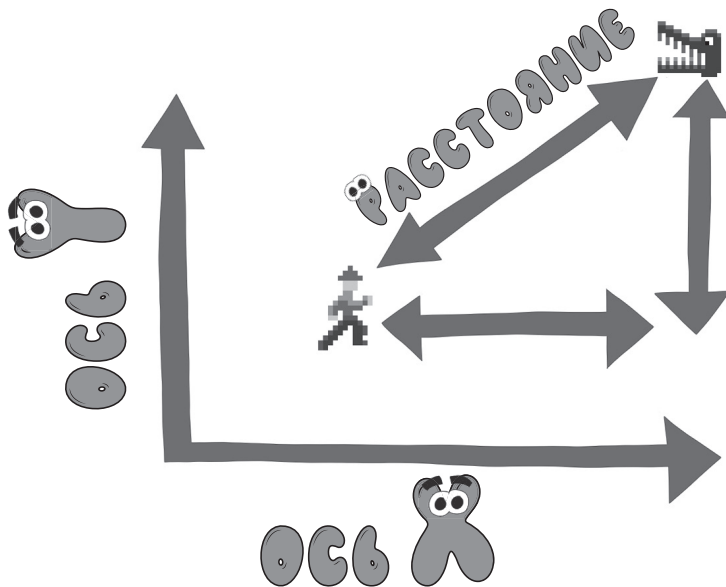


Рис. 10.1. Расчет расстояния между игровыми объектами с помощью теоремы Пифагора

Математика в этом примере не так уж и важна. Важная деталь заключается в том, что мы намеренно допустили ошибку в этом коде, прибавив значения y_1 и y_2 вместо их вычитания, чтобы получить расстояние между двумя координатами по оси y . Скомпилируйте `pythagoras.wat` в `pythagoras.wasm` и создайте новый файл с именем `pythagoras.html`. Затем добавьте код из листинга 10.2 в `pythagoras.html`.

Листинг 10.2. Веб-приложение, которое вызывает функцию расстояния WebAssembly

`pythagoras.html`

```
<!DOCTYPE html>
<html lang="en">
<body>
  ❶ X1: <input type="number" id="x1" value="0">
  ❷ Y1: <input type="number" id="y1" value="0">
  ❸ X2: <input type="number" id="x2" value="4">
  ❹ Y2: <input type="number" id="y2" value="3">
  <br><br>
  ❺ DISTANCE: <span id="dist_out">??</span>
  <script>
```



```

var distance = null;
let importObject = {
  js: {
    ❸ log_f64: function(message_index, value) {
      console.log(`message #${message_index} value=${value}`);
    }
  }
};

( async () => {
  let obj = await WebAssembly.instantiateStreaming(
    fetch('pythagoras.wasm'), importObject );
  distance = obj.instance.exports.distance;
})();

❷ function set_distance() {
  ❹ let dist_out = document.getElementById('dist_out');
  let x1 = document.getElementById('x1');
  let x2 = document.getElementById('x2');
  let y1 = document.getElementById('y1');
  let y2 = document.getElementById('y2');

  ❺ let dist = distance(x1.value, y1.value, x2.value, y2.value);
  dist_out.innerHTML = dist;
}
</script>
<br>
<br>
❻ <button onmousedown="set_distance()">Find Distance</button>
</body>
</html>

```

Внутри тега `<body>` мы настраиваем пользовательский интерфейс с помощью тегов ввода числового типа для координат `x1` ❶, `y1` ❷, `x2` ❸ и `y2` ❹. Мы добавляем тег `span`, который будет содержать расстояние ❺ между двумя точками после запуска функции `WebAssembly`.

`importObject` внутри тега `<script>` содержит функцию `log_f64` ❸, которая в качестве параметров получает индекс сообщения и значение. Эта функция выводит полученные значения в консоль браузера. `WebAssembly` не может напрямую передавать строки в JavaScript (он должен передавать индекс в линейную память), поэтому чаще бывает проще использовать код сообщения и определять строки, которые вы хотите регистрировать из JavaScript. Эта функция использует шаблонную строку ``message #${message_index} value=${value}`` для записи `message_index` и значения в консоль. В качестве альтернативы вы можете выбрать другую шаблонную строку на основе переменной `message_index`. Функция `set_distance` ❷ выполняется, когда пользователь нажимает кнопку **Find Distance** (Найти расстояние) ❻. Эта функция получает идентификаторы элементов для тега `span dist_out` ❹, а также поля ввода `x1`, `x2`, `y1` и `y2`. Затем она выполняет функцию `WebAssembly distance` ❺, используя значения в этих полях ввода.

Запустите веб-сервер и загрузите страницу *pythagoras.html* в браузер; вы должны увидеть то, что изображено на рис. 10.2.

X1: 0	Y1: 0	X2: 4	Y2: 3
DISTANCE: ??			
Find Distance			

Рис. 10.2. Снимок экрана веб-страницы *pythagoras.html*

Значения в форме, которые вы видите на рис. 10.2, являются значениями по умолчанию. Под полями для ввода координат указано значение расстояния «??». После нажатия кнопки **Find Distance** там должно появиться значение 5. Для проверки калькулятора расстояний мы используем треугольник со сторонами 3–4–5. Пока расстояние по оси *x* равно 3, а расстояние по оси *y* равно 4, расстояние между двумя точками будет равно 5, потому что $3^2 + 4^2 = 5^2$, как показано на рис. 10.3.

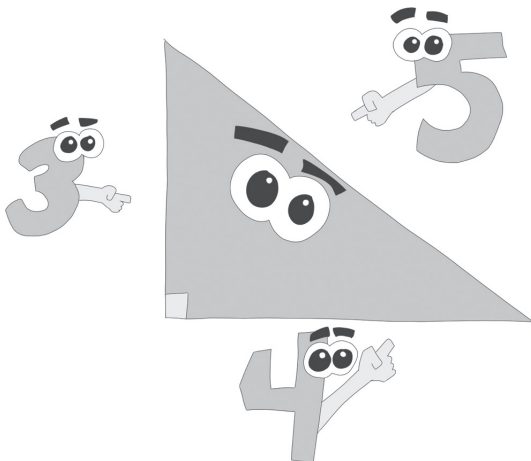


Рис. 10.3. Треугольник со сторонами 3–4–5

Когда вы нажмете кнопку **Find Distance**, то увидите, что в поле **DISTANCE** появилось значение 5, как показано на рис. 10.4.

X1: 0	Y1: 0	X2: 4	Y2: 3
DISTANCE: 5			
Find Distance			

Рис. 10.4. Расстояние, рассчитанное для треугольника со сторонами 3–4–5

Когда мы изменяем значения *X* и *Y* на одинаковую величину, расстояние между двумя точками должно оставаться неизменным. Од-

нако из-за ошибки, которую мы допустили намеренно, прибавление 1 к Y1 и Y2 приводит к неправильному значению, отображаемому в поле DISTANCE (рис. 10.5).

X1: 0	Y1: 1	X2: 4	Y2: 4
DISTANCE: 6.4031242374328485			
Find Distance			

Рис 10.5. Ошибка в вычисленном расстоянии

Вместо цифры 5 в поле DISTANCE мы видим совсем другой результат. Нам нужно отследить, что пошло не так; первый шаг – добавить операторы `log` в несколько мест в нашей функции `distance`.

Как мы знаем, работа со строками непосредственно в WAT – непростая задача. Поэтому для пошагового выполнения и отладки этого кода мы используем идентификатор сообщения вместе со значением, переданным в JavaScript из модуля WebAssembly. Измените файл `pythagoras.wat` для вызова `$log_f64` из функции `$distance`, как в листинге 10.3.

Листинг 10.3. Файл `pythagoras.wat` обновлен с помощью вызовов функций JavaScript для записи переменных `f64`

```
pythagoras.wat ...
(func $distance (export "distance")
  (param $x1 f64) (param $y1 f64) (param $x2 f64) (param $y2 f64) (result f64)
  (local $x_dist f64)
  (local $y_dist f64)
  (local $temp_f64 f64)

  local.get $x1
  local.get $x2
  f64.sub                ;; $x1 - $x2

  local.tee $x_dist      ;; $x_dist = $x1 - $x2

  ❶(call $log_f64 (i32.const 1) (local.get $x_dist))

  local.get $x_dist
  f64.mul                ;; $x_dist * $x_dist в стеке

  ❷local.tee $temp_f64    ;; для удержания вершины стека без изменений
  ❸(call $log_f64 (i32.const 2) (local.get $temp_f64))

  local.get $y1
  local.get $y2
  f64.add                ;; должно быть $y1 - $y2
  local.tee $y_dist      ;; $y_dist = $y1 - $y2

  ❹(call $log_f64 (i32.const 3) (local.get $y_dist))

  local.get $y_dist
```

```

f64.mul          ;; $y_dist * $y_dist в стеке

❷ local.tee $temp_f64 ;; для удержания вершины стека без изменений
❸ (call $log_f64 (i32.const 4) (local.get $temp_f64))

f64.add          ;; $x_dist * $x_dist + $y_dist * $y_dist в стеке

❹ local.tee $temp_f64 ;; для удержания вершины стека без изменений
❺ (call $log_f64 (i32.const 5) (local.get $temp_f64))

f64.sqrt         ;; извлечь квадратный корень из x2 плюс y2

❻ local.tee $temp_f64 ;; для удержания вершины стека без изменений
❼ (call $log_f64 (i32.const 6) (local.get $temp_f64))
)
...

```

Мы добавили здесь вызовы функции `$log_f64` в нескольких местах (❶❷❸❹❺❻). Первый параметр в `$log_f64` – это идентификатор сообщения, являющийся целым числом, которое мы будем использовать в качестве уникального идентификатора для этого сообщения. Позже мы используем этот идентификатор для вывода конкретного сообщения из JavaScript.

Второй параметр – это 64-битное значение с плавающей запятой, которое может показать нам значение на нескольких различных этапах нашего расчета расстояния. В некоторых из этих вызовов мы хотим записать значение в верхней части стека, но не извлекать его, поэтому мы используем `local.tee` (❷❹❺❻), чтобы установить значение `$temp_f64` без его удаления из стека. Затем используем значение в `$temp_f64` при вызове `$log_f64` (❸❹❺❻).

Запись сообщений в консоль

Как упоминалось ранее, модули WebAssembly не могут записывать сообщения напрямую в консоль браузера, а WAT не имеет собственных библиотек для обработки строк. Используемая нами функция `log_f64` импортируется из JavaScript модулем WebAssembly. Итак, в листинге 10.4 мы реализуем эту функцию в JavaScript.

Листинг 10.4. Функция JavaScript, вызываемая `pythagoras.wat`

```

pythagoras.html log_f64: function(message_index, value) {
                  console.log(`message #${message_index} value=${value}`);
                  }

```

Это довольно простая версия, которая записывает индекс сообщения и значение, но не настраивает сообщение для любого из значений `message_index`. В Chrome зайдите в меню браузера и нажмите **More tools** (Дополнительные инструменты), далее откройте инструменты разработчика (рис. 10.6).

Примечание По мере выпуска обновлений отладчика Chrome или Firefox то, что вы видите на своем экране, может немного отличаться от снимков экрана, показанных в этой главе.

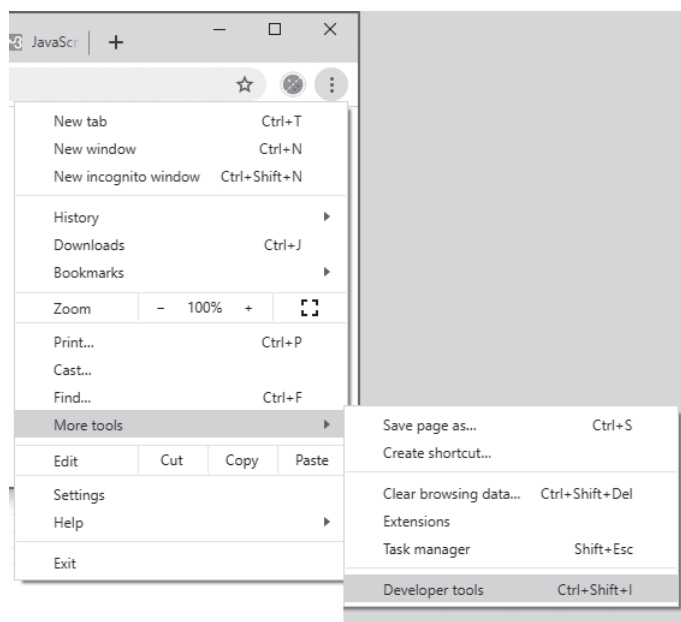


Рис. 10.6. Открытие инструментов разработчика Chrome

Зайдите в **Developer tools** (Инструменты разработчика), а затем нажмите вкладку **Console** (Консоль), чтобы увидеть консоль, как на рис. 10.7.

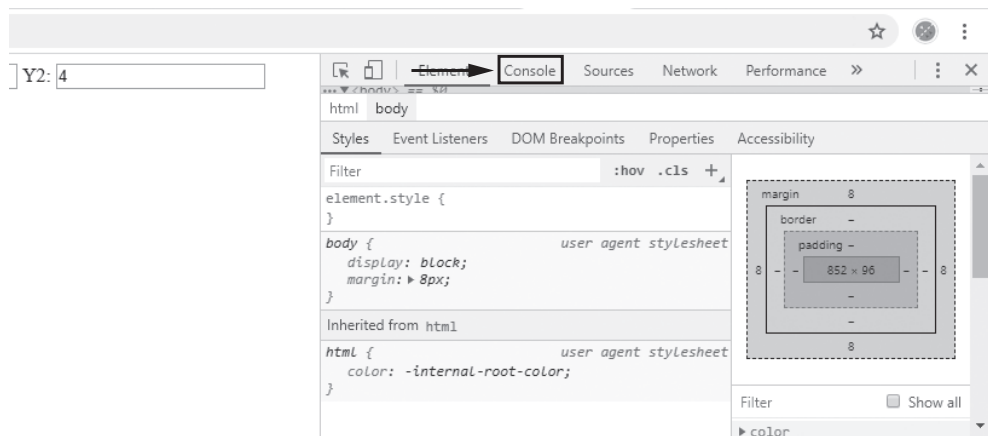


Рис. 10.7. Запуск консоли Chrome

Чтобы открыть консоль в Firefox, нажмите **Web Developer** (Веб-разработчик) в меню браузера, как показано на рис. 10.8.

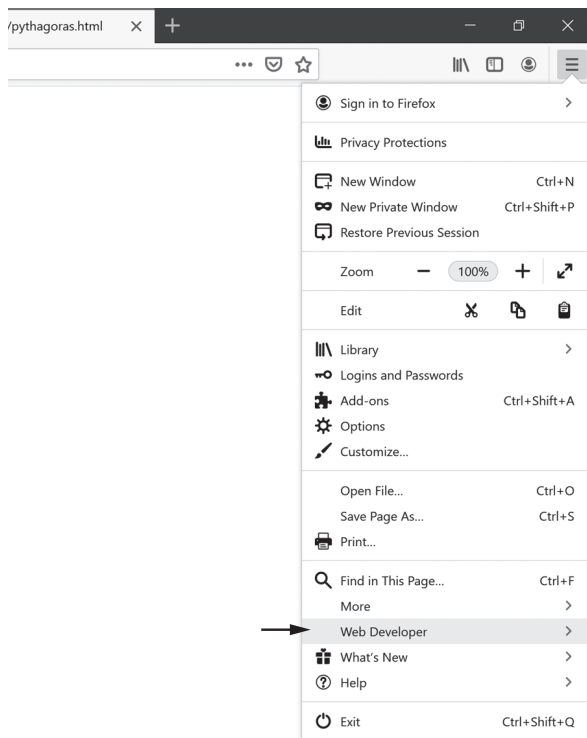


Рис 10.8. Меню веб-разработчика в Firefox

Выберите **Web Console** (Веб-консоль), как показано на рис. 10.9.

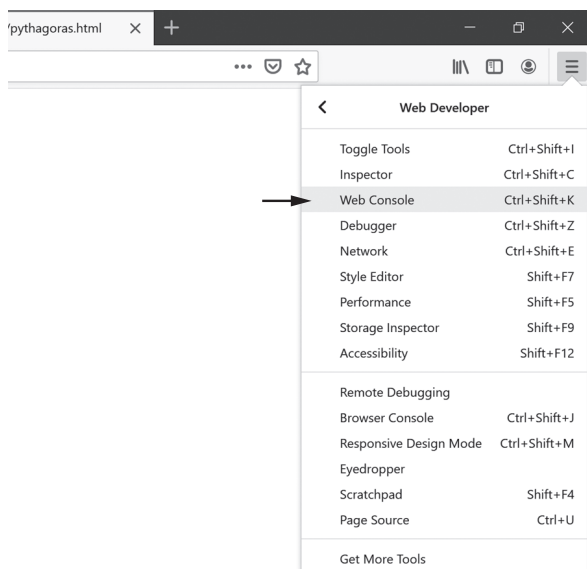


Рис. 10.9. Запуск веб-консоли в Firefox

На экране вы должны увидеть то, что показано на рис. 10.10.



Рис. 10.10. Отображение сообщений в веб-консоли

Все сообщения начинаются с `message #`, за которым следует идентификатор сообщения. Часто такого рода сообщений для нас вполне достаточно, но мы внесем изменения в функцию, чтобы записывать более конкретные сообщения. Например, если у вас возникли проблемы с отслеживанием конкретной проблемы, которая вас беспокоит, возможно, вам понадобится конкретизировать их. Один из вариантов реализации этого подхода показан в листинге 10.5. В других обстоятельствах вы можете использовать иной набор функций журналирования.

Листинг 10.5. Обновление `pythagoras.html` для получения более подробного сообщения

```
pythagoras.html log_f64: function(message_index, value) {  
    switch( message_index ) {  
        case 1:  
            console.log(`x_dist=${value}`);  
            break;  
        case 2:  
            console.log(`x_dist*x_dist=${value}`);  
            break;  
        case 3:  
            console.log(`y_dist=${value}`);  
            break;  
        case 4:  
            console.log(`y_dist*y_dist=${value}`);  
            break;  
        case 5:  
            console.log(`y_dist*y_dist + x_dist*x_dist=${value}`);  
            break;  
        case 6:  
            console.log(`dist=${value}`);  
            break;  
        default:  
            console.log(`message #${message_index} value=${value}`);  
    }  
}
```

Всего у нас есть шесть сообщений, поэтому мы создаем переключатель (оператор `switch/case`) для параметра `message_index`, который выводит на консоль разные сообщения для каждого значения `message_index`. Переключатель имеет значение по умолчанию, которое отображает исходное сообщение, если в журнал заносится непредусмотренное значение `message_index`. После срабатывания этих сообщений обновленный вывод консоли должен выглядеть так, как показано на рис. 10.11.



Рис. 10.11. Сообщения, записываемые в консоль

Предупреждения об ошибках

Далее мы воспользуемся предупреждениями об ошибках (alerts) JavaScript, которые приостанавливают выполнение кода, чтобы дать вам время просмотреть записанные сообщения. Для этой задачи мы воспользуемся функцией `alert`, которая открывает диалоговое окно с текстом ошибки. Помните, что чрезмерное использование предупреждений об ошибках может сделать проверку журналов затратной по времени, поэтому лучше использовать их с осторожностью.

В предыдущем примере функции `log_f64` можно было немедленно предупредить пользователя, если возникла определенная ситуация. Сообщение `alert` останавливает выполнение кода и создает всплывающее окно для уведомления пользователя. Вы можете использовать вызов `alert` только для предупреждения о необычных обстоятельствах, требующих немедленного внимания при отладке. В листинге 10.6 мы изменяем код `case 1`: для вывода предупреждения во всплывающем окне, а не в консоли. Измените начало функции `log_f64`, чтобы оно выглядело как в листинге 10.6.

Листинг 10.6. Обновленный файл `pythagoras.html` для вызова предупреждения об ошибке из `log_f64`

```
pythagoras.html log_f64: function(message_index, value) {
  switch( message_index ) {
    case 1:
      alert(`$x_dist=${value}`);
      break;
```


Мы заменили вызов функции `console.log` на `alert` ❶, чтобы отображать окно предупреждения, когда `message_index` равен 1. Результат, показанный на рис. 10.12, должен отображаться в браузере.

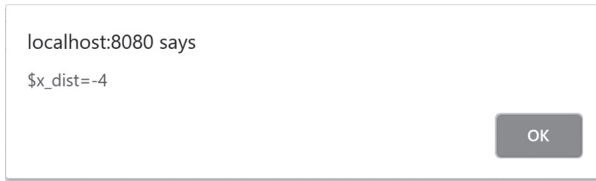


Рис. 10.12. Отображение окна предупреждения

Трассировка стека

Трассировка стека отображает список функций, которые были вызваны для перехода к текущей точке кода. Например, если функция А вызывает функцию В, которая вызывает функцию С, которая затем выполняет трассировку стека, то трассировка стека покажет функции в обратном порядке (С, В и А), а также строки, которые вызывали эти функции. WebAssembly не предоставляет эту функцию напрямую, поэтому при входе в консоль мы вызываем трассировку стека из JavaScript. Последовательность вызываемых функций должна выглядеть так, как показано на рис. 10.13.

Мы отображаем трассировку стека с помощью вызова функции JavaScript `console.trace`. Firefox и Chrome в настоящее время предлагают трассировки стека, которые сильно отличаются друг от друга. Использование `console.trace` в Firefox дает вам больше полезной информации о WAT-файле, чем браузер Chrome. Firefox преобразует двоичный файл WebAssembly в файл WAT и предоставляет трассировку стека, которая ссылается на строку в этом дизассемблированном файле WAT. Chrome, в свою очередь, дает вам ссылку на указатель функции, который может показаться довольно непонятным, если вы с ним незнакомы.

Создайте файл с именем `stack_trace.wat` и сохраните код из листинга 10.7.

Листинг 10.7. Вызовы трассировки стека в модуле WebAssembly

```
stack_trace.wat (module
  ❶(import "js" "log_stack_trace" (func $log_stack_trace (param i32)))
  ❷(func $call_level_1 (param $level i32)
    local.get $level
    call $log_stack_trace
  )
  ❸(func $call_level_2 (param $level i32)
```

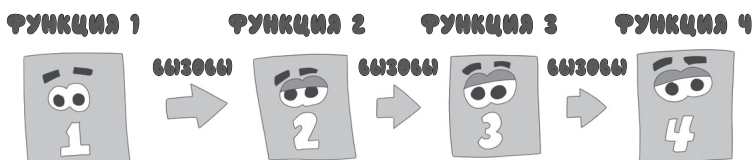
```

    local.get $level
    call $call_level_1
  )

  ④(func $call_level_3 (param $level i32)
    local.get $level
    call $call_level_2
  )

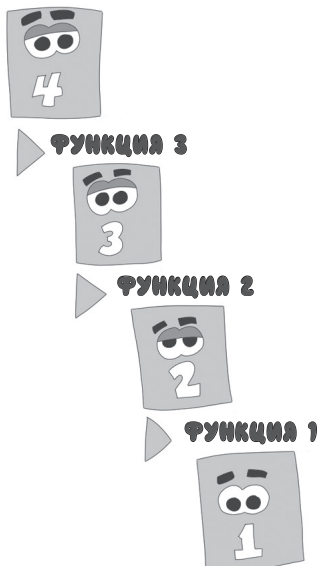
  ⑤(func $call_stack_trace (export "call_stack_trace")
    ⑥(call $log_stack_trace (i32.const 0))
    (call $call_level_1 (i32.const 1))
    (call $call_level_2 (i32.const 2))
    (call $call_level_3 (i32.const 3))
  )
)

```



ТРАССИРОВКА СТЕКА

ФУНКЦИЯ 4



**КАЖДАЯ ФУНКЦИЯ
БЫЛА ВЫЗВАНА
ПРЕДЫДУЩЕЙ ФУНКЦИЕЙ
В ТРАССИРОВКЕ СТЕКА**

Рис. 10.13. Функция 1 вызывает функцию 2, которая вызывает функцию 3, которая вызывает функцию 4 в трассировке стека

Этот модуль WebAssembly импортирует из JavaScript функцию `log_stack_trace` ①, которая будет вызывать `console.trace` из встроенного

JavaScript. Мы определяем еще четыре функции, которые демонстрируют, как каждый браузер записывает стек вызовов WebAssembly. Импортированная функция `$log_stack_trace` вызывается посредством функций `$call_stack_trace` и `$call_level_1` ❷. `$call_level_1` вызывается функциями `$call_stack_trace` и `$call_level_2` ❸. `$call_level_2` вызывается функциями `$call_stack_trace` и `$call_level_3` ❹. Наконец, `$call_level_3` вызывается функцией `$call_stack_trace`. Мы вкладываем эти вызовы функций, чтобы продемонстрировать, как выглядят трассировки стека при вызове с разных уровней функций.

Обратите внимание, что `$call_stack_trace` ❺ вызывает все остальные функции. Сначала она напрямую вызывает `$log_stack_trace`, передавая константу 0. Затем вызывает `$call_level_1`, которая вызывает `$log_stack_trace`, передавая постоянное значение 1. Когда трассировка стека записывается в журнал, она должна показывать `$call_level_1`, `$log_stack_trace` ❻ и `$call_stack_trace` в стеке вызовов. Каждая из функций `$call_level_2` и `$call_level_3` добавляет дополнительные слои, которые будут отображаться в трассировке стека.

Теперь создайте новый файл с именем `stack_trace.html` с кодом из листинга 10.8.

Листинг 10.8. HTML-файл с вызовами JavaScript для трассировки стека

```
stack_trace.html <!DOCTYPE html>
<html lang="en">
<body>
  <h1>Stack Trace</h1>
  <script>
    let importObject = {
      js: {
        ❶ log_stack_trace: function( level ) {
          console.trace(`level=${level}`);
        }
      }
    };

    ( async () => {
      let obj =
        await WebAssembly.instantiateStreaming( fetch('stack_trace.wasm'),
          importObject );

      obj.instance.exports.call_stack_trace();

    })();
  </script>
</body>
</html>
```

Это очень простой HTML-файл, похожий на `pythagoras.html`. Основной код – это функция `log_stack_trace` ❶, определенная внутри `importObject`, которая вызывает функцию JavaScript `console.trace`, передавая ей строку для вывода на консоль перед трассировкой стека.

После сохранения этого HTML-файла откройте его в браузере Firefox; вы должны увидеть журналы консоли, аналогичные показанным на рис. 10.14.

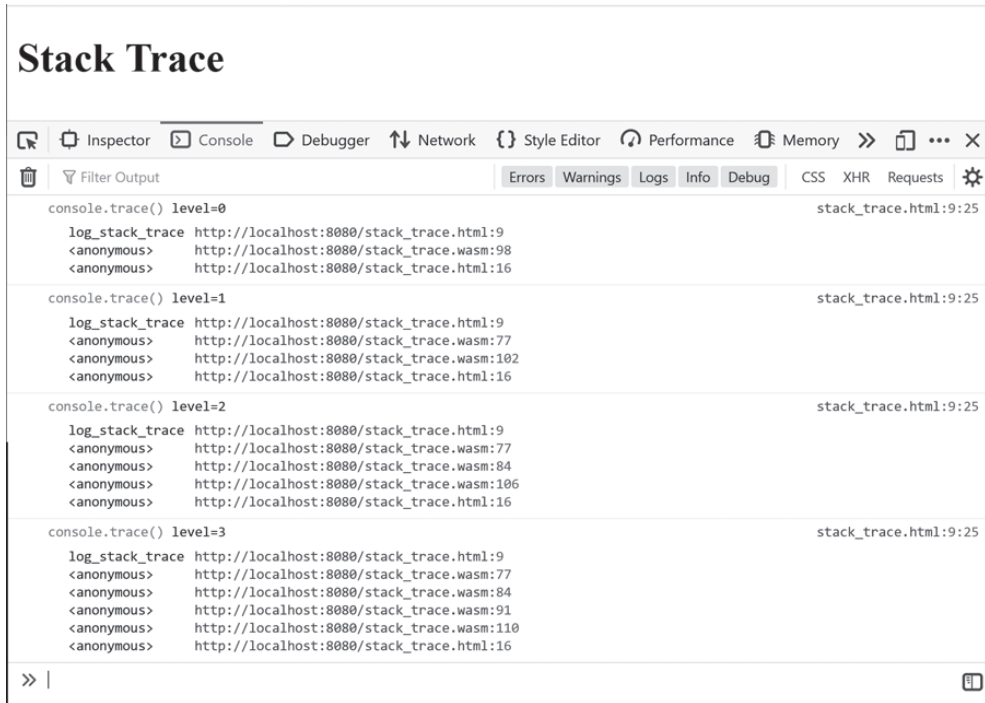


Рис. 10.14. Отображение трассировки стека в Firefox

Как видите, первая трассировка стека была записана с `level=0`, потому что мы передали значение 0 непосредственно в первый вызов `$log_stack_trace` в коде WAT. Это был прямой вызов функции WebAssembly `$call_stack_trace` в импортированную функцию JavaScript. Поскольку этот первый вызов был направлен непосредственно к `$log_stack_trace` для файла `stack_trace.wasm`, в этой первой трассировке стека записывается только один фрейм стека. В этом журнале показано, что трассировка стека была выполнена из строки 98 файла `stack_trace.wasm`. Однако в вашем WAT-файле может быть другая строка; вам нужно будет посмотреть на WAT в браузере, чтобы увидеть нужную строку. Каждая трассировка добавляет дополнительный вызов функции в файл WebAssembly, потому что мы добавили дополнительный уровень функций к каждому вызову `$log_stack_trace` в WAT. Обратите внимание, что в каждой трассировке стека есть дополнительная строка внутри `stack_trace.wasm`.

Щелкните одну из этих строк; Firefox открывает файл `stack_trace.wasm` в том месте кода, где произошел вызов функции.

Если вы еще не открыли `stack_trace.wasm` в отладчике Firefox, вам может быть предложено обновить страницу браузера, чтобы просмотреть

реть содержимое как дизассемблированный WAT. Когда *stack_trace.wasm* открывается с 98-го байта, вы должны увидеть в консоли отладчика Firefox содержимое, аналогичное рис. 10.15.

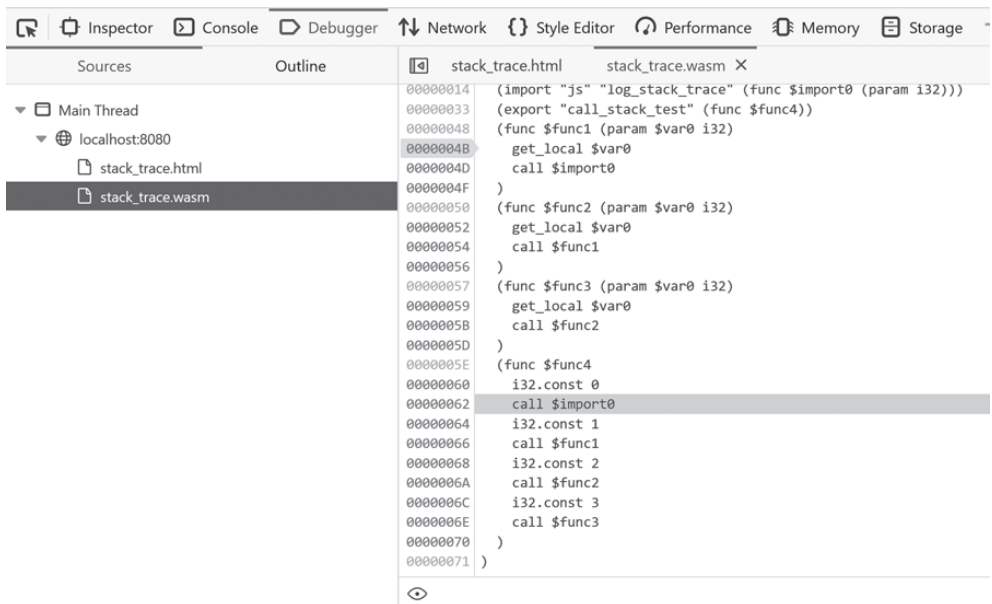


Рис. 10.15. Отображение WAT-кода в *stack_trace.wasm*

Строка, в которой выполняется вызов, временно выделяется серым цветом. Обратите внимание, что номер байта слева (62) представлен в шестнадцатеричном формате, в отличие от журнала консоли, где байтом является десятичное число 98.

Chrome не отображает номер байта внутри файла WAT для каждой трассировки стека; его представление похоже на рис. 10.16.

В браузере Chrome номер строки всегда равен 1. Однако когда вы нажимаете на ссылку в консоли, Chrome открывает дизассемблированную версию этой конкретной функции. Все функции WebAssembly начинаются с префикса *wasm-* и заканчиваются индексом функции, за которым следует *:1*. На рис. 10.17 показано, как приблизительно выглядит вывод кода, когда вы нажимаете на первую функцию WebAssembly, появившуюся в трассировке стека.

Дизассемблированная функция в Chrome отличается от функции Firefox. Мы рассмотрим эти различия более подробно в следующем разделе. А пока обратите внимание, что Chrome использует для дизассемблирования индексы переменных и функций, а не метки, которые сложнее читать.

Когда вы хотите выяснить, как выполняются и вызываются определенные функции, трассировка стека может помочь вам. Теперь посмотрим на код в отладчиках Firefox и Chrome.

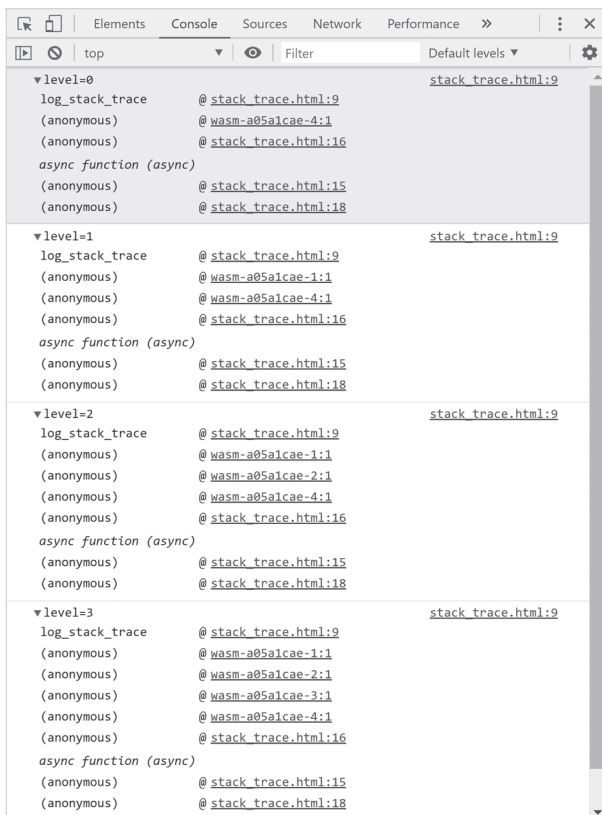


Рис. 10.16. Отображение трассировки стека в Chrome

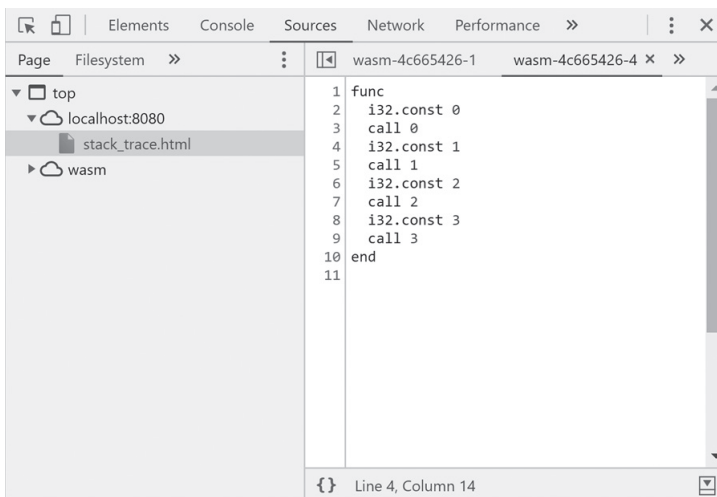


Рис. 10.17. Нажатие на трассировку стека в Chrome отображает функцию WebAssembly

Отладчик Firefox

В этом разделе мы напишем код, который вы можете пошагово выполнить в нашем отладчике. Сначала найдите время заново просмотреть файлы *pythagoras.html* и *pythagoras.wat*. В них мы намеренно допустили ошибку, чтобы отследить ее в отладчике. Мы изменим *pythagoras.wat*, удалив вызовы журнала вывода в JavaScript, чтобы отладчик смог проверить его. Создайте файл с именем *debugger.wat* и добавьте код из листинга 10.9 или просто удалите вызовы журнала в *pythagoras.wat* и повторно сохраните файл.

Листинг 10.9. Удаление вызовов журнала в файле *pythagoras.wat*

```
debugger.wat (module
  (func $distance (export "distance")
    (param $x1 f64) (param $y1 f64) (param $x2 f64) (param $y2 f64)
    (result f64)
    (local $x_dist f64)
    (local $y_dist f64)

    local.get $x1
    local.get $x2
    f64.sub          ;; $x1 - $x2

    local.tee $x_dist ;; $x_dist = $x1 - $x2
    local.get $x_dist
    f64.mul          ;; $x_dist * $x_dist в стеке

    local.get $y1
    local.get $y2
    f64.add          ;; Должно быть $y1 - $y2
    local.tee $y_dist ;; $y_dist = $y1 - $y2

    local.get $y_dist
    f64.mul          ;; $y_dist * $y_dist в стеке

    f64.add          ;; $x_dist * $x_dist + $y_dist * $y_dist в стеке

    f64.sqrt         ;; извлечь квадратный корень из x2 плюс y2
  )
)
```

Допущенная нами ошибка давала неверный результат, прибавляя \$y1 к \$y2 вместо вычитания. Скопируйте *pythagoras.html* в новый файл с именем *debugger.html* и измените код JavaScript внутри тегов `<script>`, чтобы запрашивать *debugger.wasm*. Затем удалите `importObject`, чтобы получился код, как в листинге 10.10.

Листинг 10.10. HTML-файл для тестирования *debugger.wasm*

```
pythagoras.html ...
<script>
```

```

var distance = null;

( async () => {
    let obj = await WebAssembly.instantiateStreaming( fetch('debugger.wasm')
);

    distance = obj.instance.exports.distance;

})();

function set_distance() {
    let dist_out = document.getElementById('dist_out');
    let x1 = document.getElementById('x1');
    let x2 = document.getElementById('x2');
    let y1 = document.getElementById('y1');
    let y2 = document.getElementById('y2');

    let dist = distance(x1.value, y1.value, x2.value, y2.value);
    dist_out.innerHTML = dist;
}
</script>
...

```

Загрузите *debugger.html* в Firefox и откройте консоль; затем нажмите вкладку **Debugger** (Отладчик), чтобы получить доступ к отладчику Firefox. На вкладке **Sources** (Источники) слева выберите *debugger.wasm*, чтобы увидеть дизассемблированную версию вашего WAT-кода, которая должна выглядеть, как на рис. 10.18.

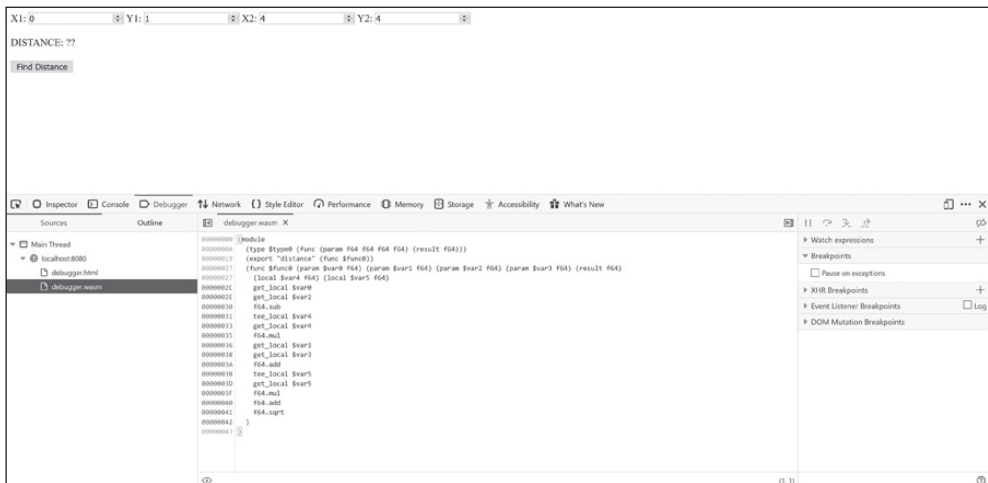


Рис. 10.18. Код WAT в отладчике Firefox

Этот код представляет собой дизассемблированный двоичный файл WebAssembly, поэтому теперь имена функций и переменных больше не доступны. Такой результат аналогичен тому, который вы бы увидели, если бы разобрали двоичный файл из интернета. По-

сколько в wat2wasm пока недоступна карта сопоставлений с исходным кодом, мы не можем пошагово выполнить исходный код в отладчике. Вместо этого вам нужно провести параллельное сравнение исходного кода и дизассемблированного кода. В листинге 10.11 показано, как выглядит этот дизассемблированный код.

Листинг 10.11. Код WAT, сгенерированный дизассемблированием Firefox

```
(module
  (type $type0 (func (param f64 f64 f64 f64) (result f64)))
  (export "distance" (func $func0))
  ❶ (func $func0
    (param ❷$var0 f64)(param ❸$var1 f64)(param ❹$var2 f64)(param ❺$var3 f64)
    (result f64)
    (local ❻$var4 f64) (local ❼$var5 f64)
    local.get $var0
    local.get $var2
    f64.sub
    local.tee $var4
    local.get $var4
    f64.mul
    local.get $var1
    local.get $var3
    f64.add
    local.tee $var5
    local.get $var5
    f64.mul
    f64.add
    f64.sqrt
  )
)
```

После дизассемблирования из двоичного файла WebAssembly не знает, какие ярлыки мы присвоили переменным или функциям. Ему также неизвестны какие-либо комментарии в коде. Если вы посмотрите на исходный код WAT (листинг 10.9), то увидите, что функция \$distance заменилась на \$func0 ❶. Переменные параметра \$x1, \$y1, \$x2 и \$y2 заменены на \$var0 ❷, \$var1 ❸, \$var2 ❹ и \$var3 ❺ соответственно. Локальные переменные \$x_dist и \$y_dist – на \$var4 ❻ и \$var5 ❼. Как только вы узнаете, какая из исходных переменных соответствует своей дизассемблированной версии, вам будет легче понимать код. Чтобы просмотреть значения этих переменных, вы можете ввести их в окне **Watch expressions** (Отслеживание выражений) без символа \$. В этом окне вы можете наблюдать за переменной \$var0, введя **var0**. Для отслеживания переменных я использую простой трюк. Я добавляю комментарий JavaScript вместе со своим отслеживаемым выражением, маркируя переменную ее исходным именем. Например, я могу ввести \$var0 в **Watch expressions** как var0 // \$x1. На рис. 10.19 показано, как это выглядит.



Рис. 10.19. Использование комментариев на вкладке **Watch expressions** в Firefox

Для пошагового выполнения кода WAT убедитесь, что выбран файл WebAssembly. Нам нужно создать точку останова, то есть точку, в которой отладчик прекращает выполнение кода, чтобы вы всегда могли перемещаться на одну строку за шаг. Чтобы установить точку останова, нажмите на номер байта слева от кода WAT. Вы можете наблюдать за изменением переменных в окне справа. Установив точку останова, выполните код WebAssembly, нажав **Find Distance** (рис. 10.20).

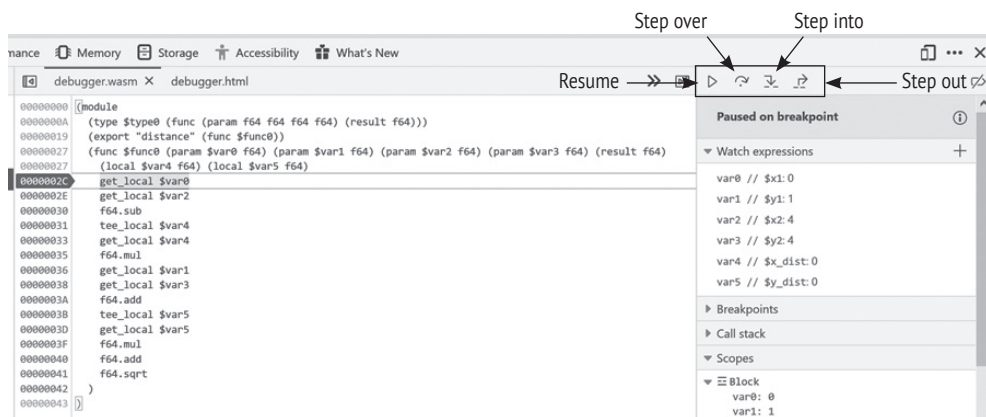


Рис. 10.20. Установка точки останова в отладчике Firefox

Когда выполнение достигает точки останова, нажмите кнопку **Step over** (Перешагнуть) , расположенную над вкладкой **Watch expressions**, что позволит вам выполнять код по одной строке за раз. Чтобы перейти к функции вместо ее выполнения, нажмите кнопку **Step into** (Войти) , расположенную рядом с кнопкой **Step over**. Справа от кнопки **Step into** расположена кнопка **Step out** (Выйти) , если вы хотите выйти из текущей функции. Чтобы указать отладчику, что нужно продолжить выполнение, пока он не достигнет еще одной точки останова, нажмите кнопку **Resume** (Продолжить) , которая выглядит как значок воспроизведения.

Чтобы найти ошибку в коде, нажимайте кнопку **Resume**, пока не дойдете до строки 3D. На этом этапе задана переменная `var5`, и мы можем видеть значение в окне **Watch expressions**, как показано на рис. 10.21.

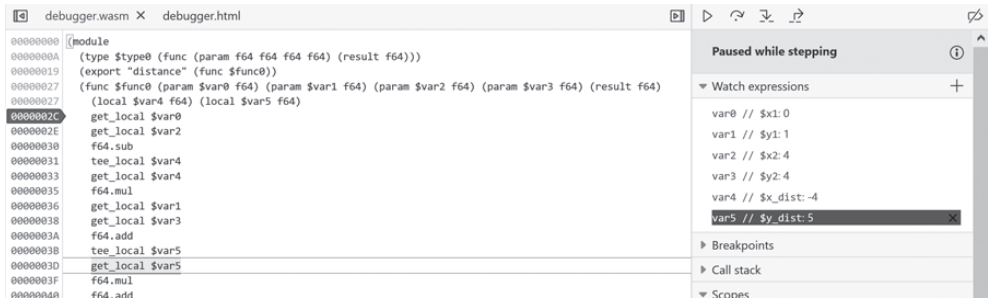


Рис. 10.21. Пошаговое выполнение кода в отладчике Firefox

Обратите внимание, что для `$y_dist` задано значение 5, когда `Y1` было задано равным 1, а `Y2` равным 4. Это означает, что `$y_dist` должно было быть равным 3. Ранее мы изменили строку с номером 3A с `f64.sub` на `f64.add`, чтобы ввести эту ошибку. Построчное прохождение по коду в отладчике помогло нам отследить проблему.

Отладчик Chrome

Отладка WebAssembly в Chrome несколько отличается от отладки того же кода в Firefox. Код WAT не разбивается на файл WebAssembly; напротив, Chrome группирует код WAT по функциям. Число в конце функции WebAssembly – это порядковый номер, основанный на том, где вы определили функцию в своем коде.

Чтобы перейти к отладчику, откройте **Developer tools** в Chrome и перейдите на вкладку **Sources**. В разделе **Page** (Страница) вы должны увидеть значок облака с надписью **wasm**. Разверните эту ветку, чтобы увидеть страницу для каждой функции, определенной в вашем модуле WebAssembly. Поскольку мы определили только одну функцию в этом модуле, существует лишь одна функция. Щелкните на эту функцию, чтобы отобразить ее код в окне справа. В этом окне задайте точку останова в строке 3, которая содержит код `local.get 0` (рис. 10.22).

Обратите внимание, что `local.get` получает число вместо имени переменной. Причина в том, что `local.get` получает локальную переменную на основе индекса, а не имени. Использование `local.get 0` в Chrome эквивалентно `local.get $var0` в браузере Firefox. Как и в Firefox, вы можете посмотреть код и сопоставить его с кодом в вашей функции. В листинге 10.12 показан код в том виде, в котором он отображается в отладчике Chrome.

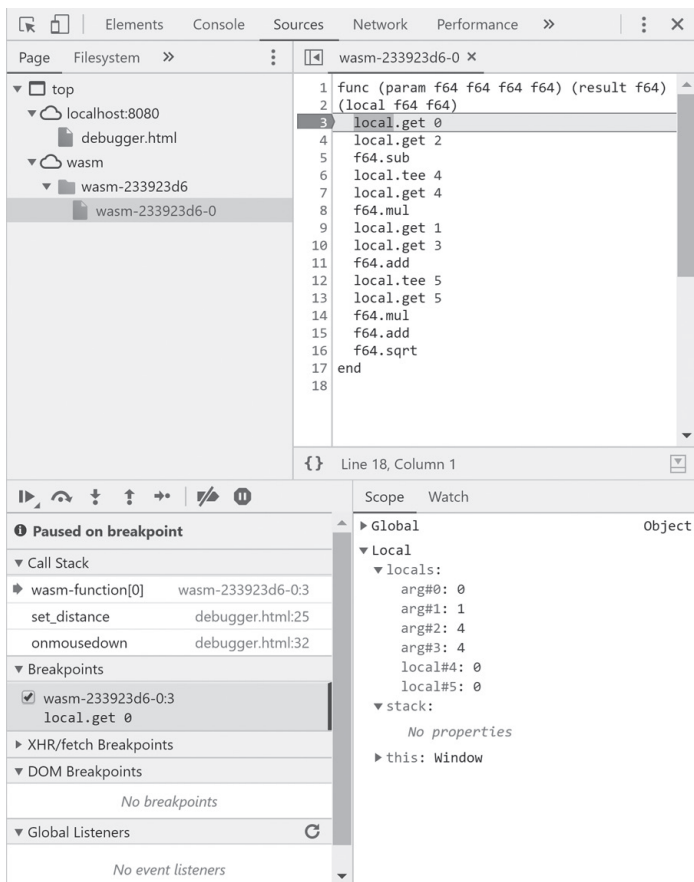


Рис. 10.22. Задаем точку останова в отладчике Chrome

Листинг 10.12. Дизассемблирование WAT в отладчике Chrome

```

❶ func (param f64 f64 f64 f64) (result f64)
❷ (local f64 f64)
  local.get 0
  local.get 2
  f64.sub
  local.tee 4
  local.get 4
  f64.mul
  local.get 1
  local.get 3
  f64.add
  local.tee 5
  local.get 5
  f64.mul
  f64.add
  f64.sqrt
end

```

Обратите внимание, что Chrome использует индексы для локальных переменных, параметров и функций. У функции ❶ нет имен ее параметров или локальных переменных ❷. То же самое с глобальными переменными и типами. Для глобальных переменных мы бы использовали `global.get` и `global.set`, передав номер индекса, соответствующий порядку, в котором были определены переменные.

Примечание Начиная с Chrome версии v86 отладчик показывает имена функций.

Одной из приятных особенностей функции отладки Chrome является то, что у вас есть доступ к стеку в окне **Scope** (Область действия). По мере прохождения по коду вы можете наблюдать, как значения проталкиваются и выталкиваются из стека. Одним из недостатков консоли Chrome является то, что его окно **Watch** менее полезно, чем в Firefox, потому что Chrome не делает переменные WAT простыми для понимания, по аналогии с переменными JavaScript.

Как и в Firefox, в Chrome есть кнопка **Resume** ▶, кнопки **Step over** ↻, **Step into** ⚡ и **Step out** ⬆, как показано на рис. 10.23.

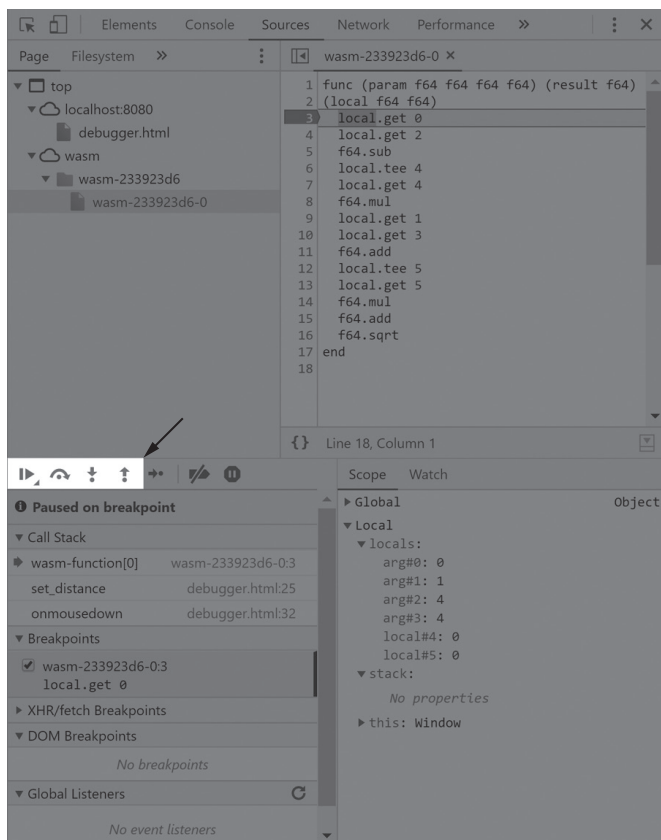


Рис. 10.23. Просмотр стека в отладчике Chrome

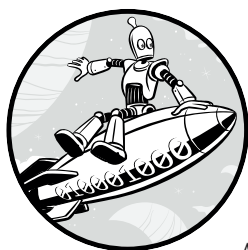
Заключение

В этой главе мы отладили код WAT, используя различные методы в Chrome и Firefox. Мы рассмотрели ведение журнала в консоли более подробно, чем в предыдущих главах. Затем использовали функцию JavaScript `alert`, чтобы остановить выполнение кода и дождаться инструкций пользователя. Мы также исследовали применение `console.trace` для записи трассировки стека и обсудили различия между тем, как трассировка стека работает в Chrome и Firefox. Наконец, использовали встроенные отладчики в Chrome и Firefox.

Для отладки WebAssembly доступно множество возможностей. Некоторые возможности, например применение отладчиков Chrome или Firefox, все еще находятся в стадии разработки. Использование тех или иных инструментов при отладке будет зависеть от кода и вашей цели. В следующей главе мы будем пользоваться WebAssembly для создания модулей Node.js.

11

ASSEMBLYSCRIPT



AssemblyScript – это язык высокого уровня, специально разработанный для компиляции кода в WebAssembly или WAT. AssemblyScript более функционален, чем WAT, но все же может компилироваться в него. Когда вы используете AssemblyScript, то теряете некоторые возможности тонкой оптимизации, которые у вас есть с WAT, но код на нем создавать проще.

Мы начнем эту главу с создания простой функции `AddInt`, подобной той, которую создали в главе 1. Напишем приложение Hello World на AssemblyScript и скомпилируем его в WAT, чтобы увидеть код WebAssembly, сгенерированный компилятором AssemblyScript. Мы рассмотрим использование строк AssemblyScript с префиксом длины, а затем установим загрузчик AssemblyScript, чтобы узнать, как он может упростить передачу строк между AssemblyScript и JavaScript. Передадим строки в AssemblyScript, создав приложение для объединения строк. Мы также изучим ООП в AssemblyScript. Создадим пару классов, чтобы продемонстрировать наследование классов, и обсудим атрибут `private`, который не позволяет AssemblyScript экспортировать атрибуты в среду встраивания. Затем разработаем код на JavaScript, который позволит нам напрямую создавать модификаторы доступа `public` (общедоступный), `private` (частный) и `protected`

(защищенный), а также использовать загрузчик AssemblyScript. Затем мы сравним производительность прямого вызова функций и вызова через загрузчик.

Разработчики сделали язык AssemblyScript похожим на TypeScript и JavaScript. В отличие от WAT, AssemblyScript – это язык высокого уровня с такими опциями, как классы, строки и массивы. Наряду с высокоуровневыми функциями AssemblyScript позволяет пользователям использовать в коде низкоуровневые команды памяти, подобные WAT. AssemblyScript имеет *интерфейс командной строки* (CLI, *Command Line Interface*), который может компилировать AssemblyScript в модуль WebAssembly для использования из приложений JavaScript.

AssemblyScript – отличный инструмент для разработчиков, работающих с JavaScript, которые заинтересованы в использовании WebAssembly для повышения производительности своих приложений. К сожалению, как и все в WebAssembly, простая настройка TypeScript до его компиляции при помощи компилятора AssemblyScript может не привести к значительному повышению производительности. Понимание того, как работает AssemblyScript, позволяет создавать программы на языке, который выглядит как JavaScript, но работает как C++. Чтобы разобраться в этом, мы скомпилируем код AssemblyScript в WAT и изучим выходной код компилятора AssemblyScript.

Интерфейс командной строки в AssemblyScript

Установите AssemblyScript с помощью следующей команды:

```
npm install assemblyscript -g
```

Команда `npm` устанавливает AssemblyScript глобально, что позволяет использовать команду `asc` компилятора AssemblyScript из командной строки. Запуск `asc -h` предоставляет список примеров и параметров команд компилятора.

Я не буду объяснять все аргументы командной строки, но упомяну несколько полезных. Параметр `-O` оптимизирует так же, как `wasm-opt` в главе 9. После `-O` вы вводите числа от 0 до 3, `s` или `z`, указывая компилятору на оптимизацию размера файла или производительности, а также степень оптимизации. Флаг `-o`, за которым следует имя файла в формате `.wat`, будет генерировать код WAT из AssemblyScript, а когда за ним следует имя файла `.wasm`, будет создан двоичный модуль WebAssembly. Флаг `--sourceMap` создает файл карты исходного кода, который поможет вам отладить свой AssemblyScript из браузера.

Сначала мы создадим простой модуль AssemblyScript. Создайте файл `as_add.ts` и добавьте код из листинга 11.1. Это упрощенная версия функции `AddInt` из главы 1.

Листинг 11.1. Сложение двух целых чисел

```
as_add.ts ❶export function AddInts(❷a: i32, ❸b: i32 ): i32 {  
    ❹return a + b;  
}
```

Мы делаем function доступной для встраиваемого JavaScript с помощью ключевого слова `export` ❶, которое получает два параметра `i32` – `a` ❷ и `b` ❸ и возвращает `a + b` ❹ как `i32`. Скомпилируйте файл `as_add.ts` с помощью команды из листинга 11.2.

Листинг 11.2. Компиляция AddInt в WAT

```
asc as_add.ts -Oz -o as_add.wat
```

Флаг `-Oz` максимально сокращает размер выходного двоичного файла. Последний флаг, `-o as_add.wat`, указывает компилятору вывести WAT. В качестве альтернативы мы могли бы скомпилировать файл в формат `.wasm`, например `as_add.wasm`, который выводил бы двоичный файл WebAssembly. В выводимом файле `as_add.wat` мы видим WAT-код, как в листинге 11.3.

Листинг 11.3. Функция AssemblyScript AddInts, скомпилированная в WAT

```
as_add.wat (module  
  (type $i32_i32_=>i32 (func (param i32 i32) (result i32)))  
  (memory $0 0)  
  ❶(export "AddInts" (func $as_add/AddInts))  
  (export "memory" (memory $0))  
  ❷(func $as_add/AddInts (param $0 i32) (param $1 i32) (result i32)  
    ❸local.get $0  
    ❹local.get $1  
    ❺i32.add  
  )  
)
```

Разрабатывать программы на AssemblyScript намного проще, чем на WAT. Этот код создает функцию `AddInts` ❶, которая экспортирует функцию ❷, принимающую два параметра `i32` и возвращающую значение `i32`. Функция вывода использует `local.get` для получения первых ❸ и вторых ❹ параметров и `i32.add` ❺ для сложения этих двух значений.

AssemblyScript – это красивый и простой язык, который относительно легко изучить любому, кто знаком с TypeScript или JavaScript. Понимание WAT – отличный способ получить максимальную отдачу от AssemblyScript или любого другого высокоуровневого языка, который вы выберете для разработки WebAssembly.

Приложение Hello World на AssemblyScript

Затем мы создадим ассемблерную версию приложения WAT Hello World из главы 2. Создайте новый файл AssemblyScript с именем `as_hello.ts` и добавьте код из листинга 11.4.

Листинг 11.4. Приложение Hello World на AssemblyScript

```
as_hello.ts ❶ declare function console_log( msg: string ):void;

❷ export function HelloWorld():void {
  ❸ console_log("hello world!");
}
```

Объявление функции в AssemblyScript должно соответствовать функции JavaScript, переданной в модуль WebAssembly. Поэтому нам нужно передать через `importObject` функцию, которая записывает нашу строку в консоль. `declare function` ❶ импортирует функцию `console_log` из JavaScript. Эта функция передаст обратно строку из AssemblyScript в вызывающее приложение JavaScript. Мы создаем `export function` под названием `HelloWorld` ❷, которая вызывает импортированную функцию `console_log` ❸, передавая ей строку `"hello world!"`. Прежде чем скомпилируем этот код в модуль WebAssembly, мы воспользуемся расширением `asc` для компиляции файла WAT, чтобы можно было посмотреть на созданный код WebAssembly (листинг 11.5).

Листинг 11.5. Скомпилируйте AssemblyScript-файл as_hello.ts в as_hello.wat.

```
asc as_hello.ts -Oz -o as_hello.wat
```

Затем мы можем открыть `as_hello.wat` (листинг 11.6), чтобы увидеть код WebAssembly, сгенерированный AssemblyScript.

Листинг 11.6. Файл as_hello.wat, сгенерированный из AssemblyScript as_hello.ts

```
;; Все комментарии были добавлены автором, а не сгенерированы asc ❶
(module
  (type $none=>_none (func))
  (type $i32=>_none (func (param i32)))
  ;; команда declare в верхней части AssemblyScript создала импорт объекта,
  ;; который импортирует функцию console_log внутри внешнего объекта as_hello
  ;; AssemblyScript требует для импорта имя файла AssemblyScript,
  ;; не включая расширение .ts
  (import "as_hello" "console_log" (func $as_hello/console_log (param i32))) ❷
  ;; использование строки автоматически создает выражение memory
  (memory $0 1) ❸
  ;; строка данных ниже переносится, потому что она слишком длинная
```

```

;; строка "hello world!" начинается с заголовка и имеет шестнадцатеричный байт 00
;; между каждой буквой в строке. Это потому, что AssemblyScript использует
;; набор символов UTF-16 вместо ASCII, как мы это делали,
;; когда работали со строковыми данными в WAT.
(data (i32.const 16) ❹
  "\18\00\00\00\01\00\00\00\01\00\00\00\18\00\00\00h\00e\00l\00o\00
\00w\00o\00r\00l\00d\00!")
(exports "memory" (memory $0))
;; Модуль экспортирует нашу функцию AssemblyScript с именем, которое мы задали.
(exports "HelloWorld" (func $as_hello/HelloWorld)) ❺
;; имя функции, которое мы дали AssemblyScript, имеет префикс с именем нашего файла
;; без расширения .ts
(func $as_hello/HelloWorld (; 1 ;) ❻
  ;; 32 – это местоположение в линейной памяти байта 'h' в "hello world"
  i32.const 32 ❼
  ;; вызывается функция console_log, передающая адрес "hello world" в линейную память
  call $as_hello/console_log ❸
)
)

```

Я добавил комментарии, чтобы внести ясность в код ❶. Этот модуль импортирует `console_log` ❷, обернутый в объект `as_hello` без расширения `.ts`, который является именем нашего файла `AssemblyScript`. Это соглашение об именах, которое `AssemblyScript` использует для своего `importObject`; когда вы разрабатываете свой код на JavaScript, вы должны соответствующим образом назвать свой объект внутри импортированного объекта.

`AssemblyScript` создает выражение `memory` ❸ для хранения строковых данных. Строка имеет заголовок с префиксом своей длины, которую `AssemblyScript` использует для управления данными из `WebAssembly`. Строковые данные `data` ❹ используют два байта на символ, и поскольку `AssemblyScript` применяет UTF-16, каждый символ в этом примере разделен нулевым байтом `\00`. UTF-16 – это 16-битная версия набора символов Unicode, которая позволяет использовать многие дополнительные символы, недоступные в ASCII.

После `data` WAT экспортирует функцию ❺ с именем, которое мы дали ей в `AssemblyScript`, с префиксом символа `$` и без расширения `.ts`. Функция `HelloWorld` ❻ вызывает `console_log` ❸, передавая местоположение первого символа в нашей строке `hello world!` в линейной памяти, которое равно 32 ❼.

Когда наш WAT-файл скомпилирован, мы можем использовать команду `asc` из листинга 11.7 для компиляции нашего модуля `WebAssembly`.

Листинг 11.7. Компиляция нашего `AssemblyScript` в двоичный файл `WebAssembly`

```
asc as_hello.ts -Oz -o as_hello.wasm
```

Далее мы создадим наш код на JavaScript.

Код JavaScript для приложения Hello World

В настоящее время у нас есть модуль WebAssembly с именем *as_hello.wasm*. Далее мы напишем приложение Node.js, которое загрузит и запустит этот модуль. В этом разделе, чтобы понять, как AssemblyScript передает строки в JavaScript, мы декодируем строковые данные так же, как в главе 5. Затем воспользуемся инструментом загрузчика AssemblyScript, который сделает большую часть работы за нас.

Сначала мы напишем функцию для извлечения строки из линейной памяти посредством индекса, переданного из модуля WebAssembly. AssemblyScript помещает длину строки в четыре байта, непосредственно предшествующих строковым данным. Мы используем Uint32Array для получения целочисленной длины строки и с помощью этой длины создадим наши строки в JavaScript. Создайте файл с именем *as_hello.js* и повторите код из листинга 11.8.

Листинг 11.8. Вызов AssemblyScript HelloWorld из JavaScript

```
as_hello.js  const fs = require('fs');
              const bytes = fs.readFileSync(__dirname + '/as_hello.wasm');

              // Объект памяти экспортируется из AssemblyScript
              ❶ var memory = null;

              let importObject = {
                // имя файла модуля без расширения используется как имя внешнего объекта
                ❷ as_hello: {
                  // AssemblyScript передает строку с префиксом длины и простым индексом
                  ❸ console_log: function (index) {
                    // на случай вызова до выделения памяти
                    if (memory == null) {
                      console.log('memory buffer is null');
                      return;
                    }
                    ❹ const len_index = index - 4;

                    // нужно поделить на 2, чтобы перейти от байтов к 16-битным символам Unicode
                    ❺ const len = new Uint32Array(memory.buffer, len_index, 4)[0];
                    ❻ const str_bytes = new Uint16Array(memory.buffer,
                      index, len);

                    // декодировать байтовый массив utf-16 в строку JS
                    ❼ const log_string = new TextDecoder('utf-16').decode(str_bytes);
                    console.log(log_string);
                  }
                },
                env: {
                  abort: () => { }
                }
              }
```

```

};

(async () => {
  let obj = await WebAssembly.instantiate(new Uint8Array(bytes),
    importObject);

  // объект памяти, экспортированный из AssemblyScript
  ❶ memory = obj.instance.exports.memory;
  // вызов функции HelloWorld
  ❷ obj.instance.exports.HelloWorld();
})();

```

Модули WebAssembly, которые генерирует AssemblyScript, всегда создают и экспортируют свою собственную память, если они не скомпилированы с флагом `--import-Memory`. AssemblyScript по умолчанию создает собственную линейную память в модуле WebAssembly. Поэтому создавать линейный объект памяти в JavaScript не нужно. Вместо этого мы создаем переменную (`var`) с именем `memory` ❶, которую позже зададим объекту линейной памяти, экспортируемому модулем WebAssembly.

Внутри `importObject` объект, содержащий данные для импорта, должен иметь то же имя, что и импортирующий его файл AssemblyScript: `as_hello` ❷ для нашего файла AssemblyScript `as_hello.ts`. Внутри `as_hello` находится функция `console_log` ❸, которой передается строковый параметр при вызове из AssemblyScript. Когда модуль WebAssembly вызывает `as_hello`, функция JavaScript получает только один числовой индекс в линейную память WebAssembly, который является местоположением части строковых данных в строке с префиксом длины, которую AssemblyScript использует для определения своего строкового типа.

Длина – это 32-битное целое число в четырех байтах, предшествующее индексу. Чтобы получить индекс целого числа длины, мы вычитаем 4 из индекса строки. Используем значение длины, расположенное в линейной памяти, создавая новый массив `Uint32Array` с помощью передачи в `memory.buffer` значения `len_index` ❹ и значения 4 для количества байтов. Поскольку `Uint32Array` ❺ представляет собой массив 32-битных целых чисел, нам нужно получить первый и единственный элемент в массиве, используя `[0]`.

Мы извлекаем строковые байтовые данные из линейной памяти с помощью `new Uint16Array` ❻ и конвертируем этот байтовый массив в строку JavaScript с помощью нового `TextDecoder`, который декодирует текстовые данные utf-16. Код вызывает функцию декодирования `TextDecoder` ❼ `decode`, передавая строковые данные, возвращающие строку JavaScript, которую мы затем записываем в консоль. Используем IIFE для создания экземпляра модуля AssemblyScript WebAssembly. Обратите внимание, что перед вызовом функции `HelloWorld` ❶ мы должны установить объект `memory` ❶ в объект памяти, экспортированный из модуля WebAssembly. Функция `console_log` использует объект памяти, и если он не установлен, вызов `HelloWorld` ничего не даст.

К счастью, есть более простой способ перемещать строковые данные между AssemblyScript и JavaScript – использовать загрузчик AssemblyScript. Этот способ предоставлен командой разработчиков AssemblyScript. В разделе «Производительность загрузчика и прямые вызовы WebAssembly» мы увидим, сможем ли повысить производительность загрузчика AssemblyScript с помощью кода.

Приложение Hello World в загрузчике AssemblyScript

Загрузчик AssemblyScript – это набор вспомогательных функций от команды разработчиков AssemblyScript, призванный упростить выполнение вызовов AssemblyScript из JavaScript. Мы сравним код, который создали ранее, с кодом, созданным с помощью загрузчика AssemblyScript. Сначала обсудим простоту использования, а затем рассмотрим, насколько влияет на производительность использование загрузчика.

Мы используем загрузчик AssemblyScript, чтобы отправить строку из AssemblyScript обратно в JavaScript. Вспомогательная функция загрузчика преобразует индекс, поступающий из WebAssembly, в строку JavaScript. Теперь установим загрузчик с помощью npm:

```
npm install @assemblyscript/loader --save
```

Примечание Возможно, вам потребуется обновить Node.js, чтобы использовать загрузчик. На момент написания этой книги последней версии загрузчика AssemblyScript требуется Node.js версии 14.

Теперь мы создадим файл JavaScript для загрузки и запуска нашего модуля WebAssembly. Создайте файл с именем `as_hello_loader.js` и добавьте код из листинга 11.9.

Листинг 11.9. Использование загрузчика AssemblyScript для вызова модуля WebAssembly

```
as_hello ❶ const loader = require("@assemblyscript/loader");  
_loader.js const fs = require('fs');  
          ❷ var module;  
  
          const importObject = {  
            ❸ as_hello: {  
              ❹ console_log: (str_index) => {  
                ❺ console.log(module.exports.__getString(str_index));  
              }  
            }  
          };  
  
          (async () => {  
            let wasm = fs.readFileSync('as_hello.wasm');
```

```
❹ module = await loader.instantiate(wasm, importObject);
❺ module.exports.HelloWorld();
})();
```

Эта функция JavaScript сначала запрашивает ❶ загрузчик `AssemblyScript`. Мы используем объект загрузчика для загрузки объекта `module` ❷, который объявляем глобально. Объект `module` – это модуль загрузчика `AssemblyScript` с дополнительными вспомогательными функциями. Внутри `importObject` находится дочерний объект нашего модуля `AssemblyScript` с именем `as_hello` ❸. Именно здесь код `AssemblyScript` ожидает найти импортированные функции. Внутри объекта `as_hello` находится `console_log` ❹, который получает индекс строки `str_index` как единственный параметр. Эта функция использует функцию `__getString` ❺ для объекта `module`, созданного загрузчиком. Когда задается индекс строки, функция `__getString` извлекает строку JavaScript из линейной памяти. Эта строка выводится на консоль с `console.log`. Функция IIFE загружает модуль `AssemblyScript` с помощью объекта `loader` ❹. Наконец, IIFE вызывает функцию `HelloWorld` ❺. Когда вы запустите этот файл JavaScript с помощью `node`, то увидите результат, показанный в листинге 11.10.

Листинг 11.10. Выходные данные приложения Hello World на AssemblyScript

```
hello world!
```

Использование загрузчика `AssemblyScript` значительно упрощает код JavaScript. Позже, в разделе «Производительность загрузчика и прямые вызовы `WebAssembly`», мы рассмотрим его влияние на производительность.

Объединение строк `AssemblyScript`

Теперь, когда мы знаем, как получать строки из `AssemblyScript`, отправим строку в модуль `AssemblyScript`. Следующая функция объединяет две строки, разделенные вертикальной чертой (`|`). Мы воспользуемся загрузчиком, чтобы упростить написание кода на стороне JavaScript. Объединение строк – это такая функциональность, которую сложно реализовать непосредственно в WAT, но очень просто в `AssemblyScript`. Создайте новый файл с именем `as_concat.ts` и перенесите в него код из листинга 11.11.

Листинг 11.11. Объединение строк с помощью AssemblyScript

```
as_concat.ts ❶ export function cat( str1: string, str2: string ): string {
               ❷ return str1 + "|" + str2;
               }
```

Мы экспортируем **1** `cat`, который принимает два строковых параметра и возвращает строку. Эта функция объединяет **2** строки при помощи оператора «вертикальная черта» (`()`), расположенного между ними.

Теперь мы можем скомпилировать `as_concat.ts`, используя команду `asc` из листинга 11.12.

Листинг 11.12. Компиляция файла `as_concat.ts` с использованием `asc`

```
asc as_concat.ts --exportRuntime -Oz -o as_concat.wasm
```

Мы передаем флаг `--exportRuntime`, который необходим для передачи строк в модуль `WebAssembly`. Компиляция с флагом `--exportRuntime` добавляет код, который позволяет вызывать функцию `__allocString` из JavaScript. Если нам не удастся экспортировать среду выполнения, при выполнении приложения возникнет следующая ошибка:

```
TypeError: alloc is not a function
```

Когда вы компилируете `as_concat.ts` в WAT, обратите внимание, что файл WAT намного больше, чем наш `as_hello.ts`. Причина в том, что среда выполнения добавляет несколько строковых функций, которые выполняют важные задачи, такие как копирование памяти, объединение строк и методы получения/установки длины строки.

Теперь мы можем написать наше приложение на JavaScript. Код в листинге 11.13 создает две строки в линейной памяти и вызывает функцию `WebAssembly cat`. Создайте новый файл JavaScript с именем `as_concat.js` и добавьте в него код из листинга 11.13.

Листинг 11.13. JavaScript использует загрузчик `AssemblyScript` для вызова функции `AssemblyScript cat`

```
as_concat.js const fs = require('fs');
const loader = require("@assemblyscript/loader");

(async () => {
  let module = await loader.instantiate(fs.readFileSync('as_concat.wasm'));

  // необходимо использовать __newString, __getString
  // компиляция с флагом --exportRuntime
  1 let first_str_index = module.exports.__newString("first string");
  2 let second_str_index = module.exports.__newString("second string");
  3 let cat_str_index = module.exports.cat(first_str_index, second_str_index);
  4 let cat_string = module.exports.__getString(cat_str_index);
  5 console.log(cat_string);
})();
```

Функция `cat`, которую мы определили в модуле `WebAssembly`, не получает строку в качестве параметра напрямую, поэтому ей тре-

буется индекс в линейной памяти, определяющий местоположение строки. Вспомогательная функция загрузчика `module.exports.__newString` принимает строку JavaScript, копирует ее в линейную память и возвращает индекс для передачи в `module.cat`. Мы дважды вызываем `module.exports.__newString`, передавая "first string" ❶ и затем передавая "second string" ❷. Каждый из этих вызовов возвращает индекс, который мы храним в `first_str_index` и `second_str_index`. Затем вызываем `module.exports.cat`, передавая туда индексы, из которых получаем строковый индекс JavaScript, сохраняемый в `cat_str_index` ❸. Потом вызываем `module.exports.__getString` ❹, передавая `cat_str_index` и сохраняя эту строку в `cat_string` ❺, которую выводим в консоль.

Теперь, когда у нас есть коды на JavaScript и WebAssembly, мы можем запустить наше приложение с помощью `node`:

```
node as_concat.js
```

А вот вывод на вашу консоль:

```
first string|second string
```

Несмотря на все сказанное выше, AssemblyScript нужно многому научиться. Как видите, писать код для работы со строками в AssemblyScript намного проще, чем в WAT. Работать со строковыми данными именно в WebAssembly не обязательно, у вас всегда есть возможность сделать это в AssemblyScript. AssemblyScript, как и WebAssembly в более широком смысле, – быстро развивающийся проект. Уделите время на то, чтобы узнать больше об этом на домашней странице проекта по адресу assemblyscript.org, оно того стоит.

Объектно-ориентированное программирование на AssemblyScript

Объектно-ориентированное программирование (ООП) практически невозможно использовать в формате WAT, но поскольку AssemblyScript смоделирован на TypeScript, он предлагает значительно больше возможностей для ООП. В этом разделе мы рассмотрим некоторые основы ООП в AssemblyScript, а также некоторые его ограничения, которые могут быть исправлены в будущих версиях.

Начнем с создания нового файла AssemblyScript с именем `vector.ts`. Прямо сейчас AssemblyScript совмещает форматирование файлов TypeScript, которое работает в большинстве случаев.

Сауле Кабрера (Saule Cabrera) создал плагин сервера языка AssemblyScript для VS Code, который доступен по адресу <https://marketplace.visualstudio.com/items?itemName=saulecabrera.asls>.

Далее мы напишем класс `AssemblyScript Vector2D` для хранения координат объектов столкновения, аналогичных тому, что мы написали для приложения обнаружения столкновений в главе 8. Мы скомпилируем код в WAT, чтобы можно было изучить вывод компилятора `AssemblyScript`. Чем лучше вы разберетесь в компиляторе и его выводе, тем проще будет оптимизировать код `WebAssembly`. Добавьте код из листинга 11.14 в свой файл, чтобы создать класс `Vector2D`.

Листинг 11.14. Создание векторного класса в `AssemblyScript`

```
vector.ts ❶ export class Vector2D {  
    ❷ x: f32;  
    ❸ y: f32;  
  
    ❹ constructor(x: f32, y: f32) {  
        this.x = x;  
        this.y = y;  
    }  
  
    ❺ Magnitude(): f32 {  
        return Mathf.sqrt(this.x * this.x + this.y * this.y);  
    }  
}
```

Мы экспортируем класс с именем `Vector2D` ❶, который имеет два атрибута `x` ❷ и `y` ❸. Он также имеет `constructor` ❹, который создает новый объект `Vector2D` из параметров `x` и `y`. Метод `Magnitude` ❺ вычисляет величину вектора путем суммирования квадратов `x` и `y` и извлечения квадратного корня из этой суммы.

Если вы знакомы с `TypeScript`, то заметите, что этот код выглядит точно так же, как структуры `class`. Однако вместо типа `number` `TypeScript` мы используем тип `f32` для 32-битных чисел с плавающей запятой. Если вы используете тип `number` в своем `AssemblyScript`, это то же самое, что использовать 64-битные числа с плавающей запятой `f64`, которые в большинстве случаев дают худшую производительность среди всех типов `WebAssembly`.

Следующая команда `asc` компилирует `vector.ts` в файл WAT:

```
asc vector.ts -o vector.wat
```

Будет создан файл WAT, который мы можем изучить в VS Code. В `asc` мы передаем имя файла `AssemblyScript`, а затем передаем флаг `-o` с именем выходного файла `vector.wat`. Расширение определяет, будет ли вывод представлен в формате WAT или в виде двоичного файла `WebAssembly`. Откройте `vector.wat` и прокрутите немного вниз до экспорта, показанного в листинге 11.15.

```
vector.wat    ...
              (export "memory" (memory $0))
              (export "Vector2D" (global $vector/Vector2D))
              ❶ (export "Vector2D#get:x" (func $vector/Vector2D#get:x))
              ❷ (export "Vector2D#set:x" (func $vector/Vector2D#set:x))
              ❸ (export "Vector2D#get:y" (func $vector/Vector2D#get:y))
                (export "Vector2D#set:y" (func $vector/Vector2D#set:y))
              ❹ (export "Vector2D#constructor" (func $vector/Vector2D#constructor))
              ❺ (export "Vector2D#Magnitude" (func $vector/Vector2D#Magnitude))
              ...
```

Обратите внимание, как компилятор сгенерировал функции доступа `get` ❶ и `set` ❷ для атрибутов `x` ❷ и `y` ❸ и экспортировал их, чтобы вы могли получить к ним доступ из среды встраивания. Это говорит о том, что, когда пользователь задает атрибут объекта с помощью загрузчика, он вызывает функцию в модуле WebAssembly. Если вы задаете несколько атрибутов одновременно, лучше создать функцию, которая бы выполняла все это одновременно по соображениям производительности. Благодаря этому вам не придется выполнять несколько вызовов функций модуля WebAssembly. Вы также можете видеть, что модуль WebAssembly экспортировал функции `constructor` ❹ и `Magnitude` ❺.

В соглашениях об именах необходимо разбираться, если вы хотите вызывать функции в модуле WebAssembly из JavaScript. Все методы имеют префикс с именем класса и символом решетки (#) – (`Vector2D#`). У методов `set` и `get` есть суффикс, указывающий, какой атрибут они задают и получают, например `:x` или `:y`. Чтобы получить доступ к этим функциям и атрибутам из нашего JavaScript без применения загрузчика AssemblyScript, нам нужно использовать соглашение об именах.

Приватные атрибуты

Если вы не хотите экспортировать все атрибуты в среду встраивания, вам нужно использовать ключевое слово `private` перед атрибутами `x` и `y`. Сделайте это сейчас в своем AssemblyScript и перекомпилируйте с помощью команды `asc`. В листинге 11.16 показана новая версия.

Листинг 11.16. Создание приватных функций в AssemblyScript

```
vector.js    export class Vector2D {
              ❶ private x: f32;
              ❷ private y: f32;

              constructor(x: f32, y: f32) {
                this.x = x;
              }
            }
```

```

    this.y = y;
}

Magnitude(): f32 {
    return Mathf.sqrt(this.x * this.x + this.y * this.y);
}
}

```

Модификатор `private` ❶ перед `x` ❶ и `y` ❷ сообщает компилятору `AssemblyScript`, что эти атрибуты не должны быть общедоступными. Перекомпилируйте модуль `WebAssembly`, который больше не экспортирует методы доступа, задающие и получающие переменные `x` и `y`, в среду встраивания, как показано в листинге 11.17.

Листинг 11.17. Экспорт в файл `WAT`

```

vector.wat ...
(export "memory" (memory $0))
(export "Vector2D" (global $vector/Vector2D))
(export "Vector2D#constructor" (func $vector/Vector2D#constructor))
(export "Vector2D#Magnitude" (func $vector/Vector2D#Magnitude))
...

```

`TypeScript` имеет три модификатора: `public`, `private` и `protected`, – которые помогают определить, как можно получить доступ к атрибутам. Эти модификаторы в `AssemblyScript` ведут себя немного иначе, чем в других языках, таких как `TypeScript`. В большинстве языков защищенные атрибуты доступны для расширяющих классов, но недоступны за пределами родительского или дочернего класса. Метод `protected` в `AssemblyScript` реализован не полностью и ведет себя так же, как модификатор `public`. На данный момент вам следует избегать его использования, чтобы уклониться от путаницы. Ключевые слова могут со временем начать работать так же, как в `TypeScript`, но имейте в виду, что эти ограничения все еще существуют в `AssemblyScript` версии 0.17.7.

Модификатор `private` не позволяет `AssemblyScript` экспортировать методы `get` и `set` при компиляции модуля.

В отличие от других языков ООП, в `AssemblyScript` модификатор `private` не запрещает классам, расширяющим оригинал, обращаться к этому атрибуту.

Давайте воспользуемся следующей командой, чтобы скомпилировать наш `AssemblyScript` в модуль `WebAssembly`, дабы мы могли вызывать его из `JavaScript`:

```
asc vector.ts -o vector.wasm
```

Когда в `vector.wasm` мы меняем флаг `-o`, то указываем компилятору `asc` вывести двоичный файл `WebAssembly`. Это позволит нам за-

гружать и запускать модуль из среды встраивания JavaScript. Теперь давайте посмотрим, как загружать и вызывать функции WebAssembly с помощью Node.js.

Среда встраивания JavaScript

Мы будем использовать Node.js для загрузки и выполнения модуля WebAssembly. Если вместо этого для загрузки модуля WebAssembly из файловой системы и вызова `WebAssembly.instantiate` использовать браузер, JavaScript будет использовать `WebAssembly.instantiateStreaming` и `fetch` вместо `fs`.

Создайте файл `vector.js` и продублируйте код из листинга 11.18.

Листинг 11.18. Вызов функций в классе `Vector2D` AssemblyScript

```
vector.js ❶ const fs = require('fs');

❷ (async () => {
  ❸ let wasm = fs.readFileSync('vector.wasm');
  ❹ let obj = await WebAssembly.instantiate(wasm, {env: {abort: () => {}}});

  ❺ let Vector2D = {
    ❻ init: function (x, y) {
      return obj.instance.exports["Vector2D#constructor"](0, x, y)
    },
    ❼ Magnitude: obj.instance.exports["Vector2D#Magnitude"],
  }

  ❽ let vec1_id = Vector2D.init(3, 4);
  let vec2_id = Vector2D.init(4, 5);

  console.log(`
    ❾ vec1.magnitude=${Vector2D.Magnitude(vec1_id)}
    vec2.magnitude=${Vector2D.Magnitude(vec2_id)}
  `);
})();
```

Мы используем модуль Node.js `fs` ❶ для загрузки ❸ двоичных данных WebAssembly из файла внутри асинхронного IIFE ❷. Получив двоичные данные, мы передаем их в `WebAssembly.instantiate` ❹, который возвращает объект модуля WebAssembly. Затем создаем объект JavaScript `Vector2D` ❺, который отражает функции внутри модуля WebAssembly.

Мы создаем функцию `init` ❻, которая вызывает `Vector2D constructor` модуля WebAssembly, передавая значение 0 в качестве первого параметра. Передача этого значения в функцию `constructor` позволяет выбирать местоположение объекта в линейной памяти. Чтобы указать конструктору создавать новый объект в следующей доступной ячейке памяти, мы передаем 0. Затем функция вернет место в линейной памяти, где был создан этот объект. Атрибут `Magnitude` ❼ в `Vector2D` по-

лучает свое значение из `obj.instance.exports["Vector2D#Magnitude"]`, который является функцией в нашем модуле `WebAssembly`.

После определения объекта `Vector2D` в JavaScript мы дважды вызываем `Vector2D.init` ❸, чтобы создать в линейной памяти два объекта `WebAssembly Vector2D`, а также вернуть линейный адрес памяти этих объектов, который мы используем для вызовов методов. Затем дважды вызываем `Vector2D.Magnitude` внутри шаблонной строки `console.log`. Мы передаем идентификаторы векторов (`vec1_id` и `vec2_id`), которые сохранили в листинге 11.18, сообщаящие модулю `WebAssembly`, какой объект он использует. Функция `Magnitude` ❹ возвращает величину данного вектора, которую приложение записывает в консоль. Запустите это приложение с помощью `node`:

```
node vector.js
```

Вот результат:

```
vec1.magnitude=5  
vec2.magnitude=6.4031243324279785
```

Два значения – это длина нашего первого вектора, где $x = 3$ и $y = 4$, и длина второго вектора, где $x = 4$ и $y = 5$.

Теперь, когда мы знаем, как напрямую выполнять вызовы в наше приложение `AssemblyScript`, давайте посмотрите, как использовать загрузчик `AssemblyScript`, чтобы немного упростить кодирование в JavaScript.

Загрузчик *AssemblyScript*

Теперь мы изменим наш код `AssemblyScript`, чтобы использовать библиотеку загрузчика `AssemblyScript`. Это позволит нам сравнить методы взаимодействия с модулем `AssemblyScript` с точки зрения простоты использования и производительности. Как упоминалось ранее, важно понимать, когда можно повысить производительность вашего приложения и сколько усилий для этого потребуется. Данная информация поможет вам найти компромисс между временем разработки и производительностью приложения.

Откройте `vector_loader.ts` и вставьте код из листинга 11.19 для использования загрузчика `AssemblyScript`.

Листинг 11.19. Удаление модификатора `private` у атрибутов `x` и `y`

```
vector_loader.ts export class Vector2D {  
  ❶ x: f32;  
  ❷ y: f32;  
  
  constructor(x: f32, y: f32) {
```

```

    this.x = x;
    this.y = y;
  }

  Magnitude(): f32 {
    return Mathf.sqrt(this.x * this.x + this.y * this.y);
  }

  ❸ add(vec2: Vector2D): Vector2D {
    this.x += vec2.x;
    this.y += vec2.y;
    return this;
  }
}

```

В *vector_loader.ts* мы добавим пару изменений относительно *vector.ts*. Во-первых, мы удаляем модификаторы `private` у атрибутов **❶** `x` и **❷** `y`, чтобы можно было получить доступ к `x` и `y` из JavaScript. Во-вторых, создаем функцию `add` **❸**, которая прибавляет второй вектор. Эта функция позволяет нам складывать два вектора. В листинге 11.20 мы компилируем *vector_loader.ts* с помощью `asc`.

Листинг 11.20. Компиляция vector.ts в файл WebAssembly с помощью asc

```
asc vector_loader.ts -o vector_loader.wasm
```

Далее мы создадим новый файл JavaScript с именем *vector_loader.js*, чтобы могли запустить новый модуль WebAssembly. Скопируйте код из листинга 11.21 в *vector_loader.js*.

Листинг 11.21. Использование загрузчика AssemblyScript в JavaScript

```

vector_loader.ts  const fs = require('fs');
  ❶ const loader = require('@assemblyscript/loader');

  (async () => {
    let wasm = fs.readFileSync('vector_loader.wasm');
    // создать экземпляр модуля с помощью загрузчика
    ❷ let module = await loader.instantiate(wasm);

    // module.exports.Vector2D отражает класс AssemblyScript.
    ❸ let Vector2D = module.exports.Vector2D;

    ❹ let vector1 = new Vector2D(3, 4);
      let vector2 = new Vector2D(4, 5);

    ❺ vector2.y += 10;
    ❻ vector2.add(vector1);

    console.log(`
      ❼ vector1=(${vector1.x}, ${vector1.y})
    `);
  })();

```

```

    vector2=({vector2.x}, {vector2.y})
    vector1.magnitude=${vector1.Magnitude()}
    vector2.magnitude=${vector2.Magnitude()}
  `);
})();

```

При использовании загрузчика вы можете взаимодействовать с классами AssemblyScript почти так же, как и с классами JavaScript. Существует небольшая разница в том, что вы вызываете функцию конструктора `demangle` без использования оператора JavaScript `new`, необходимого, если бы эти классы были созданы в JavaScript. Однако после создания экземпляра объекта вы можете взаимодействовать с ним, как если бы он был написан на JavaScript.

Сначала нам потребуется загрузчик AssemblyScript ❶. Вместо функции `WebAssembly.instantiate` из IIFE мы вызываем функцию `loader.instantiate` ❷, которая возвращает модуль загрузчика. Этот модуль работает немного иначе, чем объект модуля `WebAssembly`, возвращаемый вызовом `WebAssembly.instantiate`. Загрузчик AssemblyScript добавляет функциональность, которая позволяет JavaScript работать с высокоуровневыми объектами AssemblyScript, такими как классы и строки.

Затем мы вызываем `loader.demangle`, передавая ему модуль, возвращенный `loader.instantiate`. Функция `demangle` возвращает структуру объекта, предоставляющую нам функции, которые мы можем использовать для создания экземпляров объектов из нашего модуля `WebAssembly`. Мы извлекаем функцию `Vector2D` ❸ из структуры объекта, чтобы использовать ее в качестве функции-конструктора для создания объектов `Vector2D` в JavaScript. Обратите внимание, что мы не использовали оператор `new` при создании экземпляра `Vector2D` ❹. Однако текущая версия загрузчика поддерживает использование оператора `new`.

Мы используем функцию `Vector2D` для создания объектов `vector1` и `vector2`, передавая для них значения `x` и `y`. Теперь можем применять эти объекты как обычные объекты JavaScript. Загрузчик все сделает за нас. Например, мы вызываем `vector2.y += 10` ❺, чтобы увеличить значение `vector2.y` на 10, а `vector2.add(vector1)` ❻ вызывает функцию `add` для объекта `vector2`, передавая `vector1`. В нашем вызове `console.log` ❼ мы можем использовать такие значения, как `vector1.x` и `vector1.y`.

Запустите JavaScript с помощью `node`:

```
node vector_loader.js
```

Вы должны увидеть следующий результат:

```

vector1=(3, 4)
vector2=(7, 19)

```

```
vector1.magnitude=5  
vector2.magnitude=20.248456954956055
```

Интерфейс загрузчика AssemblyScript позволяет вам работать с модулями WebAssembly, созданными на AssemblyScript, почти так же, как с классами, объектами и функциями, созданными в JavaScript. Это дает опыт, которого у вас может не быть при написании собственного интерфейса с модулем WebAssembly. Если у вас есть конкретные целевые показатели производительности, вам нужно будет провести дополнительное тестирование, чтобы убедиться, что загрузчик отвечает всем вашим потребностям. В следующем разделе мы расширим наш класс AssemblyScript посредством механизма наследования.

Расширение классов в AssemblyScript

ООП позволяет разработчикам расширять класс, добавляя дополнительные атрибуты или функции к базовому классу. Синтаксис расширения классов в AssemblyScript такой же, как и в TypeScript. В листинге 11.22 мы расширим класс Vector2D классом Vector3D, который добавит дополнительный атрибут z, представляющий третье измерение для нашего вектора.

Откройте файл `vector_loader.ts` и скопируйте код из листинга 11.22 после определения Vector2D.

Листинг 11.22. Расширение класса Vector2D с помощью класса Vector3D

```
vector_loader.ts ...  
❶ export class Vector3D extends Vector2D {  
  ❷ z: f32;  
  
  constructor(x: f32, y: f32, z: f32) {  
    ❸ super(x, y);  
    this.z = z;  
  }  
  
  ❹ Magnitude(): f32 {  
    return Mathf.sqrt(this.x * this.x + this.y * this.y + this.z * this.z);  
  }  
  
  add(vec3: Vector3D): Vector3D {  
    ❺ super.add(vec3);  
    ❻ this.z += vec3.z;  
    return this;  
  }  
}
```

Новый класс Vector3D ❶ сохраняет исходные атрибуты x и y, а также прибавляет третий атрибут z ❷ для третьего измерения. Конструктор класса вызывает super ❸, запускающий конструктор из класса Vector2D. Затем он задает значение this.z в параметр z, переданный

в конструктор. Мы переопределяем метод `Magnitude` ❹ из `Vector2D`, чтобы он учитывал третье измерение при вычислении величины вектора. Затем функция `add` вызывает функцию `add` класса `Vector2D`, используя `super.add` ❺, и увеличивает значение `this.z` ❻ с помощью значения атрибута `z` параметра `vec3`.

Теперь мы можем перекомпилировать наш модуль `WebAssembly`, используя `asc`:

```
asc vector_loader.ts -o vector_loader.wasm
```

Далее, в листинге 11.23, мы модифицируем файл `vector_loader.js` для извлечения класса `Vector3D`.

Листинг 11.23. JavaScript с загрузчиком AssemblyScript для загрузки классов Vector2D и Vector3D

```
vector_loader.js    const fs = require('fs');
                   const loader = require("@assemblyscript/loader");

                   (async () => {
                     let wasm = fs.readFileSync('vector_loader.wasm');
                     let module = await loader.instantiate(wasm);
                     ❶ let { Vector2D, Vector3D } = await loader.demangle(module).exports;

                     let vector1 = Vector2D(3, 4);
                     let vector2 = Vector2D(4, 5);
                     ❷ let vector3 = Vector3D(5, 6, 7);

                     vector2.y += 10;
                     vector2.add(vector1);
                     ❸ vector3.z++;

                     console.log(`
                       vector1=${vector1.x}, ${vector1.y}
                       vector2=${vector2.x}, ${vector2.y}
                       ❹ vector3=${vector3.x}, ${vector3.y}, ${vector3.z}

                       vector1.magnitude=${vector1.Magnitude()}
                       vector2.magnitude=${vector2.Magnitude()}
                       ❺ vector3.magnitude=${vector3.Magnitude()}
                     `);
                   })();
```

Мы модифицируем строку, которая получила функцию `Vector2D` из вызова `demangle`, и меняем ее, чтобы деструктурировать ❶ результат, создав переменную функций `Vector2D` и `Vector3D`. Создаем объект `vector3` ❷, используя функцию `Vector3D`, в которую передаем значения `x`, `y` и `z`. Увеличиваем `vector3.z` ❸ без какой-либо практической цели, просто чтобы показать, что можем это сделать. Внутри шаблонной строки, переданной в `console.log`, мы вставляем строку, которая отображает значения `x`, `y` и `z` ❹ в `vector3`, а также величину `vector3` ❺.

Запустив этот код JavaScript из командной строки с помощью node, вы должны получить результат, представленный в листинге 11.24.

Листинг 11.24. Вывод из vector_loader.js

```
vector1=(3, 4)
vector2=(7, 19)
vector3=(5, 6, 8)
vector1.magnitude=5
vector2.magnitude=20.248456954956055
vector3.magnitude=11.180339813232422
```

Теперь давайте сравним производительность при использовании загрузчика и прямых вызовов модуля WebAssembly.

Сравнение производительности загрузчика и прямых вызовов WebAssembly

Загрузчик AssemblyScript обеспечивает более очевидную структуру взаимодействия между модулем AssemblyScript и нашим JavaScript. В последнем разделе этой главы мы сравним использование загрузчика с прямыми вызовами модулей WebAssembly. Чтобы запустить данный тест, нам не нужно писать какой-либо дополнительный код AssemblyScript. Мы будем использовать модули WebAssembly, созданные ранее в этой главе, поэтому нам нужно только создать новый файл JavaScript для вызова существующих модулей. Создайте новый файл с именем *vector_perform.js* и добавьте код из листинга 11.25.

Листинг 11.25. Сравнение вызовов функций с помощью загрузчика и прямых вызовов

```
vector_perform.js  const fs = require('fs');
                   const loader = require("@assemblyscript/loader");

                   (async () => {
                     let importObject = {
                       env: {
                         abort: () => { }
                       }
                     };
                   };
                   let wasm = fs.readFileSync('vector_loader.wasm');
                   let module = await loader.instantiate(wasm);
                   let obj = await WebAssembly.instantiate(wasm, importObject);

                   // Этот класс JavaScript будет иметь все функции,
                   // экспортированные из AssemblyScript.
                   ❶ let dVector2D = {
                     // функция init вызывает конструктор Vector2D
                     init: function (x, y) {
                       return obj.instance.exports["Vector2D#constructor"](0, x, y)
```

```

    },
    getX: obj.instance.exports["Vector2D#x"],
    setX: obj.instance.exports["Vector2D#set:x"],
    getY: obj.instance.exports["Vector2D#y"],
    setY: obj.instance.exports["Vector2D#set:y"],
    Magnitude: obj.instance.exports["Vector2D#Magnitude"],
    add: obj.instance.exports["Vector2D#add"],
  }

  // Этот класс JavaScript будет иметь все функции,
  // экспортированные из AssemblyScript
  let dVector3D = {
    // функция init вызовет конструктор Vector3D
    init: function (x, y, z) {
      return obj.instance.exports["Vector3D#constructor"](0, x, y, z)
    },
    getX: obj.instance.exports["Vector3D#x"],
    setX: obj.instance.exports["Vector3D#set:x"],
    getY: obj.instance.exports["Vector3D#y"],
    setY: obj.instance.exports["Vector3D#set:y"],
    getZ: obj.instance.exports["Vector3D#z"],
    setZ: obj.instance.exports["Vector3D#set:z"],
    Magnitude: obj.instance.exports["Vector3D#Magnitude"],
    add: obj.instance.exports["Vector3D#add"],
  }

  // подготовка к записи в журнал времени,
  // необходимого для прямого выполнения функций
  ❷ let start_time_direct = (new Date()).getTime();

  ❸ let vec1_id = dVector2D.init(1, 2);
    let vec2_id = dVector2D.init(3, 4);
    let vec3_id = dVector3D.init(5, 6, 7);

  ❹ for (let i = 0; i < 1_000_000; i++) {
    dVector2D.add(vec1_id, vec2_id);
    dVector3D.setX(vec3_id, dVector3D.getX(vec3_id) + 10);
    dVector2D.setY(vec2_id, dVector2D.getY(vec2_id) + 1);
    dVector2D.Magnitude(vec2_id);
  }

  ❺ console.log("direct time=" + (new Date().getTime() - start_time_direct));

  ❻ let { Vector2D, Vector3D } = await loader.demangle(module).exports;

  ❼ let start_time_loader = (new Date()).getTime();

  ❽ let vector1 = Vector2D(1, 2);
    let vector2 = Vector2D(3, 4);
    let vector3 = Vector3D(5, 6, 7);

  ❾ for (i = 0; i < 1_000_000; i++) {
    vector1.add(vector2);
    vector3.x += 10;
    vector2.y++;
    vector2.Magnitude();
  }

```

```
}  
⑩ console.log("loader time=" + (new Date().getTime() - start_time_loader));  
})();
```

Теперь мы видим, во что нам обходится использование красивого синтаксиса загрузчика `AssemblyScript`. Этот JavaScript создает объект для хранения прямых вызовов класса `Vector2D` `AssemblyScript` `dVector2D` ❶ и один для класса `Vector3D` с именем `dVector3D`. Затем мы устанавливаем переменную `start_direct_time` ❷ на текущее время, которое будем использовать для отслеживания производительности, и инициализируем ❸ три векторных объекта. Два векторных объекта – это `Vector2D`, и один – `Vector3D`.

После инициализации векторов мы выполняем цикл один миллион раз ❹, обращаясь к этим объектам. Так как мы не тестировали каждую функцию, получился не совсем идеальный тест производительности. Цель состоит в том, чтобы просто получить некоторые числа и посмотреть, как они соотносятся. Пока мы выполняем одни и те же прямые вызовы и вызовы загрузчика, мы сможем получить разумное сравнение. Затем используем `console.log` ❺, чтобы вывести время, необходимое для инициализации векторов и прохождения по циклу. Первый цикл проверяет производительность прямого вызова модуля `WebAssembly` без использования загрузчика `AssemblyScript`. Далее код проверяет производительность модуля с загрузчиком.

Мы используем функцию `loader.demangle` ❻ для создания рабочих функций `Vector2D` и `Vector3D`. Затем инициализируем `start_time_loader` ❼ текущим временем и вызываем функции `Vector2D` ❽ и `Vector3D` для создания трех объектов, отражающих код в первом цикле ❶, который проверял прямые вызовы инициализации. Мы повторяем цикл один миллион раз ❹, выполняя те же функции, что и раньше, но теперь через загрузчик. Наконец, записываем в журнал (`log` ❿) количество времени, которое потребовалось для выполнения кода с помощью загрузчика.

Запустите `vector_perform.js` из командной строки с помощью `node`:

```
node vector_perform.js
```

Вот результат, который я получил при запуске файла:

```
direct time=74  
loader time=153
```

Как видите, версия, применяющая загрузчик, выполнялась примерно в два раза дольше. Разница становится еще более заметной, когда мы включаем вызовы инициализации в цикл. Если вы собираетесь использовать загрузчик `AssemblyScript`, лучше всего структурировать код таким образом, чтобы между JavaScript и `AssemblyScript` выполнялось как можно меньше вызовов.

Заключение

В этой главе вы узнали о языке высокого уровня AssemblyScript, интерфейсе командной строки AssemblyScript и команде `asc`, которую можете использовать для компиляции приложений AssemblyScript.

Мы создали функцию `AddInts` и приложение `Hello World`, чтобы показать, как разработка приложения на AssemblyScript соотносится с разработкой аналогичного приложения в WAT. Мы скомпилировали его в формат WAT, просмотрели код, сгенерированный компилятором AssemblyScript, и написали приложение на JavaScript, которое напрямую запускало приложение `Hello World`. Пройдя этот путь, вы узнали, как использовать WAT, чтобы взглянуть изнутри на WebAssembly, созданный компилятором AssemblyScript.

Затем мы установили загрузчик AssemblyScript и использовали функции JavaScript, созданные командой программистов AssemblyScript для облегчения разработки кода на JavaScript.

Обсудили работу со строками в AssemblyScript, написали приложение для объединения строк и рассмотрели, как нужно использовать дополнительные флаги с компилятором `asc`, чтобы разрешить `asc` включать дополнительные библиотеки WebAssembly при компиляции.

Во второй половине главы мы исследовали ООП в AssemblyScript. Создали класс и рассмотрели экспорт из созданного им WAT-файла. Изучили атрибуты типа `private` и то, как они не позволяют AssemblyScript выполнять экспорт, чтобы их не могла использовать среда встраивания. Мы написали код на JavaScript, который позволил нам напрямую создавать связующие классы, а затем использовали загрузчик AssemblyScript для создания связующего кода. Мы сравнили производительность методов прямого вызова и вызова через загрузчик. Наконец, расширили наш класс `Vector2D` классом `Vector3D` и обсудили различия между наследованием классов в AssemblyScript и TypeScript.

ПОСЛЕСЛОВИЕ

Спасибо, что прочитали мою книгу! Я надеюсь, что теперь вы понимаете, как WebAssembly работает на низком уровне, и уже захотели начать использовать его для разработки веб-приложений на языках высокого и низкого уровней. WebAssembly – молодая технология, но она уже доступна во всех основных браузерах. Используя Node.js, вы также можете разрабатывать высокопроизводительный серверный код в виде модулей WebAssembly. Со временем становится доступным все больше языков для разработки приложений WebAssembly. Добавляются новые функции и платформы. Будущее – это мир, в котором интернет будет безопасным, надежным и быстрым с WebAssembly.

Чтобы найти обновленные версии кода, представленного в этой книге и в других руководствах по WebAssembly, посетите <https://wasm-book.com>.

Пожалуйста, свяжитесь со мной, если вам понадобится помощь или возникнут вопросы:

Твиттер: [@battagline](https://twitter.com/battagline)

LinkedIn: <https://www.linkedin.com/in/battagline>

Открытый канал AssemblyScript в Discord: <https://discord.com/invite/assemblyscript>

GitHub: <https://github.com/battlelinegames/ArtOfWasm>

ПРЕДМЕТНЫЙ УКАЗАТЕЛЬ

anyfunc, 83
AssemblyScript, 283

DOM, 83
DRY-код, 203

Escape-символ, 115

S-выражения, 52
Source map, 258

WET-код, 203

Альфа-значение, 192
Анимация, 192

Байтовый индекс, 150

Время до интерактивности, 217

Дополнительный код, 96

Загрузчик AssemblyScript, 290
Значащая часть числа. См. *Значащие разряды*
Значащие разряды, 98

Изменяемые глобальные переменные, 46
Импорт функции, 62
Инвертирование битов, 108
Интерфейс командной строки, 284

Кодировка
 ASCII, 112
 UTF, 112
Команда распределения, 141
Комментарий WAT, 40
Контекст рисования, 189

Линейная память, 40
 страница, 41

Модуль WAT, 40

Начальный адрес, 152

Объединение строк, 291
Объектно-ориентированное программирование, 78
Основной код, 271
Остаток от деления, 127

Переключатель, 268
Переменные WAT
 глобальные, 46
 локальные, 46
Побитовое маскирование, 105
Показатель степени, 98
Полубайт, 103, 131
Порядок разрядов
 обратный (big-endian), 109
 прямой (little-endian), 109
Предупреждения об ошибках, 268

Профилировщик, 217

Сдвиг, 152

Сдвиг битов

линейный, 104

циклический, 104

Смещенный порядок числа, 100

Страница, 142

Стрелочная функция, 45

Строка

с завершающим нулем, 114

с префиксом длины, 117

Субнормальные числа, 101

Сумматор, 255

Таблица

функций, 83

переходов, 64

Точка останова, 278

Трассировка стека, 269

Удаление мертвого кода, 237

Указатель, 144

Шаг, 152

Элемент canvas, 187

Книги издательства «ДМК ПРЕСС»
можно купить оптом и в розницу
в книготорговой компании «Галактика»
(представляет интересы издательств
«ДМК ПРЕСС», «СОЛОН ПРЕСС», «КТК Галактика»).

Адрес: г. Москва, пр. Андропова, 38;

тел.: **(499) 782-38-89**, электронная почта: **books@aliants-kniga.ru**.

При оформлении заказа следует указать адрес (полностью),

по которому должны быть высланы книги;

фамилию, имя и отчество получателя.

Желательно также указать свой телефон и электронный адрес.

Эти книги вы можете заказать и в интернет-магазине: <http://www.galaktika-dmk.com/>.

Рик Батталини

Искусство WebAssembly

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com

Зам. главного редактора *Сенченкова Е. А.*

Редактор *Яценков В. С.*

Перевод *Бомбакова П. М.*

Корректор *Синяева Г. И.*

Верстка *Чаннова А. А.*

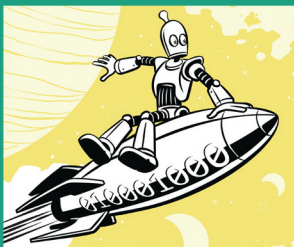
Дизайн обложки *Мовчан А. Г.*

Гарнитура PT Serif. Печать цифровая.

Усл. печ. л. 25,19. Тираж 200 экз.

Веб-сайт издательства: www.dmkpress.com

Эта книга поможет вам
разобраться в основах мощной
технологии WebAssembly
и повысить производительность
ваших веб-приложений.



WebAssembly – это быстрая компактная межплатформенная технология, которая оптимизирует производительность ресурсоемких веб-приложений и программ. В книге подробно рассматриваются принципы ее работы; показано, в каких случаях можно ее использовать, а в каких делать этого не стоит, и как создавать и развертывать приложения на основе WebAssembly. Вначале вы узнаете, как оптимизировать и компилировать низкоуровневый код, отлаживать и оценивать WebAssembly, а также представлять код в удобном для прочтения текстовом формате WebAssembly Text (WAT). Затем вы сможете создать программу обнаружения столкновений на базе браузера, поработать с технологиями рендеринга в браузере для создания графики и анимации, а также выяснить, как Web-Assembly взаимодействует с другими языками программирования.

Вы научитесь:

- встраивать приложения WebAssembly в веб-браузеры и Node.js;
- оценивать код WebAssembly с помощью браузерных программ отладки;
- форматировать переменные, циклы, функции, строки, структуры данных и условную логику в WAT;
- управлять памятью;
- создавать программы для генерации графических объектов и обнаружения столкновений между ними;
- оценивать выходные данные компилятора WebAssembly.

Рик Баттальяни – разработчик игр и основатель BattleLine Games LLC, независимой студии, специализирующейся на разработке веб-игр. Он создал сотни игровых продуктов с использованием различных веб-технологий, в числе которых WebAssembly, HTML5 и WebGL. Баттальяни входит в состав WebAssembly Community Group и AssemblyScript Community Group.

Интернет-магазин:
www.dmkpress.com
Оптовая продажа:
КТК «Галактика»
books@aliens-kniga.ru



www.dmk.pf

ISBN 978-5-97060-976-7



9 785970 609767 >